compiler-design-digitized 2
computer-architecture-digitized 27
computer-networks-digitized 104
database-systems-digitized 159
data-structures-algorithms-digitized 208
digital-logic-digitized 235
discrete-mathematics-digitized 250
engineering-mathematics-digitized 294
operating-system-digitized 312
theory-of-computation-digitized 360

Compiler Design

Comprehensive Study Notes for GATE Computer Science

Ankur Gupta

www.ankurgupta.net

Note: These notes have been prepared primarily from the following books:

(1) **Principles of Compiler Design** by Aho, Ullman

Please refer to the above books for more details.

Contents

1 Compiler Fundamentals **6**
- 1.1 Introduction to Compilers . . . 6
  - 1.1.1 Major Parts of a Compiler . . . 6
  - 1.1.2 Compiler Architecture Diagram . . . 7
- 1.2 Phases and Passes . . . 7
  - 1.2.1 Phases . . . 7
  - 1.2.2 Cross Compiler . . . 7

2 Lexical Analysis **8**
- 2.1 Overview . . . 8
  - 2.1.1 Tokens and Attributes . . . 8
  - 2.1.2 Process Flow . . . 8

3 Syntax Analysis **9**
- 3.1 Introduction . . . 9
  - 3.1.1 Key Concepts . . . 9
- 3.2 Parser Types . . . 9
  - 3.2.1 Top-Down Parsers . . . 9
  - 3.2.2 Bottom-Up Parsers . . . 9
- 3.3 Ambiguity in Grammars . . . 9
  - 3.3.1 Example of Disambiguation . . . 10

4 Parsing Techniques **11**
- 4.1 Top-Down Parsing . . . 11
  - 4.1.1 Recursive Descent Parsing . . . 11
  - 4.1.2 Predictive Parsing . . . 11
- 4.2 Predictive Parser . . . 11
  - 4.2.1 Types of Predictive Parsing . . . 11
- 4.3 Left Recursion . . . 11
  - 4.3.1 Left Factoring . . . 12
- 4.4 YACC (Yet Another Compiler Compiler) . . . 12

5 LL(1) Parsing **13**
- 5.1 LL(1) Grammars . . . 13
  - 5.1.1 Properties of LL(1) Grammars . . . 13
- 5.2 Constructing LL(1) Parsing Tables . . . 13
  - 5.2.1 Algorithm for Constructing LL(1) Parsing Table . . . 13
  - 5.2.2 Computing FIRST for Any String X . . . 14
  - 5.2.3 Computing FOLLOW (for non-terminals) . . . 14

5.3 Example: LL(1) Parsing Table Construction 14
5.3.1 LL(1) Parsing Table 15

6 Bottom-Up Parsing **16**
6.1 Introduction 16
6.1.1 Handle 16
6.2 Shift-Reduce Parsing Conflicts 16
6.2.1 Shift/Reduce Conflict 16
6.2.2 Reduce/Reduce Conflict 16

7 LR Parsing **17**
7.1 LR Grammar Types 17
7.1.1 Grammar Hierarchy 17
7.2 LR Parsing 17
7.2.1 Unambiguous Grammars 17

8 Operator Precedence Parsing **18**
8.1 Introduction 18
8.1.1 Operator Grammar Properties 18
8.2 Using Precedence Relations to Find Handles 18
8.2.1 Precedence Relation Rules 18
8.2.2 Precedence Table Example 18
8.3 LR Parsing Example 18
8.3.1 Advantages 19
8.3.2 Disadvantages 19

9 Intermediate Code Generation **20**
9.1 Overview 20
9.1.1 Three Address Code 20
9.2 Representation Forms 20
9.2.1 Quadruples 20
9.2.2 Triples 20
9.2.3 Indirect Triples 21

10 Code Optimization **22**
10.1 Common Sub-expression Elimination 22
10.2 Compile Time Evaluation 22
10.3 Variable Propagation 22
10.4 Dead Code Elimination 23
10.5 Code Motion 23
10.6 Induction Variables and Strength Reduction 23

11 Programming Language Principles **24**
11.1 Typed versus Untyped Languages 24
11.1.1 Typed Languages 24
11.1.2 Weak and Strong Typing 24
11.2 Static versus Dynamic Typing 24
11.2.1 Static Typing 24
11.3 Storage Management 24

11.3.1 Activation Records 24
11.3.2 Types of Storage Management 25

Chapter 1

Compiler Fundamentals

1.1 Introduction to Compilers

A compiler is a program that translates a high-level language program into a low-level language (machine code). The process involves multiple phases organized into two major groups.

1.1.1 Major Parts of a Compiler

The compilation process consists of two major phases:

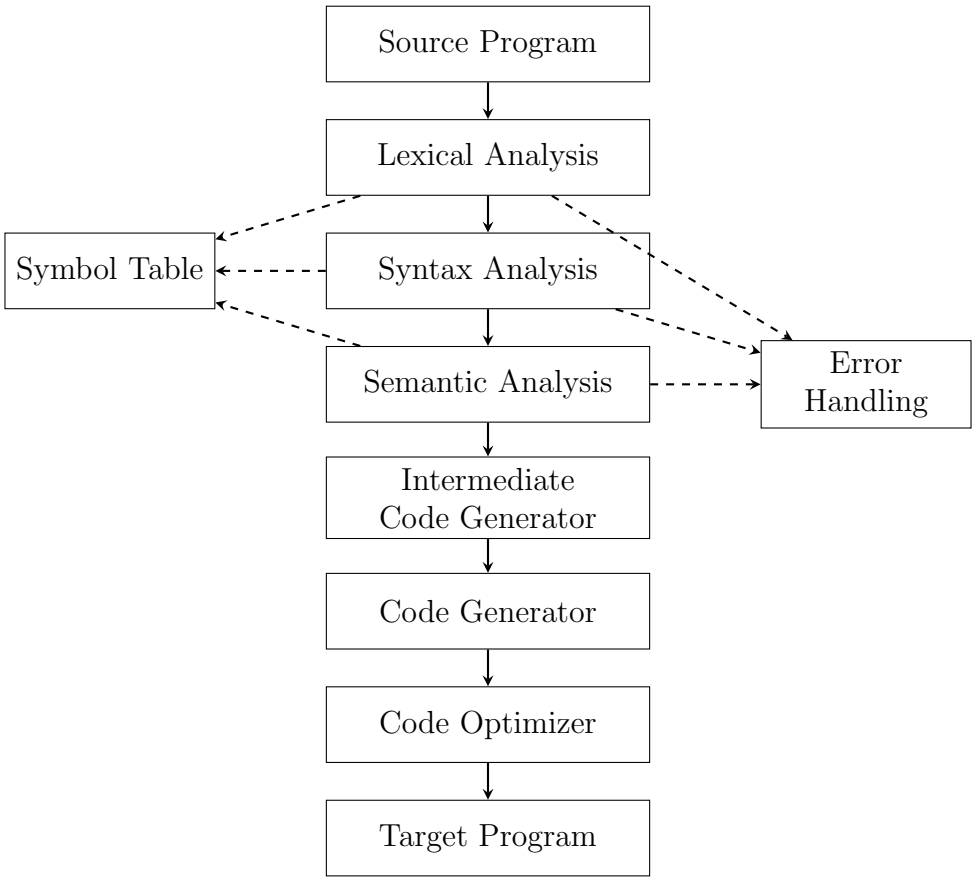
(1) **Analysis Phase:** An intermediate representation is created from the given source program

- Lexical Analyzer
- Syntax Analyzer
- Semantic Analyzer (three phases in total)

(2) **Synthesis Phase:** An equivalent target program is created from this intermediate representation

- Intermediate Code Generator
- Code Generator
- Code Optimizer (three phases in total)

1.1.2 Compiler Architecture Diagram



1.2 Phases and Passes

1.2.1 Phases

Phases refer to the number of times the compiler has to traverse through the entire program. Several phases are grouped into one pass.

1.2.2 Cross Compiler

A cross compiler is a compiler that runs on one machine (A) and produces code for another machine (B).

Chapter 2

Lexical Analysis

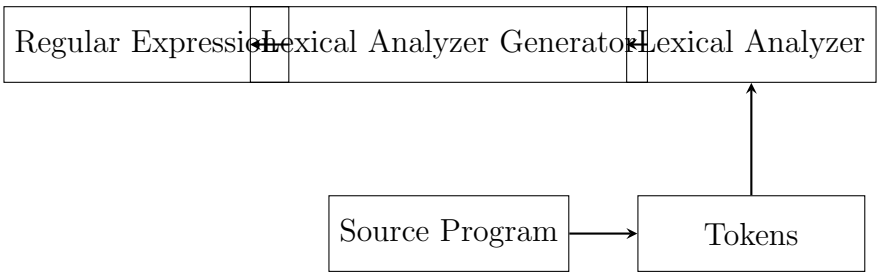
2.1 Overview

The Lexical Analyzer reads a source program character by character to produce tokens.

2.1.1 Tokens and Attributes

- A token can represent more than one lexeme; additional information should be held for that specific lexeme. This additional information is called the **attribute of the token**.
- Token type and its attribute uniquely identify a lexeme.
- Regular expressions are used to specify patterns.
- Finite Automata are used to recognize tokens.

2.1.2 Process Flow



Chapter 3

Syntax Analysis

3.1 Introduction

The Syntax Analyzer is also known as a parser. The syntax of a programming language is described by a Context-Free Grammar (CFG).

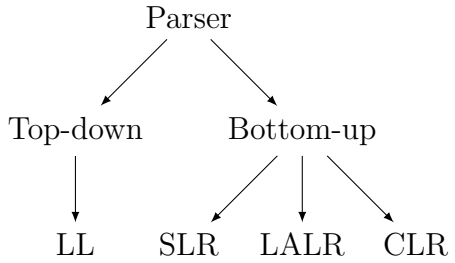
3.1.1 Key Concepts

- A CFG gives a precise syntactic specification of a programming language.
- The parser works on a stream of tokens.

3.2 Parser Types

3.2.1 Top-Down Parsers

Top-down parsers try to find the left-most derivation of the given source program.



Top-down parsers try to find the left-most derivation of the given source program.

Bottom-up parsers try to find the right-most derivation of the given source program in reverse order.

3.2.2 Bottom-Up Parsers

Both top-down and bottom-up parsers scan the input from left to right.

3.3 Ambiguity in Grammars

An unambiguous grammar should be written to eliminate ambiguity.

3.3.1 Example of Disambiguation

Consider the ambiguous grammar:

$$E \rightarrow E + E \mid E * E \mid E \wedge E \mid id \mid (E)$$

where $\wedge$ represents exponentiation (right associative) and + represents addition (left associative).

Solution: To eliminate ambiguity, we rewrite the grammar as:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow G \wedge F \mid G \\ G &\rightarrow id \mid (E) \end{aligned}$$

Chapter 4

Parsing Techniques

4.1 Top-Down Parsing

4.1.1 Recursive Descent Parsing

- Backtracking is needed, which is not efficient
- Not used for general purpose parsing

4.1.2 Predictive Parsing

- No backtracking is required, making it efficient
- Uses LL(1) grammars

4.2 Predictive Parser

A predictive parser can uniquely choose a production rule by just looking at the current symbol.

The Non-Recursive (Table Driven) Predictive parser is also known as the LL(1) parser. We eliminate the left recursion in the grammar, and left-factor it. However, it may not be suitable for predictive parsing (i.e., not an LL(1) grammar).

4.2.1 Types of Predictive Parsing

(1) Recursive Predictive Parsing

(2) Non-Recursive Predictive Parsing (LL(1))

4.3 Left Recursion

Top-down techniques cannot handle left-recursive grammars.

4.3.1 Left Factoring

A predictive parser (a top-down parser without backtracking) insists that the grammar must be left-factored.

Example: Consider the grammar:

$$A \to a\beta B \mid a\beta \mid cdg \mid cdeB \mid cdf\beta$$

After left factoring:

$$A \to aA' \mid cdg \mid cdeB \mid cdfB$$
$$A' \to \beta B \mid \beta$$

Further factoring:

$$A \to aA'TcdA', \quad A' \to \beta B \mid B$$
$$A' \to g \mid eB \mid fB$$

4.4 YACC (Yet Another Compiler Compiler)

YACC generates C code for a syntax analyzer or parser.



Chapter 5

LL(1) Parsing

5.1 LL(1) Grammars

LL(1) stands for:

- First L: Input is scanned from left to right
- Second L: Left-most derivation is used as a lookahead symbol to determine parser action
- 1: One input symbol is used

5.1.1 Properties of LL(1) Grammars

- A grammar whose parsing table has no multiple entries is said to be an LL(1) grammar.
- A left-recursive grammar cannot be an LL(1) grammar.
- A grammar that is not left-factored cannot be an LL(1) grammar.
- An ambiguous grammar cannot be an LL(1) grammar.

5.2 Constructing LL(1) Parsing Tables

5.2.1 Algorithm for Constructing LL(1) Parsing Table

For each production rule $A \to \alpha$ of a grammar G:

(1) For each terminal a in FIRST(α), add $A \to \alpha$ to $M[A, a]$

(2) If Λ (epsilon) is in FIRST(α), then:

- For each terminal a in FOLLOW(A), add $A \to \alpha$ to $M[A, a]$
- If Λ is in FIRST(α) and \$ is in FOLLOW(A), then add $A \to \alpha$ to $M[A, \$]$

All other undefined entries of the parsing table are error entries.

5.2.2 Computing FIRST for Any String X

- If X is a terminal symbol: FIRST(X) = $\{X\}$
- If X is a non-terminal symbol and $X \rightarrow A$ is a production rule, then Λ is in FIRST(X)
- If X is a non-terminal symbol and $X \rightarrow Y_1 Y_2 \dots Y_n$ is a production rule, then:
  - (i) If a terminal symbol a is in FIRST(Y_1) and Λ is in all FIRST(Y_i) for $i = 1, 2, \dots, j-1$, then a is in FIRST(X)
  - (ii) If Λ is in all FIRST(Y_i) for $i = 1, \dots, n$, then Λ is in FIRST(X)

5.2.3 Computing FOLLOW (for non-terminals)

- If S is the start symbol, then \$ is in FOLLOW(S)
- If $A \rightarrow \alpha B \beta$ is a production rule, then everything in FIRST(β) is in FOLLOW(B) except Λ
- If ($A \rightarrow \alpha B$) or ($A \rightarrow \alpha B \beta$ and Λ is in FIRST(β)), then everything in FOLLOW(A) is in FOLLOW(B)

5.3 Example: LL(1) Parsing Table Construction

Consider the grammar:

$$
\begin{aligned}
&E \rightarrow TE', \quad E' \rightarrow +TE' \mid \Lambda \\
&T \rightarrow FT', \quad T' \rightarrow *FT' \mid \Lambda \\
&F \rightarrow (E) \mid id
\end{aligned}
$$

Solution:
First, compute FIRST sets:

$$
\begin{aligned}
\text{FIRST}(F) &= \{(, id\} \\
\text{FIRST}(T) &= \{*, \Lambda\}, \quad \text{FIRST}(T) = \{(, id\} \\
\text{FIRST}(E') &= \{+, \Lambda\}, \quad \text{FIRST}(E) = \{(, id\} \\
\text{FIRST}(TE') &= \{(, id\}, \quad \text{FIRST}(+TE') = \{+\} \\
\text{FIRST}(*FT') &= \{*\}, \quad \text{FIRST}(FT') = \{(, id\} \\
\text{FIRST}((E)) &= \{(\} \\
\text{FIRST}(id) &= \{id\}
\end{aligned}
$$

Next, compute FOLLOW sets:

$$
\begin{aligned}
&\text{FOLLOW}(F) = \{\$,)\}, \quad \text{FOLLOW}(E) = \{\$,)\} \\
&\text{FOLLOW}(T) = \{+,), \$\}, \quad \text{FOLLOW}(T') = \{+,), \$\} \\
&\text{FOLLOW}(E) = \{*, +,), \$\}
\end{aligned}
$$

5.3.1 LL(1) Parsing Table

	id	+	*	(	**$**
E	$E \to TE'$			$E \to TE'$	
E'		$E' \to +TE'$			$E' \to \Lambda$
T	$T \to FT'$			$T \to FT'$	
T'		$T' \to \Lambda$	$T' \to *FT'$		$T' \to \Lambda$
F	$F \to id$			$F \to (E)$	

Chapter 6

Bottom-Up Parsing

6.1 Introduction

Bottom-up parsing is also known as shift-reduce parsing.

6.1.1 Handle

If the grammar is unambiguous, then every right-sentential form of the grammar has exactly one handle.

A right-most derivation in reverse can be obtained by handle-pruning.

6.2 Shift-Reduce Parsing Conflicts

6.2.1 Shift/Reduce Conflict

This conflict occurs when the parser cannot decide which of several reductions to make.

6.2.2 Reduce/Reduce Conflict

The parser cannot decide which of several reductions to make.

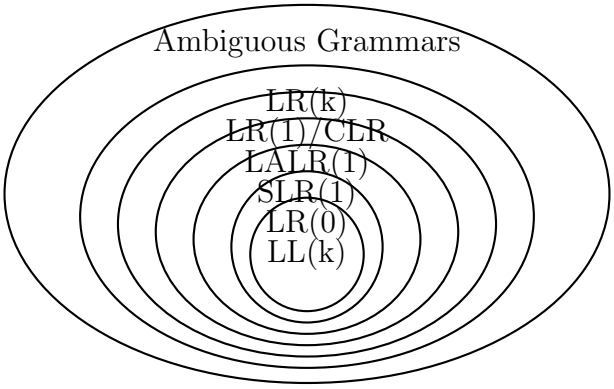
If a shift-reduce parser cannot be used for a grammar, that grammar is called a Non-LR(k) grammar.

Chapter 7

LR Parsing

7.1 LR Grammar Types

LR grammars represent a family of languages that can be parsed using LR techniques:



7.1.1 Grammar Hierarchy

$$\text{LL(k)} \subset \text{LR(k)}$$
$$\text{LR(0)} \subset \text{SLR(1)} \subset \text{LALR(1)} \subset \text{LR(1)/CLR} \subset \text{LR(k)}$$

Canonical LR is also referred to as LR(1).

7.2 LR Parsing

LR parsing is a non-backtracking shift-reduce parsing technique.

An LR parser can detect a syntactic error as soon as it is possible to do so in a left-to-right scan of the input.

7.2.1 Unambiguous Grammars

The diagram shows the relationship between different grammar classes, with unambiguous grammars forming nested subsets, and ambiguous grammars existing outside these structured families.

Chapter 8

Operator Precedence Parsing

8.1 Introduction

Operator precedence parsing is a small but important class of grammars.

8.1.1 Operator Grammar Properties

In an operator grammar, no production rule can have:

(i) Λ at the right side

(ii) Two adjacent or non-terminals at the right side

8.2 Using Precedence Relations to Find Handles

8.2.1 Precedence Relation Rules

(1) Scan the string from the left end until the first $>$ is encountered

(2) Then scan backwards (to the left) over any $=$ until a $<$ is encountered

(3) The handle contains everything to the left of the first $>$ and to the right of the $<$ is encountered

8.2.2 Precedence Table Example

	id	+	*	**$**
id		$>$	$>$	$>$
+	$<$	$>$	$<$	$>$
*	$<$	$>$	$>$	$>$
$	$<$	$<$	$<$	

8.3 LR Parsing Example

Consider the grammar: $E \to E + E$, $E \to E * E$, $E \to id$

Stack	**Input**	**Action**
$	$id + id * id$ $	Shift
$ id	$+id * id$ $	Reduce
$ E	$+id * id$ $	Shift
$ E +	$id * id$ $	Shift
$ E + id	$*id$ $	Reduce
$ E + E	$*id$ $	Shift
$ E + E *	id $	Shift
$ E + E * id	$	Reduce
$ E + E * E	$	Reduce
$ E + E	$	Reduce
$ E	$	Accept

8.3.1 Advantages

Operator precedence parsing is powerful enough for expressions in programming languages.

8.3.2 Disadvantages

(1) It cannot handle the unary minus (the lexical analyzer should handle this)

(2) It is difficult to decide which language is recognized by the grammar

Chapter 9

Intermediate Code Generation

9.1 Overview

Intermediate codes are machine-independent codes that are close to machine instructions. Syntax trees, Postfix Notation, and Three-address codes can be used as intermediate languages.

9.1.1 Three Address Code

Three address code is represented as:

$$A = B * (C + D)$$

Three address code is as follows:

$$\begin{aligned} T_1 &= -B \\ T_2 &= C + D \\ T_3 &= T_1 * T_2 \\ A &= T_3 \end{aligned}$$

9.2 Representation Forms

9.2.1 Quadruples

	Operator	**Operand1**	**Operand2**	**Result**
(1)	$-$	B		T_1
(2)	$+$	C	D	T_2
(3)	$*$	T_1	T_2	T_3
(4)	$=$	T_3		A

9.2.2 Triples

	Operator	**Operand1**	**Operand2**
(1)	$-$	B	
(2)	$+$	C	D
(3)	$*$	(1)	(2)
(4)	$=$	A	(3)

9.2.3 Indirect Triples

Statement	
(0)	(56)
(1)	(57)
(2)	(58)
(3)	(59)

	Operator	**Operand1**	**Operand2**
(56)	$-$	B	
(57)	$+$	C	D
(58)	$*$	(56)	(57)
(59)	$=$	A	(58)

Chapter 10

Code Optimization

10.1 Common Sub-expression Elimination

If an expression has already been computed and the values of variables in the expression have not changed, we can use the previously computed value.

Example:

$$\begin{aligned} a = (b * c) \quad &\Rightarrow \quad a = T_1 \\ T_1 = b * c & \\ d = (b * c) + x - y \quad &\Rightarrow \quad d = T_1 + x - y \end{aligned}$$

10.2 Compile Time Evaluation

If all operands of an expression are constants, the expression can be evaluated at compile time.

Example:

$$A = 2 * (22.0/7.0) * r$$

Perform $2 * (22.0/7.0)$ at compile time.

Example 2:

$$\begin{aligned} x = 12.4 \quad &\Rightarrow \quad \text{Evaluate } x/2.3 \text{ at compile time} \\ y = x/2.3 & \end{aligned}$$

10.3 Variable Propagation

Example:

$$\begin{array}{lcl} c = a * b & & c = a * b \\ x = a & \Rightarrow & x = a \\ d = a * b + 4 & & d = a * b + 4 \end{array}$$

After variable propagation, $a*b$ and $x*b$ will be identified as common subexpressions.

10.4 Dead Code Elimination

Variable propagation often leads to making assignment statements into dead code.

Example:

$$\begin{array}{ll} c = a * b & \\ x = a & \Rightarrow \\ d = a * b + 4 & \end{array} \qquad \begin{array}{l} c = a * b \\ \\ d = a * b + 4 \end{array}$$

10.5 Code Motion

This technique reduces the evaluation frequency of expressions by bringing loop-invariant statements out of the loop.

Example:

$$\begin{array}{l} \qquad\quad a = 200; \\ \text{while } (a > 0) \quad b = x + y; \\ \qquad\quad \{ \quad \text{while } (a > 0) \\ \qquad\quad\quad b = x + y; \quad \Rightarrow \quad \{ \\ \qquad \text{if } (a \times b == 0) \\ \qquad\quad \text{print}(\text{“x d”}, 0); \\ \qquad\quad \} \quad \} \end{array} \qquad \begin{array}{l} a = 200; \\ \\ \\ \\ \text{if } (a \times b == 0) \\ \text{print}(\text{“x d”}, 0); \end{array}$$

10.6 Induction Variables and Strength Reduction

An induction variable is used in a loop for the following kind of assignment:

$$i = i + \text{constant}$$

Strength reduction means replacing the high-strength operator by a low-strength operator.

Example:

$$\begin{array}{l} \qquad\quad i = 1; \\ \text{while } (i < 10) \quad j = 4; \\ \qquad\quad \{ \quad \text{while } (i < 40) \\ \qquad\quad\quad y = j * 4; \quad \Rightarrow \quad \{ \\ \qquad\quad \} \quad y = 1; \\ \qquad\quad\quad i = i + 4; \\ \qquad\quad\quad \} \\ \qquad\quad\quad \} \end{array} \qquad \begin{array}{l} i = 1; \end{array}$$

Chapter 11

Programming Language Principles

11.1 Typed versus Untyped Languages

11.1.1 Typed Languages

A language is typed if the specification of every operation defines the types of data to which the operation is applicable.

In contrast, an untyped language (such as most assembly languages) allows any operation to be performed on any data, which are generally considered to be sequences of bits of various lengths.

11.1.2 Weak and Strong Typing

- **Weak typing** allows a value of one type to be treated as another.
- **Strong typing** prevents the above situation.

Strongly typed languages are often termed as type-safe.

11.2 Static versus Dynamic Typing

11.2.1 Static Typing

A programming language is said to be statically typed when type checking is performed during compile time.

A programming language is said to be dynamically typed when the majority of its type checking is performed at run-time.

In dynamically typed languages, values have types but variables do not; i.e., a variable can refer to a value of any type.

11.3 Storage Management

11.3.1 Activation Records

Information needed by a single execution of a procedure is managed using a contiguous block of storage called an activation record.

An activation record is allocated when a procedure is entered, and it is deallocated when that procedure exits.

11.3.2 Types of Storage Management

(1) **Static Storage Management:**

Static allocation is done during compilation that remains fixed throughout execution. It requires no run-time storage management software. Global data in C is allocated using static storage. It is very efficient, but it is incompatible with recursive subprogram calls.

(2) **Dynamic Storage Management:**

Also termed as heap storage management. The need for heap storage arises when a language permits storage to be allocated and freed at arbitrary points during program execution.

(3) **Stack-based Storage Management:**

It is used when storage requirements are not known at compile time, but the requests obey a Last In First Out order.

Examples:

- Local variables in a procedure in C
- Procedure call information (return address, etc.)

Stack-based allocation is used in recursive sub-programs.

Computer Organization and Architecture

Comprehensive Study Notes for GATE Computer Science

Ankur Gupta

www.ankurgupta.net

Note: These notes have been prepared primarily from the following books:

(1) **Computer System Architecture** by M. Morris Mano

(2) **Computer Organization** by Carl Hamacher, Zvonko Vranesic, Safwat Zaky

(3) **Operating System Concepts** by the Silberschatz, Galvin, Gagne

Please refer to the above books for more details.

Contents

1 Number Representation **9**
- 1.1 Floating-Point Representation . . . 9
- 1.2 Normalized Floating-Point Numbers . . . 9

2 Register Transfer and Microoperations **10**
- 2.1 Micro-operation . . . 10
- 2.2 Register Transfer Language . . . 10
- 2.3 Registers . . . 10
  - 2.3.1 Register Transfer . . . 10
  - 2.3.2 Control Function . . . 11
- 2.4 Bus and Memory Transfers . . . 11
  - 2.4.1 Common Bus . . . 11
  - 2.4.2 Bus System for 4 Registers . . . 11
- 2.5 Bus Transfer . . . 11
- 2.6 Memory Transfer . . . 11
  - 2.6.1 Micro-operations . . . 12

3 Arithmetic Microoperations **13**
- 3.1 Basic Operations . . . 13
- 3.2 Arithmetic Shift . . . 13

4 Logic Microoperations **14**
- 4.1 Basic Logic Operations . . . 14
- 4.2 List of Logic Microoperations . . . 14

5 Shift Microoperations **15**
- 5.1 Shift-Left Operation . . . 15
- 5.2 Shift-Right Operation . . . 15
- 5.3 Logical Shift . . . 15
- 5.4 Circular Shift . . . 15
- 5.5 Arithmetic Shift . . . 16
  - 5.5.1 Checking for overflow . . . 16

6 Arithmetic Logic Shift Unit (ALU) **17**
- 6.1 Overview . . . 17
- 6.2 Checking for overflow . . . 17

7 Machine Instructions and Addressing Modes 18
7.1 Instruction Formats . . . 18
7.2 Types of CPU Organizations . . . 18
7.2.1 Single Accumulator Organization . . . 19
7.2.2 General Register Organization . . . 19
7.2.3 Stack Organization . . . 19

8 CISC and RISC 20
8.1 CISC . . . 20
8.2 RISC . . . 20

9 Addressing Modes 22
9.1 Implied Mode . . . 22
9.2 Immediate Mode . . . 23
9.3 Register Mode . . . 23
9.4 Register Indirect Mode . . . 23
9.5 Autoincrement or Autodecrement Mode . . . 23
9.6 Effective Address . . . 23
9.7 Direct Address Mode . . . 23
9.8 Indirect Address Mode . . . 23
9.9 Relative Address Mode . . . 24
9.10 Indexed Addressing Mode . . . 24
9.11 Base Register Addressing Mode . . . 24

10 Central Processing Unit 25
10.1 Overview . . . 25
10.2 Some Fundamental Concepts . . . 25

11 Single-Bus Organization of the Data Path Inside the CPU 26
11.1 Steps to Execute an Instruction . . . 26

12 Execution of a Complete Instruction 28
12.1 Steps Required to Execute the Above Instruction . . . 28

13 CPU Control Design 29
13.1 Hardwired Control Organization . . . 29
13.2 Microprogrammed Control Organization . . . 29

14 Micro-programmed Control 30
14.1 Overview . . . 30
14.2 Control Word . . . 30
14.3 Basic Organization of a Microprogrammed Control Unit . . . 30

15 Microinstructions 32
15.1 Overview . . . 32
15.2 Vertical Microinstructions . . . 32
15.3 The Length of the Micro-instructions Can Be Reduced Chiefly Because . . . 33

16 Prefetching Microinstructions 34
16.1 Overview . . . 34

17 Instruction Pipeline **35**
17.1 Overview . . . 35
17.2 Instruction Cycle . . . 35

18 Four-Segment Instruction Pipeline **36**

19 Pipeline Conflicts **37**
19.1 Handling of Branch Instructions or Control Hazard . . . 37
19.1.1 Prefetch Target Instruction . . . 37
19.1.2 Branch Prediction . . . 37
19.1.3 Delayed Branch . . . 37

20 Resolving Data Dependency Conflicts or Data Hazards **38**
20.1 Hardware Interlocks . . . 38
20.2 Operand Forwarding . . . 38
20.3 Delayed Load . . . 38

21 Hazards **39**

22 Synchronous Pipelines **40**
22.1 Asynchronous Pipelines . . . 40

23 I/O Interface **41**
23.1 The I/O Bus Consists Of . . . 41

24 I/O versus Memory Bus **42**
24.1 Overview . . . 42
24.2 In Isolated I/O . . . 42
24.3 In Memory Mapped I/O . . . 42

25 Types of Interrupts **43**
25.1 External Interrupts . . . 43
25.2 Internal Interrupts . . . 43
25.3 Software Interrupts . . . 43
25.4 Interrupt-Request Line . . . 43

26 Priority Interrupt **44**
26.1 Overview . . . 44
26.2 Establishing the Priority of Simultaneous Interrupts . . . 44
26.2.1 Polling . . . 44
26.2.2 Daisy-Chaining Priority . . . 44
26.2.3 Parallel Priority Interrupt . . . 44
26.3 Maskable Interrupt . . . 45
26.4 Non-maskable Interrupt . . . 45
26.5 Inter Processor Interrupt . . . 45
26.6 Spurious Interrupt . . . 45

27 Modes of Data Transfer **46**
27.1 Programmed I/O 46
27.2 Interrupt-Initiated I/O 46
27.3 Direct Memory Access (DMA) 47

28 Input-Output Processor (IOP) **48**
28.1 Overview 48

29 Direct Memory Access **49**
29.1 Overview 49
29.1.1 Bus Request 49
29.1.2 Bus Grant 49
29.1.3 Burst Transfer 49
29.1.4 Cycle Stealing 49

30 Blocking and Nonblocking I/O **50**
30.1 Blocking I/O 50

31 Asynchronous I/O **51**

32 Buffering **52**
32.1 Overview 52
32.2 Spooling 52
32.3 Expansion Bus and PCI Bus 52

33 Polling **53**

34 Memory Management **54**
34.1 Raster Display 54
34.2 Memory Hierarchy in a Computer System 54
34.3 Protection of Memory Space 54

35 Static RAM **56**
35.1 Overview 56

36 Dynamic RAM **57**
36.1 Overview 57

37 RAM and ROM Chips **58**
37.1 RAM Chip 58
37.1.1 Bi-directional Bus 58
37.2 ROM Chip 58
37.2.1 Uni-directional Bus 58
37.3 Memory Address Map 58

38 ROM **59**
38.1 Bootstrap Loader 59

39 Associative Memory **60**
39.1 Overview 60
39.2 Block Diagram of Associative Memory 60

40 Hit Ratio **61**
40.1 Overview 61

41 Cache Memory **62**
41.1 Locality of Reference 62
41.2 Mapping 62

42 Direct Mapping **63**
42.1 Overview 63

43 Set-Associative Mapping **64**
43.1 Overview 64

44 Writing into Cache **65**
44.1 Overview 65
44.2 Cache Initialization 65
44.3 Cache Coherence 65

45 Memory Binding **66**
45.1 Overview 66
45.2 Dynamic Loading 66

46 Dynamic Linking **67**
46.1 Overview 67

47 Virtual Memory **68**

48 Logical Versus Physical Address Space **69**
48.1 Overview 69

49 Contiguous Memory Allocation **70**
49.1 Overview 70
49.2 External Fragmentation 70

50 Paging **71**
50.1 Overview 71
50.2 Mapping from Address Space to Memory Space 71
50.2.1 Page (p) Line Number (d) 71

51 Translation Look-aside Buffer (TLB) **72**

52 Valid-Invalid Bit **73**
52.1 Demand Paging 73

53 Page Replacement Algorithms **74**
53.1 FIFO Page Replacement 74
53.1.1 Belady's Anomaly 74
53.2 Optimal Page Replacement 74
53.3 LRU Page Replacement 74

54 Thrashing **75**

55 Local vs. Global Page Replacement **76**

56 Byte Addressing vs. Word Addressing **77**
56.1 Byte Addressing 77
56.2 Word Addressing 77

Chapter 1

Number Representation

1.1 Floating-Point Representation

The floating-point representation of a number has two parts:

(i) **Mantissa:** Signed fixed-point number

(ii) **Exponent:** Designates the position of the decimal (or binary) point

For example, the number +6132.789 is represented in floating-point as:

$$\text{Fraction} = +0.6132789$$
$$\text{Exponent} = +04$$

The value of the exponent indicates that the actual position of the decimal point is four positions to the right of the indicated decimal point in the fraction.

Another example: The binary number +1001.11 is represented with an 8-bit fraction and 6-bit exponent as follows:

$$\text{Fraction} : +01001110$$
$$\text{Exponent} : 000100$$

The fraction has a 0 in the leftmost position to denote positive. The binary point of the fraction follows the sign bit, but is not shown in the register. The exponent has the equivalent binary number 4. The actual binary point in the fraction is four positions to the right of the indicated binary point in the fraction.

1.2 Normalized Floating-Point Numbers

A floating-point number is said to be normalized if the most significant digit of the mantissa is nonzero.

For example, the decimal number 350 is normalized, but 00035 is not.

Normalized numbers provide the maximum possible precision for the floating-point number.

A zero cannot be normalized because it does not have a nonzero digit (if a nonzero digit d is usually represented in floating-point by all 0s in the mantissa and exponent).

Chapter 2

Register Transfer and Microoperations

2.1 Micro-operation

The operations executed on data stored in registers are called microoperations. A micro-operation is an elementary operation such as shift, count, clear, and load.

2.2 Register Transfer Language

The symbolic notation used to describe the microoperation transfers among registers is called a Register Transfer Language (RTL).

The internal hardware organization of a digital computer is best defined by specifying:

(i) The set of registers it contains and their function

(ii) The sequence of microoperations performed on the binary information stored in the registers

(iii) The control that initiates the sequence of microoperations

2.3 Registers

(i) **Memory Address Register (MAR):** Holds an address of the memory unit

(ii) **PC (Program Counter):**

(iii) **IR (Instruction Register):**

(iv) **R_i (Processor Register):**

Computer registers are designated by capital letters.

2.3.1 Register Transfer

$R_2 \leftarrow R_1$

2.3.2 Control Function

$$P: \quad R_2 \leftarrow R_1$$

P represents the transfer operation to be executed by the hardware only if $P = 1$.

Note: The clock is not included as a variable in the register transfer statements. It is assumed that all transfers occur during a clock edge transition.

2.4 Bus and Memory Transfers

2.4.1 Common Bus

A common bus consists of a set of common lines, one for each bit of a register, through which binary information is transferred one at a time.

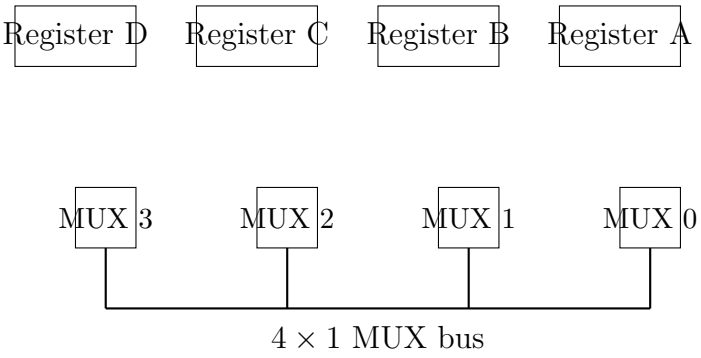
Control signals determine which register is selected by the bus during each particular register transfer.

2.4.2 Bus System for 4 Registers

In general, a bus system will multiplex k registers of n bits each to produce an n-line common bus.

The number of multiplexers needed to construct the bus is equal to n, the number of bits in each register.

The size of each multiplexer must be $k \times 1$, since it multiplexes k data lines.



2.5 Bus Transfer

$$\text{BUS} \leftarrow R_2, \quad R_1 \leftarrow \text{BUS}$$
$$\Rightarrow R_1 \leftarrow R_2$$

2.6 Memory Transfer

Read: DR $\leftarrow M$[AR]

Write: M[AR] $\leftarrow R_1$

where DR is the Data Register and AR is the Address Register.

2.6.1 Micro-operations

The microoperations are classified into four categories:

(i) Register Transfer Microoperations

(ii) Arithmetic Microoperations

(iii) Logic Microoperations

(iv) Shift Microoperations

Chapter 3

Arithmetic Microoperations

3.1 Basic Operations

(i) **Addition:** $R_3 \leftarrow R_1 + R_2$

(ii) **Subtraction:** $R_3 \leftarrow R_1 - R_2$ or $R_3 \leftarrow R_1 + \overline{R_2} + 1$

(iii) **1's complement:** $R_2 \leftarrow \overline{R_2}$

(iv) **2's complement:** $R_2 \leftarrow \overline{R_2} + 1$

(v) **Increment:** $R_1 \leftarrow R_1 + 1$

(vi) **Decrement:** $R_1 \leftarrow R_1 - 1$

3.2 Arithmetic Shift

Explained later with shift microoperations.

The increment and decrement microoperations are implemented with a binary up-down counter.

Chapter 4

Logic Microoperations

These operations consider each bit of the register separately and treat them as binary variables.

4.1 Basic Logic Operations

(i) **OR:** $R_1 \leftarrow R_2 \vee R_3$

(ii) **AND:** $R_1 \leftarrow R_2 \wedge R_3$

(iii) **Ex-OR:** $R_1 \leftarrow R_2 \oplus R_3$

(iv) **Complement:** $R_1 \leftarrow \overline{R_1}$

The symbol + occurs in a micro-operation, it will denote an arithmetic plus. When it occurs in a control function, it will denote an OR operation.

For example, in the statement:

$$P + Q: \quad R_1 \leftarrow R_2 + R_3, \quad R_4 \leftarrow R_5 \vee R_6$$

the OR operation is implied and the addition is implied in the microoperation.

4.2 List of Logic Microoperations

There are 16 different logic operations that can be performed with two binary variables:

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7
0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1

x	y	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1

Chapter 5

Shift Microoperations

There are three types of shifts:

(i) Logical Shift

(ii) Circular Shift

(iii) Arithmetic Shift

5.1 Shift-Left Operation

The serial input transfers a bit into the rightmost position.

5.2 Shift-Right Operation

The serial input transfers a bit into the leftmost position.

5.3 Logical Shift

A logical shift is one that transfers 0 through the serial input. If only all zeros and symbols are used for logical left shift and right shift microoperations:

$$R_1 \leftarrow \text{shl } R_1$$
$$R_2 \leftarrow \text{shr } R_2$$

5.4 Circular Shift

Here, the serial output of the shift register is connected to its serial input.

Examples: c_i and c_o symbols are used for circular shift left and right microoperations:

$$R_1 \leftarrow \text{cir } R_1$$
$$R_2 \leftarrow \text{cil } R_2$$

5.5 Arithmetic Shift

An arithmetic shift-left (ashl) multiplies a signed binary number by 2.

An arithmetic shift-right (ashr) divides the number by 2.

Arithmetic shifts must leave the sign bit unchanged because the sign of the number remains the same when it is multiplied or divided by 2.

The ashr leaves the sign bit unchanged and shifts the number (including the sign bit) to the right.

The ashl inserts a 0 into R_0 from right and shifts all other bits to the left. If the multiplication by 2 causes an overflow, the sign-bit in the register changes from 0 to 1 or 1 to 0.

5.5.1 Checking for overflow

If $V_s = 0$: There is no overflow.

If $V_s = 1$: There is an overflow.

Where $V_s = R_{n+1} \oplus R_{n-2}$.

Chapter 6

Arithmetic Logic Shift Unit (ALU)

6.1 Overview

To perform a microoperation, in ALU, the contents of specified registers are placed in the inputs of the common ALU. The ALU performs an operation and the result of the operation is then transferred to a destination register.

The ALU is a combinational circuit so that the entire operation can be performed during one clock pulse period.

6.2 Checking for overflow

$$\begin{aligned} \text{ashr} \rightarrow |\mathrm{R}_{n+1}|\mathrm{R}_{n+2} \quad &\longrightarrow \quad |R_1|R_0| \\ \text{ashl} \rightarrow |\mathrm{R}_{n+1}|\mathrm{R}_{n+2}| \quad &\longleftarrow \quad |R_1|R_0| \times 2 \end{aligned}$$

If $V_s = 0$: There is no overflow.
If $V_s = 1$: There is an overflow.

Chapter 7

Machine Instructions and Addressing Modes

7.1 Instruction Formats

A computer will usually have a variety of instruction code formats. It is the function of the control unit within the CPU to interpret each instruction code and provide the necessary control functions needed to process the instruction.

The bits of the instruction code are usually divided into three groups:

(i) An operation code field that specifies the operation to be performed

(ii) An address field that designates a memory address or a processor register

(iii) A mode field that specifies the way the operand or the effective address is determined

Operation Code	Address Field	Mode

A register address is a binary number of k bits that define one of 2^k registers in the CPU.

The instruction code is also called a control word.

An operation code is sometimes called a macrooperation because it specifies a set of micro-operations.

7.2 Types of CPU Organizations

The number of address fields in the instruction format of a computer depends on the internal organization of its registers.

Most computers fall into one of three types of CPU organizations:

(i) Single Accumulator organization

(ii) General Register Organization

(iii) Stack Organization

7.2.1 Single Accumulator Organization

All operations are performed with an implied accumulator register. The instruction format in this type of computer uses one address field.

For example:

$$\text{ADD } X \quad \Rightarrow \quad \text{AC} \leftarrow \text{AC} + M[X]$$

where AC is the Accumulator Register.

7.2.2 General Register Organization

The instruction format in this type of computer uses either three or two address fields.

Examples:

(i) ADD $R_1, R_2, R_3 \Rightarrow R_1 \leftarrow R_2 + R_3$

(ii) ADD $R_1, R_2 \Rightarrow R_1 \leftarrow R_1 + R_2$

(iii) ADD $R_1, X \Rightarrow R_1 \leftarrow R_1 + M[X]$

(iv) MOV $R_1, R_2 \Rightarrow R_1 \leftarrow R_2$

7.2.3 Stack Organization

Computers with stack organization have PUSH and POP instructions which require one address field.

For example: PUSH X

Operation-type instructions do not need an address field (zero address instruction).

For example: ADD

Single Accumulator Organization uses one-address instructions.

General Register Organization uses three- and two-address instructions.

Stack Organization uses zero- and one-address instructions.

In a general register's CPU organization, there are eight general purpose registers and ALU can perform 32 different operations.

The number of selection lines for selecting the operand is: $3 = (2^3 = 8)$

The number of bits in operation code $= 5$ $(2^5 = 32)$

The length of the control word $= 5 + 3 + 3 + 3 = 14$ (operation + instruction)

Chapter 8

CISC and RISC

8.1 CISC

A computer with a large number of instructions is classified as a complex instruction set computer (CISC).

The central goal of a CISC architecture is to attempt to provide a single machine instruction for each statement that is written in a high-level language.

The major characteristics of CISC architecture are:

(1) A large number of instructions - typically from 100 to 250 instructions

(2) Some instructions that perform specialized tasks and are used infrequently

(3) A large variety of addressing modes - typically from 5 to 20 different modes

(4) Variable-length instruction formats

(5) Instructions that manipulate operands in memory

8.2 RISC

The Reduced Instruction Set Computer is a computer containing fewer instruction with simple constructs that can be executed much faster within the CPU without having to use memory as often.

The major characteristics of a RISC processor are:

(1) Relatively few instructions

(2) Relatively few addressing modes

(3) Memory access limited to load and store instructions

(4) All operations done within the registers of the CPU

(5) Fixed-length, easily decoded instruction format

(6) Single-cycle instruction execution

(7) Hardwired rather than microprogrammed control

(8) Delayed Load: When there is a data dependency conflict in instruction pipeline

(9) Delayed Branch: When there is a branch instruction in instruction pipeline

Chapter 9

Addressing Modes

The way the operands are chosen during program execution is dependent on the addressing mode of the instruction.

The addressing mode may reduce the number of bits in the addressing field of the instruction.

Different types of addressing modes are given below:

(1) Implied Mode

(2) Immediate Mode

(3) Register Mode

(4) Register Indirect Mode

(5) Autoincrement or Autodecrement Mode

(6) Direct Address Mode

(7) Indirect Address Mode

(8) Relative Address Mode

(9) Indexed Addressing Mode

(10) Base Register Addressing Mode

9.1 Implied Mode

In this mode, the operands are specified implicitly in the definition of the instruction.

For example, all register reference instructions that use only an accumulator are implied-mode instructions.

Zero-address instructions in a stack-organized computer are implied-mode instructions.

9.2 Immediate Mode

In this mode, the operand is specified in the instruction itself.

In other words, an immediate-mode instruction has an operand field rather than an address field.

For example: LD #NBR $\Rightarrow$ AC $\leftarrow$ NBR

9.3 Register Mode

In this mode, the operands are in registers that reside within the CPU. The particular register is selected from a register field in the instruction.

A k-bit field can specify any one of 2^k registers.

Example: LD R_1 $\Rightarrow$ AC $\leftarrow R_1$

9.4 Register Indirect Mode

In this mode, the instruction specifies a register that reside within the CPU whose contents give the address of the operand in memory.

For example: LD (R_1) $\Rightarrow$ AC $\leftarrow M[R_1]$

9.5 Autoincrement or Autodecrement Mode

This is similar to the register indirect mode except that the register is incremented or decremented after (or before) its value is used to access the memory.

For example:

$$\text{LD } (R_1)+ \quad \Rightarrow \quad \text{AC} \leftarrow M[R_1], \quad R_1 \leftarrow R_1 + 1$$

9.6 Effective Address

The effective address is defined to be the memory address obtained from the computation dictated by the given addressing mode.

9.7 Direct Address Mode

In this mode, the effective address is equal to the address part of the instruction.

For example: LD ADR $\Rightarrow$ AC $\leftarrow M$[ADR]

9.8 Indirect Address Mode

In this mode, the address field of the instruction gives the address where the effective address is stored in memory.

For example: LD @ADR $\Rightarrow$ AC $\leftarrow M[M$[ADR]]

9.9 Relative Address Mode

In this mode, the content of the program counter (PC) is added to the address part of the instruction in order to obtain the effective address.

For example: LD \$ADR $\Rightarrow$ AC $\leftarrow M[\text{PC} + \text{ADR}]$

Relative addressing is often used with branch-type instructions.

9.10 Indexed Addressing Mode

In this mode, the content of an index register is added to the address part of the instruction to obtain the effective address.

For example: LD ADR(x) $\Rightarrow$ AC $\leftarrow M[\text{ADR} + x]$

9.11 Base Register Addressing Mode

In this mode, the content of a base register is added to the address part of the instruction to obtain the effective address.

For example:

The base register addressing mode is used for relocation of programs in memory.

Chapter 10

Central Processing Unit

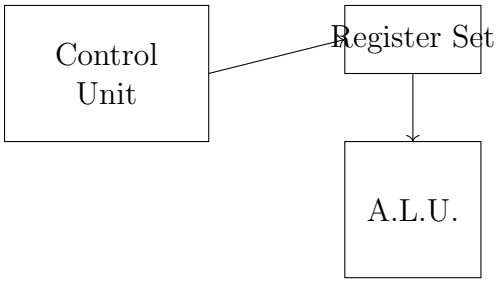
10.1 Overview

The CPU is made up of three major parts:

(i) Register Set

(ii) Arithmetic-Logic Unit (ALU)

(iii) Control Unit (CU)



10.2 Some Fundamental Concepts

The instructions constituting a program to be executed are loaded in sequential locations in the main memory.

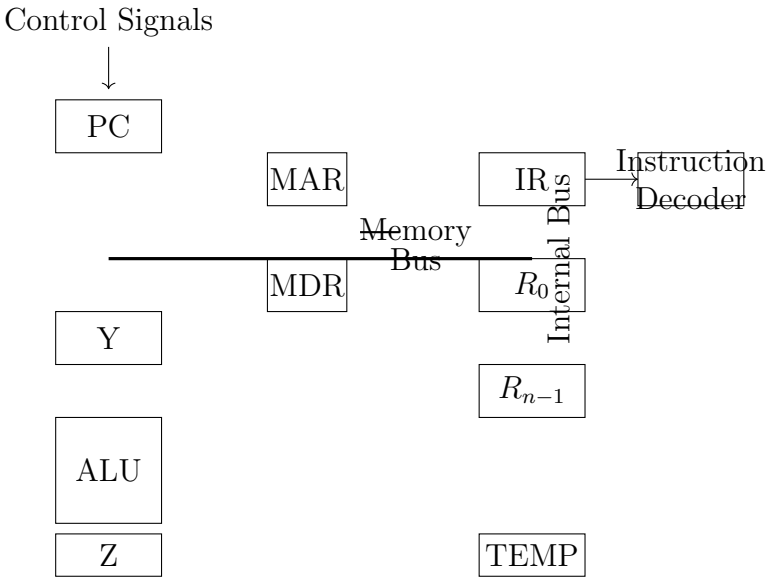
Instructions are fetched from successive memory locations until a branch or a jump instruction is executed.

A dedicated CPU register, referred to as the program counter (PC), contains the address of the memory location containing the next instruction.

After fetching an instruction, the contents of the PC are updated to point to the next instruction in sequence.

Chapter 11

Single-Bus Organization of the Data Path Inside the CPU



Here, the single common bus is internal to the CPU and should not be confused with the external bus.

The ALU and the registers are used for storing and manipulating data and referred to as a datapath.

11.1 Steps to Execute an Instruction

(i) Fetch the contents of the memory location pointed to by PC (program counter) and load them into IR (instruction register):

$$\text{IR} \leftarrow M[\text{PC}]$$

(ii) Increment the contents of PC by 1:

$$\text{PC} \leftarrow \text{PC} + 1$$

(iii) Carry out the actions specified by the instruction in the IR

Steps 1 and 2 are usually referred to as the fetch phase.

Step 3 constitutes the execution phase.

In cases where an instruction occupies more than one word, steps 1 and 2 must be repeated as many times as necessary to fetch the complete instruction.

The width of the data bus between the CPU and the main memory determines the number of bytes that can be transferred in one access. This width is likely to be $\frac{1}{2}$, 4, or 8 bytes, so the PC must be incremented by 2, 4, or 8 after each fetch step.

Chapter 12

Execution of a Complete Instruction

Consider the instruction:

$$\text{ADD } (R_3), R_1 \quad \Rightarrow \quad R_1 \leftarrow M[R_3] + R_1$$

12.1 Steps Required to Execute the Above Instruction

Step	Action
1	PC_{out}, MAR_{in}, Read, Clear Y, Set Carry-in to ALU ADD_{in}
2	Z_{out}, PC_{in}, WMFC
3	MDR_{out}, IR_{in}
4	$R3_{out}$, MAR_{in}, Read
5	$R1_{out}$, Y_{in}, WMFC
6	MDR_{out}, Add, Z_{in}
7	Z_{out}, $R1_{in}$, End.

WMFC $\rightarrow$ Wait for Memory Function Completed.

For above instruction execution, 7 non-overlapping time-slots are required. Each time slot must be at least long enough for the functions specified in the corresponding to be completed.

Chapter 13

CPU Control Design

There are two major types of control organization:

(i) Hardwired Control

(ii) Microprogrammed Control

13.1 Hardwired Control Organization

In the hardwired organization, the control logic is implemented with gates, flip-flops, decoders, and other digital circuits.

It has the advantage that it can be optimized to produce a fast mode of operation.

It requires changes in the wiring among the various components if the design has to be modified or changed.

13.2 Microprogrammed Control Organization

In the micro-programmed organization, the control information is stored in a control memory. The control memory is programmed to initiate the required sequence of micro-operations.

In the micro-programmed control, any required changes or modifications can be done by updating the microprogram in control memory.

Chapter 14

Micro-programmed Control

14.1 Overview

In micro-programmed control, the control signals are generated by a program similar to the machine language programs.

14.2 Control Word

A control word (CW) is a word whose individual bits represent the various control signals.

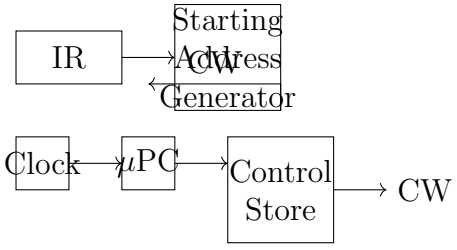
A sequence of CWs corresponding to the control sequence of a machine instruction constitutes the micro-routine.

The individual CWs in this micro-routine are referred to as micro-instructions.

The microroutines for all instructions in the instruction set of a computer are stored in a special memory called the control store.

The control unit can generate the control signals for any instruction by sequentially reading the CWs of the corresponding microinstruction from the control store.

14.3 Basic Organization of a Microprogrammed Control Unit



The microprogram counter (μPC) is used to read the control words sequentially from the control store.

Every time a new instruction is loaded into the IR, the output of the block labeled starting address generator is loaded into the μPC. The μPC is then automatically incremented by the clock, causing successive microinstructions to be read from the control store.

The μPC does not increment in the following situations:

(i) When an End microinstruction is encountered

(ii) When a new instruction is loaded into IR

(iii) When a branch micro-instruction is encountered and the branch condition is satisfied

Chapter 15

Microinstructions

15.1 Overview

A straightforward way to structure microinstructions is to assign one bit position to each control signal required in the CPU.

Control Signals F0-F7:

Micro-instruction	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7
1	0	1	1	1	0	0	0	1
2	1	0	0	0	0	0	0	0
3	0	0	0	0	1	1	0	0
4	0	0	1	1	0	0	0	0
5	0	0	0	0	0	0	1	0
6	0	0	0	0	1	0	0	0
7	0	0	0	0	0	0	0	0

Control Signals F8-F15:

Micro-instruction	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
1	1	1	1	0	0	0	0	0
2	0	0	0	1	0	0	0	1
3	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	1	0
5	0	0	0	0	1	0	0	0
6	0	1	1	0	0	0	0	0
7	0	1	0	1	0	0	1	0

This scheme has one serious drawback: assigning individual bits to each control signal results in long microinstructions.

The above scheme is called a horizontal organization.

15.2 Vertical Microinstructions

Highly encoded schemes that use compact codes to specify only a small number of control functions in each microinstruction are referred to as a vertical organization.

Grouping control signals into fields requires a little more hardware because decoding circuits must be used to decode the bit patterns of each field into individual control signals.

15.3 The Length of the Micro-instructions Can Be Reduced Chiefly Because

(i) Most signals are not needed simultaneously

(ii) Many signals are mutually exclusive

Chapter 16

Prefetching Microinstructions

16.1 Overview

The major drawback of microprogrammed control is that it is slow.

Fetching a microinstruction from the control store takes considerably longer than generating control signals using hardwired circuits.

The performance of the microprogrammed control can be improved by prefetching the microinstruction while the current one is being executed.

Prefetching microinstructions presents some organizational difficulties because sometimes the status flags and the results of the currently executed microinstructions are needed to determine the address of the next microinstruction.

Chapter 17

Instruction Pipeline

17.1 Overview

An instruction pipeline reads consecutive instructions from memory while previous instructions are being executed in other segments.

This causes the instruction fetch and execute phases to overlap and perform simultaneous operations.

If an instruction causes a branch out of sequence, then the pipeline must be emptied and all the instructions that have been in memory after the branch instruction must be discarded.

17.2 Instruction Cycle

Generally, the computer needs to process each instruction with the following set sequence of steps:

(1) Fetch the instruction from memory

(2) Decode the instruction

(3) Calculate the effective address

(4) Fetch the operands from memory

(5) Execute the instruction

(6) Store the results in the proper place

Chapter 18

Four-Segment Instruction Pipeline

Suppose the instruction cycle is divided into four segments as given below:

(i) FI is the segment that fetches an instruction

(ii) DA is the segment that decodes the instruction and calculates the effective address

(iii) FO is the segment that fetches the operand

(iv) EX is the segment that executes the instruction

It is assumed that the processor has separate instruction and data memories so that the operation in FI and FO can proceed at the same time.

Step →	1	2	3	4	5	6	7	8	9	10	11	12
Instruction: 1	FI	DA	FO	EX								
2		FI	DA	FO	EX							
(Branch) 3			FI	DA	FO	EX						
4				FI	-	-	FI	DA	FO	EX		
5					-	-	-	FI	DA	FO	EX	
6									FI	DA	FO	EX
7										FI	DA	FO

Assume that instruction 3 is a branch instruction. As soon as this instruction is decoded in segment DA in step 4, the transfer from FI to DA of the other instructions is halted until the branch instruction is executed in step 6.

If the branch is taken, a new instruction is fetched in step 7.

If the branch is not taken, the instruction fetched previously in step 4 can be used.

Chapter 19

Pipeline Conflicts

In general, there are three major difficulties that cause the instruction pipeline to deviate from its normal operation:

(i) **Resource Conflicts:** Caused by access to memory by two segments at the same time

(ii) **Data dependency conflicts:** Arise when an instruction depends on the result of a previous instruction, but this result is not yet available

(iii) **Branch difficulties:** Arise from branch and other instructions that change the value of PC

19.1 Handling of Branch Instructions or Control Hazard

19.1.1 Prefetch Target Instruction

Prefetch the target instruction in addition to the instruction following the branch. Both are saved until the branch is executed.

19.1.2 Branch Prediction

Some additional logic is used to guess the outcome of a conditional branch instruction before it is executed. The pipeline then begins prefetching the instruction stream from the predicted path.

19.1.3 Delayed Branch

Used in most RISC processors. In this procedure, the compiler detects the branch instructions and rearranges the machine language code sequence by inserting useful instructions that keep the pipeline operating without interruptions.

Chapter 20

Resolving Data Dependency Conflicts or Data Hazards

20.1 Hardware Interlocks

An interlock is a circuit that detects instructions whose source operands are destinations of instructions farther up in the pipeline. After detecting, we delay the instruction by enough clock cycles to resolve the conflict.

20.2 Operand Forwarding

Avoid the conflict by routing the data through special paths between pipeline segments.

20.3 Delayed Load

Delay the loading of the conflicting data by inserting no-operation instructions (in compiler).

Chapter 21

Hazards

There are situations, called hazards, that prevent the next instruction in the instruction stream from being executing during its designated clock cycle. Hazards reduce the performance from the ideal speedup gained by the pipeline.

There are three classes of hazards:

(i) **Structural Hazards:** They arise from resource conflicts when the hardware can not support all possible combinations of instructions in simultaneous overlapped execution

(ii) **Data Hazards:** They arise when an instruction depends on the result of a previous instruction in a way that is exposed by the overlapping of instructions in the pipeline

(iii) **Control Hazards:** They arise from the pipelining of branches and other instructions that change the PC

Hazards in pipeline can make it necessary to stall the pipeline.

Chapter 22

Synchronous Pipelines

These are clocked synchronously. The time between each clock signal is set to be greater than the longest delay between pipeline stages.

22.1 Asynchronous Pipelines

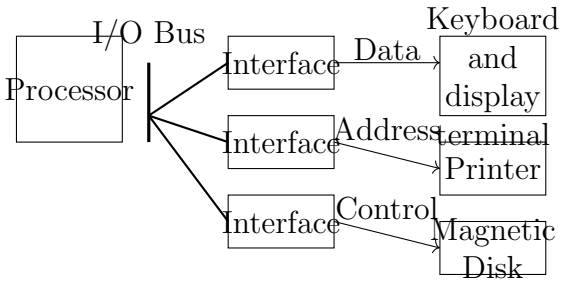
They use request/acknowledge system, wherein each stage can detect when it's finished.

Chapter 23

I/O Interface

I/O interfaces connect the peripherals to the CPU.

I/O interfaces supervise and synchronize all input and output transfers.



23.1 The I/O Bus Consists Of

(i) Data Lines

(ii) Address Lines

(iii) Control Lines

Each interface decodes the address and control received from the I/O bus, interprets them for the peripheral, and provides signals for the peripheral controller.

Each peripheral has its own controller that operates the particular electromechanical device.

Address lines are used to select a peripheral.

Control lines are used to send commands to an interface.

Chapter 24

I/O versus Memory Bus

24.1 Overview

The memory bus contains data, address, and read/write control lines.

There are three ways that computer buses can be used to communicate with memory and I/O:

(i) Use two separate buses, one for memory and the other for I/O (Using IOP - Input Output Processor)

(ii) Use one common bus for both memory and I/O, but have separate control lines for each (Isolated I/O)

(iii) Use one common bus for memory and I/O with common control lines (memory mapped)

24.2 In Isolated I/O

The I/O read and I/O write control lines are enabled during an I/O transfer. The memory read and memory write control lines are enabled during a memory transfer.

24.3 In Memory Mapped I/O

A segment of the total address space is reserved for interface registers.

Chapter 25

Types of Interrupts

There are three major types of interrupts that cause a break in the normal program execution:

(i) External Interrupts

(ii) Internal Interrupts

(iii) Software Interrupts

25.1 External Interrupts

External interrupts come from external sources like I/O devices, timing devices, etc.

25.2 Internal Interrupts

Internal interrupts arise from illegal or erroneous use of an instruction or data. Internal interrupts are also called traps. Examples are: register overflow, attempt to divide by zero, etc.

25.3 Software Interrupts

Software interrupt is a special call instruction that behaves like an interrupt rather than a subroutine call.

25.4 Interrupt-Request Line

The CPU has a wire called the interrupt request line that the CPU senses after executing every instruction.

Chapter 26

Priority Interrupt

26.1 Overview

A priority interrupt is a system that establishes a priority over the various sources to determine which condition is to be served first when two or more requests arrive simultaneously.

The system may also determine which conditions are permitted to interrupt the computer while another interrupt is being serviced.

26.2 Establishing the Priority of Simultaneous Interrupts

It can be done by software or hardware. There are three methods to establish priority:

(i) Polling

(ii) Daisy-Chaining Priority

(iii) Parallel-Priority Interrupt

26.2.1 Polling

A polling procedure is used to identify the highest-priority source by software means.

26.2.2 Daisy-Chaining Priority

It is used to identify the highest-priority source by hardware means. It consists of a serial connection of all devices that request an interrupt.

26.2.3 Parallel Priority Interrupt

This method uses a register whose bits are set separately by the interrupt signal from each device. Priority is established according to the position of the bits in the register.

26.3 Maskable Interrupt

A hardware interrupt that can be ignored by setting a bit in an interrupt mask register's bit mask.

26.4 Non-maskable Interrupt

It is a hardware interrupt that lacks an associated bit-mask, so that it can never be ignored. It is often used for timers.

26.5 Inter Processor Interrupt

It is generated by one processor to interrupt another processor in a multiprocessor system.

26.6 Spurious Interrupt

It is a hardware interrupt that is unwanted.

Chapter 27

Modes of Data Transfer

Data transfer between the CPU and I/O devices may be handled in a variety of modes. Some modes use the CPU as an intermediate path, others transfer the data directly to and from the memory unit.

Data transfer to and from peripherals may be handled in one of the three possible modes:

(i) Programmed I/O

(ii) Interrupt-initiated I/O

(iii) Direct memory access (DMA)

27.1 Programmed I/O

In programmed I/O, each data item transfer is initiated by an instruction in the program.

Once a data transfer is initiated, the CPU is required to monitor the interface to see when a transfer can again be made.

In the programmed I/O method, the CPU stays in a program loop until the I/O unit indicates that it is ready for data transfer. Usually, the transfer is to and from a CPU register and peripheral. Other instructions are needed to transfer the data to and from CPU and memory.

Here, the I/O device does not have direct access to memory.

27.2 Interrupt-Initiated I/O

An alternative to the CPU constantly monitoring the flag is to let the interface inform the computer when it is ready to transfer data. This mode of transfer uses the interrupt facility. In the meantime, the CPU can proceed to execute another program. The interface meanwhile keeps monitoring the device.

When the interface determines that the device is ready for data transfer, it generates an interrupt request to the computer. Upon detecting the external interrupt signal, the CPU momentarily stops the task it is processing, branches to a service program to process the I/O transfer, and then returns to the task it was originally performing.

27.3 Direct Memory Access (DMA)

In DMA, the interface transfers data into and out of the memory unit through the memory bus.

The CPU initiates the transfer by supplying the interface with the starting address and the number of words needed to be transferred and then proceeds to execute other tasks.

DMA transfer is also referred to as block mode.

A simple DMA Controller is a standard component in PCs.

Chapter 28

Input-Output Processor (IOP)

28.1 Overview

An IOP is a processor with direct memory access capability that communicates with I/O devices.

The IOP is similar to a CPU except that it is designed to handle the details of I/O processing.

Unlike the DMA controller that must be set up entirely by the CPU, the IOP can fetch and execute its own instructions. In addition, the IOP can perform other processing tasks, such as arithmetic, logic, branching, and code translation.

The memory unit occupies a central position and can communicate with each processor by means of direct memory access.

Chapter 29

Direct Memory Access

29.1 Overview

During DMA transfer, the CPU is idle and has no control of the memory buses. A DMA controller takes over the buses to manage the transfer directly between the I/O device and memory.

29.1.1 Bus Request

The bus request (BR) input is used by the DMA controller to request the CPU to relinquish control of buses.

29.1.2 Bus Grant

The CPU activates the bus grant (BG) output to inform the external DMA that the buses are in high-impedance state.

29.1.3 Burst Transfer

A block sequence consisting of a number of memory words is transferred in a continuous burst while the DMA controller is the master of the memory buses.

29.1.4 Cycle Stealing

It allows the DMA controller to transfer one data word at a time, after which it must return control of the buses to the CPU.

Chapter 30

Blocking and Nonblocking I/O

30.1 Blocking I/O

When an application issues a blocking system call, the execution of the application is suspended. The application is moved from the running queue to the waiting queue. After the system call completes, the application is moved back to the running queue.

A nonblocking call does not halt the execution of the application for an extended time. Instead, it returns quickly, with a return value that indicates how many bytes were transferred.

Chapter 31

Asynchronous I/O

An alternative to a nonblocking system call is an asynchronous system call.

An asynchronous call returns immediately, without waiting for the I/O to complete. The application continues to execute its code.

The completion of the I/O at some future time is communicated to the application through interrupt.

Chapter 32

Buffering

32.1 Overview

A buffer is a memory area that stores data while they are transferred between two devices or between a device and an application.

32.2 Spooling

A spool is a buffer that holds output for a device, such as a printer, that can not accept interleaved data streams.

32.3 Expansion Bus and PCI Bus

An expansion bus connects the internal hardware of the computer system (including the CPU and RAM) and peripheral devices.

The PCI Bus connects fast devices to processors.

Chapter 33

Polling

Polling is also referred to as busy-waiting. In this situation, when an I/O operation is required, the computer does nothing other than check the status of the I/O device until it is ready, at which point device is accessed.

It is sometimes also referred to as software-driven I/O.

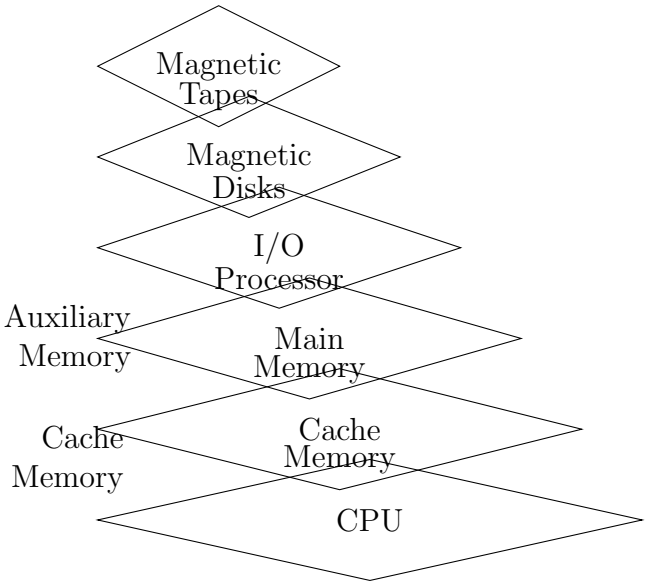
Chapter 34

Memory Management

34.1 Raster Display

The scanning process sweeps the beam from left to right across the screen n times from top to bottom. The beam is moved to the start of the next line (horizontal retrace) until position (vertical retrace).

34.2 Memory Hierarchy in a Computer System



The main memory occupies a central position by being able to communicate directly with the CPU and with auxiliary memory devices through an I/O processor.

The typical access time ratio between cache and auxiliary memory is about 1 to 7. Auxiliary memory average access time is usually 1000 times that of main memory.

34.3 Protection of Memory Space

We need to make sure that each process has a separate memory space. To do this, we need to be able to determine the range of legal addresses that the process may access and to ensure that the process can access only these legal addresses.

We can provide this protection by using two registers:

(i) Base Register holds the smallest legal physical memory address

(ii) Limit Register specifies the size of the range

Chapter 35

Static RAM

35.1 Overview

Static RAM consists of internal flip flops that store the binary information. The stored information remains valid as long as power is applied to the unit.

The Static RAM is easier to use and has shorter read and write cycles.

Chapter 36

Dynamic RAM

36.1 Overview

Dynamic RAM stores the binary information in the form of electric charge that is applied to capacitors. The capacitors are provided inside the chip by MOS transistors.

The stored charge on the capacitors tends to discharge with time and the capacitors must be periodically recharged by refreshing the dynamic memory.

The Dynamic RAM offers reduced power consumption and large storage capacity in a single memory chip.

Chapter 37

RAM and ROM Chips

37.1 RAM Chip

37.1.1 Bi-directional Bus

CS	RD	WR	Memory Function
*	*	*	Inhibit
0	1	*	Inhibit
0	*	1	Inhibit
1	0	1	Write
1	1	0	Read
1	1	*	Inhibit

37.2 ROM Chip

37.2.1 Uni-directional Bus

A ROM chip is organized externally in a similar manner; however, since a ROM can only read, the data bus can only be in an output mode.

For the same-size chip, it is possible to have more bits of ROM than of RAM, because the internal binary cells in ROM occupy less space than in RAM.

37.3 Memory Address Map

The addressing of memory can be established by means of a table that specifies the memory address assigned to each chip. This table is called a memory address map.

Component	Address Bus									
	10	9	8	7	6	5	4	3	2	1
RAM 1	0	0	0	*	*	*	*	*	*	*
RAM 2	0	0	1	*	*	*	*	*	*	*
RAM 3	0	1	0	*	*	*	*	*	*	*
RAM 4	0	1	1	*	*	*	*	*	*	*
ROM	1	*	*	*	*	*	*	*	*	*

Chapter 38

ROM

ROM is used for storing programs that are permanently resident in the computer and for tables of constants that do not change in value once the production of the computer is completed.

ROM is also Random Access Memory.

38.1 Bootstrap Loader

The bootstrap loader is a program whose function is to start the computer software operating when power is turned on. The ROM portion of main memory is used for storing an operating system (software) programs loads a portion of the operating system from disk to main memory and control is then transferred to the operating system, which prepares the computer for general use.

Chapter 39

Associative Memory

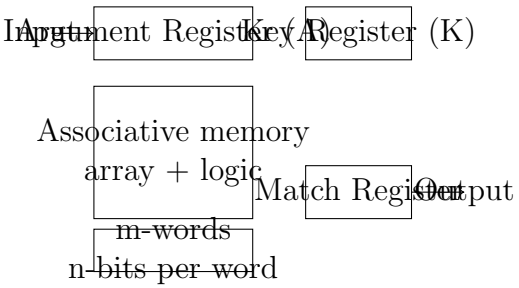
39.1 Overview

The time required to find an item stored in memory can be reduced considerably if stored data can be identified for access by the content of the data itself rather than by an address.

A memory unit accessed by content is called an associative memory or content-addressable memory.

An associative memory is more expensive than a random access memory because each cell must have storage capability as well as logic circuits for matching its content with an external argument.

39.2 Block Diagram of Associative Memory



The argument register A and key register K each have n bits, one for each bit of a word. The match register M has m bits, one for each memory word.

Each word in memory is compared in parallel with the content of the argument register. The words that match the bits of the argument register set a corresponding bit in the match register. The bits of the argument register set a corresponding bit in the match register.

Chapter 40

Hit Ratio

40.1 Overview

When the CPU refers to memory and finds the word in cache, it is said to produce a hit. If the word is not found in cache, it is in main memory and it counts as a miss.

The ratio of the number of hits divided by the total CPU references to memory (Bits plus miss) is the hit ratio.

For example: A computer with cache access time of 100ns, a main memory access time of 1000ns, and a hit ratio of 0.9 produces an average access time of:

$$0.9 \times 100 + 0.1 \times (100 + 1000) = 200 \text{ ns}$$

Chapter 41

Cache Memory

41.1 Locality of Reference

The references to memory at any given interval of time tend to be confined within a few localized area in memory. This phenomenon is known as the property of locality of reference.

If the active portions of the program and data are placed in a fast small memory, the average memory access time can be reduced, thus reducing the total execution time of the program. Such a fast small memory is referred to as a cache memory.

41.2 Mapping

The transformation of data from main memory to cache memory is referred to as a mapping process.

Three types of mapping procedures are of practical interest:

(i) Associative Mapping

(ii) Direct Mapping

(iii) Set-associative Mapping

Chapter 42

Direct Mapping

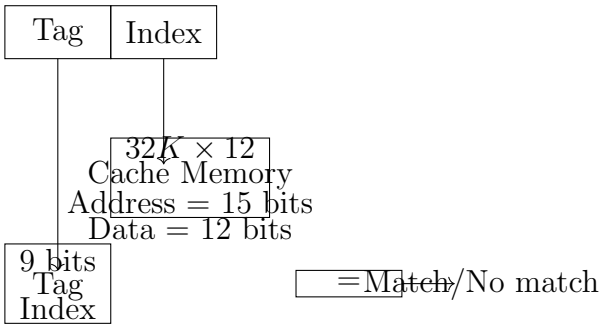
42.1 Overview

In the general case, there are 2^n words in cache memory and 2^n words in main memory address is divided into two fields:

(i) k bits for the index field

(ii) $(n - k)$ bits for the tag field

The direct mapping cache organization uses the n-bit address to access the main memory and the k-bit index to access the cache.



Each word in cache consists of the data word and its associated tag. When a new word is first brought into the cache, the tag bits are stored alongside the data bits.

When the CPU generates a memory request, the index field is used for the address to access the cache. The tag field of the CPU address is compared with the tag in the word read from the cache. If the two tags match, there is a hit. If there is no match, there is a miss.

Chapter 43

Set-Associative Mapping

43.1 Overview

The set-associative mapping is an improvement over the direct mapping in that each word of cache can store two or more words of memory under the same index address.

Each data word is stored together with its tag and the number of tag-data items in one word of cache is said to form a set.

Chapter 44

Writing into Cache

44.1 Overview

There are two ways:

(i) **Write-through:** With every memory write operation updates main memory, with cache memory being updated in parallel if it contains the word at the specified address.

(ii) **Write-back:** In this method, only the cache location is updated during a write operation. The location is then marked by a flag so that later when the word is removed from the cache, it is copied into main memory.

44.2 Cache Initialization

Each word in cache includes a valid bit to indicate whether or not the word contains valid data.

The cache is initialized by clearing all valid bits to 0. The valid bit of a particular cache word is set to 1 the first time this word is loaded from main memory and stays set unless the cache has to be initialized again.

44.3 Cache Coherence

In a shared memory multiprocessor system, with multiple processors having separate caches at its memory to keep the caches coherent by ensuring that any shared operand that is changed in any cache must be changed throughout the entire system. This is the cache coherence problem.

Chapter 45

Memory Binding

45.1 Overview

The binding of instructions and data to memory addresses can be done at any step along the way:

(i) **Compile time:** If we know at compile time where the process will reside in memory, then absolute code can be generated.

(ii) **Load time:** In this case, compiler generates relocatable code and final binding is delayed until load time.

(iii) **Execution time:** If the process can be moved during its execution from one memory segment to another, then binding must be delayed until run time.

45.2 Dynamic Loading

With dynamic loading, a routine is not loaded until it is called. All routines are kept on disk in a relocatable load format.

Dynamic loading does not require special support from the operating system. It is the responsibility of the users to design their programs to take advantage of such a method.

Chapter 46

Dynamic Linking

46.1 Overview

Some operating systems support only static linking in which system libraries are treated like any other object module and are combined by the loader into the binary program image.

The concept of dynamic linking is similar to that of dynamic loading. Here, though, linking, rather than loading, is postponed until execution time. This feature is usually used with system libraries such as language subroutine libraries.

Under this scheme, all processes that use a language library execute only one copy of the library code.

Chapter 47

Virtual Memory

Virtual memory is a concept that permits the user to construct programs as though a large memory space were available, equal to the totality of auxiliary memory.

Chapter 48

Logical Versus Physical Address Space

48.1 Overview

An address generated by the CPU is commonly referred to as a logical address or virtual address, and the set of such addresses is referred to as address space or virtual address space or logical address space.

An address in main memory is referred to as a physical address, and the set of such addresses is referred to as physical address space or memory space.

The compile-time and load-time address-binding methods generate identical logical and physical addresses.

The run-time mapping from virtual to physical addresses is done by a hardware device called the memory-management unit (MMU).

Chapter 49

Contiguous Memory Allocation

49.1 Overview

For a memory request of size n, there are three methods are used to satisfy the request from a list of free holes:

(i) **First Fit:** Allocate the first hole that is big enough

(ii) **Best Fit:** Allocate the smallest hole that is big enough

(iii) **Worst Fit:** Allocate the largest hole

Both the first-fit and best-fit strategies for memory allocation suffer from external fragmentation.

49.2 External Fragmentation

External fragmentation exists when there is enough total memory space to satisfy a request, but the available spaces are not contiguous; storage is fragmented into a large number of small holes.

One solution to the problem of external fragmentation is compaction. Compaction is possible only if relocation is dynamic and is done at execution time.

Another possible solution to the external fragmentation problem is to permit the logical address space of the processes to be non-contiguous, thus allowing a process to be allocated physical memory wherever the latter is available. This is accomplished through paging.

Chapter 50

Paging

50.1 Overview

Paging is a memory-management scheme that permits the physical address space of a process to be noncontiguous.

The basic method for implementing paging involves breaking physical memory into blocks of fixed size called frames or page-frames or blocks, and breaking logical memory into blocks of the same size called pages.

50.2 Mapping from Address Space to Memory Space

50.2.1 Page (p) Line Number (d)

1	0	1	0	0	1	0	0	0	1	1

Table Address Register

000

Page Table

P_1	P_2

Logical address

p	d

A presence bit in each location indicates whether the page has been transferred from auxiliary memory into main memory.

A 0 indicates the presence bit indicates that the page is not available in main memory.

Chapter 51

Translation Look-aside Buffer (TLB)

In paging, two memory accesses are needed to access some data: one for the page table entry and after for the data. This memory access is slowed by a factor of 2.

The solution to this problem is to use a special small fast-lookup hardware cache called a translation look-aside buffer. The TLB is associative, rapid memory.

The TLB contains only a few of the page table entries. When a logical address is generated by the CPU, its page number is presented to the TLB.

If the page number is found, its frame is immediately available and is used to access memory.

If the page number is not in the TLB (known as TLB miss), a memory reference to the page table must be made. In addition, we add the page number and frame number to the TLB so that they will be found quickly on the next reference.

Chapter 52

Valid-Invalid Bit

This bit is also kept in page table.

When this bit is set to valid, the associated page is in the process logical address space and is thus a legal page.

When the bit is set to invalid, the page is not in the process logical address space and is thus a legal page.

52.1 Demand Paging

With demand-paged virtual memory, pages are only loaded when they are demanded during program execution; pages that are never accessed are thus never loaded into physical memory.

Chapter 53

Page Replacement Algorithms

53.1 FIFO Page Replacement

A FIFO replacement algorithm associates with each page the time when that page was brought into memory. When a page must be replaced, the oldest page is chosen.

FIFO page replacement algorithm suffers from Belady's anomaly.

53.1.1 Belady's Anomaly

For some page replacement algorithms, the page-fault rate may increase as the number of allocated frames increases.

53.2 Optimal Page Replacement

Replace the page that will not be used for the longest period of time.

An optimal page replacement algorithm has the lowest page-fault rate of all algorithms and will never suffer from Belady's anomaly.

53.3 LRU Page Replacement

LRU replacement associates with each page the time of that page's last use. When a page must be replaced, LRU chooses the page that has not been used for the longest period of time.

This algorithm does not suffer from Belady's Anomaly.

Chapter 54

Thrashing

A process is thrashing if it is spending more time paging than executing.

Chapter 55

Local vs. Global Page Replacement

When a process incurs a page fault, a local page replacement algorithm selects for replacement some page that belongs to that same process.

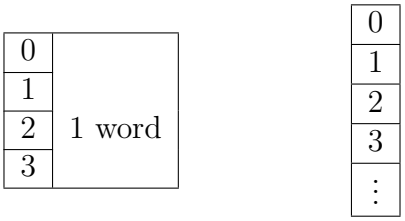
A global replacement algorithm is free to select any page in memory.

Chapter 56

Byte Addressing vs. Word Addressing

56.1 Byte Addressing

In byte addressing, (1 word = 4 bytes):



56.2 Word Addressing

In the above addressing scheme, the first word starts at address 0, and the first word are located at address 0.

The second word starts at address 4, and all bytes of the second word are located in address 4.

In the above addressing, in the first word all bytes of start at address 0 and all bytes of the second word are located in address a.

Computer Networks

Comprehensive Study Notes for GATE Computer Science

Ankur Gupta

www.ankurgupta.net

Note: These notes have been prepared primarily from the following books:

(1) **Data Communications and Networking** by Behrouz A Forouzan

Please refer to the above books for more details.

Contents

1 Physical Layer **8**

1.1 Introduction . . . 8
1.1.1 Analog versus Digital Data . . . 8
1.1.2 Periodic Analog Signals . . . 8
1.1.3 Phase and Wavelength . . . 9
1.1.4 Frequency Characteristics . . . 9
1.1.5 Single-Frequency Sine Waves . . . 9
1.2 Digital Signals . . . 9
1.2.1 Signal Levels . . . 9
1.2.2 Bit Rate . . . 9
1.2.3 Bit Length . . . 9
1.3 Transmission of Digital Signals . . . 10
1.3.1 Baseband Transmission . . . 10
1.3.2 Broadband Transmission . . . 10
1.4 Transmission Impairment . . . 10
1.4.1 Attenuation . . . 10
1.4.2 Distortion . . . 10
1.4.3 Noise . . . 10
1.5 Signal-to-Noise Ratio (SNR) . . . 11
1.5.1 Example Question . . . 11
1.6 Data Rate Limits . . . 11
1.6.1 Noiseless Channel . . . 11
1.6.2 Noisy Channel . . . 11
1.6.3 Example Problem . . . 12
1.7 Performance of a Network . . . 12
1.7.1 Bandwidth . . . 12
1.7.2 Definition . . . 12
1.7.3 Throughput . . . 12
1.7.4 Latency (Delay) . . . 12
1.7.5 Propagation Time . . . 12
1.7.6 Transmission Time . . . 13
1.7.7 Queuing Time . . . 13
1.7.8 Bandwidth-Delay Product . . . 13
1.7.9 Jitter . . . 13
1.7.10 Example Problem . . . 13
1.8 Signal Elements Versus Data Elements . . . 14
1.8.1 Definitions . . . 14
1.8.2 Data Rate Versus Signal Rate . . . 14

1.8.3 Definitions . . . 14
1.8.4 Formulas . . . 14
1.9 Asynchronous Transmission . . . 14
1.9.1 Synchronous Transmission . . . 14
1.10 Transmission Media . . . 15
1.10.1 Guided Media . . . 15
1.10.2 Twisted Pair Cable . . . 15
1.10.3 Coaxial Cable . . . 15
1.10.4 Unguided Media (Free Space) . . . 15
1.10.5 Types . . . 15
1.11 Switching . . . 16
1.11.1 Circuit-Switched Networks . . . 16
1.11.2 Datagram Networks . . . 17
1.11.3 Virtual Circuit Networks . . . 17

2 Network Topologies 18
2.1 Bus Topology . . . 18
2.1.1 Advantages . . . 18
2.1.2 Disadvantages . . . 18
2.2 Star Topology . . . 19
2.3 Ring Topology . . . 19
2.3.1 Advantages . . . 19
2.3.2 Disadvantages . . . 19
2.4 Mesh Topology . . . 20
2.4.1 Advantages . . . 20
2.4.2 Disadvantages . . . 20
2.5 Physical versus Logical Topology . . . 20
2.6 Combination of Topology and Transmission Media . . . 20

3 ISO/OSI Stack 21
3.1 Protocol Hierarchy . . . 21
3.2 Service Definitions . . . 21
3.3 Open System Interconnection Reference Model (OSI-ISO) . . . 21
3.3.1 Upper Layers of the OSI Model . . . 22
3.4 A Given Layer in the OSI Model . . . 22
3.5 Functions of the OSI Layers . . . 22
3.5.1 Physical Layer . . . 22
3.5.2 Data Link Layer . . . 23
3.5.3 Network Layer . . . 23
3.5.4 Transport Layer . . . 24
3.5.5 Session Layer . . . 24
3.5.6 Presentation Layer . . . 25
3.5.7 Application Layer . . . 25

4 Data Link Layer 26
4.1 Error Detection and Correction . . . 26
4.1.1 Types of Errors . . . 26
4.1.2 Redundancy . . . 26
4.1.3 Forward Error Correction . . . 26

4.2 Block Coding . . . 26
4.2.1 Hamming Distance . . . 27
4.2.2 Minimum Hamming Distance . . . 27
4.2.3 Minimum Distance for Error Detection . . . 27
4.3 Minimum Distance for Error Correction . . . 27
4.4 Error Detection Techniques . . . 27
4.4.1 Simple Parity Check . . . 27
4.4.2 Two-Dimensional Parity Check . . . 28
4.4.3 Checksum . . . 28
4.5 Performance of Checksum . . . 28
4.6 Cyclic Redundancy Check (CRC) . . . 29
4.6.1 Basic Approach . . . 29
4.6.2 Generating FCS . . . 29
4.6.3 Example . . . 29
4.7 Performance of CRC . . . 30
4.8 Error Correction Technique . . . 30
4.8.1 Requirement for Error Detection . . . 30
4.8.2 Requirement for Error Correction . . . 30
4.9 Hamming Code . . . 30
4.9.1 Example . . . 30
4.9.2 Calculating Parity Bits . . . 31
4.9.3 Example Problem . . . 31
4.10 Flow Control . . . 31
4.10.1 Stop-and-Wait Flow Control . . . 32
4.10.2 Sliding Window Flow Control . . . 32
4.10.3 Piggybacking . . . 32
4.10.4 Performance of Flow Control Protocols . . . 32
4.10.5 Stop and Wait Protocol . . . 32
4.11 Backward Error Control . . . 33
4.11.1 Types of Errors . . . 33
4.11.2 Three Versions of ARQ . . . 33
4.11.3 Stop-and-Wait ARQ . . . 33
4.11.4 Go-back-N ARQ . . . 34
4.11.5 Selective Repeat ARQ . . . 34
4.12 Data Link Layer Protocols . . . 34
4.13 HDLC Protocol . . . 34
4.14 Ethernet (IEEE 802.3) . . . 35
4.14.1 Multiple Access Protocols . . . 35
4.14.2 Pure ALOHA . . . 35
4.14.3 Slotted ALOHA . . . 35
4.14.4 CSMA/CD (Carrier-Sense Multiple-Access with Collision Detection) 35
4.14.5 The Binary Exponential Backoff Algorithm . . . 35
4.14.6 Limitation . . . 36

5 Routing Algorithms 37
5.1 Types of Routing Algorithms . . . 37
5.2 Fixed Routing or Non-adaptive Routing . . . 37
5.2.1 Advantages . . . 37

5.2.2 Disadvantages . . . 37
5.3 Flooding . . . 38
5.3.1 Measures to Improve Flooding . . . 38
5.4 Selective Flooding . . . 38
5.5 Flow-based Routing . . . 38
5.6 Adaptive Routing . . . 38
5.6.1 Two Principal Conditions That Change . . . 38
5.6.2 Algorithms Used . . . 38
5.7 Distance Vector Routing . . . 39
5.7.1 Key Characteristics . . . 39
5.8 Link-State Versus Distance Vector Routing . . . 39
5.9 Hierarchical Routing . . . 39

6 Congestion Control 40
6.1 Common Causes of Congestion . . . 40
6.2 Congestion . . . 40
6.3 Congestion Control . . . 40
6.4 Open Loop Congestion Control . . . 40
6.4.1 Traffic Shaping . . . 41
6.5 The Leaky Bucket Algorithm . . . 41
6.6 Flow-based Routing . . . 41
6.7 Close Loop Congestion Control . . . 42
6.7.1 Three Basic Steps . . . 42
6.8 Congestion Control Versus Flow Control . . . 42
6.9 Token Bucket Algorithm . . . 42

7 Bridge 43
7.1 Network Devices Overview . . . 43
7.1.1 Repeaters . . . 43
7.1.2 Hub . . . 43
7.2 Bridge Characteristics . . . 44
7.2.1 Types of Bridges . . . 44
7.3 Transparent Bridge . . . 44
7.3.1 Learning in Transparent Bridge . . . 44
7.3.2 Functions . . . 44
7.3.3 Loop Problem in Transparent Bridges . . . 44
7.3.4 Solution to Loop Problem (Spanning Tree Topology) . . . 44
7.4 Source Routing Bridge . . . 45
7.4.1 Switch . . . 45
7.4.2 Three Forwarding Approaches . . . 45
7.5 Router . . . 45

8 TCP/IP 46
8.1 TCP/IP . . . 46
8.1.1 TCP/IP and OSI Model . . . 46
8.2 Interworking Layers . . . 46
8.2.1 IP . . . 47
8.3 Addresses in TCP/IP . . . 47
8.4 Classful Addressing . . . 47

8.4.1 Default Masks for Classful Addressing . . . 47
8.4.2 Some Special IP Addresses . . . 47
8.5 Subnetting . . . 48
8.5.1 Classless Addressing . . . 48
8.5.2 Restrictions on Classless Address Blocks . . . 48
8.5.3 Example . . . 48
8.5.4 Network Address . . . 48
8.5.5 Two-level Hierarchy: No Subnetting . . . 48
8.5.6 Three-levels of Hierarchy: Subnetting . . . 49
8.5.7 More Level of Hierarchy . . . 49
8.5.8 Example of Three Levels of Hierarchy . . . 49
8.6 Masking . . . 49
8.7 Calculating Addresses from Mask . . . 50

9 Application Layer 51
9.1 TELNET . . . 51
9.2 MIME (Multipurpose Internet Mail Extension) . . . 51
9.3 SMTP (Simple Mail Transfer Protocol) . . . 52
9.4 FTP (File Transfer Protocol) . . . 52
9.4.1 Communication over Control Connection . . . 52
9.5 POP3 (Post Office Protocol, version 3) . . . 52
9.6 IMAP4 (Internet Mail Access Protocol version 4) . . . 53
9.7 DNS (Domain Name System) . . . 53
9.7.1 Reverse DNS Lookup . . . 53
9.7.2 Forward DNS Lookup . . . 53
9.8 HTTP Protocol . . . 53

10 Network Security 54
10.1 Public Key Cryptography . . . 54
10.1.1 RSA . . . 54
10.1.2 Diffie-Hellman Key Exchange Algorithm . . . 54

Chapter 1

Physical Layer

1.1 Introduction

To be transmitted, data must be transformed to electromagnetic signals. The Physical layer is concerned with transmitting raw bits over a communication channel. It also deals with framing.

1.1.1 Analog versus Digital Data

- **Analog data** are continuous, while **digital data** have discrete states.
- Analog signals can have an infinite number of values in a range, while digital signals can have only a limited number of values.
- In data communications, we commonly use periodic analog signals (because they need less bandwidth) and non-periodic digital signals (because they can represent variation in data).

1.1.2 Periodic Analog Signals

Periodic analog signals can be classified as simple or composite. A composite analog (periodic) signal is composed of multiple sine waves.

- **Peak amplitude** of a signal is the absolute value of its highest intensity.
- **Period** refers to the amount of time, in seconds, a signal needs to complete one cycle.
- **Frequency** refers to the number of periods in one second.

$$\text{Period} \rightarrow \text{seconds}$$
$$\text{Frequency} \rightarrow \text{Hz} \rightarrow \text{cycles per second}$$
$$f = \frac{1}{T}$$

1.1.3 Phase and Wavelength

- **Phase** describes the position of the waveform relative to time zero.
- **Wavelength** is the distance a single signal can travel in one period.

Wavelength = Propagation Speed × Period

$$\lambda = c \cdot T = \frac{c}{f}$$

1.1.4 Frequency Characteristics

- Frequency is the rate of change with respect to time. Change in a short span of time means high frequency.
- Change over a long span of time means low frequency.
- If a signal does not change at all, its frequency is zero.
- If a signal changes instantaneously, its frequency is infinite.

1.1.5 Single-Frequency Sine Waves

A single-frequency sine wave is not useful in data communications. We need to send a composite signal, a signal made of many simple sine waves.

A normal human being can create a continuous range of frequencies between 0 and 4 kHz.

1.2 Digital Signals

1.2.1 Signal Levels

If a signal has L levels, each level needs $\lceil \log_2 L \rceil$ bits.

$$\text{Number of bits per level} = \lceil \log_2 L \rceil$$

Most digital signals are non-periodic, and thus period and frequency are not appropriate characteristics for digital signals. Bit rate (instead of frequency) is used to describe digital signals.

1.2.2 Bit Rate

Bit rate is the number of bits sent in one second, expressed in bits per second (bps).

Bit Rate = Number of bits sent in 1 sec

1.2.3 Bit Length

The bit length is the distance one bit occupies on the transmission medium.

Bit Length = Propagation Speed × bit duration
= Propagation Speed / Transmission Speed

A digital signal is a composite analog signal with infinite bandwidth.

1.3 Transmission of Digital Signals

1.3.1 Baseband Transmission

Baseband transmission means sending a digital signal over a channel without changing the digital signal to an analog signal.

Baseband transmission requires that we have a low-pass channel, a channel with a bandwidth that starts from zero.

1.3.2 Broadband Transmission

Broadband transmission or modulation means changing the digital signal to an analog signal for transmission.

Modulation allows us to use a bandpass channel – a channel with a bandwidth that does not start from zero.

1.4 Transmission Impairment

$$\text{dB} = 10 \log_{10} \frac{P_2}{P_1}$$

1.4.1 Attenuation

Attenuation means a loss of energy. When a signal travels through a medium, it loses some of its energy in overcoming the resistance of the medium.

To compensate for this loss, amplifiers are used to amplify the signal.

1.4.2 Distortion

Distortion means that the signal changes its shape.

Distortion can occur in a composite signal made of different frequencies because each signal component has its own propagation speed through a medium, and therefore, its own delay in arriving at the final destination.

1.4.3 Noise

Several types of noise, such as thermal noise, induced noise, crosstalk, and impulse noise, may corrupt the signal.

- Induced noise comes from sources such as motors and appliances.
- Crosstalk is the effect of one wire on another.
- Impulse noise is a spike that comes from power lines.

1.5 Signal-to-Noise Ratio (SNR)

$$\text{SNR} = \frac{\text{Average Signal Power}}{\text{Average Noise Power}}$$

- A high SNR means the signal is less corrupted by noise.
- A low SNR means the signal is more corrupted by noise.

$$\text{SNR}_{\text{dB}} = 10 \log_{10} \text{SNR}$$

1.5.1 Example Question

Theoretically, if we have given a specific bandwidth, we can have any bit rate we want by increasing the number of signal levels.

→ Increasing the levels of a signal may reduce the reliability of the system.

1.6 Data Rate Limits

Data rate depends on three factors:

(1) The bandwidth available

(2) The level of signal we use

(3) The quality of the channel (the level of noise)

1.6.1 Noiseless Channel

Nyquist Bit Rate

$$\textbf{Bit Rate} = 2 \times \text{BW} \times \log_2 L$$

Here, BW is the bandwidth of the channel, L is the number of levels to represent data, and Bit Rate is the bit rate in bits per second.

1.6.2 Noisy Channel

Shannon Capacity

The Shannon capacity gives us the upper limit; it tells us how many signal levels we need.

$$\textbf{Capacity} = \text{BW} \times \log_2(1 + \text{SNR})$$

Capacity ⇒ Capacity of the channel in bits per second.

1.6.3 Example Problem

Question: We have a channel with a 1 MHz BW. If the SNR for this channel is 63, what is the appropriate bit rate and signal level?

Solution: Using Shannon formula:

$$\begin{aligned} C &= B \log_2(1 + \text{SNR}) \\ &= 10^6 \log_2(1 + 63) = 10^6 \times 6 \\ &= 6 \text{ Mbps, Ans} \end{aligned}$$

Shannon formula $\Rightarrow$ 6 Mbps; for the upper limit. For better performance, we should choose something lower, 4 Mbps.

Using Nyquist Formula:

$$\begin{aligned} 4 \text{ Mbps} &= 2 \times 1 \text{ MHz} \times \log_2 L \\ \Rightarrow 2 &= \log_2 L \Rightarrow L = 2^2 = 4 \text{ levels} \end{aligned}$$

1.7 Performance of a Network

1.7.1 Bandwidth

1.7.2 Definition

In networking, we use the term **bandwidth** in two contexts:

(i) Bandwidth in hertz refers to the range of frequencies in a composite signal.

(ii) Bandwidth in bits per second refers to the speed of bit transmission in a channel.

Basically, an increase in bandwidth in hertz means an increase in bandwidth in bps.

1.7.3 Throughput

Throughput is a measure of how fast we can actually send data through a network.

Throughput $<$ Bandwidth

The bandwidth is a potential measure of a link; the throughput is an actual measurement of how fast we can send data.

1.7.4 Latency (Delay)

The time taken for an entire message to completely arrive at the destination from the time the first bit is sent out from the source.

Latency = Propagation time + transmission time + queuing time + processing delay

1.7.5 Propagation Time

The time required for a bit to travel from the source to the destination.

Propagation Time = $\frac{\text{Distance}}{\text{Propagation Speed}}$

1.7.6 Transmission Time

Transmission Time = $\frac{\text{Message Size}}{\text{Bandwidth}}$

1.7.7 Queuing Time

The time needed for each intermediate or end device to hold the message before it can be processed.

1.7.8 Bandwidth-Delay Product

It is the number of bits that can fill the link.

Bandwidth-Delay Product = Bandwidth × Delay

1.7.9 Jitter

Jitter is an problem if different packets of data encounter different delays and the application using the data at the receiver site is time-sensitive (audio and video data).

1.7.10 Example Problem

Question: What are the propagation time and the transmission time for a 5 MB message if the bandwidth of the network is 1 Mbps? Assume that the distance between the sender and the receiver is 12,000 km and that light travels at 2.4×10^8 m/s.

Solution:

$$\begin{aligned}\text{Propagation Time} &= \frac{12,000 \times 10^3}{2.4 \times 10^8} \\ &= 5 \times 10^{-2} \text{ sec} \\ &= 0.05 \text{ sec}\end{aligned}$$

$$\begin{aligned}\text{Transmission Time} &= \frac{5 \times 10^6 \times 8}{1 \times 10^6} \\ &= 40 \text{ sec}\end{aligned}$$

$$\begin{aligned}\text{Latency} &= \text{Propagation time} + \text{transmission time} \\ &= 40 + 0.05 \\ &= 40.05 \text{ sec} \approx 40 \text{ sec}\end{aligned}$$

1.8 Signal Elements Versus Data Elements

1.8.1 Definitions

- A **data element** is the smallest entity that can represent a piece of information; this is the bit.
- In data communications, a **signal element** carries data elements. A signal element is the shortest unit of a digital signal.

We define a ratio r, which is the number of data elements carried by each signal element.

1.8.2 Data Rate Versus Signal Rate

1.8.3 Definitions

- The **data rate** defines the number of data elements (bits) sent in one second. The unit is bits per second (bps).
- The **signal rate** is the number of signal elements sent in one second. The unit is the baud.

1.8.4 Formulas

For digital signals:

$$S = c \times N \times \frac{1}{r} \text{ baud}$$

For analog signals:

$$S = N \times \frac{1}{r} \text{ baud}$$

where S = Signal Rate (baud), c = case factor, N = data rate (bps), and r = number of data elements per signal element.

1.9 Asynchronous Transmission

In asynchronous transmission, the timing of a signal is not important. Instead, information is received and translated by agreed-upon patterns.

In asynchronous transmission, we send one start bit (0) at the beginning and one or more stop bits (1s) at the end of each byte. There may be a gap between each byte.

1.9.1 Synchronous Transmission

In synchronous transmission, we send bits one after another without start or stop bits or gaps. It is the responsibility of the receiver to group the bits.

Synchronous transmission is faster than asynchronous transmission.

1.10 Transmission Media

Transmission media can be divided into two broad categories:

(i) Guided Media

(ii) Unguided Media

1.10.1 Guided Media

Guided media include twisted pair, coaxial cable, and fiber-optic cable.

1.10.2 Twisted Pair Cable

→ RJ45 connector is used.

→ Used in telephone lines, DSL lines, and LANs.

1.10.3 Coaxial Cable

→ Carries signals of higher frequency ranges than those in twisted-pair cable.

→ Higher frequency means higher bandwidth.

→ Used in Cable TV, Thin Ethernet, and Thick Ethernet.

→ Attenuation is much higher in coaxial cable than in a twisted-pair cable.

→ Coaxial cable has much higher bandwidth.

→ It requires the frequent use of repeaters.

1.10.4 Unguided Media (Free Space)

1.10.5 Types

Radio Waves

→ Radio waves have frequencies between 3 kHz and 1 GHz.

→ Radio waves are omnidirectional.

→ Used for long-distance broadcasting.

→ Can penetrate walls.

→ Have low data rate for digital communication.

Microwaves

→ Have frequencies between 1 and 300 GHz.

→ Microwaves are unidirectional.

→ Very high-frequency microwaves cannot penetrate walls.

→ High data rate for digital communication is possible.

→ Used for unicast (one-to-one) communication such as cellular phones, satellite networks, and wireless LANs.

Infrared

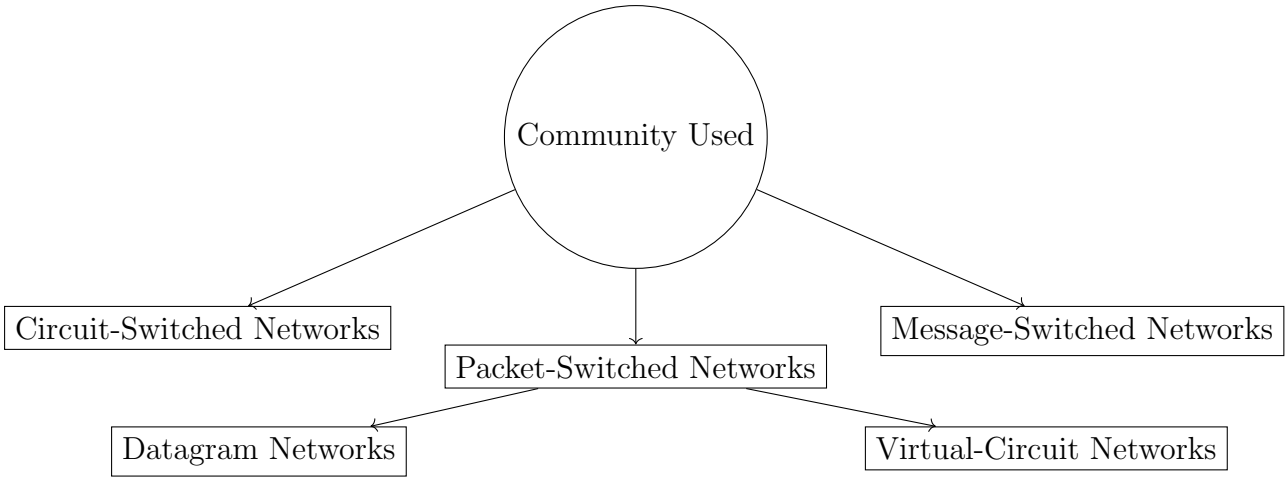
→ Have frequencies between 300 GHz and 400 THz.

→ Used for short-range communication.

→ Cannot penetrate walls.

1.11 Switching

A switched network consists of a series of interlinked nodes, called switches.



1.11.1 Circuit-Switched Networks

It is made of a set of switches connected by physical links, in which each link is divided into n channels.

Circuit switching takes place at the physical layer.

The resources need to be reserved during the setup phase and remain dedicated for the entire duration of data transfer until the teardown phase.

There is no addressing involved during data transfer.

1.11.2 Datagram Networks

In a packet-switched network, there is no resource reservation; resources are allocated on demand.

Message is divided into packets of fixed or variable size.

In a datagram network, each packet is treated independently of all others.

Datagram switching is normally done at the network layer.

The switches in a datagram network are referred to as routers.

Datagram networks are sometimes referred to as connectionless networks.

1.11.3 Virtual Circuit Networks

A virtual-circuit network is a cross between a circuit-switched network and a datagram network.

All packets follow the same path established during the connection.

It is implemented in the data link layer.

Chapter 2

Network Topologies

2.1 Bus Topology

→ Single cable connects all computers.

→ Each computer has connector to shared cable.

→ Computers must synchronize and allow only one computer to transmit at a time.

→ Cable segment must end with a terminator.

→ Uses thin coaxial cable (backbones will be thick coaxial cable).

→ Standard is IEEE 802.3.

2.1.1 Advantages

(i) Inexpensive to install

(ii) Easy to add stations

(iii) Uses less cable than other topologies

(iv) Works well for small networks

2.1.2 Disadvantages

(i) No longer recommended

(ii) Backbone breaks, while network is down

(iii) Limited number of devices can be attached

(iv) Difficult to isolate problems

(v) Sharing same cable slows response rate

2.2 Star Topology

→ Each device has a dedicated point-to-point link to a hub.

→ Easier to maintain.

→ Very flexible and the most preferred topology by industries.

→ Requires more cable than bus topology.

→ If hub goes down, the whole system is dead.

→ Used in high-speed LANs.

→ Twisted pair cable is used.

The hub can be used in two ways:

(i) As a repeater, where a frame from one node is retransmitted to all of the outgoing links.

(ii) As a switch, where a frame from one node is retransmitted only on an outgoing link to the destination station.

2.3 Ring Topology

→ Computers connected in a closed loop.

→ Most common type is Token Ring (IEEE 802.5).

→ Single ring: Data travels in one direction.

→ Double ring: Allows fault tolerance.

2.3.1 Advantages

→ Data packets travel at great speed

→ No collisions

→ Easier to find faults

→ No terminators required

2.3.2 Disadvantages

→ A break in the ring will bring it down

→ Not as common as the bus or star devices available

Widely used in Wide Area Networks.

2.4 Mesh Topology

Not common on LANs. Most often used in WANs to interconnect LANs.

Each node is connected to every other node.

It is fault-tolerant.

2.4.1 Advantages

→ Improved fault tolerance

→ Can carry more data

2.4.2 Disadvantages

→ Expensive

→ Difficult to install, manage, and troubleshoot

2.5 Physical versus Logical Topology

The actual layout of a network and its media is its physical topology.

The way in which the data accesses the medium and transmits packets is the logical topology.

A glance at a network is not always revealing. Cables emerging from a hub does not make it necessarily a star topology – it may actually be a bus or a ring.

2.6 Combination of Topology and Transmission Media

→ Twisted pair is suitable for use in star and ring topologies.

→ Coaxial cable is suitable for use in bus topology.

→ Fiber optics is suitable for use in ring and star topology.

→ Unguided media are suitable for star topology.

Chapter 3

ISO/OSI Stack

3.1 Protocol Hierarchy

Most networks are organized as a series of layers.

The task of each layer is to give some services to the upper layer. Each layer maintains a virtual connection with the corresponding layers in a peer.

The interface between the layers in the same node is well defined.

The implementation of each layer in each node is transparent to other nodes.

The protocols between the peer layers can be changed if the peers all agree. However, it need not be referred to other layers.

3.2 Service Definitions

The service definitions tell what the layer does and nothing else.

The interface tells the process above it how to access it. It specifies what the parameters are and what results to expect.

3.3 Open System Interconnection Reference Model (OSI-ISO)

The OSI reference model is a conceptual model composed of seven layers, each specifying particular network functions.

It was developed by the International Organization for Standardization (ISO).

The OSI model is not a protocol.

APPLICATION	7
PRESENTATION	6
SESSION	5
TRANSPORT	4
NETWORK	3
DATA-LINK	2
PHYSICAL	1

3.3.1 Upper Layers of the OSI Model

The upper layers of the OSI model generally are implemented only in software.

The physical layer and the data link layer are implemented in software and hardware.

3.4 A Given Layer in the OSI Model

A given layer in the OSI model generally communicates with three other OSI layers:

(i) The layer directly above it

(ii) The layer directly below it

(iii) Its peer layer in other networked systems

OSI is a model, not a protocol.

The OSI model provides a conceptual framework between computers, but the model itself is not a method of communication. Actual communication is made possible by using communication protocols.

A protocol implements the functions of one or more of the OSI layers.

3.5 Functions of the OSI Layers

3.5.1 Physical Layer

The physical layer is concerned with transmitting raw bits over a communication channel.

The physical layer is also concerned with framing.

Physical Characteristics of Interfaces and Medium

The number of pins and functions of each pin of the network connector (Mechanical).

Representation of Bits

How 0s and 1s are changed to electrical signals.

Data Rate and Synchronization of Bits

Sender and receiver must use the same bit rate and both must have synchronized clocks.

Establishing and Breaking of Connection

Transmission Mode

Simplex, half-duplex, or full-duplex.

Physical Topology

Bus/Star/Ring/Mesh.

3.5.2 Data Link Layer

The data link layer transforms the physical layer, a raw transmission facility, to a reliable link. It makes the physical layer appear error-free to the upper layer (network layer).

Framing

The data link layer divides the stream of bits received from the network layer into manageable data units called frames.

Physical Addressing

The data link layer adds a header to the frame to define the address of the next hop in the route. Next hop (node) may be the receiver or some intermediate node.

Flow Control

Deals with the data rate mismatch between the sender and the receiver.

Error Control

Uses mechanisms to detect and retransmit damaged or lost frames, and to recognize duplicate frames by using error control ARQ technique.

Access Control

Uses protocols like ALOHA and CSMA to control access on a shared medium.

Synchronization

Synchronize and initialize send and receive sequence numbers with its peer at the other end.

3.5.3 Network Layer

The network layer is responsible for the source-to-destination delivery of a packet.

If two systems are connected to the same link, there is usually no need for a network layer.

This source-to-destination delivery is performed using two basic approaches known as connection-oriented or connectionless network layer services.

Logical Addressing

The network layer adds a packet header to the frame coming from the upper layer that, among other things, includes the logical addresses of the sender and receiver.

Routing

The routing algorithm is the piece of software that decides where a packet goes next.

Dealing with Congestion

Congestion occurs in a network when more packets enter an area than can be processed.

Dealing with Internetworking

Packets may travel through many different networks. Each network may have a different frame format.

Some networks may be connectionless, others connection-oriented.

Source-to-destination delivery is also called as end-to-end delivery or host-to-host delivery.

3.5.4 Transport Layer

The transport layer is responsible for process-to-process delivery of the entire message.

The transport layer ensures that the whole message arrives intact and in order, overseeing both error control and flow control at the source-to-destination level.

Service Point Addressing

The transport layer header includes a type of address called a Service Point Address or Port Address to deliver the entire message to the correct process on the computer.

Segmentation and Reassembly

A message is divided into transmittable segments with each segment containing a sequence number, that is used for reassembly at the message correctly.

Connection Control

The transport layer can be either connectionless or connection-oriented. A connectionless transport layer treats each segment as an independent packet. A connection-oriented transport layer makes a connection with the transport layer at the destination machine first before delivering the packets.

Flow Control

Flow control at this layer is performed end-to-end.

Error Control

Error control at this layer is performed process to process. Error correction is usually achieved through retransmission.

3.5.5 Session Layer

This layer allows users on different machines to establish sessions between them. A session allows ordinary data transport but it also provides enhanced services useful in some applications.

Some of the session-related services are:

(i) **Dialogue Control:** Session can allow traffic to go in both directions at the same time, or in only one direction at one time. It also decides who speaks when and how long.

(ii) **Token Management:** Allows only one side that is holding token to perform the critical operation.

(iii) **Synchronization:** Allows a process to add checkpoints, or synchronization points in a stream of data, so that after a crash, only the data transferred after the last checkpoint have to be repeated.

Session layer establishes, maintains, and synchronizes the interaction among communicating systems.

3.5.6 Presentation Layer

This layer is concerned with syntax and semantics of the information transmitted, unlike other layers which are interested in moving data reliably from one machine to other.

Few of the services that presentation layer provides are:

(i) **Translation:** Encoding data in a standard agreed-upon way.

(ii) **Encryption and Decryption**

(iii) **Compression and Decompression**

3.5.7 Application Layer

The application layer enables the user to access the network.

Various services provided by application layer are:

(i) File Transfer (FTP)

(ii) Remote Login (Telnet)

(iii) Mail (SMTP)

(iv) News (NNTP)

(v) Web (HTTP)

Chapter 4

Data Link Layer

Specific responsibilities of the data link layer include framing, addressing, flow control, error control, and media access control.

4.1 Error Detection and Correction

4.1.1 Types of Errors

(i) Single-bit error

(ii) Burst error

If the length of the burst is measured from the first corrupted bit to the last corrupted bit, the burst length would be determined.

4.1.2 Redundancy

To detect or correct errors, we need to send extra (redundant) bits with data.

4.1.3 Forward Error Correction

The receiver tries to guess the message by using redundant bits.

In backward error correction, we retransmit the frame.

Modulo-2 Arithmetic

$$\begin{aligned} &0+0=0, \quad 0+1=1 \\ &0-0=0, \quad 0-1=1 \\ &1+0=1, \quad 1-0=1 \\ &1-1=0 \end{aligned}$$

4.2 Block Coding

In block coding, we divide our message into blocks, each of k bits, called datawords. We add r redundant bits to each block to make the length $n = k + r$. The resulting n-bit blocks are called codewords.

4.2.1 Hamming Distance

The Hamming distance between two words is the number of differences between corresponding bits.

$$d(10101, 11110) = 3$$

4.2.2 Minimum Hamming Distance

The smallest Hamming distance between all possible pairs in a set of words.

4.2.3 Minimum Distance for Error Detection

To guarantee the detection of up to s errors in all cases, the minimum Hamming distance in a block code must be $d_{\min} = s + 1$.

4.3 Minimum Distance for Error Correction

To guarantee correction of up to t errors in all cases, the minimum Hamming distance in a block code must be more than two.

Number of s.

If R is the number of additional bits, the condition is:

$$2^R \geq m + R + 1$$

Number of Data Bits (m)	Number of Redundancy Bits (R)	Total Bits (m+R)
1	2	3
2	3	5
3	3	6
4	3	7
5	4	9
6	4	10
7	4	11
8	4	12

4.4 Error Detection Techniques

4.4.1 Simple Parity Check

Append a parity bit to the end of the data.

There are two types of parity: even parity and odd parity.

It can detect all single-bit errors.

It can also detect burst errors, if the number of bits in error is odd.

4.4.2 Two-Dimensional Parity Check

Organizes the block of bits in the form of a table.

Parity check bits are calculated for each row, and after that for all columns.

It can detect up to three errors that occur anywhere in the table. However, errors affecting 4 bits may not be detected.

4.4.3 Checksum

The Sender's End

The data is divided into k segments each of m bits. The segments are added using 1's complement to get the sum.

The sum is complemented to get the checksum.

The checksum segment is sent along with data segments.

The Receiver's End

All received segments are added using 1's complement arithmetic to get the sum.

The sum is complemented.

If the result is zero, the received data is accepted; otherwise discarded.

Example 4.4.1. Sender:

$$
\begin{aligned}
E = 4, m = 8 \\
(i) &\rightarrow 10110011 \\
(ii) &\rightarrow 10101010 \\
&\quad 01011110 \\
&\quad C_1C_2C_3C_4C_5C_6 \\
\text{Sum} &: 00101110
\end{aligned}
$$

Receiver:

$$
\begin{aligned}
(i) &\rightarrow 10110011 \\
(ii) &\rightarrow 10101010 \\
(iii) &\rightarrow 01011110 \\
&\quad 00101110 \\
&\quad 10100101 \\
\text{Sum} &: 11111111 \\
\text{Complement} &: 00000000 \\
\text{Conclusion} &: \text{Accept data}
\end{aligned}
$$

4.5 Performance of Checksum

$\rightarrow$ CRC can detect all single-bit errors.

→ CRC can detect all double-bit errors (provided that the minimum three terms in the characteristic polynomial $P(x)$).

→ CRC can detect all odd-number of errors (provided that $P(x)$ should be divisible by $c + 1$).

→ CRC can detect all burst errors of length less than the degree of the characteristic polynomial (n).

→ CRC detects most of the larger burst errors with a high probability.

4.6 Cyclic Redundancy Check (CRC)

4.6.1 Basic Approach

Given an m-bit block of data, the sender generates an n-bit sequence, known as a Frame Check Sequence (FCS), so that the resulting frame consists of $m + n$ bits.

The receiver divides the incoming frame by the number used for generating FCS. If there is no remainder, the receiver assumes that there is no error.

4.6.2 Generating FCS

To generate an n-bit FCS, the following steps are followed:

(i) Append n 0s to the right-end side of the m-bit dataword.

(ii) The $(m + n)$-bit result is fed into the generator.

(iii) The generator divides the augmented dataword by the divisor (modulo-2 division), of size $n + 1$, that is predefined and agreed-upon.

(iv) The n-bit remainder of the division is called the frame check sequence (FCS).

(v) The FCS is appended to the dataword to create a codeword.

4.6.3 Example

Data: 1010, Divisor: 1011

At sender:

1001 → Quotient

1011 | 1010000

1010

<u>1011</u>

0010

<u>0000</u>

1000

<u>1011</u>

011 → Remainder

Data to be sent: 1010011
At receiver:

Since remainder is 0, there is no error

4.7 Performance of CRC

→ CRC can detect all single-bit errors

→ CRC can detect all double-bit errors

→ CRC can detect all odd number of errors

→ CRC can detect all burst errors

→ CRC detects most of the larger burst errors with a high probability

4.8 Error Correction Technique

4.8.1 Requirement for Error Detection

The minimum Hamming distance between any two codewords should be two.

4.8.2 Requirement for Error Correction

The minimum Hamming distance between any two codewords must be more than two.

Number of additional bits should be such that it can point to the position of the bit in error.

If R is the number of additional bits, the condition is:

$$2^R \geq m + R + 1$$

4.9 Hamming Code

4.9.1 Example

Data ⇒ 1011

Hamming Code ⇒

7	6	5	4	3	2	1
d_4	d_3	d_2	p_3	d_1	p_2	p_1

Codeword ⇒ 1010101

To each group of m information bits, R parity bits are added to form $(m + R)$ bit codeword.

Location of each of the $(m + R)$ bits is assigned a decimal value.

The R parity bits are placed in positions $1, 2, 4, \ldots, 2^{R-1}$.

R parity checks are performed on selected digits of each codeword.

At the receiving end, the parity bits are recalculated. The decimal value of R parity bits provides the bit position in error, if any.

4.9.2 Calculating Parity Bits

$$P_1 \rightarrow 1, 3, 5, 7$$
$$P_2 \rightarrow 2, 3, 6, 7$$
$$P_3 \rightarrow 4, 5, 6, 7$$

4.9.3 Example Problem

Data $\Rightarrow$ 1011

5		4		3		2	1
d_4	d_3	d_2	p_3	d_1	p_2	p_1	
1	0	1	0	1	0	1	

Sent to Receiver $\rightarrow$

Received Codeword: 1 0 1 0 1 0 1

Error

Calculating parity bits:

$$C_3\ C_2\ C_1$$
$$1\quad 0\quad 1 = 5$$

Location of error

After error correction:

$$1\quad 0\quad 1\quad 1\quad 0\quad 1$$

Data to be sent: $x^4 + x^3 + x + 1$

CRC:

$$\frac{x^6 + x^4 + x^3 + x + 1}{\text{CRC}}$$

At Receiver:

$$x^4 + x^4 + x + 1 \rightarrow M(x) = x^3(x + 1)$$
$$\text{Divisor} = 1011 = x^3 + x + 1$$

4.10 Flow Control

Flow control is the set of procedures used to restrict the amount of data that the sender can send before waiting for acknowledgment.

Various protocols used in flow control are:

(1) Stop-and-Wait Flow Control

(2) Sliding Window Flow Control

4.10.1 Stop-and-Wait Flow Control

This is the simplest form of flow control.

The source transmits a data frame.

After receiving the frame, the destination indicates its willingness to accept another frame by sending back an ACK frame acknowledging the frame just received.

The source must wait until it receives the ACK frame before sending the next data frame.

With the use of multiple frames for a single message, the stop-and-wait protocol does not perform well because only one frame can be in transit at a time.

4.10.2 Sliding Window Flow Control

In sliding window protocol, we allow multiple frames to be in transit at the same time.

To keep track of the frames, sender sends sequentially numbered frames.

The sequence numbers range from 0 to $2^k - 1$.

The sender maintains a list of sequence numbers that it is allowed to send (sender window).

The size of the sender window is at most 2^{k-1}.

For $R = 3$,

$$0 \quad 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6 \quad 7 \quad 0$$

The receiver also maintains a window of size 1.

The receiver acknowledges a frame by sending an ACK frame that includes the sequence number of the next frame expected.

This scheme can be used to actually acknowledge multiple frames.

4.10.3 Piggybacking

If two stations exchange data, each need to maintain two windows: one for transmit and one for receive. To save communication capacity, a technique called piggybacking is used.

When a frame is carrying data from A to B, it can also carry control information about arrived (or lost) frames from B and vice-versa.

If a station has an ACK but no data to send, it sends a separate ACK frame.

4.10.4 Performance of Flow Control Protocols

4.10.5 Stop and Wait Protocol

Utilization or Efficiency of the Line

$$U = \frac{\text{transmission}}{2 \times \text{propagation} + \text{transmission}}$$

Assuming transmission time of ACK to be negligible (tACK = 0):

$$U = \frac{\text{transmission}}{2 \times \text{propagation} + \text{transmission}}$$

Sliding Window Protocol

$$U = \frac{W \times \text{transmission}}{2 \times \text{propagation} + \text{transmission}}$$

where W = Window size (2^{k-1}), propagation = Propagation time, transmission = Transmission time.

For maximum utilization, $U = 1$:

$$W \times \text{transmission} \geq 2 \times \text{propagation} + \text{transmission}$$

then $U = 1$.

4.11 Backward Error Control

During frame transmission, following two types of errors may occur:

4.11.1 Types of Errors

Lost Frame

A noise burst may damage a frame to such an extent that it is not recognizable at the receiving end.

Damaged Frame

A frame does arrive, but some of the bits are in error.

Error control in the data link layer is based on automatic repeat request (ARQ), which is the retransmission of data.

4.11.2 Three Versions of ARQ

(i) Stop-and-Wait ARQ

(ii) Go-back-N ARQ

(iii) Selective Repeat ARQ

4.11.3 Stop-and-Wait ARQ

Based on the stop-and-wait flow control technique.

The source transmits a single frame and then waits for an ACK.

No other data frame can be sent until the ACK arrives at the source.

A timer is used to take care of lost and damaged frames and lost ACK.

The sender will timeout and resend the same frame, if the ACK is lost.

The receiver receives the same frame twice. How to identify duplicate frames?

A 1-modulo-2 numbering scheme is used, where frames are alternately labeled with 0 or 1, and positive acknowledgments are of the form ACK0 or ACK1.

The ACK always announces the sequence number of the next frame accepted.

4.11.4 Go-back-N ARQ

Based on sliding window protocol and most widely used.

The source station may send a series of frames sequentially up to a maximum number, without hearing about any ACK.

If no error, ACK as usual.

In case of no error, the destination will acknowledge incoming frames as usual.

If the destination detects error in frame, or it received a frame out of order (frame lost), it sends a NAK for that frame (using reject or REJ frame).

The destination will discard that frame and all future frames until the frame in error is correctly received.

The source, on receiving a REJ, must retransmit the frame in error plus all succeeding frames.

For lost ACK, the sender after timeout resends the frames that are still unacknowledged.

For 8-bit sequence number, the maximum sender window size is limited to $(2^k - 1)$.

The size of the receiver window is 1.

4.11.5 Selective Repeat ARQ

In this case, only those frames are retransmitted for which negative ACK has been received (called selective reject SREJ) or time-out has occurred.

Receiver requires storage buffers to contain out-of-order frames until the frame in error is correctly received.

The sender and receiver window must be at most one-half of 2^k:

$$\Rightarrow \frac{2^k}{2} = 2^{k-1}$$

4.12 Data Link Layer Protocols

4.13 HDLC Protocol

HDLC

→ High-Level Data Link Control

→ Bit-oriented protocol

→ Communication over point-to-point and multipoint links

Point-to-Point Protocol

→ Point-to-point access

→ Byte-Oriented Protocol

4.14 Ethernet (IEEE 802.3)

4.14.1 Multiple Access Protocols

4.14.2 Pure ALOHA

A station may transmit a frame at any time. The station then listens for an amount of time equal to the maximum possible round-trip propagation delay ($2t_p$) for an acknowledgment.

If the station does not hear an acknowledgment, it resends the frame.

The maximum utilization of the channel is only about 18%.

4.14.3 Slotted ALOHA

Here, the channel is organized into uniform slots whose size equals the frame transmission time.

Transmission is permitted only at a slot boundary.

The maximum utilization of the system is about 37%.

Both ALOHA and slotted ALOHA fail to take advantage of one of the key properties of networks, which is that propagation delay; the stations may be very small compared to frame transmission time.

4.14.4 CSMA/CD (Carrier-Sense Multiple-Access with Collision Detection)

In CSMA/CD, the station listens to the medium while transmitting (Listen while talking).

The steps in CSMA/CD are:

(i) If the medium is idle, transmit.

(ii) If the medium is busy, continue to listen until the channel is idle, then transmit immediately (1-persistent).

(iii) If a collision is detected during transmission, transmit a jamming signal to ensure that all stations know that there has been a collision and then cease transmission.

(iv) After transmitting the jamming signal, wait a random amount of time, referred to as backoff, then attempt to transmit again.

4.14.5 The Binary Exponential Backoff Algorithm

This algorithm is used to calculate random amount of time (backoff) to wait before attempting to retransmit.

In this algorithm, after k-th collision, each station waits x-slot times where $0 \leq x \leq 2^k$.

We choose x randomly from the interval 0 to 2^{k-1}.

If R-stations transmit during a contention slot with probability p, the probability A that some station acquires the channel in that slot is:

$$A = Rp(1-p)^{R-1}$$

Frames should be long enough to allow collision detection prior to end of transmission.

Frame transmission time $(t_f) \geq 2t_p$

4.14.6 Limitation

Enforces rigid output pattern even when the traffic on the network is small.

Chapter 5

Routing Algorithms

5.1 Types of Routing Algorithms

(1) Non-adaptive / Fixed / Static Routing

(2) Flooding

(3) Random Routing

(4) Flow-based Routing

(5) Adaptive / Dynamic Routing

5.2 Fixed Routing or Non-adaptive Routing

A route is selected for each source-destination pair of nodes in the network.

The routes are fixed; they may only change if there is a change in the topology of the network.

Algorithms used are:

(i) Dijkstra's Algorithm

(ii) Bellman-Ford Algorithm

5.2.1 Advantages

→ Simple

→ Works well in a reliable network with stable load

→ Same for virtual circuit and datagram

5.2.2 Disadvantages

→ Lack of flexibility

→ Does not react to failure or network congestion

5.3 Flooding

→ Requires no network information.

→ Every incoming packet to a node is sent out on every outgoing line except the one it arrived on.

5.3.1 Measures to Improve Flooding

→ A hop counter – a hop counter may be contained in the packet header, which is decremented at each hop, with the packet being discarded when the counter becomes zero.

→ Use sequence numbers to avoid sending them out a second time.

5.4 Selective Flooding

The routers do not send every incoming packet out on every line, only on those lines that go approximately in the direction of the destination.

5.5 Flow-based Routing

Uses both topology and load information for routing.

Flow between a pair of nodes is relatively stable and predictable.

For a given line, if the capacity and average flow is known, it is possible to compute mean packet delay using queuing theory.

From the mean delays of all the lines, it is easy to calculate the flow-weighted average to get the mean delay for the whole subnet.

Then the routing algorithm finds the path of minimum average delay.

5.6 Adaptive Routing

Routing decisions change as conditions on the network change.

5.6.1 Two Principal Conditions That Change

(i) **Failure:** When a node fails.

(ii) **Congestion**

For adaptive routing to be possible, network state information must be exchanged among the nodes.

5.6.2 Algorithms Used

(i) Distance Vector Routing

(ii) Link-state Routing

5.7 Distance Vector Routing

5.7.1 Key Characteristics

→ Knowledge about the entire network is not necessary – only knowledge to neighbours.

→ Information sharing at regular interval.

→ Each node maintains a routing table having one entry for each node with two other fields: preferred next node and a cost estimate of the distance, based on one of the metrics.

Network Id	Cost	Next Router

This algorithm is also called as the Distributed Bellman-Ford algorithm and the Ford-Fulkerson algorithm.

Here each node receives the routing tables from all of its neighbours and updates its own routing table accordingly.

The main issue with Distance Vector Routing algorithm is that it reacts rapidly to good news but leisurely to bad news (count-to-infinity problem).

5.8 Link-State Versus Distance Vector Routing

→ Link-state routing converges more quickly.

→ Link-state routing requires more CPU power and memory than distance vector routing algorithm.

→ Link-state protocols are generally more scalable than distance vector protocols.

5.9 Hierarchical Routing

The routers are divided into regions, with each router knowing all the details about how to route packets to destinations within its own region but knowing nothing about the internal structure of other regions.

For large networks, we usually create multilevel hierarchy.

Chapter 6

Congestion Control

6.1 Common Causes of Congestion

→ As packets arrive at a node, they are stored in an input buffer. If packets arrive too fast, an incoming packet may find that there is no available buffer space.

→ Even very large buffers cannot prevent congestion, because of delay, timeouts, and retransmissions.

→ Slow processors may lead to congestion.

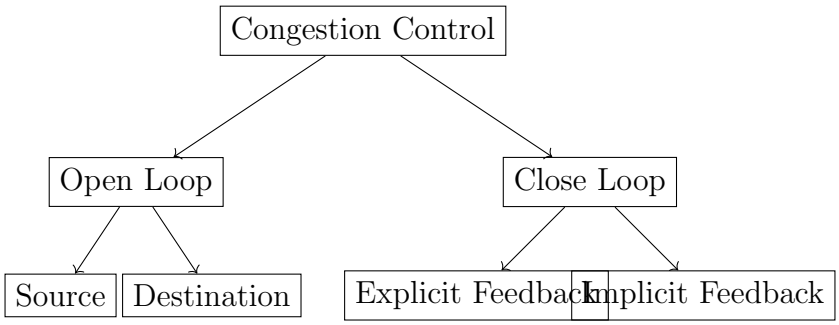
→ Low bandwidth line may also lead to congestion.

6.2 Congestion

When too many packets arrive in a part of the subnet, performance degrades. This situation is called congestion.

6.3 Congestion Control

Congestion Control refers to the mechanisms and techniques used to control congestion and keep the traffic below the capacity of the network.



6.4 Open Loop Congestion Control

The basic objective is to prevent congestion by adopting suitable policies for:

→ Flow Control

→ Acknowledgment

→ Retransmission (Timeout intervals)

→ Caching

→ Packet discard

→ Routing

6.4.1 Traffic Shaping

→ Leaky bucket algorithm

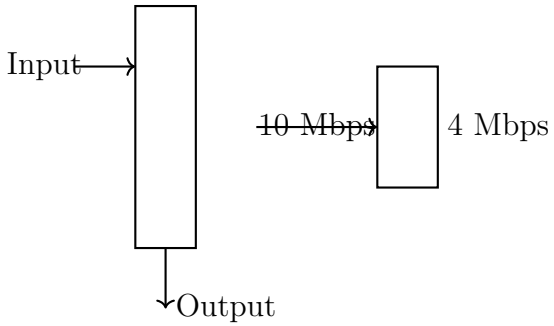
→ Token bucket algorithm

Traffic shaping is about regulating the average rate of data transmission.

6.5 The Leaky Bucket Algorithm

It shapes bursty traffic into fixed-rate traffic.

Packets are dropped when the bucket is full.



Limitation: Enforces rigid output pattern even when the traffic on the network is small.

6.6 Flow-based Routing

Uses both topology and load information for routing.

Flow between a pair of nodes is relatively stable and predictable.

For a given line, if the capacity and average flow is known, it is possible to compute mean packet delay using queuing theory.

From the mean delays of all the lines, it is easy to calculate the flow-weighted average to get the mean delay for the whole subnet.

Then the routing algorithm finds the path of minimum average delay.

6.7 Close Loop Congestion Control

6.7.1 Three Basic Steps

(1) Monitor the system for detection of congestion

(2) Pass the information to proper place for taking action to control congestion

(3) Adjust system parameters to get out of congestion

6.8 Congestion Control Versus Flow Control

Congestion Control	**Flow Control**
It is a global issue. It is a joint responsibility of the users and the network.	Flow control is local issue between a sender-receiver pair.
It occurs because of the combined behavior of the stations, switches, routing, etc.	Its job is to ensure that a fast sender does not overwhelm a slow receiver.
Its job is to ensure that the subnet is able to carry the offered load.	Its primary function is to ensure that a fast sender does not overwhelm a slow receiver.

6.9 Token Bucket Algorithm

In this algorithm, the leaky bucket holds tokens, generated by a clock at the rate of one token every ΔT sec.

For a packet to be transmitted, it must capture and destroy a token.

The token bucket algorithm allows idle hosts to save up permission to send large bursts later.

Chapter 7

Bridge

7.1 Network Devices Overview

→ Repeaters and hubs work in Physical Layer.

→ Bridge works in Data-Link Layer.

→ Router works in Network Layer.

→ Gateway works in Application Layer.

7.1.1 Repeaters

→ Connect different segments of a LAN.

→ It forwards every frame it receives.

→ It is a regenerator, not an amplifier.

7.1.2 Hub

→ Hub is a generic term but commonly refers to a multiport repeater.

→ It can be used to create multiple levels of hierarchy.

→ Easy to maintain and diagnose.

A bridge operates both in physical and data-link layers.
A bridge uses a table for filtering/routing.
It does not change the physical (MAC) addresses in a frame.
The difference between hub and bridge is that the hub forwards a frame from the input port to all other ports, while bridge forwards a frame only to the destination port.
It does not change the physical addresses in a frame.

7.2 Bridge Characteristics

7.2.1 Types of Bridges

(1) Transparent Bridges

(2) Source Routing Bridges

7.3 Transparent Bridge

7.3.1 Learning in Transparent Bridge

The stations are unaware of the presence of a transparent bridge.

Reconfiguration of the bridge is not necessary; it can be added/removed without being noticed.

7.3.2 Functions

(i) Forwarding of frames

(ii) Learning to create the forwarding table

7.3.3 Loop Problem in Transparent Bridges

Forwarding and learning processes work fine as long as there is no redundant bridge in the system.

On the other hand, redundancy is desirable from the viewpoint of reliability, so that the function of a failed bridge is taken over by a redundant bridge.

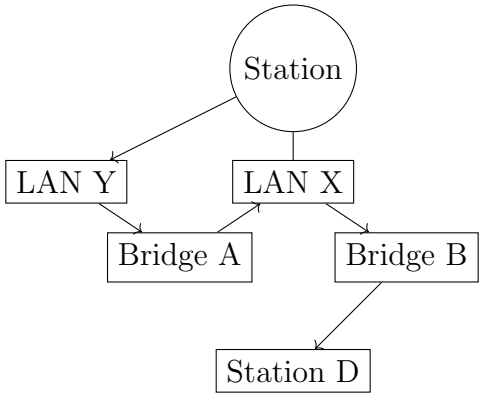
As redundancy creates a loop problem in the system, it is very undesirable.

7.3.4 Solution to Loop Problem (Spanning Tree Topology)

To solve the looping problem, the IEEE specification requires that the bridges use a special topology.

Such topology is known as spanning tree (a graph where there is no loop).

Without changing the physical topology, a logical topology is created that overlays the physical one.



If a frame is sent from station C, and if the destination of frame F is not present in the network, then the frame will start looping indefinitely between the bridges in the network.

7.4 Source Routing Bridge

7.4.1 Switch

→ Switching is a fast bridge.

→ Switch performs the forwarding operation using hardware, while bridges use software.

→ Ports are provided with buffer.

→ Each frame is forwarded after examining the address and forwarded to the proper port.

7.4.2 Three Forwarding Approaches

(i) Cut-through → No collision or error detection

(ii) Collision-free → No error detection

(iii) Fully buffered → Both collision and error detection

7.5 Router

Works in Network Layer.
It uses IP addresses.
A router has four basic components:

(i) Input Ports

(ii) Output Ports

(iii) Routing Processor

(iv) Switching Fabric

→ Input Port performs physical and data-link layer functions of the router.

→ Output Port performs the same functions as the input port, but in reverse order.

→ The routing processor performs the function of the network layer. The process involves the table lookup.

→ The switching fabric moves the packet from the input queue to the output queue by using specialized mechanisms.

Chapter 8

TCP/IP

8.1 TCP/IP

Acts as a glue to link different types of LAN and WAN to provide Internet; a single integrated network for seamless communication.

8.1.1 TCP/IP and OSI Model

APPLICATION	**APPLICATIONS**
PRESENTATION	
SESSION	
TRANSPORT	TCP / UDP
NETWORK	ICMP IP IGMP / ARP / RARP
DATA-LINK	ARPANET / LAN / SATNET / SONET
PHYSICAL	SLIP/PPP

(OSI) TCP/IP

8.2 Interworking Layers

(1) Addressing

(2) Packetizing

(3) Fragmentation and Reassembly

(4) Routing

(5) Flow Control

(6) Error detection / Error Control

(7) Congestion Control and QoS

(8) Security

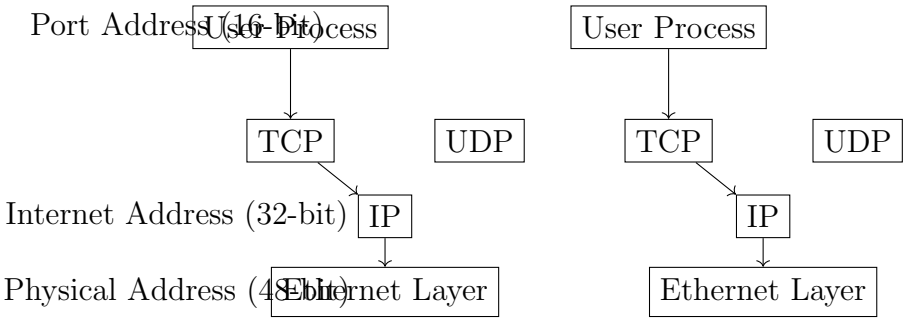
(9) Naming

8.2.1 IP

IP provides unreliable, connectionless best-effort datagram delivery service.

TCP provides reliable, efficient, and cost-effective end-to-end delivery of data.

8.3 Addresses in TCP/IP



8.4 Classful Addressing

Class A: 1.0.0.0 to 126.255.255.255

Class B: 128.0.0.0 to 191.255.255.255

Class C: 192.0.0.0 to 223.255.255.255

Class D: 224.0.0.0 to 239.255.255.255

Class E: 240.0.0.0 to 254.255.255.255

8.4.1 Default Masks for Classful Addressing

Class	Binary	Dotted-Decimal
A	11111111 00000000 00000000 00000000	255.0.0.0
B	11111111 11111111 00000000 00000000	255.255.0.0
C	11111111 11111111 11111111 00000000	255.255.255.0

8.4.2 Some Special IP Addresses

$\rightarrow$ 0.0.0.0 $\Rightarrow$ This host

$\rightarrow$ 255.255.255.255 $\Rightarrow$ Broadcast on this network (when netid is unknown)

first id **host id**

$00---00$ $\Rightarrow$ A host on this network

netid $111---111$

$\Rightarrow$ Broadcast on a distant network

$127 \cdot x \cdot y \cdot z$ $\Rightarrow$ Loopback

8.5 Subnetting

A subnet is a logically visible subdivision of an IP network.

The practice of dividing a network into two or more networks is called subnetting.

Subnetting increases the number of 1s in the mask.

8.5.1 Classless Addressing

In this scheme, there are no classes, but the addresses are still granted in blocks.

8.5.2 Restrictions on Classless Address Blocks

(i) The addresses in a block must be contiguous.

(ii) The number of addresses in a block must be a power of 2 (1, 2, 4, —).

(iii) The first address must be evenly divisible by the number of addresses.

8.5.3 Example

Calculate first address and number of Addresses for 205.16.37.39/28.

Solution: $n = 28$, $(32 - n) = 4$

$(39)_{10} = (00100111)_2$

First Address $\Rightarrow$

$$(00100000)_2 = (32)_{10}$$

$\Rightarrow$ 205.16.37.32

Last Address $\Rightarrow$

$$(00101111)_2 = (47)_{10}$$

$\Rightarrow$ 205.16.37.47

Number of Addresses:

$$\Rightarrow 2^4 = 16 \text{ Ans}$$

8.5.4 Network Address

The first address in a block is normally not assigned to any device; it is used as the network address.

It is used as a YP network address.

Hierarchy:

8.5.5 Two-level Hierarchy: No Subnetting

Left-most bits	**Address X.Y.Z.t/n**	
n	defines the network	(32-n) right-most
bits define the particular host		

8.5.6 Three-levels of Hierarchy: Subnetting

Here, the rest of the world sees the organization as a single entity, however, internally, there are several subnets.

All messages are sent to the router address that connects the organization to the rest of the Internet.

The router routes the message to the appropriate subnets.

8.5.7 More Level of Hierarchy

There can be any number of hierarchical levels in classless addressing.

8.5.8 Example of Three Levels of Hierarchy

An organization is given the block 0/26, which contains 64 addresses. The organization has three offices and needs to divide the addresses into three subnets of 32, 16, and 16 addresses.

Solution:

(1) Suppose the mask for the first subnet is h_1, then $n_1 = 32 - h_1 = 27$

(2) Suppose the mask for the second subnet is h_2, then $2^{32-h_2} = 16 \Rightarrow h_2 = 28$

(3) Similarly for third subnet: $h_2 = 28$

This means we have the masks 27, 28, 28 with the organization mask being 26.

Address range for subnet 1:

17.12.14.0 to 17.12.14.31/27

Address range for subnet 2:

17.12.14.32 to 17.12.14.47/28

Address range for subnet 3:

17.12.14.48 to 17.12.14.63/28

8.6 Masking

A block of addresses can be defined as $x.y.z.t/n$ in which $x.y.z.t$ defines one of the addresses and the /n defines the mask.

A mask is a 32-bit number made of contiguous 1s followed by contiguous 0s.

The mask helps to find the netId and hostId.

Default masks for classful addressing:

Class A: $x.y.z.t/n$

in which $x.y.z.t$ defines one of the addresses and the /n defines the mask.

8.7 Calculating Addresses from Mask

First Address

Set the rightmost $(32 - n)$ bits to 1s.

Last Address

Set the rightmost $(32 - n)$ bits to 1s.

Number of Addresses

$$2^{(32-n)}$$

Example

Calculate first address and number of addresses for 205.16.37.39/28.

Solution: $n = 28$, $(32 - n) = 4$

$$(39)_{10} = (00100111)_2$$

First Address:

$$(00100000)_2 = (32)_{10}$$
$$\Rightarrow 205.16.37.32$$

Last Address:

$$(00101111)_2 = (47)_{10}$$
$$\Rightarrow 205.16.37.47$$

Number of Addresses:

$$2^4 = 16 \text{ addresses}$$

Chapter 9

Application Layer

9.1 TELNET

TELNET (Terminal Network) is a general-purpose client-server application program for remote log-in.

- Communication takes place using NVT (Network Virtual Terminal) character set.
- Telnet uses only one TCP connection.
- The server uses the well-known port 23 and the client uses an ephemeral port.
- To distinguish data from control characters, each sequence of control characters is preceded by a special control character called Interpret as Control (IAC).
- NVT uses two sets of characters, each 8 bits:
  - For Data (highest-order bit is 0)
  - For Control (highest-order bit is 1)

9.2 MIME (Multipurpose Internet Mail Extension)

MIME is a supplementary protocol that allows non-ASCII data to be sent through e-mail.
MIME defines 5 headers:

(i) MIME-Version – Current is 1.1

(ii) Content-Type – Content-Type/Content-Subtype

(iii) Content-Transfer-Encoding

(iv) Content-Id

(v) Content-Description

9.3 SMTP (Simple Mail Transfer Protocol)

SMTP simply defines four commands and responses must be sent back and forth.

- Uses Port 25 of destination machine.
- Commands are sent from the client to the server.
- It defines 14 commands:
  - HELO – Sender's host name
  - MAIL FROM – Sender of the message
  - RCPT TO – Intended recipient of message
  - DATA – Body of the mail
- Responses are sent from the server to the client. It is a 3-digit code.
- The process of transferring a mail message occurs in three phases:
  - Connection Establishment
  - Mail Transfer
  - Connection Termination

9.4 FTP (File Transfer Protocol)

FTP uses TCP/IP. It establishes two connections between the hosts:

- For Data transfer (Port 20)
- For Control Information (Port 21)

The control connection uses very simple rules of communication, while the data connection uses more complex rules due to the variety of data types transferred.

9.4.1 Communication over Control Connection

FTP uses the same approach as SMTP to communicate across the control connection. It uses the 7-bit ASCII character set.

9.5 POP3 (Post Office Protocol, version 3)

The client POP3 software is installed on the recipient's computer; the server POP3 software is installed on the mail server.

Uses TCP port 110 on the server.

Two modes of operation:

- Delete Mode
- Keep Mode

9.6 IMAP4 (Internet Mail Access Protocol version 4)

Uses TCP port 143.

IMAP4 provides the following extra functions:

(i) Checking email header prior to downloading.

(ii) Searching in the contents of the e-mail prior to downloading.

(iii) Partially download e-mail.

(iv) Create, delete, or rename mailboxes on the mail server.

(v) Creating a hierarchy of mailboxes in a folder for email storage.

The control connection remains connected during the entire FTP session. The data connection is opened and then closed for each file transferred.

9.7 DNS (Domain Name System)

DNS Lookup:

9.7.1 Reverse DNS Lookup

Determination of domain name from IP address.

9.7.2 Forward DNS Lookup

Determination of IP address from domain name.

9.8 HTTP Protocol

HTTP (HyperText Transfer Protocol) is a stateless protocol.

HTTP methods are case-sensitive, i.e., GET is a legal method but get is not.

HTTP 1.1 supports persistence connection. It is also possible to pipeline requests, i.e., sending request-2 before the response to request-1 has arrived.

HTTP supports proxy servers. A proxy server is a computer that keeps copies of responses to recent requests.

Chapter 10

Network Security

10.1 Public Key Cryptography

The public key used for encryption is different from the private key used for decryption.

10.1.1 RSA

Inventors: Rivest, Shamir, and Adleman.

Based on number theory.

The private key is a pair of numbers (d, n) and the public key is also a pair of numbers (e, n).

Steps:

(i) Choose two large primes p and q.

(ii) Compute:

$$n = p \times q \quad \text{and} \quad z = (p-1) \times (q-1)$$

(iii) Choose a number d, relatively prime to z.

(iv) Find e, such that:

$$(e \cdot d) \bmod z = 1$$

(v) For encryption:

$$C = P^e (\text{mod} n)$$

(vi) For decryption:

$$P = C^d (\text{mod} n)$$

10.1.2 Diffie-Hellman Key Exchange Algorithm

Given:

- Prime Number $= p = 7$
- Base $= g = 3$

Alice has a secret integer a, Bob has a secret integer b.
Secret Key $= g^{ab} \bmod p$
Example: Modulus = 7, Primitive Root = 3, $a = 2$, $b = 5$

$$\text{Key} = 3^{(2\times5)} \bmod 7 = 3^{10} \bmod 7 = 4$$

At Alice:

- Primitive Root $= g$
- Prime Number = Modulus $= p$

Database Management Systems

Comprehensive Study Notes for GATE Computer Science

Ankur Gupta

www.ankurgupta.net

Note: These notes have been prepared primarily from the following books:

(1) **Fundamentals of Database Systems** by Ramez Elmasri, Shamkant B. Navathe

(2) **Database Systems Concepts** by Henry F. Korth, Abraham Silberschatz, S. Sudarshan

Please refer to the above books for more details.

Contents

1 Introduction to Database Systems **7**
1.1 Database Management Systems 7
1.2 Characteristics of the Database Approach 7

2 Database Architecture **8**
2.1 The Three-Schema Architecture 8
2.1.1 Schema Levels 8
2.2 Data Independence 9
2.2.1 Types of Data Independence 9

3 Database Design **10**
3.1 Database Design Techniques 10
3.1.1 Top-down Approach 10
3.1.2 Bottom-up Approach 10
3.2 Database Design Approach 10
3.2.1 Entity-Relationship Modeling 10

4 Conceptual Design **11**
4.1 Conceptual Schema 11
4.1.1 Entity 11
4.1.2 Attribute Types 11
4.1.3 Entity Types and Entity Sets 12
4.1.4 Key Attributes 12
4.1.5 Domains of Attributes 12
4.2 Relationships 12
4.2.1 Degree of a Relationship Type 12
4.3 Relationships versus Attributes 13

5 Constraints on Relationships **14**
5.1 Cardinality Ratios 14
5.1.1 Types of Cardinality Ratios 14
5.2 Participation Constraints 14
5.2.1 Types of Participation Constraints 14
5.2.2 Cardinality Ratio 14
5.2.3 Participation Constraint 15
5.3 Attributes of Relationship Types 15
5.3.1 Budget 15
5.4 Identifying Relationships 15
5.4.1 Example 15

5.5 Recursive Relationship 15

6 Enhanced Entity-Relationship (EER) 16
6.1 Subclasses and Inheritance 16
6.1.1 Superclass/Subclass Relationships 16
6.2 Specialization and Generalization 16
6.3 Predicate-Defined Subclasses 16
6.4 Disjoint and Overlapping Subclasses 17
6.5 Union Types or Categories 17
6.6 Specialization and Generalization Hierarchies and Lattices 17

7 Alternative Notations for ER Diagrams 18
7.1 Structural Constraint 18

8 Relational Model 19
8.1 Background 19
8.2 Relational Model Concepts 19
8.3 Some Definitions 19
8.4 Characteristics of Relations 19
8.4.1 Ordering of Tuples 19
8.4.2 Ordering of Attributes 20
8.4.3 Values of Tuples 20
8.5 Key 20

9 Relational Constraints 21
9.1 Domain Constraints 21
9.2 Key Constraints 21
9.3 Entity Integrity Constraint 21
9.4 Referential Integrity Constraint 21
9.5 Semantic Integrity Constraints 22

10 Relational Algebra and Relational Calculus 23
10.1 The SELECT Operation 23
10.2 Properties of SELECT Operation 23
10.3 The Project Operation 23
10.4 Properties of Project Operation 24
10.5 Composition and Assignment 24
10.6 Cartesian Join ($\times$) 24
10.7 Properties of Cartesian Join 24
10.8 Theta Join ($\bowtie_\theta$) 25
10.9 Properties of Theta Join 25
10.10 Properties of Cartesian Join 25
10.11 Compatibility 25

11 Other Relational Operators 26
11.1 Division Operator ($\div$) 26
11.2 Outer Join 26
11.3 Outer Union 26

12 Relational Calculus **27**
12.1 Tuple Calculus . . . 27
12.2 Informally . . . 27
12.3 Expressions and Formulas in Tuple Relational Calculus . . . 28
12.4 Formulas (wff) are Built from Atoms . . . 28
12.5 The Logical Implication Expression . . . 28
12.6 Transforming the Universal and Existential Quantifiers . . . 28
12.7 Safe Expressions . . . 29
12.8 Domain Calculus . . . 29

13 Relational Completeness **30**

14 Relational Database Design **31**
14.1 Informal Design Guidelines for Relation Schemas . . . 31
14.1.1 Semantics of the Relation Attributes . . . 31
14.1.2 Reducing the Redundant Values in Tuples . . . 31
14.1.3 Reducing the Null Values in Tuples . . . 31
14.1.4 Disallowing the Possibility of Generating Spurious Tuples . . . 31

15 Functional Dependencies **32**
15.1 Framework for Automatic Design and Optimization of Relational Schemas 32
15.2 Definition of Functional Dependency . . . 32
15.3 Trivial and Non-trivial Functional Dependency . . . 32
15.4 Inference Rules for Functional Dependencies . . . 32
15.4.1 Inference Rule 1: Reflexive Rule . . . 33
15.4.2 Inference Rule 2: Augmentation Rule . . . 33
15.4.3 Inference Rule 3: Transitive Rule . . . 33
15.4.4 Inference Rule 4: Decomposition Rule . . . 33
15.4.5 Inference Rule 5: Union Rule (Additive Rule) . . . 33
15.4.6 Inference Rule 6: Pseudotransitive Rule . . . 33
15.5 Specified Functional Dependency . . . 33
15.6 Closure of FDs . . . 34
15.7 Computing Closures . . . 34
15.8 Definition of X_F^+ . . . 34
15.9 Algorithm to Compute X^+ . . . 34
15.10 Covers and Equivalence of FDs . . . 34
15.11 Determining whether F covers E . . . 35
15.12 Algorithm to Find a Non-Redundant Cover . . . 35
15.13 Canonical Cover or Minimal Cover . . . 35
15.14 Full Functional Dependency . . . 35

16 Normalization of Relations **36**
16.1 First Normal Form (1NF) . . . 36
16.2 Second Normal Form (2NF) . . . 36
16.3 Third Normal Form (3NF) . . . 36
16.3.1 General Definition of 2NF . . . 37
16.3.2 General Definition of 3NF . . . 37
16.4 Boyce-Codd Normal Form (BCNF) . . . 37
16.4.1 Note . . . 37

16.4.2 General Definition of 2NF (Candidate Keys) 37
16.4.3 General Definition of 3NF (Candidate Keys) 38
16.4.4 Note 38
16.4.5 2-attribute Relations 38

17 Properties of Relational Decompositions 39
17.1 Attribute Preservation Property of a Decomposition 39
17.2 Dependency Preservation Property of a Decomposition 39
17.3 Lossless (Nonadditive) Join Property of a Decomposition 39
17.4 Algorithm for Testing for Lossless (Nonadditive) Join Property 40
17.5 Successive Lossless (Nonadditive) Join Decompositions 40
17.6 Testing Binary Decomposition for the Nonadditive Join Property 40

18 Algorithms for Creating a Relational Decomposition 41
18.1 Dependency-Preserving Decomposition into 3NF Schemas 41
18.2 Algorithm to Find a Key K of R 41
18.3 Lossless (Nonadditive) Join Decomposition into BCNF Schemas 41
18.4 We Use 3NF if We Require Efficiency 42
18.5 Normal Forms are are Insufficient on Their Own as Criteria for a Good Relational Database Schema Design 42

19 Nested Loop Join and Block Nested Join 43
19.1 Nested Loop Join 43
19.2 Block Nested Loop Join 43

20 Transaction Processing and Concurrency Control 44
20.1 Properties of Transaction (ACID) 44
20.2 Transaction State 44
20.3 Concurrent Execution of Transactions Allows 45
20.4 Schedules 45
20.5 Serial Schedule 45
20.6 Example 45
20.7 Conflict Serializability 45
20.8 View Serializability 45
20.9 Example 46
20.10Recoverable Schedules 46
20.11Cascadeless Schedules 46
20.12Testing for Conflict Serializability 46
20.13Example 47

21 Transaction Log 48
21.1 Techniques to Maintain Log Files 48
21.2 Occurrence of Failure in Deferred Update Scheme 48
21.3 Occurrence of Failure in Immediate Update Scheme 48

Chapter 1

Introduction to Database Systems

1.1 Database Management Systems

A database management system (DBMS) is a collection of programs that enables users to create and maintain a database.

1.2 Characteristics of the Database Approach

The main characteristics of the database approach are the following:

(1) Self-describing nature of a database system

(2) Insulation between programs and data, and data abstraction

(3) Support of multiple views of the data

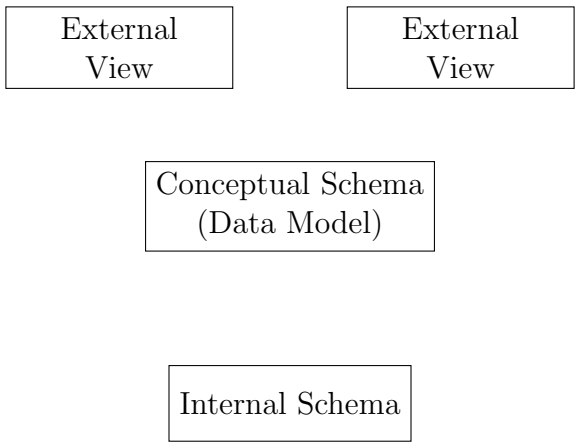
(4) Storing of data and multiuser transaction processing

Chapter 2

Database Architecture

2.1 The Three-Schema Architecture

The goal of the three-schema architecture is to separate the user applications and the physical database.



2.1.1 Schema Levels

(1) The internal level has an internal schema which describes the physical storage structure of the database

(2) The conceptual level has a conceptual schema which describes the structure of the whole database for a community of users

(3) The external or view level includes a number of external schemas or user views. Each external schema describes the part of the database that a particular user group is interested in and hides the rest of the database from that user group

2.2 Data Independence

Data independence can be defined as the capacity to change the schema at one level of a database system without having to change the schema at the next higher level.

2.2.1 Types of Data Independence

(1) **Logical data independence:** Capacity to change the conceptual schema without having to change external schemas or user programs

(2) **Physical data independence:** Capacity to change the internal schema without having to change the internal conceptual schema

Chapter 3

Database Design

3.1 Database Design Techniques

3.1.1 Top-down Approach

We start by defining the data set and then we go on defining data elements in the set.

Example: Entity-Relationship Modeling

3.1.2 Bottom-up Approach

We start by defining required attributes first and then group those attributes to form the entities.

Example: Normalization

3.2 Database Design Approach

3.2.1 Entity-Relationship Modeling

The process of creating subclasses out of a given entity type is called specialization.

The reverse process of taking two or more entity types and clubbing them under a common super class is called generalization.

Chapter 4

Conceptual Design

4.1 Conceptual Schema

4.1.1 Entity

An entity represents an object of the real world that has an independent existence.
An entity has attributes or properties that describes it.

4.1.2 Attribute Types

Simple versus Composite

A composite attribute is made of one or more simple or composite attributes.
Example: Name is made of first name and last name.

Single-valued versus Multivalued

A single-valued attribute is described by one value (e.g., age of a person). A multivalued attribute is described by many values (e.g., the color of a multicolored bird).

Stored versus Derived

An attribute whose value can be inferred from the value of other attributes is called a derived attribute (e.g., age can be derived from date of birth and current date). If this is not the case, then it is a stored attribute.

Null Values

Attributes that do not have any applicable values are said to have null values. Different from 0, unknown, and missing values.

Complex Attributes

Combination of composite and multivalued attributes. For example, a person can have more than one address and each address can have different parts.

4.1.3 Entity Types and Entity Sets

An **entity type** defines a collection of different entities having the same type.

Example: "Department" defined earlier is an entity type defining several departments having the same attribute structure.

Any specific collection of entities of a particular type is called an entity set.

An entity type describes the schema or intension for an entity set.

4.1.4 Key Attributes

An attribute or a set of attributes that uniquely identify entities in an entity set are called **key attributes**.

When a collection of attributes define the key, the set is called a **composite key**.

Composite keys should be minimal. No subset of a composite key should be a key itself.

Key attributes are shown underlined in the Entity-Relationship (ER) Diagram.

Key attributes should retain the unique definition property for all extensions (EER model) of the entity type.

There may be more than one key in a relation. All such keys are known as **candidate keys**. One of the candidate keys is chosen as the **primary key**; the others are known as **alternate keys**.

4.1.5 Domains of Attributes

The domain of an attribute is the set of all valid values that the attribute can have.

Example: The domain of the attribute "age" of an employee is the set of all integers between 18 and 65.

If the composite attribute can have values with domains $D_1, D_2, \ldots, D_n$ then the domain of the attribute is:

$$D_1 \times D_2 \times \ldots \times D_n$$

4.2 Relationships

Relationship types relate two (or more) entity types and defines a set of associations (a relationship set) among entities of this type.

Formally:

$$R \subseteq E_1 \times E_2 \times \ldots \times E_n$$

A relationship instance is any $r_i \in R_i$ where $r_i = (e_1, e_2, \ldots, e_n)$ such that:

$$e_1 \in E_1,\ e_2 \in E_2,\ \ldots,\ e_n \in E_n$$

4.2.1 Degree of a Relationship Type

The number of participating entity types in this relationship type.

Example: Degree of the relationship "department managed by Employee" is 2.

Unary, binary, and ternary relationships are special forms for relationships of degree 1, 2, and 3, respectively.

4.3 Relationships versus Attributes

Relationships as attributes are a concept used in data models called Functional Data Model.

In object databases, relationships are represented by object references.

In relational databases, foreign keys represent relationships as attributes.

Note: A one-to-one relationship between two entity sets does not imply that for an occurrence of an entity from one set at any time, there must be an occurrence of an entity in the other set.

Chapter 5

Constraints on Relationships

5.1 Cardinality Ratios

The cardinality ratio for a binary relationship specifies the maximum number of relationship instances that an entity can participate in.

5.1.1 Types of Cardinality Ratios

- One-to-one (1:1)
- One-to-many (1:N) or Many-to-one (N:1)
- Many-to-many (M:N)

5.2 Participation Constraints

Constraints limit the set of possible combinations of entities that can participate in the relationship type.

5.2.1 Types of Participation Constraints

(a) **Cardinality Ratios**

(b) **Participation Constraints**

(c) Structural constraints (both (a) and (b) taken together)

5.2.2 Cardinality Ratio

The cardinality ratio for a binary relationship specifies the maximum number of relationship instances that an entity can participate in.

Example: An employee works in exactly one department and each department should have a minimum 4 employees.

5.2.3 Participation Constraint

If every project has to be associated with a department, then the existence of an instance of Project entity type depends on the existence of an instance of the Handles relationship type. The entity type Project is said to **totally participate** in the relationship type Handles. The relationship is also said to introduce an **existence dependency** on project.

The participation constraint specifies whether the existence of an entity depends on its being related to another entity via the relationship type. This constraint specifies the minimum number of relationship instances that each entity can participate in, and is sometimes called the **minimum cardinality constraint**.

Types

(i) Total Participation

(ii) Partial Participation

5.3 Attributes of Relationship Types

5.3.1 Budget

Budget refers to the allocated budget for a specific department for handling a specific project.

If belongs neither to the department nor to the project in its entirety.

Budget refers to the allocated budget for a specific department for handling a specific project.

Example: 1:N or 1:1 relationship attributes can be moved to a corresponding participating entity type.

5.4 Identifying Relationships

The relationship type that relates a weak entity type to its owner is called the identifying relationship.

PAN

5.4.1 Example

Insurance record is a weak entity type with no key. It has no existence without its association with entity type Employee.

The relationship *insurance details* is an identifying relationship.

5.5 Recursive Relationship

A relationship type where the same entity type participates more than once in a different role is called a recursive relationship.

Chapter 6

Enhanced Entity-Relationship (EER)

6.1 Subclasses and Inheritance

An entity class B is said to be a subclass of another entity class A if it shares an "is-a" relationship with A.

Example: "Car is a vehicle" or "Manager is a Employee".

An entity of class B is said to be a specialization of entities of class A. Conversely, entities of class A are generalizations of class B entities.

An entity cannot exist in the database but merely belongs to a subclass. It has to belong to the super class as well.

6.1.1 Superclass/Subclass Relationships

Subclass entities undergo type inheritance of the super class. That is, each member of the subclass has the same attributes as the super class entities and participates in the same relationship types.

Example: A decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R has the lossless (nonadditive) join property with respect to the set of dependencies F on R if, for every relation state r of R that satisfies F, the following holds where $*$ is the Natural Join of all the relations in D:

$$*(\pi_{R_1}(r), \pi_{R_2}(r), \ldots, \pi_{R_m}(r)) = r$$

The word *loss* refers to loss of information, not to loss of tuples.

6.2 Specialization and Generalization

The process of creating subclasses out of a given entity type is called **specialization**.

The reverse process of taking two or more entity types and clubbing them under a common super class is called **generalization**.

6.3 Predicate-Defined Subclasses

Members can be defined based on the value of an attribute are called predicate-defined subclasses.

6.4 Disjoint and Overlapping Subclasses

Subclasses that are not disjoint are said to overlap.

This is represented by an "o" in the inheritance circle.

Disjoint or overlap may be either partial or total.

6.5 Union Types or Categories

An account holder can be either an Individual or a Family or an Institution, each of whom have their own attributes and keys.

A union type is also called a category.

6.6 Specialization and Generalization Hierarchies and Lattices

A subclass may have further subclasses specified on it, forming a hierarchy or a lattice of specializations.

Reasons for using the concept of specialization:

(1) Some attributes may apply to some but not all entities of the superclass. A subclass is defined in order to group the entities to which these attributes apply

(2) Some relationship types may be participated in only by entities that are members of the subclass

Chapter 7

Alternative Notations for ER Diagrams

7.1 Structural Constraint

This notation involves associating a pair of integers (min, max) with each participation of an entity type E in a relationship type R, where $0 \leq min \leq max$ and $max \geq 1$.

The numbers mean that for each entity e in E, e must participate in at least min and at most max relationship instances in R at any point of time.

Here each employee must work in exactly one department and each department should have minimum 4 employees.

Here $min = 0$ implies partial participation and $min > 0$ implies total participation.

Chapter 8

Relational Model

8.1 Background

The relational model was introduced by Ted Codd of IBM Research in 1970. It introduced the concept of mathematical relation as the underlying basis for the standard data model for most transactional databases today.

8.2 Relational Model Concepts

A relational database is a collection of relations.

A relation resembles a "table" or a "flat file" of records.

Each row or a relational instance is called a tuple, and each column is called an attribute of the relation.

8.3 Some Definitions

A data value is said to be atomic if it cannot be subdivided into smaller data values.

A domain is a set of atomic values.

A relational schema denoted by $R(A_1, A_2, \ldots, A_n)$ is made up of relation name R and a list of attributes $A_1, A_2, \ldots, A_n$. Each A_i is the name of a role played by some domain D in the schema R. The domain of A_i is denoted by $\text{dom}(A_i)$.

The cardinality of the relation is the number of tuples of the relation.

The degree or arity of the relation is the number of attributes of its relation schema. It is also called as Arity.

A relation r of a relation schema $R(A_1, A_2, \ldots, A_n)$ is a set of n-tuples of the form $[t_1, t_2, \ldots, t_n]$ where each $t_i \in \text{dom}(A_i)$.

8.4 Characteristics of Relations

8.4.1 Ordering of Tuples

Tuples in relation don't have any ordering. Mathematically, they do have order when being stored in files or disks.

8.4.2 Ordering of Attributes

Attributes within a tuple have to maintain order if they are stored as tuples. However, a relation need not necessarily be maintained as a set of ordered tuples as long as a mapping between attribute and value is maintained in each relation instance.

8.4.3 Values of Tuples

Each value that makes up a tuple is atomic and cannot be further subdivided. This is also called the First Normal Form assumption.

8.5 Key

A "key" K of a relation is a subset of the superkey $R \subseteq K$ such that removal of any attribute in K removes the superkey property for K. They are also called "minimal superkeys".

A relation schema may have more than one key. Each key is called a candidate key.

Usually one of the candidate keys is chosen to uniquely identify tuples in a relation, such a key is called primary key for a relation.

Chapter 9

Relational Constraints

9.1 Domain Constraints

Domain constraints specify that value of each attribute A must be atomic and a member of dom(A).

9.2 Key Constraints

Given a relational schema $R = (A_1), i = 1, \ldots, n$, a subset of attributes K in called a superkey if the values of attributes in K is distinct for every relation instance.

This is, for any two tuples t_i, t_j:

$$(t_i \neq t_j) \rightarrow (t_i[K] \neq t_j[K])$$

9.3 Entity Integrity Constraint

The primary key of a tuple may never be null.

9.4 Referential Integrity Constraint

Informally: A tuple that refers to another tuple from another relation should refer to an existing tuple.

Given a relation R_1, a set of attributes FK is said to be a foreign key of R_1 referencing another relation R_2 if the following rules hold:

(a) The attribute in FK have the same domain as the primary key of R_2

(b) For every tuple in R_1, the attribute in its foreign key FK either reference to a tuple in R_2 or is null

Example:

Emp_id	Name	Works-in	Reports-to

Department:

Dept-id	Location	Managed by	Num members

Foreign keys are diagrammatically depicted as arrows.

A foreign key can be from a relation to itself.

9.5 Semantic Integrity Constraints

Constraints on the useless of attributes. Example: the minimum age of an employee is 18 and maximum is 65.

Chapter 10

Relational Algebra and Relational Calculus

Relational algebra is a collection of operations to manipulate relations. The result of each of these relations is also a relation.

Relational algebra is a procedural language. It specifies the operations to be performed on existing relations to derive result relations.

The select operation is used to select (retrieve) a subset of the tuples from a relation that matches the selection condition.

10.1 The SELECT Operation

$$\sigma_{\text{Salary}>3000}(\text{Employee})$$

Selects all tuples of relation Employee which matches the criteria Employee.Salary > 3000.

10.2 Properties of SELECT Operation

The SELECT operation is unary. It operates on only one relation.

Selection conditions are applied to each tuple individually in the relation. Hence the condition cannot span more than one tuple.

The degree of the relation resulting from σ_{CR} is same as that of R.

$|\sigma_{CR}| \leq |R|$

The SELECT operation is commutative:

$$\sigma_{\sigma_1(\sigma_2(R))} \Leftrightarrow \sigma_{\sigma_2(\sigma_1(R))}$$

Hence select operation can be cascaded in any manner.

10.3 The Project Operation

The project operation selects specific columns from the specified relation.

$$\pi_{\text{Name, Salary}}(\sigma_{\text{Salary}>3000}(\text{Employee}))$$

Returns Name and Salary field of all records of Employee where salary is greater than 3000.

10.4 Properties of Project Operation

The number of tuples returned by Project is less than or equal to number of tuples in the specified relation.

When attribute list of PROJECT includes the super key, then the number of tuples returned is equal to the number of tuples in the specified relation.

PROJECT is not commutative:

$$\pi_{L_2}(\pi_{L_1}(R)) \neq \pi_{L_1}(\pi_{L_2}(R))$$

if L_2 is a substring of L_1. Else the operation is an incorrect operation.

10.5 Composition and Assignment

Since the input and output of the relational operators is a single relation, they can be composed in any fashion, with the output of one operation being the input of the other.

$$\pi_{\text{Name, Salary}}(\sigma_{\text{Salary}>3000}(\text{Employee}))$$

Returns Name and Salary field of all records of Employee where salary is greater than 3000.

Results of a query can also be assigned to a name to form a relation by that name.

Salary Statement $\leftarrow \pi_{\text{Name, Salary}}(\sigma_{\text{Salary}>3000}(\text{Employee}))$

Creates a new relation Salary Statement out of the specified query.

10.6 Cartesian Join ($\times$)

The join operator allows the combining of two relations to form a single new relation.

Join is basically the Cartesian product of the relations followed by a selection operation.

Consider the Student and Lab relation:

Student: $\bowtie_{\text{Student.Lab = Lab.Name}}$ **Lab**

Returns the same result as:

$$\sigma_{\text{Student}\times\text{Lab}}(\sigma_{\text{Student.Lab = Lab.Name}})$$

10.7 Properties of Cartesian Join

The Cartesian join where the only comparison operator used is the equality (=) sign are called equijoins.

A natural join denoted by $*$ in an equijoin where the attribute names at both ends of the equality sign are the same. In this case, the attribute is not repeated in the final result.

Consider the Student and the Lab example. Let the Student relation schema is modified as Student(RollNo, Name, LabName, Faculty, Dept).

The natural join Student $*$ Lab has a schema:

(RollNo, Name, LabName, Faculty, Dept)

10.8 Theta Join ($\bowtie_\theta$)

Theta join is used to combine related tuples from two or more relations (specified by the condition θ) to form a single tuple.

10.9 Properties of Theta Join

Theta join where the only comparison operator used is the equality (=) sign are called **equijoins**.

A natural join denoted by $*$ is an equijoin where the attribute names at both ends are of the equality sign are the same. $*$ In this case, the attribute is not repeated in the final result.

10.10 Properties of Cartesian Join

The cartesian join represents a Canonical Join of five or more relations. If relation R having $|R|$ tuples and relation S having $|S|$ tuples are joined using X, then $(R \times S) = |R|.|S|$

The union operator $\cup$ returns the set of all tuples that are present in either R or S or both. Duplicate tuples will be eliminated.

The intersection operator $\cap$ returns the set of all tuples that are present in both R and S.

The Set Difference Operator $R - S$ returns the set of all tuples in R but not in S.

10.11 Compatibility

Two relations $R(A_1, A_2, \ldots, A_k)$ and $S(B_1, B_2, \ldots, B_k)$ are compatible if they have the same number of attributes and for all $A_i, B_i, 1 \leq i \leq k$, $\text{dom}(A_i) = \text{dom}(B_i)$.

Chapter 11

Other Relational Operators

11.1 Division Operator ($\div$)

The division operator is used to denote the conditions where a given relation R is to be split based on whether its association with every tuple in an another relation S.

Let the set of attributes of R be Z and the set of attributes of S be X. Let $Y = Z - X$, that is the set of all attributes of R not in S.

The result of the operation:

$$T(Y) = R(Z) \div S(X)$$

As follows:

R	S
a_1	b_1
a_2	b_1
a_3	b_2

$$T = R - S$$

11.2 Outer Join

A normal join operation by requiring referential integrity. Every attribute in R that refers to S should refer to an existing tuple in S. If the referencing attribute of S is null, the tuple is not returned.

Outer join is used when all tuples are required even when referential integrity is not met. Left outer join includes tuples even when the referencing attribute is null, and Right outer join includes tuples even when the referenced attribute is null or referenced tuple does not exist.

An outer join which is both left and right is called a Full Outer Join.

11.3 Outer Union

Outer union computes the union of partially compatible relations. Two relations $R(X)$ and $S(Z)$ are said to be partially compatible if there exist $W \subseteq X$ and $Y \subseteq Z$ such that for every $A_i \in W$ and $B_i \in Y$, $\text{dom}(A_i) = \text{dom}(B_i)$.

Chapter 12

Relational Calculus

Relational calculus is a query system wherein queries are expressed as variables and formulas on these variables.

A calculus expression specifies what to be retrieved rather than how to retrieve it. Therefore, the relational calculus is considered to be a nonprocedural language.

It has been shown that any retrieval that can be specified in the basic relational algebra can also be specified in the relational calculus and vice versa. In other words, the expressive power of the two languages is identical.

Relational calculus can be categorized in two parts:

(1) Tuple Calculus

(2) Domain Calculus

12.1 Tuple Calculus

Queries in tuple calculus are expressed by a tuple calculus expression. A tuple calculus expression is of the form:

$$\{t|\text{COND}(t)\}$$

where t is a tuple variable and COND(t) is a conditional expression involving t. The result of such a query is the set of all tuples t that satisfy COND(t).

12.2 Informally

Informally, we need to specify the following information in a tuple calculus expression:

(1) For each tuple variable t, the range relation R of t. This value is specified by a condition of the form $R(t)$.

(2) A condition to select a particular combination of tuples

(3) A set of attributes to be retrieved, the requested attributes.

Example: t.attr-name

12.3 Expressions and Formulas in Tuple Relational Calculus

A general expression of the tuple relational calculus is of the form:

$$\{t_1.A_1, t_2.A_2, \ldots, t_n.A_n | \text{COND}(t_1, t_2, \ldots, t_n, t_{n+1}, \ldots, t_m)\}$$

where $t_1, t_2, \ldots, t_n, t_{n+1}, \ldots, t_m$ are tuple variables, each A_i is an attribute of the relation on which t_i ranges, and COND is a condition or formula of the tuple relational calculus. It is also called a well formed formula (wff).

12.4 Formulas (wff) are Built from Atoms

(1) Every atom is a formula

(2) If f and g are formulas then: If $f \vee g$, $f \wedge g$, $f \rightarrow g$ are also formulas

(3) If $f(x)$ is a formula where x is in free, then $\exists x(f(x))$ and $\forall x(f(x))$ are also formulas. However x is now bound.

12.5 The Logical Implication Expression

The logical implication expression:

$$f \rightarrow g$$

means if f then g is equivalent to $\neg f \vee g$.

If the formula $\exists x(f(x))$ is TRUE if the formula $f(x)$ evaluates to true for some (at least one) tuple assigned to free occurrences of x in $f(x)$, otherwise it is false.

If the formula $\forall x(f(x))$ is TRUE if the formula $f(x)$ evaluates to true for every tuple assigned to free occurrences of x in $f(x)$, otherwise it is false.

12.6 Transforming the Universal and Existential Quantifiers

(1) $(\forall x)(P(x)) \equiv \neg(\exists x)(\neg P(x))$

(2) $(\forall x)(P(x)) \equiv (\exists x)(\neg P(x))$

(3) $(\forall x)(P(x)) \equiv \neg(\exists x)(\neg(P(x)))$

(4) $(\exists x)(P(x)) \equiv \neg(\forall x)(\neg P(x))$

(5) $(\forall x)(P(x) \wedge Q(x)) \equiv (\exists x)(\neg P(x) \vee \neg(Q(x)))$

(6) $(\forall x)(P(x) \vee Q(x)) \equiv (\exists x)(\neg P(x) \wedge \neg(Q(x)))$

(7) $(\exists x)(P(x) \wedge Q(x)) \equiv \neg(\forall x)(\neg(P(x)) \vee \neg(Q(x)))$

(8) $(\exists x)(P(x) \vee Q(x)) \equiv \neg(\forall x)(\neg P(x) \wedge \neg(Q(x)))$

(9) $(\forall x)(P(x)) \rightarrow (\exists x)(P(x))$

(10) $\neg(\exists x)(P(x)) \rightarrow \neg(\forall x)(P(x))$

12.7 Safe Expressions

A safe expression in relational calculus is one that is guaranteed to yield a finite number of tuples as its result, otherwise the expression is called unsafe.

An expression is said to be safe if all values in its result are from the domain of the expression E: $\{t|\text{NOT}(\text{Employee}(t))\}$ is unsafe.

12.8 Domain Calculus

Domain calculus differs from tuple calculus in the type of variables used in formulas. Rather than having variables range over tuples, the variables range over single values from domains of attributes.

To form a query result, we must have n of these domain variables — one for each attribute. An expression of the domain relational calculus is of the form:

$$\{x_1, x_2, \ldots, x_n|\text{COND}(x_1, x_2, \ldots, x_n, x_{n+1}, \ldots, x_m)\}$$

where $x_1, x_2, \ldots, x_n, x_{n+1}, \ldots, x_m$ are domain variables that range over domains and COND is a condition or formula of the domain relational calculus.

Note:

SQL is based on tuple relational calculus or while QBE is no related to domain relational calculus. Both of these languages were developed by IBM Research.

SQL - Structured Query Language

QBE - Query by example.

Chapter 13

Relational Completeness

Relational completeness means that a query language can express all the queries that can be expressed in the relational algebra. It does not mean that the language can express any desired query.

Chapter 14

Relational Database Design

14.1 Informal Design Guidelines for Relation Schemas

14.1.1 Semantics of the Relation Attributes

Design a relation schema so that it is easy to explain its meaning. Do not combine attributes from multiple entity types and relationship types into a single relation.

14.1.2 Reducing the Redundant Values in Tuples

Design the base relation schemas so that no insertion, deletion, or modification anomalies are present in the relations. If any anomalies are present, note them clearly and make sure that the programs that update the database will operate correctly.

14.1.3 Reducing the Null Values in Tuples

As far as possible, avoid placing attributes in a base relation whose values may frequently be null. If nulls are unavoidable, make sure that they apply in exceptional cases only and do not apply to a majority of tuples in the relation.

14.1.4 Disallowing the Possibility of Generating Spurious Tuples

Design relation schemas so that they can be joined together with equality conditions on attributes that are either primary key or foreign key in a way that guarantees that no spurious tuples are generated. Avoid relations that contain matching attributes that are not (foreign key, primary key) combinations because joining on such attributes may produce spurious tuples.

Chapter 15

Functional Dependencies

15.1 Framework for Automatic Design and Optimization of Relational Schemas

- Generalization over the notion of keys
- Crucial in obtaining correct normalized schemas

15.2 Definition of Functional Dependency

In any relation R, if there exists a set of attributes $A_1, A_2, \ldots, A_n$ and on attribute B such that if any two tuples have the same value for $A_1, A_2, \ldots, A_n$, then they also have the same value for B.

A functional dependency (FD) of the above form is written as:

$$A_1, A_2, \ldots, A_n \rightarrow B$$

Functional dependencies define properties of the schema and not of any particular legal relation state (extension) of R. Hence an FD cannot be inferred automatically from a given relation extension r, but must be defined explicitly by someone who knows the semantics of the attributes of R.

15.3 Trivial and Non-trivial Functional Dependency

Note that in the Movies relation:

title, *year* $\rightarrow$ *title*

An FD where the right-hand side is contained within the left-hand side is called a trivial FD.

Example: $(X \supseteq Y)$

If there is at least one element on R.H.S that is not contained in L.H.S, it is called a non-trivial FD.

15.4 Inference Rules for Functional Dependencies

FDs which are specified are called stated FDs, and FDs which are derived are called inferred FDs.

We use the notation $F \models X \to Y$ to denote that the functional dependency $X \to Y$ is inferred from the set of functional dependencies F.

15.4.1 Inference Rule 1: Reflexive Rule

If $X \supseteq Y$, then $X \to Y$

15.4.2 Inference Rule 2: Augmentation Rule

$\{X \to Y\} \vdash XZ \to YZ$

15.4.3 Inference Rule 3: Transitive Rule

$\{X \to Y, Y \to Z\} \vdash X \to Z$

Inference Rules (IR1), (IR2), and (IR3) are sound and complete. These are called Armstrong's Axioms (AA) or (free from defect) Soundness.

Every new FD $X \to Y$ derived by FD using Armstrong's Axioms from a given set of FDs F using Armstrong's Axioms is such that: $F \models \{X \to Y\}$

Completeness:

Any FD $X \to Y$ logically implied by F (i.e., $F \vdash \{X \to Y\}$) can be derived from F using Armstrong's Axioms.

15.4.4 Inference Rule 4: Decomposition Rule

$\{X \to YZ\} \vdash X \to Y, X \to Z$

15.4.5 Inference Rule 5: Union Rule (Additive Rule)

$\{X \to Y, X \to Z\} \vdash X \to YZ$

15.4.6 Inference Rule 6: Pseudotransitive Rule

$\{X \to Y, WY \to Z\} \vdash WX \to Z$

Proof:

$X \to Y$	(given)
$WY \to Z$	(given)
$WX \to WY$	(using IR2 on 1)
$WX \to Z$	(using IR6 on 3 and 2)

Inference Rules IR1 through IR3 are known as Armstrong's Inference Rules, or Armstrong's Axioms.

15.5 Specified Functional Dependency

Certain FDs can be specified without referring to a specific relation, but as a property of these attributes.

15.6 Closure of FDs

The set of all dependencies that include F as well as all dependencies that can be inferred from F is called the closure of F. It is denoted by F^+.

15.7 Computing Closures

The size of F^+ can sometimes be exponential in the size of F.

For instance:

$$F = \{A \to B, A \to B_2, \ldots, A \to B_n\}$$

$$F^+ = \{A \to X\}, \text{ where } X \in \{B_1, B_2, \ldots, B_n\}$$

Thus $|F^+| = 2^n$

Since the method of computing F^+ is computationally expensive, however there is an alternative method to check $F \vdash X \to Y$ without generating F^+.

Checking if $X \to Y \in F^+$ can be done by checking if $Y \subseteq X^+$, where X^+ is $*$ the closure of X under F.

15.8 Definition of X_F^+

The closure of X w.r.t. F, written as X_F^+, is the set of all attributes that are eventually defined by $* X$.

Let:

$$A \to B, B \to C, D \to E$$

Then:

$$A^+ = A \cup B \cup C \cup D \cup E$$

15.9 Algorithm to Compute X^+

Algorithm to compute X^+ w.r.t F for each FD $X \to Y$ in F:

(1) Initially $X^+ = X$

(2) For every $X' \subseteq X$, if there exists an FD of the form $X' \to Y$ and $Y \not\subseteq X$, then $X^+ = X' \cup Y$

(3) Repeat step 2 until no more attributes can be added to closure X^+

15.10 Covers and Equivalence of FDs

A set of functional dependencies F is said to cover another set of functional dependencies E if every FD in E is also in F^+, that is, if every dependency in E can be inferred from F; alternatively, we can say that E is covered by F^+.

Equivalence:

Two sets of functional dependencies E and F are equivalent if $E^+ = F^+$. Hence, equivalence means that every FD in E can be inferred from F, and every FD in F can be inferred from E; that is, E is equivalent to F if both the conditions E covers F and F covers E hold.

15.11 Determining whether F covers E

A functional dependency is a property of the relation schema (intension) R, not of a particular legal relation state (extension) of R. Hence an FD cannot be automatically inferred from a given relation extension r, but must be defined explicitly by someone who knows the semantics of the attributes of R.

15.12 Algorithm to Find a Non-Redundant Cover

(1) Initially $E = F$

(2) Replace each functional dependency $X \rightarrow \{A_1, A_2, \ldots, A_n\}$ in F by the n functional dependencies $X \rightarrow A_1, X \rightarrow A_2, \ldots, X \rightarrow A_n$

(3) For each functional dependency $X \rightarrow A$ in F, for each attribute B that is an element of X, if $\{F - \{X \rightarrow A\}\} \cup \{(X - \{B\}) \rightarrow A\}$ is equivalent to F, then replace $X \rightarrow A$ with $(X - \{B\}) \rightarrow A$ in F

(4) For each remaining FDs $X \rightarrow A$ in F, if $\{F - \{X \rightarrow A\}\}$ is equivalent to F, then remove $X \rightarrow A$ from F

(5) $E = F$

15.13 Canonical Cover or Minimal Cover

A set of FDs F is called a canonical cover or minimal cover if it satisfies the following conditions:

(1) Every dependency in F has a single attribute for its right-hand side

(2) We cannot replace any dependency $X \rightarrow A$ in F with a dependency $Y \rightarrow A$ where $Y \subset X$ and still have a set of dependencies that is equivalent to F

(3) We cannot remove any dependency from F and still have a set of dependencies that is equivalent to F

If F is a set of FDs and if F is a nonredundant cover of F, then it is not true that E has the minimum number of FDs.

15.14 Full Functional Dependency

Given a relational schema R and an FD $X \rightarrow Y$, if Y is fully functionally dependent on X if there is no Z, where $Z \subseteq X$ such that both $X \rightarrow Z$ and $Z \rightarrow Y$ hold.

Chapter 16

Normalization of Relations

Normalization of data can be looked upon as a process of analyzing the given relation schemas based on their FDs and primary keys to achieve the desirable properties of:

(1) Minimizing redundancy

(2) Minimizing the insertion, deletion, and update anomalies

The normal form of a relation refers to the highest normal form condition that it meets.

16.1 First Normal Form (1NF)

A relation schema R is said to be in First Normal Form (1NF) if the values in the domain of each attribute of the relation are atomic.

In other words, only one value is associated with each attribute and the value is not a set of values or a list of values.

16.2 Second Normal Form (2NF)

Second Normal Form (2NF) is based on the concept of full functional dependency.

A functional dependency $X \rightarrow Y$ in a relation schema R is a transitive dependency if there is a set of attributes Z that is neither a candidate key nor a subset of any key of R, and both $X \rightarrow Z$ and $Z \rightarrow Y$ hold.

A relation schema R is in 2NF if every non-prime attribute A in R is fully functionally dependent on the primary key of R.

16.3 Third Normal Form (3NF)

3NF is based on the concept of transitive dependency.

A functional dependency $X \rightarrow Y$ in a relation schema R is a transitive dependency if there is a set of attributes Z in R is neither a candidate key nor a subset of any key of R, and both $X \rightarrow Z$ and $Z \rightarrow Y$ hold.

16.3.1 General Definition of 2NF

A relation schema R is in 2NF if every non-prime attribute A in R is not partially dependent on any key of R.

16.3.2 General Definition of 3NF

A relation schema R is in third normal form (3NF) if, whenever a non-trivial functional dependency $X \rightarrow A$ holds in R, either:

(a) X is a superkey of R, or

(b) A is a prime attribute of R

16.4 Boyce-Codd Normal Form (BCNF)

According to Codd's original definition, a relation schema R is in 3NF if it satisfies 2NF and no nonprime attribute of R is transitively dependent on the primary key.

Emp-Dept:

ENAME	ENO	DOB	ADDRESS	DNUMBER	DNAME	DMGRENO

3NF Normalization:

ENO	PNUMBER	HOURS

FD2:

PNUMBER	PNAME	PLOCATION

FD3:

ENO	ENAME

Third Normal Form: (Considers only primary keys)

3NF is based on the concept of transitive dependency.

A functional dependency $X \rightarrow Y$ in a relation schema R is a transitive dependency if there is a set of attributes Z that is neither a candidate key nor a subset of any key of R, and both $X \rightarrow Z$ and $Z \rightarrow Y$ hold.

16.4.1 Note

In general definitions of 2NF and 3NF, partial and full functional dependencies and transitive dependencies are considered with respect to all candidate keys of a relation.

16.4.2 General Definition of 2NF (Candidate Keys)

A relation schema R is in 2NF if every nonprime attribute A in R is fully functionally dependent on every key of R.

16.4.3 General Definition of 3NF (Candidate Keys)

A relation schema R is in third normal form (3NF) if, whenever a nontrivial functional dependency $X \rightarrow A$ holds in R, either (a) X is a superkey of R, or (b) A is a prime attribute of R.

16.4.4 Note

In the definition of normal forms, an attribute that is part of any candidate key will be considered as prime attribute.

16.4.5 2-attribute Relations

Any 2-attribute relation of the form $R(A, B)$ is always in BCNF.

Proof: Consider the following cases:

(1) There are no FDs between A and B, in which case only trivial FDs exist and R is in BCNF

(2) $A \rightarrow B$ but there is no FD of the form $B \rightarrow A$. In this case, A is a key and R is in BCNF

(3) $B \rightarrow A$ and $B \rightarrow A$. Both A and B are keys, thus does not violate the BCNF condition

(4) $A \rightarrow B$ and $B \rightarrow A$. Both A and B are keys, thus does not violate the BCNF condition

Chapter 17

Properties of Relational Decompositions

17.1 Attribute Preservation Property of a Decomposition

Let R be a relation schema and let $D = \{R_1, R_2, \ldots, R_m\}$ be the decomposition of R into $R_1, R_2, \ldots, R_m$. Then:

We must make sure that each attribute in R will appear in at least one relation schema R_i in the decomposition so that no attributes are lost.

Formally, we have:

$$\bigcup_{i=1}^{m} R_i = R$$

This is called the attribute preservation property of decomposition.

17.2 Dependency Preservation Property of a Decomposition

If each functional dependency $X \rightarrow Y$ specified in F either appears directly in one of the relation schemas R_i in the decomposition D or could be inferred from the dependencies that appear in some R_i.

A decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R is dependency preserving with respect to F if the union of the projections of F on each R_i in D is equivalent to F; that is:

$$((\pi_{R_1}(F)) \cup (\pi_{R_2}(F)) \cup \ldots \cup (\pi_{R_m}(F)))^+ = F^+$$

17.3 Lossless (Nonadditive) Join Property of a Decomposition

If F is a set of FDs on R, all legal relation states r of R that satisfy F must also satisfy the lossless join property of decomposition. If a decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R has the lossless (nonadditive) join property with respect to the set of dependencies F

on R if, for every relation state r of R that satisfies F, the following holds where $*$ is the Natural Join of all the relations in D:

$$*(\pi_{R_1}(r), \pi_{R_2}(r), \ldots, \pi_{R_m}(r)) = r$$

The word *loss* refers to loss of information, not to loss of tuples.

17.4 Algorithm for Testing for Lossless (Nonadditive) Join Property

Input: A universal relation R, a decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R, and a set F of functional dependencies.

(1) Create an initial matrix S with one row i for each relation R_i in D and one column j for each attribute A_j in R

(2) Set $S(i, j) = a_j$ for all matrix entries

(3) For each row i representing relation schema R_i, if relation R_i includes attribute A_j, then set $S(i, j) = a_j$

(4) Repeat the loop until a complete loop execution results in no change to S: for each FD $X \rightarrow Y$ in F, for each column j representing attribute A_j, if make the symbols in each column that correspond to an attribute in Y be the same in all three rows or follows

If relation R_i includes attribute A_j, then set $S(i, j) = a_j$.

If a row is not made up entirely of "a" symbols, then the decomposition has the lossless join property, otherwise it does not.

17.5 Successive Lossless (Nonadditive) Join Decompositions

If a decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R has the lossless (nonadditive) join property with respect to a set of F-s F on R, and if a decomposition $D_i = \{Q_1, Q_2, \ldots, Q_k\}$ of R_i has the nonadditive join property w.r.t. the projection of F on R_i, then the decomposition $D_2 = \{R_1, R_2, \ldots, R_{i-1}, Q_1, Q_2, \ldots, Q_k, R_{i+1}, \ldots, R_m\}$ of R has the nonadditive join property w.r.t. F.

17.6 Testing Binary Decomposition for the Nonadditive Join Property

A decomposition $D = \{R_1, R_2\}$ of R has the lossless join property w.r.t. a set of FDs F on R if and only if either:

The FD $(R_1 \cap R_2) \rightarrow (R_1 - R_2)$ is in F^+, or

The FD $(R_1 \cap R_2) \rightarrow (R_2 - R_1)$ is in F^+

Chapter 18

Algorithms for Creating a Relational Decomposition

18.1 Dependency-Preserving Decomposition into 3NF Schemas

Input: A universal relation R and a set of FDs F on the attributes of R.

(1) Find a minimal cover G for F

(2) For each left-hand side X of an FD that appears in G, create a relation schema in D with attributes $\{X \cup \{A_1\} \cup \{A_2\} \cup \ldots \cup \{A_k\}\}$, where $X \rightarrow A_1, X \rightarrow A_2, \ldots, X \rightarrow A_k$ are the only dependencies in G with X as left-hand side (X is the key of this relation)

(3) If none of the relation schemas in D contains a key of R, then create one more relation schema in D that contains attributes that form a key of R

18.2 Algorithm to Find a Key K of R

Input: A relation R and a set of FDs F.

(1) Set $K = R$

(2) For each attribute A in K, compute $(K - A)^+$ w.r.t. F. If $(K - A)^+$ contains all attributes in R, then set $K = K - \{A\}$

18.3 Lossless (Nonadditive) Join Decomposition into BCNF Schemas

Input: A universal relation R and a set of FDs F.

(1) Set $D = \{R\}$

(2) While there is a relation schema Q in D that is not in BCNF, do:

(i) Choose a relation schema Q in D that is not in BCNF

(ii) Find an FD $X \rightarrow Y$ in Q that violates BCNF

(iii) Replace Q in D by two relation schemas $(Q - Y)$ and $(X \cup Y)$

18.4 We Use 3NF if We Require Efficiency

We use 2NF if we have more data retrieval operations (SELECT), then insert, update, or delete operations.

18.5 Normal Forms are are Insufficient on Their Own as Criteria for a Good Relational Database Schema Design

The relation must collectively satisfy these two additional properties:

(i) Dependency Preservation Property

(ii) Lossless or nonadditive join property

Chapter 19

Nested Loop Join and Block Nested Join

19.1 Nested Loop Join

For each tuple r in R do
For each tuple s in S do
If r and s satisfy the join condition, then output the tuple $\langle r, s \rangle$
This algorithm involves $n_r \times b_s + b_r$ block transfers, where:
n_r = No. of records in R
b_s = No. of blocks used by S

19.2 Block Nested Loop Join

For each block b in R do
For each tuple s in S do
For each tuple r in block b do
If r and s satisfy the join condition, then output the tuple $\langle r, s \rangle$
This algorithm involves $b_r \times b_s + b_r$ block transfers.

Chapter 20

Transaction Processing and Concurrency Control

20.1 Properties of Transaction (ACID)

(1) **Atomicity:** Transaction Management Component

(2) **Consistency**

(3) **Isolation:** Concurrency Control Component

(4) **Durability:** Recovery Management Component

20.2 Transaction State

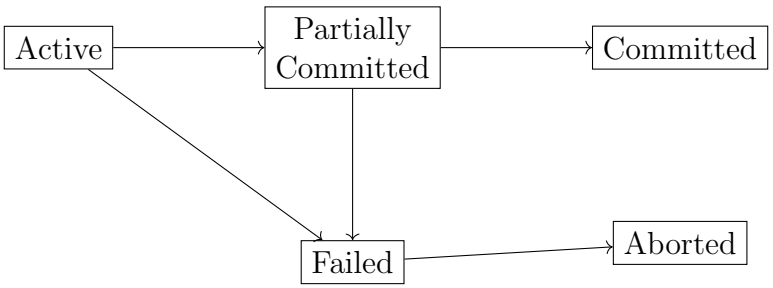
(1) Active: While executing

(2) Partially Committed: After the final statement has been executed

(3) Failed: When normal operation can no longer proceed

(4) Aborted: After the transaction has been rolled back and database has been restored to its state prior to start of the transaction

(5) Committed: After successful completion



Terminated means if the transaction has either committed or aborted.

20.3 Concurrent Execution of Transactions Allows

(1) Improved throughput and resource utilization

(2) Reduced waiting time

20.4 Schedules

Schedules represent the order in which instructions are executed in the system.

20.5 Serial Schedule

It consists of a sequence of instructions from various transactions, where the instructions belonging to one single transaction appear together in that schedule.

20.6 Example

T_1	T_2
Read (A)	
A = A - 50	
Write (A)	
Read (B)	
B = B + 50	
Write (B)	
	Read (A)
	Write (A)
	Read (B)
	Write (B)
	Read (C)
	Write (C)

20.7 Conflict Serializability

Two instruction I_i and I_j conflict if they are operations by different transactions on the same data item and at least one of these instructions is a write operation.

If a schedule S can be transformed into a schedule S' by a series of swaps of nonconflicting instructions, we say that S and S' are conflict equivalent.

A schedule S is conflict serializable if it is conflict equivalent to a serial schedule.

20.8 View Serializability

The schedules S and S' are said to be view equivalent if three conditions are met:

(1) For each data item Q, if transaction T_i reads the initial value of Q in schedule S, then transaction T_i must, in schedule S', also read the initial value of Q

(2) For each data item Q, if transaction T_i executes read (Q) in schedule S, and if that value was produced by a write (Q) operation executed by transaction T_j, then the read (Q) operation of transaction T_i must, in schedule S', also read the value of Q that was produced by the same write (Q) operation of transaction T_j

(3) For each data item Q, the transaction (if any) that performs the final write (Q) operation in schedule S must perform the final write (Q) operation in schedule S'

A schedule S is view serializable if it is view equivalent to a serial schedule.

20.9 Example

T_3	T_4	T_6
Read (Q)		
	Write (Q)	
Write (Q)		
		Write (Q)

Every conflict-serializable schedule is also view-serializable, but there are view-serializable schedules that are not conflict serializable.

20.10 Recoverable Schedules

A recoverable schedule is one where, for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i, the commit operation of T_i appears before the commit operation of T_j.

20.11 Cascadeless Schedules

The phenomenon in which a single transaction failure leads to a series of transaction rollbacks is called cascading rollback.

A cascadeless schedule is one where for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i, the commit operation of T_i appears before the read operation of T_j.

Every cascadeless schedule is also recoverable.

20.12 Testing for Conflict Serializability

Construct a directed graph called a precedence graph from schedule S. This graph consists of a pair $G = (V, E)$, where V is a set of vertices and E is a set of edges.

The set of vertices consists of all the transactions participating in the schedule.

The set of edges consists of all edges $T_i \rightarrow T_j$ for which one of three conditions hold:

(1) T_i executes write (Q) before T_j executes read (Q)

(2) T_i executes read (Q) before T_j executes write (Q)

(3) T_i executes write (Q) before T_j executes write (Q)

If an edge $T_i \rightarrow T_j$ exists in the precedence graph, then in any serial schedule S' equivalent to S, T_i must appear before T_j.

If the precedence graph for S has a cycle, then schedule S is not conflict serializable.

20.13 Example

T_1	T_2
Read (A)	
Write (A)	
	Read (A)
	Write (A)
	Read (C)
	Write (C)
Read (B)	
Write (B)	

Recovery:
T_1: No changes T_2: Redo C, T_3: Redo ABC
T_4: Undo AB T_5: Discard AB

Chapter 21

Transaction Log

21.1 Techniques to Maintain Log Files

(1) Deferred Update

(2) Immediate Update

21.2 Occurrence of Failure in Deferred Update Scheme

$\langle T_0$ Start$\rangle$	$\langle T_0$ Start$\rangle$	$\langle T_0$ Start$\rangle$
$\langle T_0, A, 950\rangle$	$\langle T_0, A, 950\rangle$	$\langle T_0, A, 950\rangle$
$\langle T_0, B, 2050\rangle$	$\langle T_0, B, 2050\rangle$	$\langle T_0, B, 2050\rangle$
CRASH	$\langle T_0$ Commit$\rangle$	$\langle T_0$ Commit$\rangle$
	$\langle T_1$ Start$\rangle$	$\langle T_1$ Start$\rangle$
Recovery:	$\langle T_1, C, 600\rangle$	$\langle T_1, C, 600\rangle$
No Action	CRASH	$\langle T_1$ Commit$\rangle$
		CRASH
	Recovery:	Recovery:
	Redo CT_0	Redo CT_0
		Redo CT_1

21.3 Occurrence of Failure in Immediate Update Scheme

Checkpoints:

$\langle T_0$ Start$\rangle$
$\langle T_0, A, 950\rangle$
$\langle T_0, B, 2050\rangle$
$\langle T_0$ Commit$\rangle$

$\langle T_1$ ABC
T_1: AB
T_2: AB
T_3: C
T_4: ABC
T_5: AB

Recovery:
T_1: No changes T_2: Redo C, T_3: Redo ABC
T_4: Undo AB T_5: Discard AB

Design and Analysis of Algorithms
+
Data Structures

Comprehensive Study Notes for GATE Computer Science

Ankur Gupta

www.ankurgupta.net

Note: These notes have been prepared primarily from the following books:

(1) **Introduction to Algorithms** by Cormen, Leiserson, Rivest, Stein

(2) **Data Structures** by Seymour Lipschutz

Please refer to the above books for more details.

Contents

1 Growth of Functions **6**
- 1.1 Asymptotic Notation . . . 6
  - 1.1.1 Big-O Notation (Asymptotic Tight Bound) . . . 6
  - 1.1.2 Big-O Notation (Asymptotic Upper Bound) . . . 6
  - 1.1.3 Big-Omega Notation (Asymptotic Lower Bound) . . . 6
  - 1.1.4 Small-o Notation (Upper Bound that is not Asymptotically Tight) 7
  - 1.1.5 Small-omega Notation (Lower Bound that is not Asymptotically Tight) . . . 7
  - 1.1.6 Important Results . . . 7
  - 1.1.7 Comparison of Functions . . . 7
- 1.2 Comparison Tables . . . 8
- 1.3 Important Examples . . . 8
  - 1.3.1 Polynomial Functions . . . 8
- 1.4 Logarithms . . . 8
  - 1.4.1 Properties of Logarithms . . . 8
  - 1.4.2 Iterated Logarithm Function . . . 8

2 Recurrences **9**
- 2.1 Master Method . . . 9
  - 2.1.1 Solution Cases . . . 9
- 2.2 Changing Variables . . . 9

3 Sorting Algorithms **10**
- 3.1 Insertion Sort . . . 10
  - 3.1.1 Algorithm Description . . . 10
- 3.2 Merge Sort . . . 10
  - 3.2.1 Algorithm Description . . . 10
- 3.3 Bubble Sort . . . 10
  - 3.3.1 Algorithm Description . . . 10
- 3.4 Selection Sort . . . 11
  - 3.4.1 Algorithm Description . . . 11
- 3.5 Heap Sort . . . 11
  - 3.5.1 Max-Heapify . . . 11
  - 3.5.2 Build-Max-Heap . . . 11
  - 3.5.3 Heapsort . . . 11
  - 3.5.4 Heap-Extracts-Max . . . 11
  - 3.5.5 Heap-Increase-Key . . . 12
  - 3.5.6 Max-Heap-Insert . . . 12

3.6 Quick Sort . . . 12
3.6.1 Algorithm Description . . . 12
3.6.2 Performance Analysis . . . 12
3.7 Counting Sort . . . 13
3.7.1 Algorithm Description . . . 13
3.8 Radix Sort . . . 13
3.8.1 Algorithm Description . . . 13
3.9 Bucket Sort . . . 13
3.9.1 Algorithm Description . . . 13

4 Graph Algorithms 14
4.1 Representation of Graphs . . . 14
4.1.1 Adjacency List Representation . . . 14
4.1.2 Adjacency Matrix Representation . . . 14
4.2 Breadth-First Search . . . 14
4.2.1 Algorithm Description . . . 15
4.3 Depth-First Search . . . 15
4.3.1 Contains Two Methods . . . 15
4.4 Topological Sorting . . . 15
4.5 Strongly Connected Components . . . 15
4.6 Minimum Spanning Trees . . . 16
4.6.1 Kruskal's Algorithm . . . 16
4.6.2 Prim's Algorithm . . . 16
4.7 Single Source Shortest Paths . . . 17
4.7.1 Bellman-Ford Algorithm . . . 17
4.7.2 Dijkstra's Algorithm . . . 17

5 Data Structures 18
5.1 Arrays . . . 18
5.1.1 Representation of Linear Array in Memory . . . 18
5.1.2 Representation of 2-D Array in Memory . . . 18
5.2 Stacks . . . 19
5.2.1 Arithmetic Expressions . . . 19
5.3 Queues . . . 19
5.4 Linked Lists . . . 19
5.5 Binary Trees . . . 20
5.5.1 Definition . . . 20
5.5.2 Complete Binary Trees . . . 20
5.5.3 Extended Binary Tree (2-Tree) . . . 20
5.6 Binary Search Trees . . . 20
5.6.1 Operations on a BST . . . 20
5.7 AVL Search Trees . . . 21
5.7.1 Representation . . . 21
5.7.2 Insertion in AVL Search Tree . . . 21
5.7.3 Types of Rotations . . . 21
5.7.4 LR and RL Rotations . . . 22
5.8 B-Trees . . . 22
5.8.1 Minimum degree of the B-tree . . . 22

5.8.2 Height of a B-tree . . . 22
5.8.3 Searching a B-Tree . . . 23
5.8.4 Inserting a Key into a B-Tree . . . 23
5.8.5 Deleting a Key from a B-Tree . . . 23
5.9 Hash Tables . . . 24
5.9.1 Direct Address Tables . . . 24
5.9.2 Hash Tables . . . 24
5.9.3 Hash Functions . . . 24
5.9.4 Types of Hashing Schemes . . . 24
5.9.5 Collision Resolution . . . 24
5.9.6 Types of Probing . . . 25
5.9.7 Perfect Hashing . . . 25

6 Additional Topics **26**
6.1 Tower of Hanoi Problem . . . 26
6.1.1 Algorithm . . . 26
6.2 Assembly Line Scheduling . . . 26
6.3 Longest Common Subsequence . . . 26
6.4 Matrix Chain Multiplication . . . 27
6.4.1 Solution . . . 27
6.4.2 Inversions . . . 27

Chapter 1

Growth of Functions

1.1 Asymptotic Notation

Asymptotic notation provides a framework for analyzing the efficiency of algorithms. We use these notations to describe the limiting behavior of a function when the argument tends toward infinity.

1.1.1 Big-O Notation (Asymptotic Tight Bound)

Definition 1.1.1 (Big-O Notation)**.** A function $f(n)$ belongs to $\mathcal{O}(g(n))$ if there exist positive constants c_1, c_2, and n_0 such that:

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0$$

This means that $f(n)$ is bounded both above and below by $g(n)$ for sufficiently large n.

Key properties:

- $f(n) = \Theta(g(n))$ if and only if $f(n) = \mathcal{O}(g(n))$ and $f(n) = \Omega(g(n))$
- $\mathcal{O}(g(n)) \cap \Omega(g(n)) = \Theta(g(n))$

1.1.2 Big-O Notation (Asymptotic Upper Bound)

Definition 1.1.2 (O-notation)**.** A function $f(n)$ belongs to $\mathcal{O}(g(n))$ if there exist positive constants c and n_0 such that:

$$0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

Example: $f(n) = O(n^2)$ means that the worst-case running time is $O(n^2)$.

1.1.3 Big-Omega Notation (Asymptotic Lower Bound)

Definition 1.1.3 (Ω-notation)**.** A function $f(n)$ belongs to $\Omega(g(n))$ if there exist positive constants c and n_0 such that:

$$0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0$$

This means that for all values of n greater than or equal to n_0, the value $f(n)$ is on or above $c \cdot g(n)$.

1.1.4 Small-o Notation (Upper Bound that is not Asymptotically Tight)

Definition 1.1.4 (o-notation)**.** A function $f(n)$ belongs to $o(g(n))$ if for any positive constant $c > 0$, there exists a constant $n_0 > 0$ such that:

$$0 \leq f(n) < c \cdot g(n) \text{ for all } n \geq n_0$$

1.1.5 Small-omega Notation (Lower Bound that is not Asymptotically Tight)

Definition 1.1.5 (ω-notation)**.** A function $f(n)$ belongs to $\omega(g(n))$ if for any positive constant $c > 0$, there exists a constant $n_0 > 0$ such that:

$$0 \leq c \cdot g(n) < f(n) \text{ for all } n \geq n_0$$

1.1.6 Important Results

#1 $f(n) = \Theta(g(n))$ if and only if $f(n) = \mathcal{O}(g(n))$ and $f(n) = \Omega(g(n))$

#2 $\mathcal{O}(g(n)) \cap \Omega(g(n)) = \Theta(g(n))$

#3 $\mathcal{O}(g(n)) \cap \omega(g(n)) = \emptyset$

1.1.7 Comparison of Functions

The following relationships hold for comparing asymptotic growth rates:

Transitivity:

- If $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n))$, then $f(n) = \Theta(h(n))$

Reflexivity:

$$f(n) = \mathcal{O}(f(n))$$
$$f(n) = \Omega(f(n))$$
$$f(n) = \Theta(f(n))$$

Symmetry:

$$f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$$

Transpose Symmetry:

$$f(n) = \mathcal{O}(g(n)) \iff g(n) = \Omega(f(n))$$
$$f(n) = o(g(n)) \implies g(n) = \omega(f(n))$$

Condition	Relation
$f(n) = \mathcal{O}(g(n))$	$a \leq b$
$f(n) = \Omega(g(n))$	$a \geq b$
$f(n) = \Theta(g(n))$	$a = b$
$f(n) = o(g(n))$	$a < b$
$f(n) = \omega(g(n))$	$a > b$

Table 1.1: Analogy between asymptotic comparisons and numerical comparisons

1.2 Comparison Tables

1.3 Important Examples

1.3.1 Polynomial Functions

Example 1.3.1. Consider $(n + a)^b = \Theta(n^b)$ where a and b are constants and $b > 0$.

Example 1.3.2. $2^{n+1} = \mathcal{O}(2^n)$ but $2^{2n} \neq \mathcal{O}(2^n)$

1.4 Logarithms

1.4.1 Properties of Logarithms

(1) $\log_b a^k = k \log_b a$

(2) $\log_b \log_c n = \log_b(\log_c n)$

(3) $\log_b(n^a) = a \log_b n$

(4) $\log_b a = \frac{1}{\log_a b}$

(5) $\frac{\log_a c}{\log_b c} = \frac{\log_a b}{1}$

1.4.2 Iterated Logarithm Function

The iterated logarithm function is defined as:

$$\lg^* n = \min\{i \geq 0 : \lg^{(i)} n \leq 1\}$$

Example 1.4.1.

$$\begin{aligned} \lg^* 2 &= 1 \\ \lg^* 4 &= 2 \\ \lg^* 16 &= 3 \\ \lg^* 65536 &= 4 \\ \lg^* 2^{65536} &= 5 \end{aligned}$$

Chapter 2

Recurrences

2.1 Master Method

The Master Method is used for solving recurrences of the form:

$$T(n) = aT(n/b) + f(n)$$

where $a \geq 1$ and $b > 1$ are constants, and $f(n)$ is an asymptotically positive function.

2.1.1 Solution Cases

(1) If $f(n) = \mathcal{O}(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then:

$$T(n) = \Theta(n^{\log_b a})$$

(2) If $f(n) = \Theta(n^{\log_b a})$, then:

$$T(n) = \Theta(f(n) \lg n)$$

(3) If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for $c < 1$ and all sufficiently large n, then:

$$T(n) = \Theta(f(n))$$

Note. Here, $f(n)$ must be polynomially smaller or greater than $n^{\log_b a}$ in cases 1 and 3.

2.2 Changing Variables

We can use the technique of changing variables to simplify recurrences.

Example 2.2.1. Given $T(n) = 2T(\sqrt{n}) + \lg n$, let $n = 2^m$:

$$T(2^m) = 2T(2^{m/2}) + m$$

Put $T(2^m) = S(m)$:

$$S(m) = 2S(m/2) + m$$

Using the Master Method:

$$S(m) = \mathcal{O}(m \lg m)$$

Putting $m = \lg n$:

$$T(n) = \mathcal{O}(\lg n \cdot \lg \lg n)$$

Chapter 3

Sorting Algorithms

3.1 Insertion Sort

3.1.1 Algorithm Description

Insertion sort builds the final sorted array one item at a time. It is much less efficient on large lists than more advanced algorithms such as quicksort, heapsort, or merge sort.

Time Complexity:

- Best Case: $O(n)$
- Worst Case: $O(n^2)$
- Average Case: $O(n^2)$
- Maximum number of swaps required: $O(n^2)$

3.2 Merge Sort

3.2.1 Algorithm Description

Merge sort is a divide-and-conquer algorithm that divides the input array into two halves, calls itself for the two halves, and then merges the two sorted halves.

Recurrence Relation:

$$T(n) = 2T(n/2) + \Theta(n)$$

Time Complexity:

- Worst Case: $\Theta(n \lg n)$

3.3 Bubble Sort

3.3.1 Algorithm Description

Bubble sort repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.

Time Complexity:

- Average and Worst Case: $O(n^2)$

3.4 Selection Sort

3.4.1 Algorithm Description

Selection sort divides the input list into two parts: a sorted portion at the left end and an unsorted portion at the right end. The algorithm proceeds by finding the smallest element in the unsorted subarray and swapping it with the leftmost unsorted element.

Time Complexity:

- Worst Case: $O(n^2)$
- Average Case: $O(n^2)$
- Maximum swaps required: $O(n)$

3.5 Heap Sort

3.5.1 Max-Heapify

Max-Heapify is used for maintaining the heap property. The input is an array A and an index i into the array. When MAX-HEAPIFY is called, it is assumed that the binary trees rooted at LEFT(i) and RIGHT(i) are max-heaps, but that $A[i]$ may be smaller than its children.

Running Time: $O(\lg n)$ for a node of height h. For a node at RIGHT R, running time is $O(R)$.

3.5.2 Build-Max-Heap

Build-Max-Heap is used for building a max-heap from an array. It applies the MAX-HEAPIFY operation on all internal nodes from the leaf to the root node.

Running Time: $O(n)$

3.5.3 Heapsort

Heapsort uses the heap to sort the elements of the resultant array in non-decreasing order.

Running Time: $O(n \lg n)$

3.5.4 Heap-Extracts-Max

This operation removes the maximum element of the heap and returns it.

Running Time: $O(\lg n)$

3.5.5 Heap-Increase-Key

This operation increases the value at location i by the value key and then traverses the heap upward to maintain the heap property.

Running Time: $O(\lg n)$

3.5.6 Max-Heap-Insert

This operation inserts the element at the end of the heap and then traverses upward to maintain the heap property.

Running Time: $O(\lg n)$

3.6 Quick Sort

3.6.1 Algorithm Description

Quick sort uses a divide-and-conquer approach. It uses the PARTITION algorithm to partition the array into two sub-arrays. The PARTITION algorithm selects the last element of the input array as a pivot element and partitions the array such that the elements to the left are smaller than or equal to the pivot element, and the elements to the right of the pivot element are greater than it.

3.6.2 Performance Analysis

The running time of quicksort depends on whether the partitioning is balanced or unbalanced.

Worst Case Partitioning:

The worst case occurs when the partitioning routine produces one sub-problem with $n-1$ elements and one with 0 elements.

$$T(n) = T(n-1) + T(0) + \Theta(n) = T(n-1) + \Theta(n) = \mathcal{O}(n^2)$$

Best Case Partitioning:

If the average-case running time of quicksort is much closer to the best case than to the worst case:

$$T(n) \leq 2T(n/2) + \Theta(n)$$
$$T(n) = \mathcal{O}(n \lg n)$$

Balanced Partitioning:

If the average-case running time of quicksort is much closer to the best case than to the worst case:

$$T(n) \leq T(9n/10) + T(n/10) + cn$$

Running Time: $\mathcal{O}(n \lg n)$

3.7 Counting Sort

3.7.1 Algorithm Description

Counting sort assumes that each of n input elements is an integer in the range 0 to k, for some integer k, where $k = O(n)$.

Running Time: $\Theta(n + k) = \mathcal{O}(n)$

It is a stable sorting algorithm. This property is generally used for sorting suitable data.

3.8 Radix Sort

3.8.1 Algorithm Description

Radix sort uses a stable sorting algorithm.

Running Time: $\Theta(d(n + k))$, where d is the number of digits and k is 10 (0, 1, ..., 9) if d is constant and $k = O(n)$.

Running Time: $O(n) \rightarrow$ if d is constant.

3.9 Bucket Sort

3.9.1 Algorithm Description

Bucket sort divides the interval [0, 1) into n equal-sized sub-intervals or buckets and then distributes the n input numbers into the buckets. Here, if n numbers are in the range [0, 1), then:

Running Time: $\Theta(n)$

Chapter 4

Graph Algorithms

4.1 Representation of Graphs

A graph $G = (V, E)$ consists of a set of vertices V and a set of edges E.

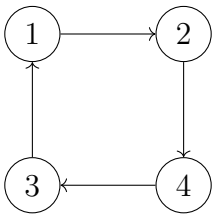
4.1.1 Adjacency List Representation

Adjacency list representation is used to represent sparse graphs, i.e., those for which $|E| \ll |V|^2$.

For an adjacency list representation, if G is a directed graph, the sum of all the adjacency lists is $|E|$. If G is an undirected graph, the sum is $2|E|$.

Amount of memory required for Adjacency List Representation: $\Theta(V + E)$

If the transpose of a directed graph $G = (V, E^T)$ is the graph $G^T = (V, E^T)$:



4.1.2 Adjacency Matrix Representation

Adjacency matrix representation is used for representing dense graphs, i.e., $|E| \leq |V|^2$.

For adjacency list representation, if G is a directed graph, the sum of all adjacency lists is $|E|$ and if G is an undirected graph, the sum is $2|E|$.

Amount of memory required: $\Theta(V + E)$

4.2 Breadth-First Search

Breadth-First Search (BFS) is used for searching a graph. It finds a minimum-spanning tree algorithm and Dijkstra's single-source shortest-path algorithm. We use ideas similar to those in BFS.

4.2.1 Algorithm Description

BFS produces a breadth-first tree with a root that contains all reachable nodes.

It works on both directed and undirected graphs.

To keep track of progress, breadth-first search colors each vertex white, gray, or black. All vertices start out white and may later become gray and then black.

- It assumes that the input graph $G = (V, E)$ is represented using adjacency lists.
- The color of each vertex u is stored in the variable color[u], and the predecessor of u is stored in variable $\pi[u]$. The distance from the source s to vertex u is stored in $d[u]$. The algorithm also uses a FIFO queue Q to manage the set of gray vertices.

Running Time of BFS: $O(V + E)$

4.3 Depth-First Search

Depth-First Search (DFS) produces a depth-first forest composed of several depth-first trees.

Each vertex is initially white, is grayed when it is discovered in the search, and is blackened when it is finished.

It timestamps each vertex. Each vertex v has two timestamps: the first timestamp $d[v]$ records when v is first discovered (and grayed), and the second timestamp $f[v]$ records when the search finishes examining v's adjacency list (and blackens v).

Vertex u is white before $d[u]$, gray between $d[u]$ and $f[u]$, and black thereafter.

4.3.1 Contains Two Methods

DFS-VISIT and DFS-G.

Running Time of DFS: $\Theta(V + E)$ (only for DAG)

4.4 Topological Sorting

Topological sorting is applicable only if the graph is acyclic. A topological sort of a graph is an ordering of its vertices along a horizontal line so that all directed edges go from left to right.

Running Time: $\Theta(V + E)$

Note. This is applicable only if the graph is a directed acyclic graph (DAG). We can sort the vertices in order of decreasing finishing times.

4.5 Strongly Connected Components

A graph is strongly connected if for every pair of vertices u, v: $u \to v$ and $v \to u$, i.e., u and v are reachable from each other.

STRONGLY-CONNECTED-COMPONENTS(G)**:**

(1) Call DFS(G) to compute finishing times $f[u]$ for each vertex u.

(2) Compute G^T.

(3) Call DFS(G^T), but in the main loop of DFS, consider the vertices in order of decreasing $f[u]$.

(4) Output the vertices of each tree in the depth-first forest formed in Line 3 as a strongly-connected component.

Running Time: $\Theta(V + E)$

4.6 Minimum Spanning Trees

Kruskal's and Prim's Algorithms are used to find minimum spanning trees from a weighted graph. Both are greedy algorithms.

4.6.1 Kruskal's Algorithm

Kruskal's algorithm is a greedy algorithm because at each step it adds to the forest an edge of least possible weight.

Running Time of Kruskal's algorithm for a graph $G = (V, E)$ depends on the implementation of the disjoint-set data structure.
Running Time: $O(E \lg E)$. Since $|E| < |V|^2 \Rightarrow \lg |E| = O(\lg V)$.
Running Time: $O(E \lg V)$

If we use a binary min-heap, then:
Running Time: $O(V \lg V + E \lg V) = O(E \lg V)$
If we use Fibonacci heap, then:
Running Time: $O(E + V \lg V)$

4.6.2 Prim's Algorithm

Prim's algorithm operates much like Dijkstra's algorithm for finding shortest paths in a graph.

The edges in the set A always form a single tree. The tree starts from an arbitrary root vertex and grows until the tree spans all the vertices in V. At each step, a light edge is added to the tree A that connects A to an isolated vertex of $G_A = (V, A)$.

Running Time:

(i) Using Binary Min-Heap:

$$O((V + E) \lg V) = O(E \lg V)$$

(ii) Using Fibonacci Min-Heap:

$$O(E + V \lg V)$$

4.7 Single Source Shortest Paths

4.7.1 Bellman-Ford Algorithm

The Bellman-Ford algorithm solves the single-source shortest-path problem in which edge weights may be negative.

It uses relaxation to find shortest paths. It returns TRUE if the graph contains no negative-weight cycles that are reachable from the source.

Running Time: $O(VE)$

Note: This algorithm does not find any negative-weight cycle. It only finds a relaxable $-ve$ weight cycle. If such a cycle is reachable from the source, it returns FALSE, indicating that no solution exists for the source.

Single Source Shortest Paths in Directed Acyclic Graphs:

By relaxing the edges of a weighted DAG according to a topological sort of its vertices, we can compute shortest paths from a single source in $\Theta(V + E)$ time.

4.7.2 Dijkstra's Algorithm

Dijkstra's algorithm solves the single-source shortest-paths problem in which all edge weights are non-negative.

It maintains a set S of vertices whose shortest-path weights from the source have been determined. The algorithm repeatedly selects the vertex $u \in V - S$ with the minimum shortest-path estimate, adds u to S, and relaxes all edges leaving u.

The running time of Dijkstra's algorithm depends on how the min-priority queue is implemented.

Running Time:

(i) Using Binary Min-Heap:

$$O((V + E) \lg V) = O(E \lg V)$$

(ii) Using Fibonacci Min-Heap:

$$O(E + V \lg V)$$

Chapter 5

Data Structures

5.1 Arrays

A linear array is a list of a finite number n of homogeneous data elements such that:

(a) The elements of the array are referenced respectively by an index set consisting of n consecutive numbers.

(b) The elements of the array are stored respectively in successive memory locations.

Length of the array:

$$\text{Length} = UB - LB + 1$$

5.1.1 Representation of Linear Array in Memory

$$\text{LOC(A[K])} = \text{Base(A)} + w(K - LB)$$

where w is the size of word/memory cell, K is the index of the element in the array, and LOC is the location of an element.

5.1.2 Representation of 2-D Array in Memory

(a) **Row-Major Order:**

$$\text{LOC(A[J, K])} = \text{Base(A)} + w[N(J - 1) + (K - 1)]$$

(b) **Column-Major Order:**

$$\text{LOC(A[J, K])} = \text{Base(A)} + w[M(K - 1) + (J - 1)]$$

where A is an $M \times N$ array and Base(A) is the base address of A.

5.2 Stacks

In a stack, the element deleted from the set is the one most recently inserted. The stack implements a Last-In, First-Out (LIFO) policy.

The INSERT operation on a stack is often called PUSH, and the DELETE operation, which does not take an element argument, is often called POP.

We can implement a stack of at most n elements with an array $S[1..n]$. The array has an attribute top[S] that indexes the most recently inserted element. When top[S] = 0, the stack contains no elements and is empty.

5.2.1 Arithmetic Expressions

(a) **Infix Notation:** $A + B$, $C - D$, $E * F$, G/H

(b) **Prefix Notation** or **Polish Notation:** $+AB$, $-CD$, $*EF$, $/GH$

(c) **Reverse Polish Notation** or **Postfix Notation:** $AB+$, $CD-$, $EF*$, $GH/$

5.3 Queues

In a queue, the element deleted is always the one that has been in the set for the longest time. The queue implements a First-In, First-Out (FIFO) policy.

The insert operation of a queue is called ENQUEUE and the delete operation is called DEQUEUE.

The queue has an attribute Read[Q] that indexes, or points to, its head. The attribute tail[Q] indexes the next location at which a newly existing element will be inserted into the queue.

When Read[Q] = tail[Q], the queue is empty.

Initially, we have Read[Q] = tail[Q] = 1.

When Read[Q] = tail[Q] + 1, the queue is full.

5.4 Linked Lists

A linked list is a linear collection of data elements, called nodes, where the linear order is given by means of pointers.

A doubly linked list L is an object with a key field and two other pointer fields: next and prev. Given an element x in the list, next[x] points to its successor in the linked list, and prev[x] points to its predecessor. If prev[x] = NIL, the element x has no predecessor and is therefore the first element, or head, of the list. If next[x] = NIL, the element x has no successor and is therefore the last element, or tail, of the list. An attribute head[L] points to the first element of the list. If head[L] = NIL, the list is empty.

- If a list is singly linked, we omit the prev pointer in each element.
- If a list is sorted, the linear order of the list corresponds to the linear order of keys stored in elements of the list. The minimum element is then the head of the list.
- In a circular list, the prev pointer of the head of the list points to the tail, and the next pointer of the tail of the list points to the head.

5.5 Binary Trees

5.5.1 Definition

Binary trees T and T' are said to be similar if they have the same structure or, in other words, if they have the same shape. The trees are said to be copies if they are similar and if they have the same contents at corresponding nodes.

A node L is called a descendant of a node N (and N is called an ancestor of L) if there is a succession of children from N to L. In particular, L is called a left or right descendant of N according to whether L belongs to the left or right subtree of N.

5.5.2 Complete Binary Trees

The tree T is said to be complete if all its levels, except possibly the last, have the maximum number of possible nodes and if all the nodes at the last level appear as far left as possible.

5.5.3 Extended Binary Tree (2-Tree)

A binary tree T is said to be a 2-tree or an extended binary tree if each node N has either 0 or 2 children. In such a case, the nodes with 2 children are called internal nodes, and the nodes with 0 children are called external nodes.

In an extended k-ary tree, the number of leaves with n internal nodes is:

$$L = n(k-1) + 1$$

5.6 Binary Search Trees

A Binary Search Tree (BST) is organized in a binary tree. In addition to a key field, each node contains fields left, right, and p that point to the nodes corresponding to its left child, right child, and its parent.

The keys in BST always satisfy the Binary-Search-Tree Property: Let x be a node in BST. If y is a node in the left subtree of x, then key[y] $\leq$ key[x]. If y is a node in the right subtree of x, then key[x] $\leq$ key[y].

The in-order walk of BST prints the keys in non-decreasing order.

It takes $O(h)$ time to walk an n-node BST, since after the initial call, the procedure is called recursively exactly twice for each node in the tree–once for its left child and once for its right child.

5.6.1 Operations on a BST

(1) **Searching:** Running Time = $O(h)$, where h is the height of the tree.

(2) **Minimum and Maximum:** To find the minimum, follow left child pointers and to find the maximum, follow right child pointers from the root until a NIL is encountered.

Running Time = $O(h)$.

Number of different binary search trees: Let b_n denote the number of different binary trees with n nodes, then:

$$b_n = \frac{n-1}{\sum_{k=0}^{n-1}} b_k \cdot b_{n-1-k} \quad \text{where } b_0 = 1 \ \& \ b_1 = 1$$

$$\Rightarrow n \geq 2^h$$

5.7 AVL Search Trees

An AVL tree is a binary search tree that is height-balanced. For each node x, the heights of the left and right subtrees of x differ by at most 1.

$$B.F. = h(T^L) - h(T^R)$$

where $B.F.$ is the Balance Factor and T^L and T^R are the heights of the left and right subtrees.

For an AVL tree:

$$|h(T^L) - h(T^R)| \leq 1$$

5.7.1 Representation

AVL search trees like BST are represented using a linked representation. However, every node registers its balance factor.

5.7.2 Insertion in AVL Search Tree

Inserting an element into an AVL tree in its first phase is similar to that of the one used in a binary search tree. However, if after insertion of the element, the balance factor of any node in the tree is affected so as to render the binary search tree unbalanced, we resort to techniques called Rotations to restore the balance of the tree.

To perform rotations, it is necessary to identify a specific node A whose BF(A) is neither 0, 1, or -1, which is the nearest ancestor to the inserted node on the path from the inserted node to the root.

5.7.3 Types of Rotations

1. **LL Rotation:** Inserted node is in the left subtree of the left subtree of node A.
2. **RR Rotation:** Inserted node is in the right subtree of the right subtree of node B.
3. **LR Rotation:** Inserted node is in the right subtree of the left subtree of node A.
4. **RL Rotation:** Inserted node is in the left subtree of the right subtree of node A.

5.7.4 LR and RL Rotations

The balancing methodology of LR and RL rotations are similar in nature but are mirror images of one another.

LR can be accomplished by RR followed by LL rotation, and RL can be accomplished by LL followed by RR rotation.

5.8 B-Trees

A B-tree T is a rooted tree having the following properties:

(1) Every node x has the following fields:

  (a) $n[x]$, the number of keys currently stored in node x.

  (b) The $n[x]$ keys themselves, stored in non-decreasing order, so that $\text{key}_1[x] \leq \text{key}_2[x] \leq \cdots \leq \text{key}_{n[x]}[x]$.

  (c) leaf[x], a boolean value that is true if x is a leaf and false if x is an internal node.

(2) Each internal node x also contains $n[x]+1$ pointers $c_1[x], c_2[x], \ldots, c_{n[x]+1}[x]$ to its children. Leaf nodes have no children, so their c_i fields are undefined.

(3) The keys $\text{key}_j[x]$ separate the ranges of keys stored in each subtree: if k_i is any key stored in the subtree with root $c_i[x]$, then:

$$k_1 \leq \text{key}_1[x] \leq k_2 \leq \text{key}_2[x] \leq \cdots \leq \text{key}_{n[x]}[x] \leq k_{n[x]+1}$$

(4) All leaves have the same depth, which is the tree's height h.

(5) There are lower and upper bounds on the number of keys a node can contain. It is represented by an integer $t \geq 2$ called the minimum degree of the B-tree.

5.8.1 Minimum degree of the B-tree

(a) Every node other than the root must have at least $t-1$ keys. Every internal node other than the root thus has at least t children. If the tree is non-empty, the root must have at least one key.

(b) Every node can contain at most $2t-1$ keys. Therefore, an internal node can have at most $2t$ children. A node is full if it contains $(2t-1)$ keys.

5.8.2 Height of a B-tree

If $n \geq 1$, then for any n-key B-tree T of height h and minimum degree $t \geq 2$:

$$h \leq \log_t \left(\frac{n+1}{2} \right)$$

or

$$n \geq 2t^h - 1$$

5.8.3 Searching a B-Tree

The running time of a B-tree search is $O(t \log_t n)$ and the number of keys in a page is less than $2t$.

$$\text{So, total search time } = O(t \log_t n) \approx O(\log_t n)$$

and the number of keys in a page less than $2t$ is:

$$= O(t)$$

5.8.4 Inserting a Key into a B-Tree

To insert a key into a B-tree, we travel down the tree searching for the position where the new key belongs, we split each full node we come to along the way (including the leaf itself). Thus, whenever we want to split a full node y, we are assured that its parent is not full.

Running Time: $O(t \log_t n)$

5.8.5 Deleting a Key from a B-Tree

Here, we must ensure that a node doesn't get too small during deletion (except the root).

Procedure:

(1) If the key k is in node x and x is a leaf, delete the key k from x.

(2) If the key k is in node x and x is an internal node, do the following:

   (a) If the child y that precedes k in node x has at least t keys, then find the predecessor k' of k in the subtree rooted at y. Recursively delete k' and replace k by k' in x.

   (b) Symmetrically, if the child z that follows k in node x has at least t keys, then find the successor k' of k in the subtree rooted at z. Recursively delete k' and replace k by k' in x.

   (c) Otherwise, if both y and z have only $t-1$ keys, merge k and all of z into y, so that x loses both k and the pointer to z, and y now contains $2t-1$ keys. Then free z and recursively delete k from y.

(3) If the key k is not present in internal node x, determine the root $c_i[x]$ of the appropriate subtree that must contain k. If $c_i[x]$ has only $t-1$ keys, execute step 3a or 3b as necessary to guarantee that we descend to a node containing at least t keys. Then finish by recursing on the appropriate child of x.

   (a) If $c_i[x]$ has only $t-1$ keys but has an immediate sibling with at least t keys, give $c_i[x]$ an extra key by moving a key from x down into $c_i[x]$, moving a key from $c_i[x]$'s immediate left or right up into x, and moving the appropriate child pointer from the sibling into $c_i[x]$.

   (b) If $c_i[x]$ and both of $c_i[x]$'s immediate siblings have $t-1$ keys, merge $c_i[x]$ with one sibling, which involves moving a key from x down into the new merged node to become the median key for that node.

5.9 Hash Tables

5.9.1 Direct Address Tables

Direct address tables work well when the universe U of keys is reasonably small.

Let universe $U = \{0, 1, \ldots, m-1\}$, where m is not too large.

To represent the dynamic set, we use an array, or Direct-address table, denoted by $T[0..m-1]$ in which each position, or slot, corresponds to a key in the universe U.

5.9.2 Hash Tables

With direct addressing, an element with key k is stored in slot k. With hashing, this element is stored in slot $h(k)$, where h is the hash function. The hash function h maps the universe U of keys into the slots of a hash table $T[0..m-1]$:

$$h : U \to \{0, 1, \ldots, m-1\}$$

Here:

$$k \to U \to \{0, 1, \ldots, m-1\}$$

5.9.3 Hash Functions

A good hash function satisfies the assumption of simple uniform hashing: each key is equally likely to hash to any of the m slots independently of where any other key has hashed to.

A hash function h must be deterministic in that a given input k should always produce the same output $h(k)$.

5.9.4 Types of Hashing Schemes

Division Method

$$h(k) = k \mod m$$

A prime not too close to an exact power of 2 is often a good choice for m.

Multiplication Method

$$h(k) = \lfloor m(kA \mod 1) \rfloor$$

where $0 < A < 1$ and $kA \mod 1$ is the fractional part.

5.9.5 Collision Resolution

When two keys hash to the same slot, this situation is called collision. Collision is also called an open hashing.

Collision Resolution by Chaining

In chaining, we put all the elements that hash to the same slot in a linked list as shown in the figure.

Running Time: $O(1 + \log_2 m)$

Open Addressing

In open addressing, all elements are stored in the hash table itself.

To perform insertion using open addressing, we successively examine or probe the hash table until we find an empty slot to put the key.

The algo for searching for key k probes the same sequence of slots used during insertion.

Deletion from an open-address hash table is difficult. When we delete a key from slot i, we can't simply mark that slot as empty by storing NIL in it. Doing so might make it impossible to retrieve any key k during whose insertion we had probed slot i and found it occupied.

We use a small secondary table hash table S_j with an associated hash function h_j.

5.9.6 Types of Probing

Linear Probing

Given an ordinary hash function $h' : U \to \{0, 1, \ldots, m-1\}$, which we refer to as an auxiliary hash function, the method of linear probing uses the hash function:

$$h(k, i) = (h'(k) + i) \mod m$$

for $i = 0, 1, \ldots, m-1$.

Quadratic Probing

Quadratic probing uses a hash function of the form:

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \mod m$$

where h' is an auxiliary function, c_1 and $c_2 \neq 0$ are auxiliary constants, and $i = 0, 1, \ldots, m-1$.

Double Hashing

Double hashing uses the hash function of the form:

$$h(k, i) = (h_1(k) + i h_2(k)) \mod m$$

where h_1 and h_2 are auxiliary functions.

5.9.7 Perfect Hashing

Perfect hashing is a technique that allows a search in $O(1)$ worst case.

Instead of making a list of the keys hashing to slot j, however, we use a small secondary hash table S_j with an associated hash function h_j.

If we use a binary min-heap, then:
Running Time: $O(V \lg V + E \lg V) = O(E \lg V)$
If we use Fibonacci heap, then:
Running Time: $O(E + V \lg V)$

Chapter 6

Additional Topics

6.1 Tower of Hanoi Problem

Suppose there are three pegs labeled A, B, and C and suppose on peg A there is a finite number n of disks with decreasing size. The object of the game is to move the disks from peg A to peg C using peg B as an auxiliary peg.

(a) Only one disk may be moved at a time, i.e., the top disk on any peg.

(b) At no time a larger disk be placed on a smaller disk.

6.1.1 Algorithm

TOWER(N, BEG, AUX, END):

(1) If $N = 1$, then:

  (a) Write BEG $\rightarrow$ END

  (b) Return

(2) Call TOWER($N - 1$, BEG, END, AUX)

(3) Write BEG $\rightarrow$ END

(4) Call TOWER($N - 1$, AUX, BEG, END)

(5) Return

Complexity:

$$T(n) = 2T(n-1) + \Theta(1) = \Theta(2^n)$$

6.2 Assembly Line Scheduling

Assembly line scheduling is a problem in dynamic programming.

6.3 Longest Common Subsequence

The Longest Common Subsequence (LCS) problem is a classic problem in dynamic programming.

6.4 Matrix Chain Multiplication

Matrix chain multiplication is an optimization problem that determines the most efficient way to multiply a given sequence of matrices.

6.4.1 Solution

Given matrices:

$$M_1 = (10 \times 100)$$
$$M_2 = (100 \times 20)$$
$$M_3 = (20 \times 5)$$
$$M_4 = (5 \times 80)$$

The number of multiplications required = 19,000

6.4.2 Inversions

Let $A[1 \ldots n]$ be an array of n distinct numbers. If $i < j$ and $A[i] > A[j]$, then the pair (i, j) is called an inversion of A.

The maximum number of inversions in an array with n distinct elements is $\frac{n(n-1)}{2} = C_2 = \binom{n}{2}$.

As the number of inversions increases, the running time of insertion sort also increases.

To determine the number of inversions in any permutation on n-elements takes $O(n \lg n)$ time in the worst case (using modified merge sort).

Digital Logic

Comprehensive Study Notes for GATE Computer Science

Ankur Gupta

www.ankurgupta.net

Contents

1 Number Representation **4**
- 1.1 Range of n-bit 2's Complement Numbers . . . 4
- 1.2 Range of n-bit 1's Complement Numbers . . . 4

2 Boolean Algebra and Logic Minimization **5**
- 2.1 Implicants . . . 5
  - 2.1.1 Implicant . . . 5
  - 2.1.2 Prime Implicant . . . 5
  - 2.1.3 Essential Prime Implicant . . . 5
- 2.2 Finite State Machine to Calculate 2's Complement . . . 6

3 Sequential Logic: Flip-Flops **7**
- 3.1 SR (Set-Reset) Flip-Flop . . . 7
- 3.2 T (Toggle) Flip-Flop . . . 7
- 3.3 D (Delay) Flip-Flop . . . 7
- 3.4 JK Flip-Flop . . . 8
- 3.5 Characteristic Equation . . . 8

4 Advanced Sequential Logic **9**
- 4.1 Clocked Flip-Flops with Clear and Reset Inputs . . . 9
- 4.2 Logic Levels . . . 9
  - 4.2.1 Positive Logic . . . 9
  - 4.2.2 Negative Logic . . . 9
- 4.3 Minimum Number of Gates Required . . . 9
- 4.4 Finite State Machines . . . 9
  - 4.4.1 Moore Machine . . . 9
  - 4.4.2 Mealy Machine . . . 10

5 Combinational Logic Components **11**
- 5.1 Multiplexers . . . 11
- 5.2 Decoders . . . 11
- 5.3 Encoders . . . 12
- 5.4 Canonical Logic Forms . . . 12
  - 5.4.1 Half Adder . . . 12
  - 5.4.2 Full Adder . . . 12
- 5.5 Look-Ahead Carry Adder . . . 12

6 Arithmetic Circuits **13**
6.1 Half Subtractor 13
6.2 Full Subtractor 13

7 State Machine Minimization **14**
7.1 Reduction of State Tables 14
7.1.1 Example 14

8 Code Conversion **15**
8.1 Binary Code to Gray Code Conversion 15
8.2 Gray Code to Binary Code Conversion 15

Chapter 1

Number Representation

1.1 Range of n-bit 2's Complement Numbers

The range of an n-bit 2's complement number is:

$$-2^{(n-1)} \text{ to } (2^{(n-1)} - 1)$$

1.2 Range of n-bit 1's Complement Numbers

The range of an n-bit 1's complement number is:

$$-(2^{(n-1)} - 1) \text{ to } (2^{(n-1)} - 1)$$

Chapter 2

Boolean Algebra and Logic Minimization

2.1 Implicants

2.1.1 Implicant

A normal product term that implies a Boolean function Y is called an **implicant**.

$$Y = AB + AB\bar{C} + \bar{B}C$$

All terms AB, $AB\bar{C}$, and $\bar{B}C$ are implicants of Y.

2.1.2 Prime Implicant

An implicant of Y is called a **prime implicant** if any variable is removed from the implicant, the resulting term does not imply Y.

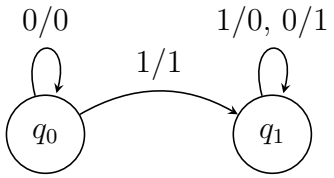
Example: Consider $Y = AB + \bar{B}C + ABC$

- This is not a prime implicant because if we remove variable A from ABC, we get $\bar{B}C$, which still implies Y.
- Prime implicant: $AB + \bar{B}C$

2.1.3 Essential Prime Implicant

If a minterm is covered only by a prime implicant, then that prime implicant is called an **essential prime implicant**.

2.2 Finite State Machine to Calculate 2's Complement



Input is from least significant bit

Chapter 3

Sequential Logic: Flip-Flops

3.1 SR (Set-Reset) Flip-Flop

$S(t)$	$R(t)$	$Q(t)$	$Q(t+1)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	Inputs not allowed
1	1	1	

Characteristic Equation:

$$Q^+ = S + \bar{R}Q$$

where $SR = 0$ (condition for the type of flip-flop to be employed).

3.2 T (Toggle) Flip-Flop

T	Q	Q^+
0	0	0
0	1	1
1	0	1
1	1	0

Characteristic Equation:

$$Q^+ = T\bar{Q} + \bar{T}Q = T \oplus Q$$

3.3 D (Delay) Flip-Flop

D	Q	Q^+
0	0	0
0	1	0
1	0	1
1	1	1

Characteristic Equation:

$$Q^+ = D$$

3.4 JK Flip-Flop

J	K	Q	Q^+
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Characteristic Equation:

$$Q^+ = J\bar{Q} + \bar{K}Q = Q\bar{K} + \bar{Q}J$$

3.5 Characteristic Equation

A characteristic equation is an equation that expresses the next state of a flip-flop in terms of its present state and inputs.

Chapter 4

Advanced Sequential Logic

4.1 Clocked Flip-Flops with Clear and Reset Inputs

- If a 0 is applied to the **clear** input, the flip-flop will reset to $Q = 0$.
- If a 0 is applied to the **preset** input, the flip-flop will set to $Q = 1$.
- These inputs override the clock and J, K inputs.

4.2 Logic Levels

4.2.1 Positive Logic

- $+V$ represents a logic 1
- 0 volts represents a logic 0

4.2.2 Negative Logic

- $+V$ represents a logic 0
- 0 volts represents a logic 1

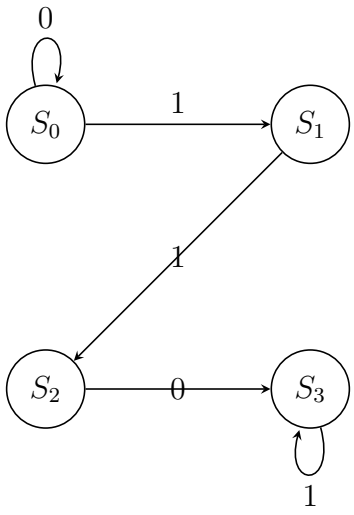
4.3 Minimum Number of Gates Required

To find a minimum solution for an expression, one must find both the network with the AND-gate output and the one with the OR-gate output.

4.4 Finite State Machines

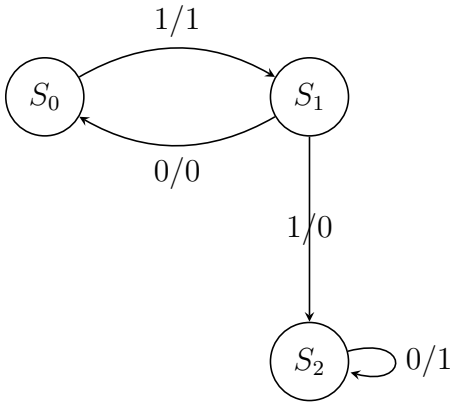
4.4.1 Moore Machine

If the output of a sequential network is a function of the present state only, the network is often referred to as a **Moore Machine**.



4.4.2 Mealy Machine

If the output is a function of both the present state and the input, the network is referred to as a **Mealy Machine**.



Chapter 5

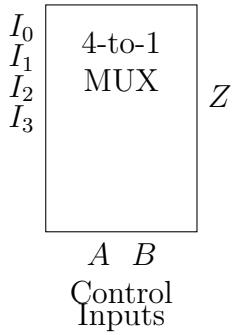
Combinational Logic Components

5.1 Multiplexers

A multiplexer has a group of data inputs and a group of control inputs. The control inputs are used to select one of the data inputs and connect it to the output terminal.

For an n-variable function:

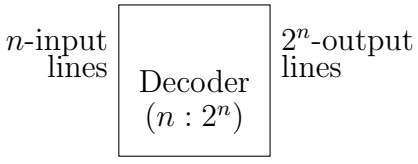
$$Z = A'B'I_0 + A'BI_1 + AB'I_2 + ABI_3$$



A 2^n-to-1 multiplexer can be used to realize any $(n+1)$ variable function with no added gates. n of the variables are used as control inputs and the remaining variable is used as required on the data inputs.

5.2 Decoders

A decoder is a combinational logic circuit that converts binary information from n input lines to a maximum of 2^n unique output lines.



5.3 Encoders

An encoder is a combinational logic circuit. It is the reverse of a decoder function. It has 2^n (or fewer) input lines and n output lines.

5.4 Canonical Logic Forms

Each term in SOP and POS forms contains all the literals. These forms are called **Canonical** (Standard) SOP and POS expressions.

5.4.1 Half Adder

$$S = A \oplus B \qquad C = AB$$

5.4.2 Full Adder

$$S = A \oplus B \oplus C_{in} \qquad C_{out} = (A \oplus B)C_{in} + AB$$

5.5 Look-Ahead Carry Adder

For a 4-bit adder:

$$\begin{aligned} S_i &= P_i \oplus C_i \qquad G_i = A_i \cdot B_i \\ C_{i+1} &= G_i + P_i C_i \end{aligned}$$

where $i = 0, 1, 2, 3$.

$$\begin{aligned} C_1 &= G_0 + P_0 C_0 \\ C_2 &= G_1 + P_1 C_1 \\ &= G_1 + P_1(G_0 + P_0 C_0) \\ C_3 &= G_2 + P_2 C_2 \\ &= G_2 + P_2(G_1 + P_1(G_0 + P_0 C_1)) \\ C_4 &= G_3 + P_3 C_3 \\ &= G_3 + P_3(G_2 + P_2(G_1 + P_1(G_0 + P_0 C_1))) \end{aligned}$$

Chapter 6

Arithmetic Circuits

6.1 Half Subtractor

$$D = A \oplus B \qquad B_{out} = \bar{A}B$$

6.2 Full Subtractor

$$D = A \oplus B \oplus B_{in} \qquad B_{out} = (A \oplus B)B_{in} + \bar{A}B$$

Chapter 7

State Machine Minimization

7.1 Reduction of State Tables

The process involves two steps:

(1) Elimination of redundant states

(2) Determination of state equivalence using an implication table

7.1.1 Example

Present State	$I=0$		$I=1$	
	Q	Z	Q	Z
A	B	0	C	1
B	C	0	A	0
C	A	0	B	1
D	A	1	C	1

Implication Table:

$$\begin{array}{cccc} & & & A \\ & B-C & & \\ C & A-D & B-C & \\ & & & \\ D & X & X & \\ \hline & A & B & \end{array}$$

Since $C \neq D$ due to output difference, the next state is different.
From the implication table:

$$A \equiv B \equiv C$$

New State Table:

	$I=0$		$I=1$	
A	A	0	B	1
B	A	0	B	1
D	A	1	B	1

Chapter 8

Code Conversion

8.1 Binary Code to Gray Code Conversion

Binary Code:

(1) (0) (1) (1) (0)

Gray Code:

(1) (0) (1) (1) (0) Same

8.2 Gray Code to Binary Code Conversion

Gray Code:

(1) (0) (1) (1) (0)

Binary Code:

(1) (0) (1) (1) (1) Same

Discrete Mathematics

Comprehensive Study Notes for GATE Computer Science

Ankur Gupta

www.ankurgupta.net

Note: These notes have been prepared primarily from the following books:

(1) **Discrete Mathematics and Its Applications** by Kenneth H. Rosen

(2) **Discrete Mathematics** by Seymour Lipschutz, Marc Lars Lipson

Please refer to the above books for more details.

Contents

1 Logic and Proofs **7**

1.1 Propositional Logic . . . 7
1.1.1 The Rules of Logic . . . 7
1.1.2 Propositions and Compound Propositions . . . 7
1.1.3 Logical Operators . . . 7
1.1.4 Dual of a Compound Proposition . . . 8
1.1.5 Conditional Statements . . . 9
1.1.6 Functionally Complete Collection of Logical Operators . . . 10
1.1.7 Biconditionals . . . 10
1.1.8 Implicit use of Biconditionals . . . 10
1.1.9 Precedence of Logical Operators . . . 10
1.1.10 Tautology . . . 10
1.1.11 Contradiction . . . 10
1.1.12 Contingency . . . 11
1.2 Logical Equivalences . . . 11
1.2.1 Logical Equivalences Involving Logical Operators . . . 11
1.2.2 Logical Equivalences Involving Conditional Statements . . . 12
1.2.3 Logical Equivalences Involving Biconditionals . . . 12
1.2.4 If — Then — Unless — Rule . . . 12
1.3 Predicates and Quantifiers . . . 12
1.3.1 Predicates . . . 12
1.3.2 Quantifiers . . . 12
1.3.3 Universal Quantifiers . . . 13
1.3.4 Existential Quantifier . . . 13
1.3.5 Quantifiers with Restricted Domains . . . 13
1.3.6 Precedence of Quantifiers . . . 14
1.3.7 Binding Variables . . . 14
1.3.8 Logical Equivalences Involving Quantifiers . . . 14
1.3.9 Negating Quantified Expressions . . . 14
1.3.10 Nested Quantifiers . . . 15
1.3.11 Negating Nested Quantifiers . . . 15
1.4 Important Points to Remember . . . 15
1.5 Example: Evaluation of the formula $\forall p \exists q(p \leftrightarrow q)$ in the interpretation $I = \{p \mapsto 1, q \mapsto 0\}$ or I_0. . . . 17

2 Set Theory **18**

2.1 Basic Set Concepts . . . 18
2.1.1 Set . . . 18

2.1.2 Equal Sets . . . 18
2.1.3 Empty Set or Null Set . . . 18
2.1.4 Singleton Set . . . 18
2.1.5 Subset . . . 19
2.1.6 Proper Subset . . . 19
2.1.7 Finite and Infinite Sets . . . 19
2.1.8 Power Set . . . 19
2.1.9 Ordered n-tuples . . . 19
2.1.10 Cartesian Product . . . 20
2.1.11 Using Set Notation with Quantifiers . . . 20
2.1.12 Truth Set of Quantifiers . . . 20
2.2 Set Operations . . . 20
2.2.1 Union . . . 20
2.2.2 Intersection . . . 20
2.2.3 Difference . . . 20
2.2.4 Complement . . . 20
2.2.5 Set Identities . . . 21
2.2.6 Symmetric Difference . . . 21
2.2.7 Principle of Inclusion-Exclusion . . . 21
2.2.8 Multisets . . . 21

3 Functions 22
3.1 Basic Function Concepts . . . 22
3.1.1 Domain and Co-domain . . . 22
3.1.2 Image and Preimage . . . 22
3.1.3 Range . . . 22
3.1.4 Equality of functions . . . 23
3.1.5 Addition and Multiplication of functions . . . 23
3.1.6 Increasing and Strictly Increasing Functions . . . 23
3.1.7 Decreasing and Strictly Decreasing Functions . . . 23
3.1.8 One-to-One Functions (Injective) . . . 23
3.1.9 Onto Functions (Surjective) . . . 24
3.1.10 One-to-One Correspondence (Bijective) . . . 24
3.1.11 Inverse Functions . . . 24
3.1.12 Composition of Functions . . . 25
3.1.13 Important Points about Functions . . . 25

4 Relations 26
4.1 Basic Relations . . . 26
4.1.1 Binary Relation . . . 26
4.1.2 Functions as Relations . . . 26
4.1.3 Relations on a Set . . . 26
4.2 Properties of Relations . . . 26
4.2.1 Reflexive Relation . . . 26
4.2.2 Symmetric Relation . . . 27
4.2.3 Antisymmetric Relation . . . 27
4.2.4 Asymmetric Relation . . . 27
4.2.5 Irreflexive Relation . . . 27

4.2.6 Transitive Relation . . . 27
4.3 Combining Relations . . . 28
4.3.1 Inverse Relation . . . 28
4.3.2 Complementary Relation . . . 28
4.3.3 Representing Relations Using Matrices . . . 28
4.3.4 Some Important Properties: . . . 28
4.3.5 Representing Relations Using Digraphs . . . 29
4.4 Closures of Relations . . . 29
4.4.1 Reflexive Closure . . . 29
4.4.2 Symmetric Closure . . . 29
4.4.3 Transitive Closure . . . 29
4.5 Equivalence Relations . . . 29
4.5.1 Equivalence Classes . . . 30
4.5.2 Refinements . . . 30
4.5.3 Partitions . . . 30
4.5.4 Refinements . . . 30

5 Order Theory and Lattices 31
5.1 Partial Orderings . . . 31
5.1.1 Comparable and Incomparable Elements . . . 31
5.1.2 Total Order . . . 31
5.1.3 Well-Ordered Set . . . 31
5.1.4 Lexicographic Order . . . 31
5.2 Hasse Diagrams . . . 32
5.2.1 Maximal Element . . . 32
5.2.2 Minimal Element . . . 32
5.2.3 Greatest Element . . . 32
5.2.4 Least Element . . . 32
5.2.5 Upper Bound and Lower Bound . . . 33
5.2.6 Least Upper Bound . . . 33
5.2.7 Greatest Lower Bound . . . 33
5.3 Lattices . . . 33
5.4 Topological Sorting . . . 34
5.4.1 Algorithm for Topological Sorting . . . 34
5.4.2 Additional Lattice Concepts . . . 34
5.4.3 Lattices and Order . . . 34
5.4.4 Sublattices . . . 34
5.4.5 Bounded Lattices . . . 35
5.4.6 Complemented Lattice . . . 35

6 Algebraic Structures 36
6.1 Groups . . . 36
6.1.1 Semi-Groups and Monoids . . . 36
6.1.2 Groups . . . 36
6.1.3 Abelian Group . . . 36
6.2 Homomorphisms . . . 36
6.2.1 Example of Homomorphism . . . 37
6.3 Cyclic Subgroups . . . 37

6.3.1 Cyclic Group 37

7 Combinatorics **38**
7.1 Permutations and Combinations 38
7.1.1 Permutation and Combination 38
7.1.2 Distributing r distinct objects into n cells: 38
7.1.3 Distributing r similar objects into n cells: 39
7.1.4 Circular Permutation 39
7.1.5 Permutation and Combination under certain conditions: 40
7.1.6 Division into groups 40
7.1.7 If some or all of n things be taken at a time, where all n things are of different kind, then the number of combinations will be: 40
7.1.8 Selection out of identical things 40
7.1.9 Gap Method 41

8 Advanced Topics **42**
8.1 Counting and Summation 42
8.1.1 Exponent of prime p in $n!$: 42
8.1.2 Summation: 42
8.2 Recurrence Relations 42
8.2.1 The Pigeonhole Principle 42
8.2.2 Recurrence Relation 43
8.2.3 Solving Recurrence Relations 43

Chapter 1

Logic and Proofs

1.1 Propositional Logic

1.1.1 The Rules of Logic

▶ The rules of logic specify the meaning of mathematical statements.
▶ Once we prove a mathematical statement is true, we call it a **theorem**.
▶ A correct mathematical argument about a theorem is a **proof**.

1.1.2 Propositions and Compound Propositions

▶ The rules of logic give precise meaning to mathematical statements. These rules are used to distinguish between valid and invalid mathematical arguments.
▶ A proposition is a declarative sentence that is either true or false, but not both.
E.g.:

(i) $1 + 1 = 2$ is a proposition

(ii) What time is it? is not a proposition

▶ The area of logic that deals with propositions is called the propositional calculus or propositional logic.
▶ New propositions, called **compound propositions**, are formed from existing propositions using logical operators.

1.1.3 Logical Operators

Negation

$\neg p$, $\bar{p}$ not p

p	$\neg p$
T	F
F	T

Conjunction

$p \wedge q$, p and q

p	q	$p \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F

▶ Sometimes the word "but" is used instead of "and" in a conjunction.
E.g.: The sun is shining, but it is raining.

Disjunction (Inclusive OR)

$p \vee q$, p or q

p	q	$p \vee q$
T	T	T
T	F	T
F	T	T
F	F	F

▶ When used in one of two ways, the word "or" is used in English in an inclusive way.

Exclusive OR

$p \oplus q$ $[p \oplus q \equiv (p \wedge \neg q) \vee (\neg p \wedge q)]$

p	q	$p \oplus q$
T	T	F
T	F	T
F	T	T
F	F	F

E.g.: Students who have taken calculus or computer science can take this class.
▶ The logical operators Conjunction, Disjunction and Exclusive OR are called as connectives, since they form new propositions using two or more existing propositions.

1.1.4 Dual of a Compound Proposition

The dual of a compound proposition that contains only the logical operators $\vee$, $\wedge$ and $\neg$ is the compound proposition obtained by replacing each $\vee$ by $\wedge$, each $\wedge$ by $\vee$, each T by F, and each F by T. The dual of S is denoted by S^*.

Duals of two equivalent compound propositions:
Two compound propositions that are equivalent have duals that are also equivalent. For instance, $S \Rightarrow \bar{p} \wedge T \equiv \bar{p}$ and $S^* \Rightarrow p \vee F \equiv p$

$$S \Rightarrow \neg(p \wedge q) \equiv \neg p \vee \neg q$$
$$S^* \Rightarrow \neg(p \vee q) \equiv \neg p \wedge \neg q$$

1.1.5 Conditional Statements

$p \to q$ / if p, then q
► A conditional statement is also called an implication.
► Here p is called the hypothesis or antecedent or premise and q is called the conclusion or consequence.

p	q	$p \to q$
T	T	T
T	F	F
F	T	T
F	F	T

Ways to express the conditional statement $(p \to q)$**:**

(i) if p, then q

(ii) p implies q

(iii) if p, q

(iv) p only if q

(v) p is sufficient for q

(vi) a sufficient condition for q is p

(vii) q if p

(viii) q whenever p

(ix) (ix) q when p

(x) (x) q is necessary for p

(xi) (xi) a necessary condition for p is q

(xii) (xii) q follows from p

(xiii) (xiii) q unless $\neg p$

► "p only if q" says that p cannot be true when q is not true. That is, the statement is false if p is true, but q is false.
► "q unless $\neg p$" means that if $\neg p$ is false, then q must be true.

Converse:

The proposition $q \to p$ is called the converse of $p \to q$.

Contrapositive:

The proposition $\neg q \to \neg p$ is called the contrapositive of $p \to q$.

Inverse:

The proposition $\neg p \rightarrow \neg q$ is called the inverse of $p \rightarrow q$.
▶ A conditional statement and its contrapositive are equivalent.
▶ The converse and the inverse of a conditional statement are also equivalent.

1.1.6 Functionally Complete Collection of Logical Operators

A collection of logical operators is called functionally complete if every compound proposition is logically equivalent to a compound proposition involving only these logical operators.

E.g.: $\{\neg, \wedge\}$, $\{\neg, \vee\}$, $\{NAND\}$, $\{NOR\}$

1.1.7 Biconditionals

$p \leftrightarrow q$ / p if and only if q

p	q	$p \leftrightarrow q$
T	T	T
T	F	F
F	T	F
F	F	T

Other ways to express $p \leftrightarrow q$:

(i) p is necessary and sufficient for q

(ii) if p then q, and conversely

(iii) p iff q

▶ $p \leftrightarrow q$ has exactly the same truth value as $(p \rightarrow q) \wedge (q \rightarrow p)$.

1.1.8 Implicit use of Biconditionals

Biconditionals are often expressed using an "if, then" or an "only if" construction in natural language. The other part of the "if and only if" is implied, but not explicitly stated.

1.1.9 Precedence of Logical Operators

$$\neg > \wedge > \vee > \rightarrow > \leftrightarrow$$

1.1.10 Tautology

A compound proposition that is always true, no matter what the truth values of the propositions that occur in it, is called a tautology.

1.1.11 Contradiction

A compound proposition that is always false is called a contradiction.

1.1.12 Contingency

A compound proposition that is neither a tautology nor a contradiction is called a contingency.

Tautology	$\Rightarrow$ Valid
Contingency	$\Rightarrow$ Satisfiable
Contradiction	$\Rightarrow$ Unsatisfiable

1.2 Logical Equivalences

The compound propositions p and q are called logically equivalent if $p \leftrightarrow q$ is a tautology.

OR

Compound propositions that have the same truth values in all possible cases are called logically equivalent.

▶ The notation $p \equiv q$ denotes that p and q are logically equivalent. Sometimes, the symbol $\Leftrightarrow$ is also used.

1.2.1 Logical Equivalences Involving Logical Operators

Equivalence	**Name**
$p \wedge T \equiv p$ $p \vee F \equiv p$	Identity Laws
$p \vee T \equiv T$ $p \wedge F \equiv F$	Domination Laws
$p \vee p \equiv p$ $p \wedge p \equiv p$	Idempotent Laws
$\neg(\neg p) \equiv p$	Double Negation Law
$p \vee q \equiv q \vee p$ $p \wedge q \equiv q \wedge p$	Commutative Laws
$(p \vee q) \vee r \equiv p \vee (q \vee r)$ $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$	Associative Laws
$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$ $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$	Distributive Laws
$\neg(p \wedge q) \equiv \neg p \vee \neg q$ $\neg(p \vee q) \equiv \neg p \wedge \neg q$	De Morgan's Laws
$p \vee (p \wedge q) \equiv p$ $p \wedge (p \vee q) \equiv p$	Absorption Laws
$p \vee \neg p \equiv T$ $p \wedge \neg p \equiv F$	Negation Laws

1.2.2 Logical Equivalences Involving Conditional Statements

$$
\begin{aligned}
p \to q &\equiv \neg p \vee q \\
p \to q &\equiv \neg q \to \neg p \\
p \vee q &\equiv \neg p \to q \\
p \wedge q &\equiv \neg(p \to \neg q) \\
\neg(p \to q) &\equiv p \wedge \neg q \\
(p \to q) \wedge (p \to r) &\equiv p \to (q \wedge r) \\
(p \to r) \wedge (q \to r) &\equiv (p \vee q) \to r \\
(p \to q) \vee (p \to r) &\equiv p \to (q \vee r) \\
(p \to r) \vee (q \to r) &\equiv (p \wedge q) \to r
\end{aligned}
$$

1.2.3 Logical Equivalences Involving Biconditionals

$$
\begin{aligned}
p \leftrightarrow q &\equiv (p \to q) \wedge (q \to p) \\
p \leftrightarrow q &\equiv \neg p \leftrightarrow \neg q \\
p \leftrightarrow q &\equiv (p \wedge q) \vee (\neg p \wedge \neg q) \\
(p \leftrightarrow q) &\equiv \neg p \leftrightarrow \neg q \\
p \leftrightarrow q &\equiv \neg(p \oplus q)
\end{aligned}
$$

1.2.4 If — Then — Unless — Rule

If x is A then y is B unless z is C.
$\Rightarrow$ If x is A and z is not C then y is B.
$\Rightarrow$ If x is A and z is C then y is not B.
If X, then Y unless Z
$\Rightarrow (X \wedge \neg Z) \to Y$
$\Rightarrow X \to (\neg Z \to Y)$

1.3 Predicates and Quantifiers

1.3.1 Predicates

We can denote the statement "x is greater than 3" by $P(x)$, where P denotes the predicate "is greater than" and x is a variable.
▶ A statement of the form $P(x_1, x_2, \ldots, x_n)$ is the value of the propositional function P at the n-tuple $(x_1, x_2, \ldots, x_n)$, and P is also called an n-place predicate or an n-ary predicate.
▶ The area of logic that deals with predicates and quantifiers is called the predicate calculus.

1.3.2 Quantifiers

Quantification expresses the extent to which a predicate is true over a range of elements.
In English, the words all, some, many, none, and few are used in quantifications.

1.3.3 Universal Quantifiers

The universal quantification of $P(x)$ is:

$\forall x P(x)$ / for all $xP(x)$ / for every x $P(x)$

$\rightarrow$ An element for which $P(x)$ is false is called a counterexample of $\forall x P(x)$.

▶ If the domain x is empty, then $\forall x P(x)$ is true for any propositional function $P(x)$ because there are no elements x in the domain for which $P(x)$ is false.

Other ways to express 'Universal Quantification':

all of / for each / given any / for arbitrary / for any.

▶ When all the elements in the domain can be listed - say $x_1, x_2, \ldots x_n$ - it follows that the universal quantification $\forall x P(x)$ is the same as the conjunction:

$$P(x_1) \wedge P(x_2) \wedge \cdots \wedge P(x_n)$$

because this conjunction is true if and only if $P(x_1), P(x_2), \ldots P(x_n)$ are all true.

1.3.4 Existential Quantifier

The existential quantification of $P(x)$ is: "There exists an element x such that $P(x)$" or $\exists x P(x)$.

Other ways to express 'Existential Quantifier':

for some / for at least one / there is

▶ If the domain x is empty, then $\exists x Q(x)$ is false whenever $Q(x)$ is a propositional function because when the domain is empty, there can be no element x in the domain for which $Q(x)$ is true.

▶ When all elements in the domain can be listed - say $x_1, x_2, \ldots x_n$ - the existential quantifier $\exists x P(x)$ is the same as the disjunction:

$P(x_1) \vee P(x_2) \vee \cdots \vee P(x_n)$

because this disjunction is true if and only if at least one of the $P(x_1), P(x_2), \ldots, P(x_n)$ is true.

1.3.5 Quantifiers with Restricted Domains

E.g.: $\forall x < 0\ (x^2 > 0)$

$\exists z > 0\ (z^2 = 2)$.

▶ The restriction of a universal quantification is the same as the universal quantification of a conditional statement.

$\forall x < 0\ (x^2 > 0) \equiv \forall x\ (x < 0 \rightarrow x^2 > 0)$

▶ The restriction of an existential quantification is the same as the existential quantification of a conjunction.

$\exists z > 0\ (z^2 = 2) \equiv \exists z\ (z > 0 \wedge z^2 = 2)$

1.3.6 Precedence of Quantifiers

The quantifiers $\forall$ and $\exists$ have higher precedence than all logical operators from propositional calculus.

E.g.: $\forall x P(x) \vee Q(x)$ is equivalent to $(\forall x P(x)) \vee Q(x)$

1.3.7 Binding Variables

▶ When a quantifier is used on the variable x, we say that this occurrence of the variable is bound.
▶ An occurrence of a variable that is not bound by a quantifier or set equal to a particular value is said to be free.
▶ The part of a logical expression to which a quantifier is applied is called the scope of this quantifier. Consequently, a variable is free if it is outside the scope of all quantifiers in the formula that specifies this variable.

E.g.:

(i) $\exists x(x + y = 1)$

(i) $\rightarrow x$ is bound

(ii) $\rightarrow y$ is free

(ii) $\exists x(P(x) \wedge Q(x)) \vee \forall x R(x)$

- $\rightarrow$ Scope of first quantifier, $\exists x$, is the expression $P(x) \wedge Q(x)$.
- $\rightarrow$ Scope of second quantifier $\forall x$ is the expression $R(x)$.

Note:
In common usage, the same letter is often used to represent variables bound by different quantifiers with scopes that do not overlap.

1.3.8 Logical Equivalences Involving Quantifiers

$$\forall x(P(x) \wedge Q(x)) \equiv \forall x P(x) \wedge \forall x Q(x)$$
$$\exists x(P(x) \vee Q(x)) \equiv \exists x P(x) \vee \exists x Q(x)$$
$$\forall x(P(x) \vee Q(x)) \not\equiv \forall x P(x) \vee \forall x Q(x)$$
$$\exists x(P(x) \wedge Q(x)) \not\equiv \exists x P(x) \wedge \exists x Q(x)$$

Some Important Results:

$$\forall x P(x) \vee \forall x Q(x) \rightarrow \forall x(P(x) \vee Q(x))$$
$$\exists x(P(x) \wedge Q(x)) \rightarrow \exists x P(x) \wedge \exists x Q(x)$$
$$\forall x(P(x) \rightarrow Q(x)) \rightarrow \forall x P(x) \rightarrow \forall x Q(x)$$

1.3.9 Negating Quantified Expressions

$$\neg\forall x P(x) \equiv \exists x \neg P(x) \quad \text{De Morgan's Laws}$$
$$\neg\exists x P(x) \equiv \forall x \neg P(x) \quad \text{for quantifiers}$$

1.3.10 Nested Quantifiers

Two quantifiers are nested if one is within the scope of the other, such as:

$\forall x \exists y(x + y = 0)$

The above statement is same as $\forall x Q(x)$, where $Q(x)$ is $\exists y P(x, y)$, where $P(x, y)$ is $x + y = 0$.

The order of Quantifiers:

The order of the quantifiers is important, unless all the quantifiers are universal quantifiers or all are existential quantifiers.

$$\forall x \forall y P(x, y) \equiv \forall y \forall x P(x, y)$$
$$\exists x \exists y P(x, y) \equiv \exists y \exists x P(x, y)$$
$$\forall x \exists y P(x, y) \not\equiv \exists y \forall x P(x, y)$$
$$\exists y \forall x P(x, y) \rightarrow \forall x \exists y P(x, y)$$

1.3.11 Negating Nested Quantifiers

Successively applying the rules for negating statements involving a single quantifier.

E.g.: $\neg \forall x \exists y \forall z P(x, y, z)$

$$\equiv \exists x \neg \exists y \forall z P(x, y, z)$$
$$\equiv \exists x \forall y \neg \forall z P(x, y, z)$$
$$\equiv \exists x \forall y \exists z \neg P(x, y, z)$$

1.4 Important Points to Remember

(1) How many different truth tables of compound propositions are there that involve the propositional variables p and q?

Answer: $2^{2^2} = 16$ (2^n) where n is number of propositional variables

(2) Disjunctive Normal Form or Sum of Products:

$(p \wedge q \wedge r) \vee (\bar{p} \wedge \neg q \wedge r) \vee (\bar{p} \wedge q \wedge \neg r)$

(3) Conjunctive Normal Form or Product of Sums:

$(p \vee q \vee s) \wedge (p \vee \neg q \vee s) \wedge (\bar{p} \vee \neg q \vee \neg s)$ - - -

(4) Satisfiable:

A compound proposition is satisfiable if there is an assignment of truth values to its variables that makes the compound proposition true.

(5) Some important translations from English language statements into logical expressions:

(i) No one is perfect.

$$\forall x \neg P(x)$$

(ii) Not everyone is perfect.

$$\neg \forall x P(x) \ / \ \exists x \neg P(x)$$

(iii) All your friends are perfect.

$$\forall x(F(x) \rightarrow P(x))$$

(iv) At least one of your friends is perfect.

$$\exists x(F(x) \wedge P(x))$$

(v) Everyone is your friend and is perfect.

$$\forall x(F(x) \wedge P(x))$$

(vi) The negation of a contradiction is a tautology.

$$\forall x(C(x) \rightarrow \neg T(x))$$

(vii) The disjunction of two contingencies can be a tautology.

$$\exists x \exists y(\neg T(x) \wedge \neg C(x) \wedge \neg T(y) \wedge \neg C(y) \wedge T(x \vee y))$$

(viii) The conjunction of two tautologies is a tautology.

$$\forall x \forall y((T(x) \wedge T(y)) \rightarrow T(x \wedge y))$$

► **(5)** $\exists[xP(x)]$ (Used for unique)
→ There exists a unique x such that $P(x)$ is true.
(7) Valid:
A well formed formula (wff) is called valid, if it is true for all assignment of truth values to the variables in wff.
(8) Closed Formula:
A formula with no free variables.
(9) Only Free Variables Matter:
The truth values of a formula depends only on the truth values of free variables of the formula.
► The truth value of a propositional formula (F) in an interpretation I depends on the truth values of all variables p occurring in F.
► The truth value of a quantified boolean formula in an interpretation I does not depend on the values in I of variables occurring in quantifiers, called bound variables.
► $I \models A$ represents interpretation I satisfies formula A.
(10) Satisfiability of quantified formulas:
► For every interpretation I and closed formula F of the following propositions are equivalent:

(i) $I \models F$

(ii) F is satisfiable

(iii) F is valid

► The values of quantified boolean formulae does not depend on the values of its quantified (bound) variables.

1.5 Example: Evaluation of the formula $\forall p \exists q(p \leftrightarrow q)$ in the interpretation $I = \{p \mapsto 1, q \mapsto 0\}$ or I_0.

$I_{pq} \models \forall p \exists q(p \leftrightarrow q)$

$I_{0q} \models \exists q(p \leftrightarrow q)$ $\qquad$ $I_{1q} \models \exists q(p \leftrightarrow q)$

$I_{00} \models (p \leftrightarrow q)$ $\quad$ $I_{01} \models (p \leftrightarrow q)$ $\quad$ $I_{10} \models (p \leftrightarrow q)$ $\quad$ $I_{11} \models (p \leftrightarrow q)$

$\Rightarrow I_{pq} = [(I_{00} \vee I_{01}) \wedge (I_{10} \vee I_{11})]$
(for all p, q)

Chapter 2

Set Theory

2.1 Basic Set Concepts

2.1.1 Set

A set is an unordered collection of objects.
▶ The objects in a set are called the elements or members of the set.

- $a \in A \Rightarrow a$ is an element of the set A
- $a \notin A \Rightarrow a$ is not an element of the set A

Some Important Sets:

$$
\begin{aligned}
N &= \{0, 1, 2, \ldots\}, \text{ the set of natural numbers} \\
Z &= \{\ldots, -2, -1, 0, 1, 2, \ldots\}, \text{ the set of integers} \\
Z^+ &= \{1, 2, 3, \ldots\}, \text{ the set of positive integers} \\
Q &= \{p/q \mid p \in Z, q \in Z \text{ and } q \neq 0\}, \text{ the set of rational numbers} \\
R &= \text{the set of real numbers}
\end{aligned}
$$

Note: Some people do not consider 0 a natural number.
▶ Sets can have other sets as members.

2.1.2 Equal Sets

Two sets are equal if and only if they have the same elements. We write $A = B$ if A and B are equal sets.

2.1.3 Empty Set or Null Set

An empty set (or null set) is a set that has no elements, denoted by $\emptyset$ or $\{\}$.

2.1.4 Singleton Set

A singleton set is a set with only one element.

2.1.5 Subset

A set A is said to be a subset of B if and only if every element of A is also an element of B.

$A \subseteq B$

▶ $A \subseteq B$ if and only if $\forall x(x \in A \rightarrow x \in B)$

▶ For every set S:

(i) $\emptyset \subseteq S$

(ii) $S \subseteq S$

Important:

- $\emptyset \notin \{0, 1, \ldots\}$
- $\emptyset \in \{\emptyset, 0, 1, \ldots\}$
- $\emptyset \subset \{0, 1, \ldots\}$

2.1.6 Proper Subset

If a set A is a subset of the set B, but that $A \neq B$, we write $A \subset B$ and say that A is a proper subset of B.

▶ $A \subset B$, if

$\forall x(x \in A \rightarrow x \in B) \wedge \exists x(x \in B \wedge x \notin A)$ is true.

2.1.7 Finite and Infinite Sets

A set S with exactly n distinct elements, where n is a non-negative integer, is called a finite set. Here n is the cardinality of S. The cardinality of S is denoted by $|S|$.

A set is said to be infinite if it is not finite.

2.1.8 Power Set

Given a set S, the power set of S is the set of all subsets of the set S. The power set of S is denoted by $P(S)$.

▶ If a set has n elements, then its power set has 2^n elements.

2.1.9 Ordered n-tuples

The ordered n-tuple $(a_1, a_2, \ldots, a_n)$ is the ordered collection that has a_1 as its first element, a_2 as its second element, and so on up to a_n as its nth element.

▶ Two ordered n-tuples are equal if and only if each corresponding pair of their elements is equal.

▶ 2-tuples are called ordered pairs. E.g.: (a, b).

2.1.10 Cartesian Product

The cartesian product of the sets $A_1, A_2, \ldots, A_n$, denoted by $A_1 \times A_2 \times \ldots \times A_n$, is the set of ordered n-tuples $(a_1, a_2, \ldots, a_n)$, where a_i belongs to A_i for $i = 1, 2, \ldots n$.

$A_1 \times A_2 \times \ldots \times A_n = \{(a_1, a_2, \ldots, a_n) \mid a_i \in A_i \text{ for } i = 1, 2, \ldots, n\}$

or $A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$

2.1.11 Using Set Notation with Quantifiers

(a) $\forall x \in S(P(x))$

$\rightarrow$ It is shorthand for $\forall x(x \in S \rightarrow P(x))$

(b) $\exists x \in S(P(x))$

$\rightarrow$ It is shorthand for $\exists x(x \in S \wedge P(x))$

▶ $A \times \emptyset = \emptyset \times A = \emptyset$

2.1.12 Truth Set of Quantifiers

Given a predicate P, and a domain D, the truth set of P is the set of elements x in D for which $P(x)$ is true. The truth set of $P(x)$ is denoted by:

$\{x \in D \mid P(x)\}$

▶ $\forall x P(x)$ is true over the domain U if and only if the truth set of P is U.

▶ $\exists x P(x)$ is true over the domain U if and only if the truth set of P is nonempty.

2.2 Set Operations

2.2.1 Union

$A \cup B = \{x \mid x \in A \vee x \in B\}$

2.2.2 Intersection

$A \cap B = \{x \mid x \in A \wedge x \in B\}$

2.2.3 Difference

The difference of sets A and B, denoted by $A - B$, is the set containing those elements that are in A but not in B.

The difference of A and B is also called the complement of B with respect to A.

$A - B = \{x \mid x \in A \wedge x \notin B\}$

2.2.4 Complement

Let U be the universal set. The complement of the set, denoted by $\bar{A}$, is the complement of A with respect to U.

$\bar{A} = U - A$

or

$\bar{A} = \{x \mid x \notin A\}$

▶ Two sets are called disjoint if their intersection is empty set.

2.2.5 Set Identities

Identity	Name
$A \cup \phi = A$ $A \cap U = A$	Identity Laws
$A \cup U = U$ $A \cap \phi = \emptyset$	Domination Laws
$A \cup A = A$ $A \cap A = A$	Idempotent Laws
$\overline{(\bar{A})} = A$	Complementation Laws
$A \cup B = B \cup A$ $A \cap B = B \cap A$	Commutative Laws
$A \cup (B \cup C) = (A \cup B) \cup C$ $A \cap (B \cap C) = (A \cap B) \cap C$	Associative Laws
$\overline{(A)} = A$	
$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$	Distributive Laws
$\overline{A \cup B} = \bar{A} \cap \bar{B}$ $\overline{A \cap B} = \bar{A} \cup \bar{B}$	De Morgan's Laws
$A \cup (A \cap B) = A$ $A \cap (A \cup B) = A$	Absorption Laws
$A \cup \bar{A} = U$ $A \cap \bar{A} = \emptyset$	Complement Laws

2.2.6 Symmetric Difference

The symmetric difference of A and B, denoted by $A \oplus B$, is the set containing those elements in either A or B, but not in both A and B.

$A \oplus B = (A \cup B) - (A \cap B)$
$A \oplus B = (A - B) \cup (B - A)$
$A \oplus (B \oplus C) = (A \oplus B) \oplus C$

2.2.7 Principle of Inclusion-Exclusion

$|A \cup B| = |A| + |B| - |A \cap B|$
$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |B \cap C|$
$-(C \cap A) + |A \cap B \cap C|$

2.2.8 Multisets

Multisets are unordered collections of elements where an element can occur as a member more than once.

→ The union of the multisets P and Q is the multiset where the multiplicity of an element is the maximum of its multiplicities in P and Q. (PUQ)

→ The intersection of P and Q is the multiset where the multiplicity of an element is the minimum of its multiplicities in P and Q. (PNQ)

Chapter 3

Functions

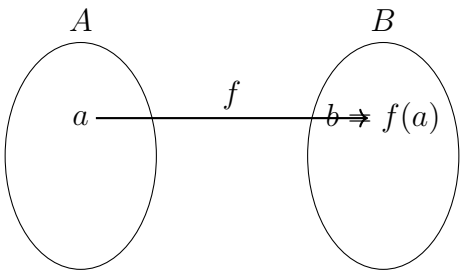
3.1 Basic Function Concepts

Function:

Let A and B be nonempty sets. A function f from A to B is an assignment of exactly one element of B to each element of A. We write $f(a) = b$ if b is the unique element of B assigned by the function f to the element a of A.

If f is a function from A to B, we write $f : A \to B$, and say: f maps A to B.

$\to$ Functions are sometimes also called mappings or transformations.



3.1.1 Domain and Co-domain

If f is a function from A to B, we say that A is the domain of f and B is the codomain of f.

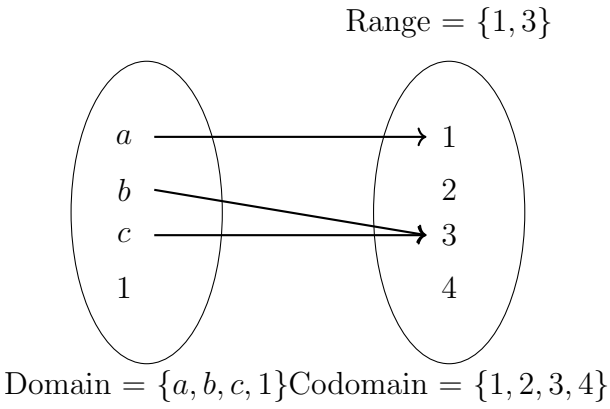
3.1.2 Image and Preimage

If $f(a) = b$, we say that b is the image of a and a is the preimage of b.

3.1.3 Range

The range of f is the set of all images of elements of A.

Range $\subseteq$ Codomain



3.1.4 Equality of functions

Two functions are equal when they have the same domain, have the same codomain and map elements of their common domain to the same elements in their common codomain.

3.1.5 Addition and Multiplication of functions

Let f_1 and f_2 be functions from A to R. Then $f_1 + f_2$ and $f_1 f_2$ are also functions from A to R defined by:

$(f_1 + f_2)(x) = f_1(x) + f_2(x)$.

$(f_1 f_2)(x) = f_1(x) f_2(x)$

3.1.6 Increasing and Strictly Increasing Functions

A function f is called increasing if $f(x) \leq f(y)$, and strictly increasing if $f(x) < f(y)$, whenever $x < y$ and x and y are in the domain of f.

Increasing: $\forall x \forall y (x < y \rightarrow f(x) \leq f(y))$

Strictly Increasing: $\forall x \forall y (x < y \rightarrow f(x) < f(y))$

3.1.7 Decreasing and Strictly Decreasing Functions

A function f is called decreasing if $f(x) \geq f(y)$, and strictly decreasing if $f(x) > f(y)$, whenever $x < y$ and x and y are in the domain of f.

Decreasing: $\forall x \forall y (x < y \rightarrow f(x) \geq f(y))$

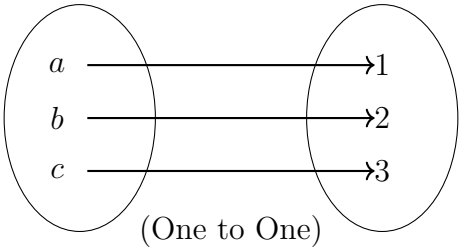
Strictly Decreasing: $\forall x \forall y (x < y \rightarrow f(x) > f(y))$

▶ Here domain and codomain are subsets of the set of real numbers.

3.1.8 One-to-One Functions (Injective)

A function f is said to be one-to-one, or injective, if and only if $f(a) = f(b)$ implies that $a = b$ for all a and b in the domain of f.

$\forall a \forall b (f(a) = f(b) \rightarrow a = b)$



(One to One)

▶ A function that is either strictly increasing or strictly decreasing must be one-to-one.
▶ A function $f : R \to R$ is increasing, but not strictly increasing, is not necessarily one-to-one.

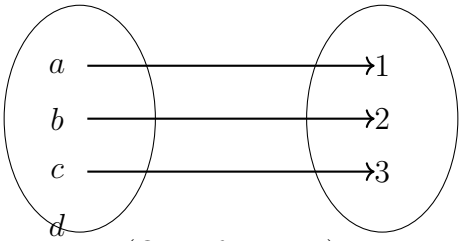
→ One to One functions never assign the same value to two different domain elements.

3.1.9 Onto Functions (Surjective)

A function f from A to B is called onto, or surjective, if and only if for every element $b \in B$, there is an element $a \in A$ with $f(a) = b$.

$\forall y \exists x(f(x) = y)$

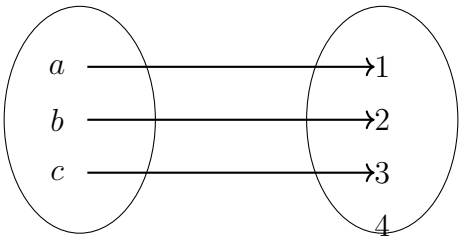
▶ For onto functions, the range and the codomain are equal, i.e. every member of the codomain is the image of some element of the domain.



(Onto function)

3.1.10 One-to-One Correspondence (Bijective)

A function f is a one-to-one correspondence, or a bijection, if it is both one-to-one and onto.



▶ $f(x) = x^3$ (from R to R) is a bijection.

3.1.11 Inverse Functions

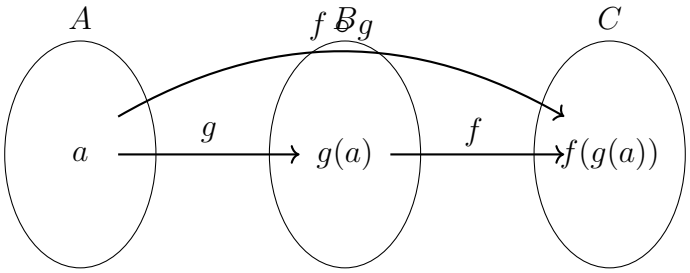
Let f be a one-to-one correspondence from the set A to the set B. The inverse function of f is the function that assigns to an element b belonging to B, the unique element a in A, such that $f(a) = b$. The inverse function of f is denoted by f^{-1}.

Hence, $f^{-1}(b) = a$, when $f(a) = b$.

▶ $f^{-1} \neq 1/f$
▶ If a function f is not a one-to-one correspondence, we cannot define an inverse function of f.
▶ A one-to-one correspondence is called invertible because we can define an inverse of this function.

3.1.12 Composition of Functions

Let g be a function from the set A to the set B and let f be a function from the set B to the set C. The composition of the functions f and g, denoted by $f \circ g$, is defined by:
$(f \circ g)(a) = f(g(a))$



▶ $f \circ g \neq g \circ f$
▶ $f^{-1} \circ f = f^{-1}(f(a)) = f^{-1}(b) = a$
▶ $f \circ f^{-1} = f(f^{-1}(b)) = f(a) = b$
▶ $(f^{-1})^{-1} = f$
Note:

(i) If both f and g are one-to-one functions, then $f \circ g$ is also one-to-one.

(ii) If both f and g are onto functions, then $f \circ g$ is also onto.

(iii) If f and $f \circ g$ are one-to-one, then g is also one-to-one.

(iv) If f and $f \circ g$ are onto, then it does not mean that g is also onto.

3.1.13 Important Points about Functions

(1) Let f be a function from the set A to set B. Let S and T be subsets of A, then:

$$f(S \cup T) = f(S) \cup f(T)$$
$$f(S \cap T) \subseteq f(S) \cap f(T)$$

(2) No. of functions or N_f on a set with n elements:
$N_f = m^n$
(3) No. of functions N_f from a set A with n elements to another set B with m elements:
$N_f = m^n$

Chapter 4

Relations

4.1 Basic Relations

4.1.1 Binary Relation

Let A and B be sets. A binary relation from A to B is a subset of $A \times B$.

$aRb \quad \Leftrightarrow \quad (a, b) \in R$

$aKb \quad \Leftrightarrow \quad (a, b) \notin R$

4.1.2 Functions as Relations

▶ A function is always a relation, but a relation can be a function or not.

▶ A relation can be used to express a one-to-many relationship between the elements of the set A and set B, while a function represents a relation where exactly one element of B is related to each element of A.

4.1.3 Relations on a Set

A relation on the set A is a relation from A to A.

Important:

▶ How many relations are there on a set with n elements?

Answer: A relation on a set A is a subset of $A \times A$. Because $A \times A$ has n^2 elements and a set with m elements has 2^m subsets, there are 2^{n^2} subsets of $A \times A$.

Thus there are 2^{n^2} relations on a set with n elements.

▶ $2^{n^2} > n^n$ (No. of relations > No. of functions)

4.2 Properties of Relations

4.2.1 Reflexive Relation

A relation R on a set A is called reflexive, if $(a, a) \in R$ for every element $a \in A$. $[\forall a((a, a) \in R)]$

4.2.2 Symmetric Relation

A relation R on a set A is called symmetric if $(b, a) \in R$ whenever $(a, b) \in R$, for all $a, b \in A$.

$\forall a \forall b((a, b) \in R \rightarrow (b, a) \in R)$

4.2.3 Antisymmetric Relation

A relation R on a set A such that for all $a, b \in A$, if $(a, b) \in R$ and $(b, a) \in R$, then $a = b$ is called antisymmetric.

$\forall a \forall b(((a, b) \in R \wedge (b, a) \in R) \rightarrow (a = b))$

▶ The terms symmetric and antisymmetric are not opposite, because a relation can have both of these properties or may lack both.

4.2.4 Asymmetric Relation

An asymmetric relation is a binary relation which is not a symmetric relation.

OR

A relation R on a set A is called asymmetric, if $(b, a) \notin R$, whenever $(a, b) \in R$ for all $a, b \in A$.

$\forall a \forall b((a, b) \in R \rightarrow \neg(b, a) \in R)$

▶ A relation is asymmetric if and only if it is both antisymmetric and irreflexive.

4.2.5 Irreflexive Relation

A relation R on a set A is called irreflexive if for every $a \in A$, $(a, a) \notin R$. That is, R is irreflexive if no element in A is related to itself.

$\forall a((a, a) \notin R)$

▶ A relation on a set can be neither reflexive nor irreflexive.

4.2.6 Transitive Relation

A relation R on a set A is called transitive if whenever $(a, b) \in R$ and $(b, c) \in R$, then $(a, c) \in R$, for all $a, b, c \in A$.

$\forall a \forall b \forall c(((a, b) \in R \wedge (b, c) \in R) \rightarrow (a, c) \in R)$

▶ How many relations are there on a set with n elements, that are:

(i) Reflexive $\rightarrow 2^{n(n-1)}$

(ii) Symmetric $\rightarrow 2^{n(n+1)/2}$

(iii) Antisymmetric $\rightarrow$

(iv) Asymmetric $\rightarrow$

(v) Irreflexive $\rightarrow 2^{n(n-1)}$

(vi) Reflexive & Symmetric $\rightarrow 2^{n(n+1)/2}$

(vii) Neither reflexive nor irreflexive $\rightarrow$

▶ No. of different relations from a set with m elements to a set with n elements $\rightarrow 2^{mn}$

4.3 Combining Relations

Let R be a relation from a set A to a set B and S a relation from B to a set C. The composition of R and S is the relation consisting of ordered pairs (a, c), where $a \in A$, $c \in C$, and for which there exist an element $b \in B$ such that $(a, b) \in R$ and $(b, c) \in S$. We denote the composition of R and S by $S \circ R$.

▶ Let R be a relation on a set A. The powers R^n, $n = 1, 2, 3, \ldots$, are defined recursively by:

$R^1 = R$ and $R^{n+1} = R^n \circ R$.

▶ The relation R on a set A is transitive if and only if $R^n \subseteq R$ for $n = 1, 2, 3, \ldots$

4.3.1 Inverse Relation

Let R be a relation from a set A to a set B. The inverse relation from B to A, denoted by R^{-1}, is the set of ordered pairs:

$\{(b, a) \mid (a, b) \in R\}$

4.3.2 Complementary Relation

The complementary relation $\bar{R}$ is the set of ordered pairs $\{(a, b) \mid (a, b) \notin R\}$

4.3.3 Representing Relations Using Matrices

Let R be a relation from A to B. The relation R can be represented by the matrix $M_R = [m_{ij}]$, where:

$$m_{ij} = \begin{cases} 1 & \text{if } (a_i, b_j) \in R \\ 0 & \text{if } (a_i, b_j) \notin R \end{cases}$$

▶ The matrix of a relation on a set, which is a square matrix, can be used to determine whether the relation has certain properties.

▶ A relation R is reflexive if all the elements on the main diagonal of M_R are equal to 1.

▶ A relation R is irreflexive if all the elements on the main diagonal of M_R are equal to 0.

▶ A relation R is symmetric if and only if:

$m_{ij} = m_{ji}$ or $M_R = (M_R)^T$

▶ A relation R is antisymmetric if and only if, either $m_{ij} = 0$ or $m_{ji} = 0$, when $i \neq j$.

4.3.4 Some Important Properties:

(i) $M_{R_1 \cup R_2} = M_{R_1} \vee M_{R_2}$

(ii) $M_{R_1 \cap R_2} = M_{R_1} \wedge M_{R_2}$

(iii) $M_{S \circ R} = M_R \odot M_S$

4.3.5 Representing Relations Using Digraphs

▶ A relation is reflexive if and only if there is a loop at every vertex of the directed graph.
▶ A relation is irreflexive if and only if there is no loop at any vertex of the directed graph.
▶ A relation is symmetric if and only if for every edge between distinct vertices in its digraph, there is an edge in the opposite direction.
▶ A relation is antisymmetric if and only if there are never two edges in opposite directions between distinct vertices.
▶ A relation is transitive if and only if whenever there is an edge from x to y and an edge from y to z, there is an edge from x to z.

4.4 Closures of Relations

4.4.1 Reflexive Closure

The reflexive closure of R equals $R \cup \Delta$, where $\Delta = \{(a, a) \mid a \in A\}$ is the diagonal relation on A.

4.4.2 Symmetric Closure

The symmetric closure of R equals $R \cup R^{-1}$ where:
$R^{-1} = \{(b, a) \mid (a, b) \in R\}$

4.4.3 Transitive Closure

The transitive closure of a relation R equals the connectivity relation R^*, where:

$$R^* = \bigcup_{i=1}^{\infty} R^i$$

Here R^* consists of the pairs (a, b) such that there is a path of length n from a to b in R
or

$$M_{R^*} = M_R \vee M_R^{(2)} \vee M_R^{(3)} \vee \ldots \vee M_R^{(n)}$$

where $n =$ number of elements in matrix.

4.5 Equivalence Relations

A relation on a set A is called an equivalence relation if it is reflexive, symmetric and transitive.

If two elements a and b that are related by an equivalence relation are called equivalent. Its represented by $a \sim b$.

4.5.1 Equivalence Classes

Let R be an equivalence relation on a set A. The set of all elements that are related to an element a of A is called the equivalence class of a.

The equivalence class of a with respect to R is denoted by $[a]_R$.

Or

If R is an equivalence relation on a set A, the equivalence class of the element a is:

$[a]_R = \{s \mid (a, s) \in R\}$

▶ Intersection of two equivalence relations on a set is also an equivalence relation.

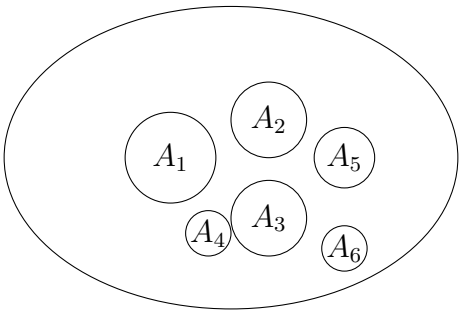
- i.e. $R \rightarrow$ Equivalence relation on A
- $S \rightarrow$ Equivalence relation on A
- $R \cap S \rightarrow$ Equivalence relation on A.

4.5.2 Refinements

A partition P_1 is called a refinement of the partition P_2 if every set in P_1 is a subset of one of the sets in P_2.

4.5.3 Partitions

The equivalence classes of an equivalence relation on a set form a partition of the set. The subsets in this partition are the equivalent classes. Two elements are equivalent with respect to this relation if and only if they are in the same subset of the partition.



A Partition of a Set

▶ Intersection of two equivalence relations on a set is also an equivalence relation.

- i.e. $R \rightarrow$ Equivalence relation on A
- $S \rightarrow$ Equivalence relation on A
- $R \cap S \rightarrow$ Equivalence relation on A.

4.5.4 Refinements

A partition P_1 is called a refinement of the partition P_2 if every set in P_1 is a subset of one of the sets in P_2.

Chapter 5

Order Theory and Lattices

5.1 Partial Orderings

A relation R on a set S is called a partial ordering or partial order if it is reflexive, antisymmetric and transitive. A set S together with a partial ordering R is called a partially ordered set, or poset, and is denoted by (S, R).

Members of S are called elements of the poset.

The notation $a \leq b$ is used to denote that $(a, b) \in R$ in an arbitrary poset (S, R).

5.1.1 Comparable and Incomparable Elements

The elements a and b of a poset $(S, \leq)$ are called comparable if either $a \leq b$ or $b \leq a$.

When a and b are elements of S such that neither $a \leq b$ nor $b \leq a$, then a and b are called incomparable.

5.1.2 Total Order

If $(S, \leq)$ is a poset and every two elements of S are comparable, S is called a totally ordered or linearly ordered set, and $\leq$ is called a total order or a linear order. A totally ordered set is also called a chain.

If there exist some elements that are not comparable, then S is called a partially ordered set, and $\leq$ is called a partial order.

5.1.3 Well-Ordered Set

$(S, \leq)$ is a well-ordered set if it is a poset such that $\leq$ is a total ordering and every nonempty subset of S has a least element.

E.g.: The set Z, with the usual $\leq$ ordering, is not well-ordered because the set of negative integers, which is a subset of Z, has no least element.

5.1.4 Lexicographic Order

The words in a dictionary are listed in lexicographic order.

E.g.: (a_1, a_2) is less than (b_1, b_2), that is

$(a_1, a_2) < (b_1, b_2)$

either if $a_1 < b_1$ or if both $a_1 = b_1$ & $a_2 < b_2$.

(i) $(3, 5) < (4, 8)$

(ii) $(3, 8) < (4, 5)$.

(iii) $(4, 11) \not< (4, 9)$

5.2 Hasse Diagrams

Steps to create Hasse Diagram:
Remove all self loops from the directed graph of the relation

(i) Start with the directed graph of the relation.

(ii) Remove all self loops.

(iii) Remove all edges that must be in the partial ordering because of the presence of other edges and transitivity.

(iv) Arrange each edge so that its initial vertex is below its terminal vertex.

(v) Remove all the arrows on the directed edges, because all edges point "upward" toward their terminal vertex.

▶ Hasse Diagrams are used to represent partial ordering relations.
▶ The Hasse Diagram contains sufficient information to find the partial ordering.

5.2.1 Maximal Element

An element of a poset is called maximal if it is not less than any element of the poset. That is, a is maximal in the poset $(S, \leq)$, if there is no $b \in S$ such that $a < b$.

5.2.2 Minimal Element

An element of a poset is called minimal if it is not greater than any element of the poset. That is, a is minimal in the poset $(S, \leq)$, if there is no $b \in S$ such that $b < a$.
▶ Maximal and Minimal elements are the top and bottom elements in the Hasse diagram.

5.2.3 Greatest Element

An element in a poset, that is greater than every other element, is called the greatest element. That is, a is the greatest element of the poset $(S, \leq)$, if $b \leq a$ for all $b \in S$.
▶ The greatest element is unique when it exists.

5.2.4 Least Element

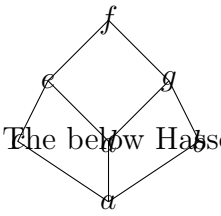
An element in a poset, that is less than every other element, is called the least element. That is, a is the least element of the poset $(S, \leq)$ if $a \leq b$ for all $b \in S$.
▶ The least element is unique when it exists.

5.2.5 Upper Bound and Lower Bound

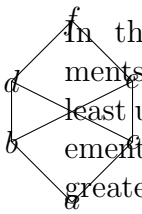
An element, that is greater than or equal to all the elements in a subset A of a poset $(S, \leq)$, is called the upper bound of A.

An element, that is less than or equal to all the elements in a subset A of a poset $(S, \leq)$, is called the lower bound of A.

E.g.:



The below Hasse Diagram is not a lattice:



In this diagram, the elements b & c don't have any least upper bound, so the elements d & e don't have any greatest lower bound.

5.2.6 Least Upper Bound

The element x is called the least upper bound of the subset A if x is an upper bound that is less than every other upper bound of A. It is denoted by $lub(A)$.

▶ The least upper bound of the subset A is unique if it exists.

5.2.7 Greatest Lower Bound

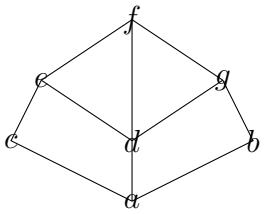
The element y is called the greatest lower bound of the subset A if y is a lower bound that is greater than every other lower bound of A. It is denoted by $glb(A)$.

▶ The greatest lower bound of the subset A is unique if it exists.

5.3 Lattices

A poset in which every pair of elements has both a least upper bound and a greatest lower bound is called a lattice.

E.g.:



5.4 Topological Sorting

A total ordering $\leq$ is said to be compatible with the partial ordering R if $a \leq b$ whenever aRb.

Constructing a compatible total ordering from a partial ordering is called topological sorting.

► It is generally used to order the tasks in such a way that if a and b are tasks where b cannot be started until a has been completed, then a comes before b.

5.4.1 Algorithm for Topological Sorting

TOPOLOGICAL-SORT($S, \leq$): {finite poset}
$R := 1$
while $S \neq \emptyset$
begin
$a_R :=$ a minimal element of S.
$S := S - \{a_R\}$
$R := R + 1$
end
Output: $\{a_1, a_2, \ldots, a_k\}$ is a compatible total ordering of S

► A poset can have multiple different topological sortings.

5.4.2 Additional Lattice Concepts

Let L be a nonempty set closed under two binary operations called meet and join, denoted respectively by $\wedge$ and $\vee$. Then L is called a lattice if the following axioms hold where a, b, c are elements:

(1) Commutative laws:
(1a) $a \wedge b = b \wedge a$ (1b) $a \vee b = b \vee a$
(2) Associative laws:
(2a) $(a \wedge b) \wedge c = a \wedge (b \wedge c)$
(2b) $(a \vee b) \vee c = a \vee (b \vee c)$
(3) Absorption laws:
(3a) $a \wedge (a \vee b) = a$ (3b) $a \vee (a \wedge b) = a$
Lattice is sometimes denoted by $(L, \wedge, \vee)$.

5.4.3 Lattices and Order

Given a lattice L, we can define a partial order on L as follows:
$a \leq b$ if $a \wedge b = a$
OR
$a \leq b$ if $a \vee b = b$

► Let P be a poset such that least upper bound and greatest lower bound exist for any $a, b \in P$, then $(P, \leq)$ is a lattice.

5.4.4 Sublattices

Suppose M is a nonempty subset of a lattice L. We say M is a sublattice of L, if M itself is a lattice (with respect to the operations of L).

5.4.5 Bounded Lattices

A lattice L is said to have a lower bound 0 if for any element x in L, we have $0 \leq x$.
Analogously, L is said to have an upper bound 1 if for any x in L, we have $x \leq 1$.
We say L is bounded if L has both a lower's bound 0 and an upper bound 1.
In such a lattice, we have the identities:
$a \vee I = I$ and $a \wedge I = 0$

▶ In a bounded distributive lattice, complements are unique if they exist.

5.4.6 Complemented Lattice

A lattice L is said to be complemented if L is bounded and every element in L has a complement.

Complement

Let L be a bounded lattice with lower bound 0 and upper bound 1. Let a be an element of L. An element x in L is called a complement of a if:
$a \vee x = I$ and $a \wedge x = 0$

▶ In a bounded lattice, complementsare unique if they exist.

Distributive Lattices

A lattice L is said to be distributive if for any elements a, b, c in L, we have the following distributive law):

(i) $a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$

(ii) $a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$

Otherwise, L is said to be nondistributive.

Chapter 6

Algebraic Structures

6.1 Groups

6.1.1 Semi-Groups and Monoids

Let S be a nonempty set with an operation, then:

(i) S is called a semigroup if the operation is associative. $(a \circ b) \circ c = a \circ (b \circ c)$.

(ii) If the operation also has an identity element, then S is called a monoid.

6.1.2 Groups

Let G be a nonempty set with a binary operation, then G is called a group if the following axioms hold:

(i) Associative Law: For any a, b, c in G, we have $(ab)c = a(bc)$.

(ii) Identity Element:

There exists an element e in G such that $ae = ea = a$ for $\forall a \in G$.

(iii) Inverses:

For each a in G, there exists an element a^{-1} in G, such that:

$a \cdot a^{-1} = a^{-1} \cdot a = e$

6.1.3 Abelian Group

A group G is said to be abelian (or commutative) if the commutative law holds, i.e. if $ab = ba$ for every $a, b \in G$.

► The number of elements in a group G, denoted by $|G|$, is called the order of G, and G is called a finite group if its order is finite.

6.2 Homomorphisms

A mapping f from a group G into a group G' is called a homomorphism if

$f(ab) = f(a)f(b)$

for every $a, b \in G$

6.2.1 Example of Homomorphism

$$f(a+b) = 2^{a+b} = 2^a 2^b = f(a)f(b)$$

6.3 Cyclic Subgroups

Let G be any group and let a be any element of G. As usual, we define $a^0 = e$ and $a^{n+1} = a^n \cdot a$.

All the powers of a:

$\ldots, a^{-3}, a^{-2}, a^{-1}, e, a, a^2, a^3, \ldots$

form a subgroup of G called the cyclic group generated by a, and will be denoted by $gp(a)$.

▶ The smallest positive integer m such that $a^m = e$ is called the order of a. If $|a| = m$, then the cyclic subgroup $gp(a)$ has m elements given by:

$gp(a) = \{e, a, a^2, \ldots, a^{m-1}\}$

6.3.1 Cyclic Group

A group G is said to be cyclic if it has an element a such that:

$G = gp(a)$.

Chapter 7

Combinatorics

7.1 Permutations and Combinations

7.1.1 Permutation and Combination

(1) $0! = 1$

(2) $P(n,r) = {}^nP_r = \frac{n!}{(n-r)!}$

(3) $C(n,r) = {}^nC_r = \frac{n!}{r!(n-r)!}$

(4) Any number means zero or more times

(5) Some or all mean one or more times.

(6) **Sum Rule:**

If one event can occur in m ways and another event in n-ways, then there are $(m+n)$ ways in which one of these events can occur.

(7) **Product Rule:**

If one event can occur in m ways, and for each of m-ways, there are associated n ways in which another event can occur, then there are mn-ways in which these two events can occur.

(8) **Number of ways of arranging r objects, when they are selected from n objects, allowing repetition** $= n^r$

(9) **Suppose there are n_1 objects of the first kind, n_2 objects of the second kind $\ldots$ n_r objects of the rth kind, then the number of ways to select one or more objects from the collection is:**

$(n_1+1)(n_2+1)\ldots(n_r+1)-1$

7.1.2 Distributing r distinct objects into n cells:

(a) If $n \geq r$ and if each cell can hold only one object, then there are ${}^nP_r = \frac{n!}{(n-r)!}$ ways to distribute these r objects.

(b) If $r \geq n$ and if each cell can hold only one object, then there are ${}^{r}C_{n} = \frac{r!}{n!(r-n)!}$ ways to distribute these r objects.

(c) If each cell can hold any number of objects, then there are n^r ways of distributing the objects.

(d) If there are n_1 objects of the first kind, n_2 objects of the second kind ... n_r objects of the rth kind, then there are $\frac{n!}{n_1!n_2!...n_r!}$ ways of distributing the objects, where $n_1 + n_2 + \ldots + n_r = n$.

(e) If there are n_1 objects of the first kind, n_2 objects of the second kind ... n_r objects of the rth kind, and if $n_1 + n_2 + \ldots + n_r \leq h$, then there are $\frac{n!}{n_1!n_2!...n_r!}$ ways of distributing the objects among n cells.

► **(f) If each cell can hold any number of objects and the r objects are ordered inside the cells, then there are $\frac{n!}{n_1!n_2!...n_k!}$ ways of distributing the r objects among n cells.**

7.1.3 Distributing r similar objects into n cells:

(a) If $r \leq n$, and each cell can hold only one object, then there are ${}^{n}C_{r}$ ways of distributing the r objects.

(b) **If $r \geq n$ and if each cell can hold only one object, then there are ${}^{r-1}C_{r-n}$ ways of distributing the r objects.**

$$\binom{r}{r-1} = \frac{r!}{(r-1)!1!} = r$$

Every cell contains at least one object, and we have to distribute $(r - n)$ - objects

(c) If $r \geq n$ and if no cells remain empty, then there are ${}^{r+1}C_{n-1}$ ways of distributing the r objects.

$$\left(\frac{r}{r-1}\right) = \frac{r!}{(r-1)!1!} = r$$

(d) If each cell should contain at least q objects and if $r \geq nq$, then there are ${}^{r-nq-1}C_{r-nq}$ ways of distributing the r objects.

7.1.4 Circular Permutation

The number of ways in which n different things can be arranged in a circle is $(n - 1)!$

If clockwise and anti-clockwise arrangements are not distinguished, then the no. of arrangements is $\frac{1}{2}(n - 1)!$

7.1.5 Permutation and Combination under certain conditions:

(a) No. of combinations of n dissimilar things taken r at a time, when p particular things always occur $= {}^{n-p}C_{r-p}$

(b) No. of combinations of n dissimilar things taken r at a time, when p particular things never occur $= {}^{n-p}C_r$

(c) No. of permutations of n dissimilar things taken r at a time, when p particular things always occur $= {}^{n-p}C_{r-p} \times r!$

(d) No. of permutations of n dissimilar things taken r at a time, when p particular things never occur $= {}^{n-p}C_r \times r!$

7.1.6 Division into groups

(a) The no. of ways in which $(m+n)$ things can be divided into two groups containing m and n things respectively is:
$\frac{(m+n)!}{m!n!}$ (if groups are equal) and otherwise $\frac{(m+n)!}{m!n!2!}$

(b) If $n = m$, the groups are equal and in this case, the no. of different ways of subdivision is:
$\frac{(2m)!}{m!m!2!}$

(c) If $2m$ things are to be divided equally between two persons, then the no. of ways of subdivision is:
$\frac{(2m)!}{m!m!}$

7.1.7 If some or all of n things be taken at a time, where all n things are of different kind, then the number of combinations will be:

${}^nC_1 + {}^nC_2 + \ldots + {}^nC_n = 2^n - 1$

7.1.8 Selection out of identical things

(a) If there are n-identical things out of which we have to draw r-things, then the number of combinations will be 1

(b) If there are n-identical things and we have to make selection of any no. of things, including none, then the no. of selections will be $n + 1$

(c) Total no. of ways in which it is possible to make a selection by taking some or all out of $p + q + s + \ldots$ things, where p are alike of one kind, q are alike of one kind while r of all different kind, then total no. of non-empty selection is:

$(p + 1)(q + 1) \cdot 2^r - 1$

(d) If there are p things, alike of one kind, q alike of one kind while r of all different kind, then total no. of non-empty selection is:

$(p + 1)(q + 1) \cdot 2^r - 1$

7.1.9 Gap Method

Suppose 5-males A, B, C, D, E are arranged in a row as:

$A \times B \times C \times D \times E \times$

Now if 3-females P, Q, R are to be arranged so that they are never together, we will arrange them in 6-gaps.

So, total no. of arrangements $= {}^5P_5 \cdot {}^6P_3$

$= 5! \cdot {}^6P_3$

Chapter 8

Advanced Topics

8.1 Counting and Summation

8.1.1 Exponent of prime p in $n!$:

$$E_p(n!) = \left[\frac{n}{p}\right] + \left[\frac{n}{p^2}\right] + \ldots + \left[\frac{n}{p^k}\right]$$

Example: Find the exponent of 15 in 1001
Solution: We have, $15 = 3 \times 5$
Now,

$$\begin{aligned} E_3(1001) &= \left[\frac{100}{3}\right] + \left[\frac{100}{3^2}\right] + \left[\frac{100}{3^3}\right] + \left[\frac{100}{3^4}\right] \\ &= 33 + 11 + 3 + 1 = 48 \\ E_5(1001) &= \left[\frac{100}{5}\right] + \left[\frac{100}{5^2}\right] \\ &= 20 + 4 = 24 \\ E_{15}(1001) &= \min(48, 24) \\ &= 24 \end{aligned}$$

8.1.2 Summation:

$$(1) \sum n = 1 + 2 + \ldots + n = \frac{n(n+1)}{2}$$
$$(2) \sum n^2 = 1^2 + 2^2 + \ldots + n^2 = \frac{1}{6}n(n+1)(2n+1)$$
$$(3) \sum n^3 = 1^3 + 2^3 + \ldots + n^3 = \left(\frac{1}{2}n(n+1)\right)^2$$

8.2 Recurrence Relations

8.2.1 The Pigeonhole Principle

If $n+1$ or more pigeons holes can be occupied by $n+1$ or more pigeons, then at least k is a positive integer, then at least one pigeonhole is occupied by $(k+1)$ or more pigeons.

Example: Find the minimum no. students in a class so that there are of them are born in the same month.

Solution:

$2 \times 12 + 1 = 25$

8.2.2 Recurrence Relation

A recurrence relation for the sequence $\{a_n\}$ is an equation that expresses a_n in terms of one or more of the previous terms of the sequence, namely $a_0, a_1, \ldots a_{n-1}$ for all $n \geq n_0$, where n_0 is a nonnegative integer.

A sequence is called a solution of a recurrence if its terms satisfy the recurrence relation.

8.2.3 Solving Recurrence Relations

Substitution Method

This is done in two steps:

(a) Apply the recurrence formula iteratively and look for a pattern to predict an explicit formula using internal conditions.

(b) Use induction to prove the predicted formula.

Example: $a_n = a_{n-1} + 3$**,** $n \geq 2$ **and** $a_1 = 2$

Now,

$$a_n = (a_{n-1} + 3) + 3 = a_{n-2} + 2 \cdot 3$$
$$a_n = (a_{n-2} + 3) + 2 \cdot 3 = a_{n-3} + 3 \cdot 3$$

$a_n = (a_{n-1} + 3) + (j-1) \cdot 3 = a_{n-j} + j \cdot 3$

Put $j = n - 1$

$$a_n = a_1 + (n-1) \cdot 3$$
$$\Rightarrow a_n = 2 + 3n - 3$$
$$\Rightarrow a_n = 3n - 1$$

Solving Linear Homogeneous Recurrence Relations with Constant Coefficients

Definition:

A linear homogeneous recurrence relation of degree k with constant coefficients is a recurrence relation of the form:

$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k}$

where $c_1, c_2, \ldots c_k$ are real numbers and $c_k \neq 0$

Examples:

(a) The recurrence relation $f_n = f_{n-1} + f_{n-2}$ is a linear homogeneous recurrence relation of degree two

(b) The recurrence relation $a_n = a_{n-5}$ is a linear homogeneous recurrence relation of degree five

Solution:

Basically, when solving such recurrence relations, we try to find out solutions of the form $a_n = r^n$ where r is a constant.

Now $a_n = r^n$ is a solution of the recurrence relation $a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k}$ if and only if:

$$\begin{aligned} & r^n = c_1 r^{n-1} + c_2 r^{n-2} + \ldots + c_k r^{n-k} \\ \Rightarrow \quad & r^k = c_1 r^{k-1} + c_2 r^{k-2} + \ldots + c_k r^0 = 0 \\ \Rightarrow \quad & r^k - c_1 r^{k-1} - c_2 r^{k-2} - \ldots - c_k = 0 \end{aligned}$$

This is called the characteristic equation of the recurrence relation.

The solutions of this equation are called the characteristic roots of the recurrence relation.

Example: What is the solution of the recurrence relation $a_n = a_{n-1} + 2a_{n-2}$ with $a_0 = 2$ and $a_1 = 7$?

Solution:

The characteristic equation of the recurrence relation is $r^2 - r - 2 = 0$

Its roots are: $r = 2 \quad$ and $\quad r = -1$

Hence, the solution of the recurrence relation is:

$a_n = \alpha_1(2)^n + \alpha_2(-1)^n$

for some constants α_1 & α_2.

Since we know that $a_0 = 2$ and $a_1 = 7$, it follows that:

$$\begin{aligned} a_0 &= 2 = \alpha_1 + \alpha_2 \\ a_1 &= 7 = \alpha_1 \cdot 2 + \alpha_2 \cdot (-1) \end{aligned}$$

Solving these two equations gives us:

$\alpha_1 = 3 \quad$ and $\quad \alpha_2 = -1$

Therefore, the solution to the recurrence relation is:

$a_n = 3 \cdot 2^n - (-1)^n$

▶ If both the roots of the characteristic equation are same:

The solution of the recurrence will be:

$a_n = \alpha_1(\alpha_0)^n + \alpha_2 \cdot n \cdot (\alpha_0)^n$

Engineering Mathematics

Comprehensive Study Notes for GATE Computer Science

Ankur Gupta

www.ankurgupta.net

Contents

1 Matrices **4**
- 1.1 Singular and Non-Singular Matrices . . . 4
- 1.2 Properties of Matrix Addition and Multiplication . . . 4
- 1.3 Transpose of a Matrix . . . 4
  - 1.3.1 Properties of Transpose of a Matrix . . . 4
- 1.4 Symmetric Matrix . . . 4
- 1.5 Skew-Symmetric Matrix . . . 5
- 1.6 Orthogonal Matrix . . . 5
- 1.7 Idempotent Matrix . . . 5
- 1.8 Involutory Matrix . . . 5
- 1.9 Adjoint of a Square Matrix . . . 5
  - 1.9.1 Properties of Adjoint . . . 5
- 1.10 Inverse of a Square Matrix . . . 5
  - 1.10.1 Properties of Inverse . . . 6
- 1.11 Elementary Matrices . . . 6
- 1.12 Rank of a Matrix . . . 6
  - 1.12.1 Echelon Form Method of Finding Rank . . . 6

2 System of Linear Equations **7**
- 2.1 Solution of a System of Linear Equations . . . 7
  - 2.1.1 For a System of Non-Homogeneous Linear Equations $AX = B$ . . . 7
  - 2.1.2 For a System of Homogeneous Linear Equations $AX = 0$ . . . 7
- 2.2 Linear Dependence and Linear Independence of Vectors . . . 7

3 Eigenvalues and Eigenvectors **9**
- 3.1 Characteristic Equation . . . 9
- 3.2 Eigenvectors and Eigenvalues . . . 9
- 3.3 Properties of Eigenvalues and Eigenvectors . . . 10

4 Limits, Continuity and Differentiability **11**
- 4.1 Trigonometry . . . 11
  - 4.1.1 Basic Identities . . . 11
  - 4.1.2 Sum and Difference Formulas . . . 11
  - 4.1.3 Product-to-Sum Formulas . . . 11
  - 4.1.4 Sum-to-Product Formulas . . . 12
  - 4.1.5 Double Angle Formulas . . . 12
  - 4.1.6 Hyperbolic Functions . . . 12
- 4.2 Continuity and Differentiability . . . 12

4.3 Differentiation 12
4.3.1 Important Formulas 12
4.4 Limits 13
4.4.1 Important Expansions to be Used 13
4.4.2 Some Important Theorems on Limits 14

5 Integration **15**
5.1 Fundamental Integration Formulas 15
5.2 Integration by Substitution 16
5.3 Integration by Parts 16

6 Numerical Methods **17**
6.1 Numerical Solution of Algebraic Equations 17
6.1.1 Bisection Method 17
6.1.2 Secant Method 17
6.1.3 Newton-Raphson Method 17
6.1.4 Regula-Falsi Method (Always Converges) 17
6.2 Numerical Integration 18
6.2.1 Trapezoidal Rule 18
6.2.2 Simpson's One-Third Rule 18
6.2.3 Simpson's Three-Eighth Rule 18
6.2.4 Error in Trapezoidal Rule 18
6.2.5 Error in Simpson's Rule 18

Chapter 1

Matrices

1.1 Singular and Non-Singular Matrices

Definition 1.1.1. A square matrix A is said to be **singular** if $|A| = 0$, and **non-singular** if $|A| \neq 0$.

1.2 Properties of Matrix Addition and Multiplication

Let A, B, and C be matrices. The following properties hold:

(i) $A + B = B + A$ (Commutative)

(ii) $(A + B) + C = A + (B + C)$ (Associative)

(iii) $AB \neq BA$ (Not Commutative)

(iv) $(AB)C = A(BC)$ (Associative)

(v) $A(B + C) = AB + AC$ (Distributive)

1.3 Transpose of a Matrix

Definition 1.3.1. If $A = [a_{ij}]_{m\times n}$, then $A' = [b_{ij}]_{n\times m}$ where $b_{ij} = a_{ji}$.

1.3.1 Properties of Transpose of a Matrix

(i) $(A')' = A$

(ii) $(A + B)' = A' + B'$

(iii) $(AB)' = B'A'$

1.4 Symmetric Matrix

Definition 1.4.1. A matrix A is **symmetric** if $A' = A$.

1.5 Skew-Symmetric Matrix

Definition 1.5.1. A matrix A is **skew-symmetric** (or anti-symmetric) if $A' = -A$. If A and B are skew-symmetric matrices, then $A' = -A$ and $B' = -B$, so $(AB - BA)' = -AB + BA$ (skew-symmetric).

1.6 Orthogonal Matrix

Definition 1.6.1. A matrix A is **orthogonal** if $AA' = A'A = I$.

1.7 Idempotent Matrix

Definition 1.7.1. A matrix A is **idempotent** if $A^2 = A$.

1.8 Involutory Matrix

Definition 1.8.1. A matrix A is **involutory** if $A^2 = I$.

Theorem 1.8.1. Every square matrix can be uniquely expressed as the sum of a symmetric matrix and a skew-symmetric matrix:

$$A = \frac{1}{2}(A + A')(\text{symmetric}) + \frac{1}{2}(A - A')(\text{skew-symmetric})$$

1.9 Adjoint of a Square Matrix

Definition 1.9.1. If $A = \begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{bmatrix}$, then the adjoint of A is

$$\text{Adj } A = \text{Transpose of } \begin{bmatrix} A_1 & B_1 & C_1 \\ A_2 & B_2 & C_2 \\ A_3 & B_3 & C_3 \end{bmatrix} = \begin{bmatrix} A_1 & A_2 & A_3 \\ B_1 & B_2 & B_3 \\ C_1 & C_2 & C_3 \end{bmatrix}$$

where A_i, B_i, C_i are cofactors of a_i, b_i, c_i respectively.

1.9.1 Properties of Adjoint

(i) $A(\text{Adj } A) = (\text{Adj } A)A = |A|I_n$

(ii) $\text{Adj}(AB) = (\text{Adj } B) \cdot (\text{Adj } A)$

1.10 Inverse of a Square Matrix

Definition 1.10.1. The inverse of a matrix A is given by

$$A^{-1} = \frac{\text{Adj } A}{|A|}, \quad |A| \neq 0$$

1.10.1 Properties of Inverse

(i) $(A^{-1})' = A$

(ii) $(AB)^{-1} = B^{-1}A^{-1}$

(iii) $(A')^{-1} = (A^{-1})'$

(iv) Only a non-singular square matrix can have an inverse.

Example: Find A^{-1} if $A = \begin{bmatrix} 3 & -3 & 4 \\ 2 & -3 & 4 \\ 0 & -1 & 1 \end{bmatrix}$.

Solution:

$$\text{Adj } A = \begin{bmatrix} 1 & -2 & -2 \\ -1 & 3 & 3 \\ 0 & -4 & -3 \end{bmatrix}' = \begin{bmatrix} 1 & -1 & 0 \\ -2 & 3 & -4 \\ -2 & 3 & -3 \end{bmatrix}$$

$$|A| = 3(-3+4) + 3(2) - 4(-1) = 1$$

$$A^{-1} = \frac{\text{Adj } A}{|A|} = \begin{bmatrix} 1 & -1 & 0 \\ -2 & 3 & -4 \\ -2 & 3 & -3 \end{bmatrix}$$

1.11 Elementary Matrices

Definition 1.11.1. The matrix obtained from a unit matrix I by subjecting it to one of the elementary operations is called an **elementary matrix**.

1.12 Rank of a Matrix

Definition 1.12.1. Let A be any $m \times n$ matrix. The rank is defined such that:

- A has square sub-matrices of different orders. The determinants of these square sub-matrices are called **minors** of A.
- A matrix is said to be of **rank** r, if
  - (i) It has at least one non-zero minor of order r
  - (ii) All its minors of order $(r+1)$ or higher than r are zero

The rank of $A = r$ is written as $\rho(A) = r$.
If A is a non-singular $n \times n$ matrix, then $\rho(A) = n$.

1.12.1 Echelon Form Method of Finding Rank

In the echelon form of a matrix, each of the first r elements of the leading diagonal is 1, and every element below this diagonal is 0. The rank of the matrix is equal to the number of non-zero diagonal elements when it has been reduced to echelon form.

Chapter 2

System of Linear Equations

2.1 Solution of a System of Linear Equations

A system of equations having no solution is called an **inconsistent system** of equations.

A system of equations having one or more solutions is called a **consistent system** of equations.

2.1.1 For a System of Non-Homogeneous Linear Equations $AX = B$

(i) If $\rho[A : B] \neq \rho(A)$, the system is inconsistent.

(ii) If $\rho[A : B] = \rho(A) =$ number of unknowns, the system has a unique solution.

(iii) If $\rho[A : B] = \rho(A) <$ number of unknowns, the system has an infinite number of solutions.

2.1.2 For a System of Homogeneous Linear Equations $AX = 0$

(i) $X = 0$ is always a solution (Trivial Solution).

(ii) If $\rho(A) =$ number of unknowns, the system has only the trivial solution.

(iii) If $\rho(A) <$ number of unknowns, the system has an infinite number of non-trivial solutions.

A homogeneous system is always consistent.

2.2 Linear Dependence and Linear Independence of Vectors

Definition 2.2.1. A set of n-vectors $X_1, X_2, \ldots, X_r$ in V is said to be **linearly dependent** if there exist r scalars (numbers) $k_1, k_2, \ldots, k_r$, not all zero, such that

$$k_1X_1 + k_2X_2 + \cdots + k_rX_r = 0$$

It is called **linearly independent** if every relation of the type

$$k_1X_1 + k_2X_2 + \cdots + k_rX_r = 0$$

implies $k_1 = k_2 = \cdots = k_r = 0$.

Chapter 3

Eigenvalues and Eigenvectors

3.1 Characteristic Equation

If A is a square matrix of order n, we can form the matrix $(A - \lambda I)$, where λ is a scalar and I is an identity matrix of order n. The determinant of this matrix equated to zero,

$$|A - \lambda I| = \begin{vmatrix} a_{11} - \lambda & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} - \lambda & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} - \lambda \end{vmatrix} = 0$$

is called the **characteristic equation** of A.

The roots of this equation are called the **characteristic roots** or **eigenvalues** of A.

3.2 Eigenvectors and Eigenvalues

Definition 3.2.1. A matrix A of size $(n \times n)$ has a column vector X of size $(n \times 1)$ called the **Eigen Vector** of A, if

$$AX = \lambda X$$
$$AX - \lambda X = 0$$
$$(A - \lambda I)X = 0$$

where λ is a nonzero scalar.

The characteristic roots of equation $(A - \lambda I) = 0$ are called the Eigen Values.

Example: Find the eigenvalues and eigenvectors of the matrix $A = \begin{bmatrix} -2 & 2 & -3 \\ 2 & 1 & -6 \\ -1 & -2 & 0 \end{bmatrix}$.

Solution: The characteristic equation of the given matrix is

$$|A - \lambda I| = 0$$

$$\begin{vmatrix} -2 - \lambda & 2 & -3 \\ 2 & 1 - \lambda & -6 \\ -1 & -2 & -\lambda \end{vmatrix} = 0$$

$$(\lambda+3)(\lambda+3)(\lambda-5)=0 \implies \lambda=-3,-3,5$$

Corresponding to $\lambda=-3$, the eigenvectors are given by $(A+3I)X_1=0$:

$$\begin{bmatrix}1 & 2 & -3\\ 2 & 4 & -6\\ -1 & -2 & 3\end{bmatrix}\begin{bmatrix}x_1\\ x_2\\ x_3\end{bmatrix}=\begin{bmatrix}0\\ 0\\ 0\end{bmatrix}$$

$$x_1=k_1, \quad x_2=2k_1, \quad x_3=-k_1$$

$$X_1=k_1\begin{bmatrix}1\\ 2\\ -1\end{bmatrix}$$

For $\lambda=5$, let $X_2=k_2$ and $x_1=k_2$, then $x_2=2k_2$, $x_3=-k_1$.

$$X_2=k_2\begin{bmatrix}1\\ 2\\ -1\end{bmatrix}$$

3.3 Properties of Eigenvalues and Eigenvectors

(1) The sum of the eigenvalues of a matrix A is equal to the trace of A.

(2) The product of the eigenvalues of a matrix A is equal to its determinant.

(3) If λ is an eigenvalue of an orthogonal matrix, then $1/\lambda$ is also its eigenvalue.

(4) The eigenvalues of an idempotent matrix are either zero or unity.

(5) The trace or spur of a square matrix is the sum of its diagonal elements.

Chapter 4

Limits, Continuity and Differentiability

4.1 Trigonometry

4.1.1 Basic Identities

(1) $\sin^2\theta + \cos^2\theta = 1$ All (+), 180° sin(+), Others (−)

(2) $1 + \tan^2\theta = \sec^2\theta$ tan, cot (+), Others (−)

(3) $1 + \cot^2\theta = \csc^2\theta$ 270° cos(+), Others (−)

(4) $\sec\theta = 1/\cos\theta, \quad \csc\theta = 1/\sin\theta$

(5) $\tan\theta = \sin\theta/\cos\theta, \quad \cot\theta = \cos\theta/\sin\theta$

4.1.2 Sum and Difference Formulas

(1) $\sin(A+B) = \sin A\cos B + \cos A\sin B$

(2) $\sin(A-B) = \sin A\cos B - \cos A\sin B$

(3) $\cos(A+B) = \cos A\cos B - \sin A\sin B$

(4) $\cos(A-B) = \cos A\cos B + \sin A\sin B$

(5) $\tan(A+B) = \frac{\tan A+\tan B}{1-\tan A\tan B}, \quad \tan(A-B) = \frac{\tan A-\tan B}{1+\tan A\tan B}$

4.1.3 Product-to-Sum Formulas

(1) $2\sin A\cos B = \sin(A+B) + \sin(A-B)$

(2) $2\cos A\sin B = \sin(A+B) - \sin(A-B)$

(3) $2\cos A\cos B = \cos(A+B) + \cos(A-B)$

(4) $2\sin A\sin B = \cos(A-B) - \cos(A+B)$

4.1.4 Sum-to-Product Formulas

(1) $\sin C + \sin D = 2\sin\left(\frac{C+D}{2}\right)\cos\left(\frac{C-D}{2}\right)$

(2) $\sin C - \sin D = 2\cos\left(\frac{C+D}{2}\right)\sin\left(\frac{C-D}{2}\right)$

(3) $\cos C + \cos D = 2\cos\left(\frac{C+D}{2}\right)\cos\left(\frac{C-D}{2}\right)$

(4) $\cos C - \cos D = -2\sin\left(\frac{C+D}{2}\right)\sin\left(\frac{C-D}{2}\right)$

4.1.5 Double Angle Formulas

(1) $\sin 2x = 2\sin x\cos x$

(2) $\cos 2x = \cos^2 x - \sin^2 x = 2\cos^2 x - 1 = 1 - 2\sin^2 x$

(3) $\tan 2x = \frac{2\tan x}{1-\tan^2 x}$

4.1.6 Hyperbolic Functions

(1) $\sin 2x = \frac{2\tan x}{1+\tan^2 x}$

(2) $\cos 2x = \frac{1-\tan^2 x}{1+\tan^2 x}$

4.2 Continuity and Differentiability

Definition 4.2.1. A function is **continuous** if its graph is a single unbroken curve with no holes or jumps.

Definition 4.2.2. A function is **differentiable** if its graph is relatively smooth and does not contain any breaks or bends.

Note: A differentiable function is always continuous, but a continuous function need not be differentiable.

4.3 Differentiation

4.3.1 Important Formulas

(i) $\frac{d}{dx}(x^n) = nx^{n-1}$

(ii) $\frac{d}{dx}(e^x) = e^x$

(iii) $\frac{d}{dx}(a^x) = a^x \log_e a$

(iv) $\frac{d}{dx}c = 0$ where c is a constant

(v) $\frac{d}{dx}\cos x = -\sin x$

(vi) $\frac{d}{dx}(\tan x) = \sec^2 x$

(vii) $\frac{d}{dx}\sec x = \sec x\tan x$

(viii) $\frac{d}{dx}(\csc x) = -\csc x \cot x$

(ix) $\frac{d}{dx}\cot x = -\csc^2 x$

(x) $\frac{d}{dx}(\log_e x) = \frac{1}{x}$

(xi) $\frac{d}{dx}(\sin^{-1} x) = \frac{1}{\sqrt{1-x^2}}$

(xii) $\frac{d}{dx}(\cos^{-1} x) = \frac{-1}{\sqrt{1-x^2}}$

(xiii) $\frac{d}{dx}(\tan^{-1} x) = \frac{1}{1+x^2}$

(xiv) $\frac{d}{dx}(\cot^{-1} x) = \frac{-1}{1+x^2}$

(xv) $\frac{d}{dx}(\sec^{-1} x) = \frac{1}{x\sqrt{x^2-1}}$

(xvi) $\frac{d}{dx}(\csc^{-1} x) = \frac{-1}{x\sqrt{x^2-1}}$

(xvii) $\frac{d}{dx}[f(x)g(x)] = f(x)g'(x) + f'(x)g(x)$

(xviii) $\frac{d}{dx}\left(\frac{f(x)}{g(x)}\right) = \frac{g(x)f'(x)-f(x)g'(x)}{[g(x)]^2}$

(xix) $\frac{d}{dx}f(g(x)) = f'(g(x)) \cdot g'(x)$

4.4 Limits

4.4.1 Important Expansions to be Used

(i) $(1+x)^n = 1 + nx + \frac{n(n-1)}{2!}x^2 + \cdots$

(ii) $\left(\frac{x^n-a^n}{x-a}\right) = (x^{n-1} + ax^{n-2} + a^2x^{n-3} + \cdots + a^{n-1})$

(iii) $e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots$

(iv) $a^x = 1 + x\log_e a + \frac{(x\log_e a)^2}{2!} + \cdots$

(v) $\log(1+x) = x - \frac{x^2}{2!} + \frac{x^3}{3!} - \frac{x^4}{4!} + \cdots$

(vi) $\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots$

(vii) $\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots$

(viii) $\tan x = x + \frac{x^3}{3} + \frac{2x^5}{15} + \cdots$

4.4.2 Some Important Theorems on Limits

(i) $\lim_{x\to a}\left(\frac{x^n-a^n}{x-a}\right) = na^{n-1}$ where $a > 0$

(ii) $\lim_{x\to 0}\left(\frac{e^x-1}{x}\right) = 1, \quad \lim_{a\to 0}\left(\frac{a^x-1}{x}\right) = \log_e a$

(iii) $\lim_{x\to\infty}(1+x)^n = e$

(iv) $\lim_{x\to 0}\frac{\log(1+x)}{x} = 1$

(v) $\lim_{x\to\infty}\left(\frac{a^x}{x}\right) = 0$

(vi) $\lim_{x\to 0}\left(\frac{\sin x}{x}\right) = 1, \quad \lim_{x\to 0}\left(\frac{\tan x}{x}\right) = 1$

Chapter 5

Integration

5.1 Fundamental Integration Formulas

(i) $\int x^n \, dx = \frac{x^{n+1}}{(n+1)} + C$

(ii) $\int \frac{1}{x} \, dx = \log x + C$

(iii) $\int e^x \, dx = e^x + C$

(iv) $\int a^x \, dx = \frac{a^x}{\log_e a} + C$

(v) $\int \sin x \, dx = -\cos x + C$

(vi) $\int \cos x \, dx = \sin x + C$

(vii) $\int \sec^2 x \, dx = \tan x + C$

(viii) $\int \csc^2 x \, dx = -\cot x + C$

(ix) $\int \sec x \tan x \, dx = \sec x + C$

(x) $\int \csc x \cot x \, dx = -\csc x + C$

(xi) $\int \cot x \, dx = \log(\sin x) + C$

(xii) $\int \tan x \, dx = -\log(\cos x) + C$

(xiii) $\int \sec x \, dx = \log(\sec x + \tan x) + C$

(xiv) $\int \csc x \, dx = \log(\csc x - \cot x) + C$

(xv) $\int \frac{dx}{a^2 - x^2} = \sin^{-1}\left(\frac{x}{a}\right) + C$

(xvi) $\int \frac{-1}{a^2 - x^2} \, dx = \cos^{-1}\left(\frac{x}{a}\right) + C$

(xvii) $\int \frac{dx}{a^2 + x^2} = \frac{1}{a} \tan^{-1}\left(\frac{x}{a}\right) + C$

(xviii) $\int \frac{-1}{a^2 + x^2} \, dx = \frac{1}{a} \cot^{-1}\left(\frac{x}{a}\right) + C$

(xix) $\int \frac{dx}{x\sqrt{x^2 - a^2}} = \frac{1}{a} \sec^{-1}\left(\frac{x}{a}\right) + C$

(xx) $\int \frac{dx}{x\sqrt{x^2 - a^2}} = \frac{1}{a} \csc^{-1}\left(\frac{x}{a}\right) + C$

5.2 Integration by Substitution

$$
\begin{aligned}
I &= \int f(g(x)) \cdot g'(x)\, dx \\
&\text{Let } g(x) = t \\
&\implies g'(x)\, dx = dt \\
\implies I &= \int f(t)\, dt
\end{aligned}
$$

Note: If $\int f(x)\, dx = g(x) + c$, then $\int f(ax + b)\, dx = \frac{1}{a} g(ax + b) + c$.

5.3 Integration by Parts

$$\int uv\, dx = u \int v\, dx - \int \left(\frac{du}{dx} \int v\, dx \right) dx$$

Chapter 6

Numerical Methods

6.1 Numerical Solution of Algebraic Equations

6.1.1 Bisection Method

Let the function $f(x)$ be continuous between a and b, and let $f(a)$ be negative and $f(b)$ be positive. Then the first approximation to the root is

$$x_1 = \frac{1}{2}(a+b)$$

If $f(x_1)$ is negative, $x_1 = \frac{1}{2}(x_1+b)$ and so on.

If $f(x_1)$ is positive, $x_2 = \frac{1}{2}(a+x_1)$ and so on.

6.1.2 Secant Method

$$x_{n+1} = x_n - \left[\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}\right] f(x_n)$$

6.1.3 Newton-Raphson Method

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

6.1.4 Regula-Falsi Method (Always Converges)

$$x_{n+1} = x_0 - \left[\frac{x_n - x_0}{f(x_n) - f(x_0)}\right] f(x_n)$$

Convergence Rate:
Newton-Raphson > Secant > Regula-Falsi (Quadratic) > (Linear)

- Newton's method does not always converge.
- Order of convergence of secant method is 1.618
- Order of convergence of Newton's method is 2
- Order of convergence of Regula-Falsi method is 1
- If the initial values are not close enough to the root, then there is no guarantee that the secant method converges.

6.2 Numerical Integration

6.2.1 Trapezoidal Rule

$$\int_{x_0}^{x_n} f(x)\,dx = \frac{h}{2}\left[(y_0 + y_n) + 2(y_1 + y_2 + \cdots + y_{n-1})\right]$$

6.2.2 Simpson's One-Third Rule

$$\int_{x_0}^{x_n} f(x)\,dx = \frac{h}{3}\left[(y_0 + y_n) + 4(y_1 + y_3 + \cdots + y_{n-1}) + 2(y_2 + y_4 + \cdots + y_{n-2})\right]$$

6.2.3 Simpson's Three-Eighth Rule

$$\int_{x_0}^{x_n} f(x)\,dx = \frac{3h}{8}\left[(y_0 + y_n) + 3(y_1 + y_2 + y_4 + y_5 + \cdots + y_{n-2}) + 2(y_3 + y_6 + \cdots + y_{n-3})\right]$$

Here h is the interval, and n is the number of intervals.

- $y_0 = f(x_0)$, $y_1 = f(x_0 + h)$, $y_2 = f(x_0 + 2h)$ and so on
- For Simpson's One-Third rule, n is even
- For Simpson's Three-Eighth rule, n should be a multiple of 3

6.2.4 Error in Trapezoidal Rule

$$\text{Error} \leq \frac{(b-a)^3}{12n^2} \max|f''(x)|$$

or

$$\text{Error} \leq \frac{(b-a)h^2}{12} \max|f''(x)|$$

6.2.5 Error in Simpson's Rule

$$\text{Error} \leq \frac{(b-a)h^4}{180} \max|f^{(4)}(x)|$$

Here, $a = x_0$, $b = x_0 + nh$, $h = \frac{b-a}{n}$, and $f'(x)$ and $f''(x)$ refer to the integrand taken on $[a, b]$.

- Simpson's rule provides exact results for any polynomial of degree three or less.
- Trapezoidal rule gives exact results for any polynomial of degree one.

Operating Systems

Comprehensive Study Notes for GATE Computer Science

Ankur Gupta

www.ankurgupta.net

Note: These notes have been prepared primarily from the following books:

(1) **Operating System Concepts** by the Silberschatz, Galvin, Gagne

Please refer to the above books for more details.

Contents

1 Processes **7**
- 1.1 Introduction to Processes . . . 7
  - 1.1.1 Components of a Process . . . 7
- 1.2 Process State . . . 7
  - 1.2.1 Process State Diagram . . . 8
- 1.3 Process Control Block . . . 8
- 1.4 Operations on Processes . . . 8
  - 1.4.1 Process Creation . . . 8
  - 1.4.2 Process Termination . . . 9
- 1.5 Process Table . . . 9
  - 1.5.1 Trap or Exception . . . 9
  - 1.5.2 Dual-Mode Operation in an Operating System . . . 10
  - 1.5.3 Timers . . . 10
  - 1.5.4 Fork System Call . . . 10
  - 1.5.5 Dual Mode Operation Support . . . 10
- 1.6 Chapter Summary . . . 10

2 Multithreading **12**
- 2.1 Problems with Traditional Processes . . . 12
- 2.2 Threads . . . 12
- 2.3 Types of Threads . . . 12
- 2.4 Multithreading Models . . . 13
- 2.5 Benefits of Multithreading . . . 13

3 Process Scheduling **14**
- 3.1 Multiprogramming . . . 14
- 3.2 Time Sharing . . . 14
- 3.3 Scheduling Queues . . . 14
- 3.4 Schedulers . . . 14
  - 3.4.1 Long-term Scheduler or Job Scheduler . . . 15
  - 3.4.2 Short-term Scheduler or CPU Scheduler . . . 15
  - 3.4.3 Medium-term Scheduler . . . 15
- 3.5 Context Switch . . . 15
- 3.6 Dispatcher . . . 15
- 3.7 Preemptive and Nonpreemptive Scheduling . . . 16
- 3.8 Scheduling Criteria . . . 16

4 Scheduling Algorithms **17**
4.1 First-Come, First-Served Scheduling . . . 17
4.2 Shortest-Job-First Scheduling . . . 17
4.2.1 Preemptive SJF Scheduling . . . 17
4.3 Priority Scheduling . . . 17
4.3.1 Aging . . . 18
4.4 Round-Robin Scheduling . . . 18
4.5 SCAN Scheduling . . . 18
4.6 Circular SCAN Scheduling . . . 18

5 Multiprocessor Scheduling **19**
5.1 Processor Affinity . . . 19
5.2 Load Balancing . . . 19

6 Deadlocks **20**
6.1 Introduction . . . 20
6.2 Necessary Conditions for Deadlock . . . 20
6.3 Resource Allocation Graph . . . 20
6.3.1 Request Edge . . . 21
6.3.2 Assignment Edge . . . 21
6.3.3 Observations . . . 21
6.4 Methods for Handling Deadlocks . . . 21
6.5 Deadlock Prevention . . . 21
6.6 Deadlock Avoidance . . . 22
6.6.1 Safe State . . . 22
6.6.2 Banker's Algorithm . . . 22
6.7 Recovery from Deadlock . . . 23

7 Deadlocks **24**
7.1 Introduction . . . 24
7.2 Necessary Conditions for Deadlock . . . 24
7.3 Resource Allocation Graph . . . 24
7.3.1 Request Edge . . . 25
7.3.2 Assignment Edge . . . 25
7.3.3 Observations . . . 25
7.3.4 Shared Memory Model . . . 25
7.4 Message-Passing Systems . . . 26
7.4.1 Synchronous Communication . . . 26
7.4.2 Asynchronous Communication . . . 26
7.4.3 Blocking Send . . . 26
7.4.4 Nonblocking Send . . . 26
7.4.5 Blocking Receive . . . 26
7.4.6 Nonblocking Receive . . . 26
7.4.7 Direct Communication . . . 27
7.4.8 Indirect Communication . . . 27
7.5 Producer-Consumer Problem . . . 27
7.6 Race Condition . . . 28

8 The Critical-Section Problem **29**
8.1 Introduction . . . 29
8.1.1 Structure of a Process . . . 29
8.2 Solution to the Critical-Section Problem . . . 29
8.3 Peterson's Solution . . . 30
8.3.1 The Structure of Process P_i in Peterson's Solution . . . 30

9 Synchronization Hardware **31**
9.1 Introduction . . . 31
9.2 TestAndSet Instruction . . . 31
9.2.1 The TestAndSet Instruction . . . 31
9.3 Swap Instruction . . . 31
9.4 Bounded-Waiting Mutual Exclusion with TestAndSet . . . 32

10 Semaphores **33**
10.1 Introduction . . . 33
10.2 Binary Semaphores . . . 33
10.3 Counting Semaphores . . . 33
10.4 Semaphore Implementation . . . 34
10.4.1 Wait Operation Implementation . . . 34
10.4.2 Signal Operation Implementation . . . 34
10.5 Deadlocks and Starvation . . . 35

11 Code for Producer and Consumer **36**
11.1 Producer-Consumer Problem with Race Condition . . . 36
11.1.1 Code for Consumer . . . 36
11.2 Modified Code for Producer . . . 36
11.3 Modified Code for Consumer . . . 37

12 File Systems **38**
12.1 File System Structure . . . 38
12.1.1 Levels of File System . . . 38
12.2 I/O Control . . . 38
12.3 Basic File System . . . 38
12.4 File Organization Module . . . 38
12.5 Logical File System . . . 39
12.6 File Systems Used by Various Operating Systems . . . 39
12.7 On-Disk and In-Memory Structures . . . 39

13 Disk Allocation Methods **40**
13.1 Contiguous Allocation . . . 40

14 File Organization and Indexes **41**
14.1 File Organization . . . 41
14.1.1 Spanned Records . . . 41
14.1.2 Unspanned Records . . . 41
14.1.3 Blocking Factor . . . 41
14.2 Types of Organization of Files . . . 41
14.3 Indexes . . . 42

14.3.1 Dense and Sparse Indexes 42
14.3.2 Types of Indexes 42
14.4 Single-Level Ordered Indexes 42
14.4.1 Primary Indexes 42
14.4.2 Clustering Indexes 42
14.4.3 Secondary Indexes 43
14.5 Multilevel Indexes 43
14.6 B-Trees 43
14.6.1 Introduction to B-Trees 43
14.6.2 B+ Tree Structure 44
14.6.3 Insertion in B+ Trees 45
14.6.4 Deletion in B+ Trees 46
14.6.5 Shortest Seek Time First Scheduling 47
14.6.6 SCAN Scheduling 47
14.6.7 Circular SCAN Scheduling 47
14.7 Disk Formatting 47
14.7.1 Physical Formatting 48
14.7.2 Logical Formatting 48
14.8 Raw Disk 48
14.9 Disk Block 48

Chapter 1

Processes

1.1 Introduction to Processes

A **process** is a program in execution. A program by itself is not a process; a program is a passive entity, whereas a process is an active entity.

1.1.1 Components of a Process

A process includes the following:

(i) **Text Section**: Contains the program code.

(ii) **Current Activity**: Represented by the value of the program counter and the contents of the processor's registers.

(iii) **Stack**: Contains temporary data such as function parameters, return addresses, and local variables.

(iv) **Data Section**: Contains global variables.

(v) **Heap**: Used for dynamic memory allocation during process runtime.

1.2 Process State

Each process may be in one of the following states during execution:

(i) **New**: The process is being created.

(ii) **Running**: Instructions are being executed.

(iii) **Waiting**: The process is waiting for some event to occur, such as I/O completion or reception of a signal.

(iv) **Ready**: The process is waiting to be assigned to a processor.

(v) **Terminated**: The process has finished execution.

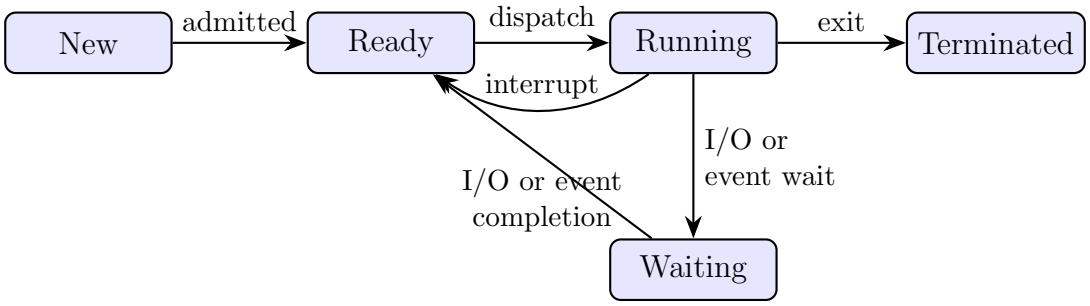


Figure 1.1: Process State Transition Diagram

1.2.1 Process State Diagram

The transitions between process states can be represented as:

Note: Only one process can be running on any processor at any instant.

1.3 Process Control Block

Each process is represented by a process control block (PCB), also called a task control block. A PCB contains the following information:

(1) **Process State**: Current state of the process.

(2) **Program Counter**: Address space for the process.

(3) **CPU Registers**: User identification.

(4) **List of Open Files**: Files currently accessed by the process.

(5) **CPU Scheduling Information**: Priority and scheduling parameters.

(6) **Memory Management Information**: Base and limit registers, page tables.

(7) **Accounting Information**: Resource usage statistics.

(8) **I/O Status**: List of I/O devices allocated to the process.

(9) **Process Identifier (PID)**: Unique identifier for the process.

(10) **Parent Process ID (PPID)**: Identifier of the parent process.

1.4 Operations on Processes

1.4.1 Process Creation

A process may create several new processes via a create-process system call during the course of execution.

- The creating process is called a **parent process**, and the new processes are called the **children** of that process.

- Each of these processes may in turn create other processes, forming a tree of processes.
- When a process creates a subprocess, the subprocess may be able to obtain its resources directly from the operating system, or it may be constrained to a subset of the resources of the parent process.
- Restricting a child process to a subset of the parent's resources prevents any process from overloading the system by creating too many subprocesses.

In UNIX systems:

- A new process is created by the `fork()` system call.
- The `exec()` system call is used after a `fork()` system call to replace the process's memory space with a new program.

In Windows systems:

- A new process is created using the `CreateProcess()` function.

1.4.2 Process Termination

A process terminates when it finishes executing its final statement and asks the operating system to delete it by using the `exit()` system call.

- Some systems do not allow a child to exist if its parent process has terminated. In such systems, if a process terminates, then all its children must also be terminated. This phenomenon is referred to as **cascading termination**.
- In UNIX, if the parent terminates, all its children are assigned `init` process as their parent.

1.5 Process Table

The process table is a data structure maintained by the operating system to facilitate context switching and scheduling. Each entry in the process table (an entry is known as a process control block PCB) contains:

- Process key (an entry in the process table).
- Entries are known as process control blocks (PCBs).

1.5.1 Trap or Exception

A **trap** is a software-generated interrupt caused either by an error or by a specific request from a user program.

1.5.2 Dual-Mode Operation in an Operating System

There are two separate modes of operation in an operating system:

(i) **User Mode**

(ii) **Kernel Mode** or **Supervisor Mode** or **System Mode** or **Privileged Mode**

Mode Bit

A mode bit is used to distinguish between user mode and kernel mode:

- For user mode: mode-bit = 1
- For kernel mode: mode-bit = 0

At system boot time, the hardware starts in kernel mode. The operating system is then loaded and starts user applications in user mode.

Whenever a trap or interrupt occurs, the hardware switches from user mode to kernel mode.

1.5.3 Timers

A timer is used to ensure that the operating system maintains control over the CPU. A timer can be set to interrupt the computer after a specified period. We can use the timer to prevent a user program from running too long.

1.5.4 Fork System Call

When a process forks, it creates a copy of itself. In the child process, the return value of the `fork()` is zero. In the parent process, the return value is the PID of the newly created child process. There is a separate address space for the child.

Both the parent and child processes pass the same code segments, but execute independently of each other.

For example, for the program with n `fork()` calls, $2^n - 1$ child processes will be created.

1.5.5 Dual Mode Operation Support

Dual mode operation support is provided by the hardware, not the software.

1.6 Chapter Summary

- At system boot time, the hardware starts in kernel mode, then loads the operating system in kernel mode, and starts user applications in user mode.
- Whenever a trap or interrupt occurs, the hardware switches from user mode to kernel mode.
- The dual mode of operation provides us with the means for protecting the operating system from errant users and errant users from one another.

- Some of the machine instructions that may cause harm are designated as privileged instructions. The hardware allows privileged instructions to be executed only in kernel mode.
- If an attempt is made to execute a privileged instruction in user mode, the hardware treats it as illegal and traps it to the operating system.
- System calls provide the means for a user program to ask the operating system to perform tasks reserved for the operating system on the user program's behalf.
- When a system call is executed, it is treated by the hardware as a software interrupt.
- System calls provide an interface between a process and the operating system to ask for operating system services or to perform tasks reserved for the operating system on the user program's behalf.

Chapter 2

Multithreading

2.1 Problems with Traditional Processes

Traditional processes have separate address spaces:

(i) Process creation is expensive.

(ii) Process switch is expensive.

(iii) Sharing memory areas among processes is non-trivial.

(iv) A process is a single thread of execution.

2.2 Threads

A **thread** is a basic unit of CPU utilization. It comprises:

(i) Thread ID

(ii) Program Counter (PC)

(iii) A register set

(iv) A stack

It shares with other threads belonging to the same process its code section, data section, and other operating-system resources, such as open files and signals.

Many operating system kernels are now multithreaded.

2.3 Types of Threads

There are two types:

(i) **User-Level Threads**: Managed by thread library. Kernel is not aware of threads.

(ii) **Kernel-Level Threads**: All thread management done in kernel. Kernel is aware of threads.

2.4 Multithreading Models

There must exist a relationship between user threads and kernel threads. There are three common ways of establishing this relationship:

(i) **Many-to-One Model**: Maps many user-level threads to one kernel thread.

(ii) **One-to-One Model**: Maps each user thread to a kernel thread. It provides more concurrency than the many-to-one model by allowing another thread to run when a thread makes a blocking system call.

(iii) **Many-to-Many Model**: It multiplexes many user-level threads to a smaller or equal number of kernel threads.

2.5 Benefits of Multithreading

(i) **Responsiveness**: Multithreading an interactive application may allow a program to continue running even if part of it is blocked, thereby increasing responsiveness to the user.

(ii) **Resource Sharing**: Threads share the address space and the resources of the process to which they belong.

(iii) **Economy**: Thread creation is much simpler than process creation. Thread switch is simpler.

(iv) **Utilization of Multiprocessor Architectures**: Threads can run in parallel on different processors.

Chapter 3

Process Scheduling

3.1 Multiprogramming

The objective of multiprogramming is to have some process running at all times, to maximize CPU utilization. In multiprogrammed systems, the user cannot interact with the system.

3.2 Time Sharing

The objective of time-sharing (also called multitasking systems) is to switch the CPU among processes so frequently that users can interact with each program while it is running.

For a time-sharing system, the response time should be short.

Time-sharing systems internally use multiprogramming.

3.3 Scheduling Queues

A process migrates among the various scheduling queues throughout its lifetime. The schedulers are used to select the processes from these queues in some fashion.

(i) **Job Queue**: As processes enter the system, they are put into a job queue, which consists of all processes in the system.

(ii) **Ready Queue**: The processes that are residing in main memory and are ready and waiting to execute are kept on a list called the ready queue.

(iii) **Device Queue**: The list of processes waiting for a particular I/O device is called a device queue. Each device has its own device queue.

3.4 Schedulers

A process migrates among the various scheduling queues throughout its lifetime. The schedulers are used to select the processes from these queues in some fashion.

There are generally three types of schedulers:

(1) **Long-term Scheduler or Job Scheduler**

(2) **Short-term Scheduler or CPU Scheduler**

(3) **Medium-term Scheduler**

3.4.1 Long-term Scheduler or Job Scheduler

The long-term scheduler selects processes from the job queue and loads them into memory for execution. The long-term scheduler controls the degree of multiprogramming.

3.4.2 Short-term Scheduler or CPU Scheduler

The short-term scheduler selects from among the processes that are ready to execute and allocates the CPU to one of them. The short-term scheduler must select a new process for the CPU frequently and it must be fast.

3.4.3 Medium-term Scheduler

The key idea behind a medium-term scheduler is that sometimes it can be advantageous to remove processes from memory (and thus reduce the degree of multiprogramming). Later, the process can be reintroduced into memory, and its execution can be continued where it left off. This scheme is called **swapping**. The process is swapped out, and is later swapped in, by the medium-term scheduler.

On some systems, the long-term scheduler may be absent or minimal, such as some time-sharing systems.

The medium-term scheduler is generally used in time-sharing systems.

3.5 Context Switch

Switching the CPU to another process requires performing a state save of the current process and a state restore of a different process. This task is known as a **context switch**.

Context-switch time is pure overhead, because the system does no useful work while switching.

Its speed varies from machine to machine, depending on the memory speed, the number of registers that must be copied, and the existence of special instructions (such as a single instruction to load or store all registers).

3.6 Dispatcher

The dispatcher is the module that gives control of the CPU to the process selected by the short-term scheduler. This function involves:

(i) Switching context

(ii) Switching to user mode

(iii) Jumping to the proper location in the user program to restart the program

The dispatcher should be as fast as possible.

The time taken by the dispatcher to stop one process and start another running is known as the **dispatch latency**.

3.7 Preemptive and Nonpreemptive Scheduling

Under nonpreemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state.

Nonpreemptive scheduling is also called **cooperative scheduling**.

Whenever a process switches from the running state to the ready state or waiting state to the ready state, and if scheduling takes place in these cases, then the scheduling is called **preemptive scheduling**.

A scheduling is preemptive if once a process has been given the CPU, it can be taken away.

Preemptive scheduling incurs a cost associated with access to shared data. The shared data can become inconsistent if one process that is updating some data is preempted by another process, and after that another process tries to read this data.

3.8 Scheduling Criteria

It is desirable to maximize CPU utilization and throughput, and to minimize turnaround time, waiting time, and response time.

(i) **CPU Utilization**: The CPU should be as busy as possible.

(ii) **Throughput**: The number of processes that are completed per unit time is called throughput.

(iii) **Turnaround Time**: The interval from the time of submission of a process to the time of completion is the turnaround time.

(iv) **Waiting Time**: Waiting time is the sum of the periods spent waiting in the ready queue.

(v) **Response Time**: The time from the submission of a request until the first response is produced is called response time.

Chapter 4

Scheduling Algorithms

4.1 First-Come, First-Served Scheduling

The process that requests the CPU first is allocated to the CPU first. It is implemented using a FIFO queue.

FCFS scheduling algorithm is non-preemptive. The average waiting time in FCFS is often quite long.

4.2 Shortest-Job-First Scheduling

This algorithm associates with each process the length of the process's next CPU burst. When the CPU is available, it is assigned to the process that has the smallest next CPU burst.

If the next CPU bursts of two processes are the same, the FCFS scheduling is used to break the tie.

The SJF algorithm is optimal, in that it gives the minimum average waiting time for a given set of processes.

The SJF algorithm cannot be implemented at the level of short-term CPU scheduling. There is no way to know the length of the next CPU burst.

The SJF algorithm can be either preemptive or nonpreemptive.

4.2.1 Preemptive SJF Scheduling

Preemptive SJF scheduling is sometimes called **Shortest-Remaining-Time-First Scheduling**.

4.3 Priority Scheduling

A priority is associated with each process, and the CPU is allocated to the process with the highest priority.

Equal-priority processes are scheduled in FCFS order.

Priority scheduling can be either preemptive or nonpreemptive.

4.3.1 Aging

A preemptive priority scheduling algorithm will preempt the CPU if the priority of the newly arrived process is higher than the priority of the currently running process.

A major problem with priority scheduling algorithms is indefinite blocking or starvation.

A solution to the problem of indefinite blocking of low-priority processes is **aging**.

Aging is a technique of gradually increasing the priority of processes that wait in the system for a long time.

4.4 Round-Robin Scheduling

The round-robin scheduling algorithm is designed especially for time-sharing systems.

It is similar to FCFS scheduling, but preemption is added to switch between processes. The ready queue is treated as a circular queue. The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum.

4.5 SCAN Scheduling

The disk arm starts at one end of the disk and moves towards the other end, servicing requests as it reaches each cylinder. At the other end, the direction of head movement is reversed. It is sometimes called the elevator algorithm.

4.6 Circular SCAN Scheduling

It is similar to SCAN scheduling. However, when the head reaches the other end, it immediately returns to the beginning of the disk without servicing any requests on the return trip.

Chapter 5

Multiprocessor Scheduling

There are two approaches to multiprocessor scheduling:

(1) **Asymmetric Multiprocessing**: All scheduling decisions, I/O processing, and other system activities are handled by a single processor, the master server.

(2) **Symmetric Multiprocessing (SMP)**: Here, each processor is self-scheduling.

5.1 Processor Affinity

When a process migrates from one processor to another processor, the contents of the cache memory must be invalidated for the processor being migrated from, and the cache must be repopulated for the processor being migrated to.

Because of the high cost of invalidating and re-populating caches, most SMP systems try to avoid migration of processes and instead attempt to keep a process running on the same processor. This is known as **processor affinity**.

5.2 Load Balancing

Load balancing attempts to keep the workload evenly distributed across all processors in an SMP system. The load balancing is typically only necessary on systems where each processor has its own private queue of eligible processes to execute.

There are two approaches to load balancing:

(1) **Push Migration**: A specific task periodically checks the load on each processor and, if it finds an imbalance, evenly distributes the load by moving processes from overloaded to idle or less-busy processors.

(2) **Pull Migration**: It occurs when an idle processor pulls a waiting task from a busy processor.

Push and pull migration need not be mutually exclusive and are often implemented in parallel.

Chapter 6

Deadlocks

6.1 Introduction

A set of processes is in a deadlock state when every process in the set is waiting for an event that can be caused only by another process in the set.

6.2 Necessary Conditions for Deadlock

A deadlock situation can arise if the following four conditions hold simultaneously in a system:

(1) **Mutual Exclusion**: At least one resource must be held in a non-sharable mode; that is, only one process at a time can use the resource.

(2) **Hold and Wait**: A process must be holding at least one resource and waiting for acquiring additional resources that are currently being held by other processes.

(3) **No Preemption**: Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.

(4) **Circular Wait**: A set $\{P_0, P_1, \ldots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource held by P_1, P_1 is waiting for a resource held by P_2, and P_n is waiting for a resource held by P_0.

These four conditions are not completely independent.

6.3 Resource Allocation Graph

A **resource allocation graph** is used for describing deadlocks precisely.

The graph consists of a set of vertices V and a set of edges E. The set of vertices V is partitioned into two different types of nodes:

- $P = \{P_1, P_2, \ldots, P_n\}$: The set consisting of all the active processes in the system.
- $R = \{R_1, R_2, \ldots, R_m\}$: The set consisting of all resource types in the system.

6.3.1 Request Edge

A directed edge from process P_i to resource type R_j, denoted by $P_i \rightarrow R_j$, is called a request edge. It signifies that process P_i has requested an instance of resource type R_j and is currently waiting for that resource.

6.3.2 Assignment Edge

A directed edge from resource type R_j to process P_i, denoted by $R_j \rightarrow P_i$, is called an assignment edge. It signifies that an instance of resource type R_j has been allocated to process P_i.

6.3.3 Observations

- If the graph contains no cycles, then no process in the system is deadlocked.
- If the graph does contain a cycle, then a deadlock may exist.
- If the cycle involves only a set of resource types, each of which has only a single instance, then a deadlock has occurred.
- If each resource type has several instances, then a cycle does not necessarily imply that a deadlock has occurred.

6.4 Methods for Handling Deadlocks

There are three ways to deal with the deadlock problem:

(i) We can use a protocol to prevent or avoid deadlocks, ensuring that the system will never enter a deadlock state.

(ii) We can allow the system to enter a deadlock state, detect it, and recover.

(iii) We can ignore the problem altogether and pretend that deadlocks never occur in the system.

6.5 Deadlock Prevention

By ensuring that at least one of these conditions cannot hold, we can prevent the occurrence of a deadlock:

(i) **Mutual Exclusion**: At least one resource must be held in a non-sharable mode. That is, only one process at a time can use the resource.

(ii) **Hold and Wait**: A process must be holding at least one resource and waiting for acquiring additional resources that are currently being held by other processes.

(iii) **No Preemption**: Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.

(iv) **Circular Wait**: A set $\{P_0, P_1, P_2, \ldots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource held by P_1, P_1 is waiting for a resource held by P_2, and P_n is waiting for a resource held by P_0.

The drawback of this method is low device utilization and reduced system throughput.

6.6 Deadlock Avoidance

A deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that a circular wait condition can never exist. The resource-allocation state is defined by the number of available and allocated resources and the maximum demands of the processes.

There are two algorithms for deadlock avoidance:

(i) Resource-Allocation-Graph Algorithm

(ii) Banker's Algorithm

6.6.1 Safe State

A state is safe if the system can allocate resources to each process in some order and still avoid a deadlock. More formally, a system is in a safe state only if there exists a safe sequence.

A sequence of processes $\langle P_1, P_2, \ldots, P_n \rangle$ is a safe sequence for the current allocation state if, for each P_i, the resource requests that P_i can still make can be satisfied by the currently available resources plus the resources held by all P_j, with $j < i$.

A safe state is not a deadlocked state. Conversely, a deadlocked state is an unsafe state.

Not all unsafe states are deadlocks; however, an unsafe state may lead to a deadlock.

6.6.2 Banker's Algorithm

This algorithm is used when a resource type may have multiple instances.

When a new process enters the system, it must declare the maximum number of instances of each resource type that it may need. This number may not exceed the total number of resources in the system.

When a user requests a set of resources, the system must determine whether the allocation of these resources will leave the system in a safe state. If it will, the resources are allocated; otherwise, the process must wait until some other process releases enough resources.

Let n be the number of processes in the system and m be the number of resource types. We need the following data structures:

(i) **Available**: A vector of length m indicates the number of available resources of each type. If Available$[j] = k$, there are k instances of resource type R_j available.

(ii) **Max**: An $n \times m$ matrix defines the maximum demand of each process. If Max$[i][j] = k$, then process P_i may request at most k instances of resource type R_j.

(iii) **Allocation**: An $n \times m$ matrix defines the number of resources of each type currently allocated to each process. If Allocation$[i][j] = k$, then process P_i is currently allocated k instances of resource type R_j.

(iv) **Need**: An $n \times m$ matrix indicates the remaining resource need of each process. If Need$[i][j] = k$, then process P_i may need k more instances of resource type R_j to complete its task.

Note: Need$[i][j]$ = Max$[i][j]$ - Allocation$[i][j]$

6.7 Recovery from Deadlock

When a deadlock detection algorithm determines that a deadlock exists, several alternatives are available:

(1) **Process Termination**: Two methods are available:

- Abort all deadlocked processes.
- Abort one process at a time until the deadlock cycle is eliminated.

(2) **Resource Preemption**: To eliminate deadlocks using resource preemption, we successively preempt some resources from processes and give these resources to other processes until the deadlock cycle is broken.

(3) **Rollback**: If we preempt a resource from a process, we must rollback the process to some safe state and restart it from that state.

(4) **Starvation**: How can we guarantee that resources will not always be preempted from the same process?

Chapter 7

Deadlocks

7.1 Introduction

A set of processes is in a deadlock state when every process in the set is waiting for an event that can be caused only by another process in the set.

7.2 Necessary Conditions for Deadlock

A deadlock situation can arise if the following four conditions hold simultaneously in a system:

(1) **Mutual Exclusion**: At least one resource must be held in a non-sharable mode; that is, only one process at a time can use the resource.

(2) **Hold and Wait**: A process must be holding at least one resource and waiting for acquiring additional resources that are currently being held by other processes.

(3) **No Preemption**: Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.

(4) **Circular Wait**: A set $\{P_0, P_1, \ldots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource held by P_1, P_1 is waiting for a resource held by P_2, and P_n is waiting for a resource held by P_0.

These four conditions are not completely independent.

7.3 Resource Allocation Graph

A **resource allocation graph** is used for describing deadlocks precisely.

The graph consists of a set of vertices V and a set of edges E. The set of vertices V is partitioned into two different types of nodes:

- $P = \{P_1, P_2, \ldots, P_n\}$: The set consisting of all the active processes in the system.
- $R = \{R_1, R_2, \ldots, R_m\}$: The set consisting of all resource types in the system.

7.3.1 Request Edge

A directed edge from process P_i to resource type R_j, denoted by $P_i \rightarrow R_j$, is called a request edge. It signifies that process P_i has requested an instance of resource type R_j and is currently waiting for that resource.

7.3.2 Assignment Edge

A directed edge from resource type R_j to process P_i, denoted by $R_j \rightarrow P_i$, is called an assignment edge. It signifies that an instance of resource type R_j has been allocated to process P_i.

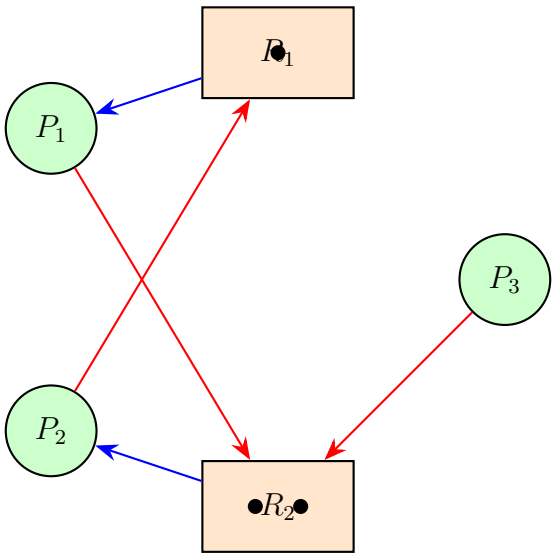


Figure 7.1: Resource Allocation Graph Example (With Cycle - Possible Deadlock)

7.3.3 Observations

- If the graph contains no cycles, then no process in the system is deadlocked.
- If the graph does contain a cycle, then a deadlock may exist.
- If the cycle involves only a set of resource types, each of which has only a single instance, then a deadlock has occurred.
- If each resource type has several instances, then a cycle does not necessarily imply that a deadlock has occurred.

7.3.4 Shared Memory Model

In this model, a region of memory that is shared by cooperating processes is established. Typically, a shared-memory region resides in the address space of the process creating the shared-memory segment. Other processes that wish to communicate using this shared-memory segment must attach it to their address space.

The processes are also responsible for ensuring that they are not writing to the same location simultaneously.

7.4 Message-Passing Systems

Message passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space. It is particularly useful in a distributed environment where the communicating processes may reside on different computers connected by a network.

A message-passing facility provides at least two operations:

(i) Send (message)

(ii) Receive (message)

The communication between different processes in a message-passing system can be of the following types:

(a) **Synchronous or Asynchronous Communication**

(b) **Direct or Indirect Communication**

7.4.1 Synchronous Communication

In synchronous communication, processes synchronize at every message. Both send and receive are blocking operations.

7.4.2 Asynchronous Communication

The send operation is almost always non-blocking. The receive operation, however, can be blocking or non-blocking.

7.4.3 Blocking Send

The sending process is blocked until the message is received by the receiving process.

7.4.4 Nonblocking Send

The sending process sends the message and resumes operation.

7.4.5 Blocking Receive

The receiver blocks until a message is available.

7.4.6 Nonblocking Receive

The receiver retrieves either a valid message or a null.

7.4.7 Direct Communication

Processes must explicitly name the receiver or sender of a message (symmetric addressing):

- send(P, message). Send message to process P.
- receive(Q, message). Receive message from Q.

In a client-server system, the server does not have to know the name of a specific client in order to receive a message. In this case, a variant of the receive operation can be used (asymmetric addressing):

- receive(ID, message). Receive a pending message from any process. When a message arrives, ID is set to the name of the sender.

7.4.8 Indirect Communication

With indirect communication, the messages are sent to and received from mailboxes, or ports.

A mailbox can be viewed abstractly as an object into which messages can be placed by processes and from which messages can be removed.

The send() and receive() primitives are defined as follows:

- send(A, message). Send a message to mailbox A.
- receive(A, message). Receive a message from mailbox A.

Generally, a mailbox is associated with many senders and receivers.

In some systems, only one receiver is associated with a particular mailbox; such a mailbox is often called a **port**.

7.5 Producer-Consumer Problem

A producer process produces information that is consumed by a consumer process.

One solution to the producer-consumer problem uses shared memory. To allow producer and consumer processes to run concurrently, we must have available a buffer of items that can be filled by the producer and emptied by the consumer.

The producer and consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been produced.

Two types of buffers can be used:

(i) **Unbounded Buffer**: It places no practical limit on the size of the buffer. The producer can always produce new items, but the consumer may have to wait for new items.

(ii) **Bounded Buffer**: It assumes a fixed buffer size. In this case, the consumer must wait if the buffer is empty, and the producer must wait if the buffer is full.

7.6 Race Condition

A situation where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a **race condition**.

To guard against the race condition, we need to ensure that only one process at a time can be manipulating the variable counter. To make such a guarantee, we require that the processes be synchronized in some way.

Chapter 8

The Critical-Section Problem

8.1 Introduction

Consider a system consisting of n processes $\{P_0, P_1, \ldots, P_{n-1}\}$. Each process has a segment of code, called a **critical section**, in which the process may be changing common variables, updating a table, writing a file, and so on.

The important feature of the system is that, when one process is executing in its critical section, no other process is to be allowed to execute in its critical section.

The **critical-section problem** is to design a protocol that the processes can use to cooperate. Each process must request permission to enter its critical section.

8.1.1 Structure of a Process

```
do {
    entry section
    critical section
    exit section
    remainder section
} while (true);
```

8.2 Solution to the Critical-Section Problem

A solution to the critical-section problem must satisfy the following three requirements:

(1) **Mutual Exclusion**: If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.

(2) **Progress**: If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in the decision on which will enter its critical section next, and this selection cannot be postponed indefinitely.

(3) **Bounded Waiting**: There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Two general approaches are used to handle critical sections in operating systems:

(i) **Preemptive Kernels**: Allows a process to be preempted while it is running in kernel mode.

(ii) **Nonpreemptive Kernels**: Does not allow a process running in kernel mode to be preempted; it will run until it exits kernel mode, blocks, or voluntarily yields control of the CPU.

A nonpreemptive kernel is essentially free from race conditions on kernel data structures, as only one process is active in the kernel at a time.

8.3 Peterson's Solution

Peterson's solution is restricted to two processes that alternate execution between their critical sections and remainder sections. The processes are numbered P_i and P_j, where $j = 1 - i$.

Peterson's solution requires two data items to be shared between the two processes:

```
int turn;
boolean flag[2];
```

The variable `turn` indicates whose turn it is to enter the critical section.

The `flag` array is used to indicate that a process is ready to enter its critical section.

8.3.1 The Structure of Process P_i in Peterson's Solution

```
do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);
    critical section
    flag[i] = false;
    remainder section
} while (true);
```

To enter the critical section, process P_i first sets `flag[i]` to be true and then sets `turn` to the value j, thereby asserting that if the other process wishes to enter the critical section, it can do so.

If both processes try to enter at the same time, turn will be set to both i and j at roughly the same time. Only one of these assignments will last; the other will occur but will be overwritten immediately. The eventual value of `turn` decides which of the two processes is allowed to enter its critical section first.

Peterson's solution is a software-based solution.

Chapter 9

Synchronization Hardware

9.1 Introduction

In general, we can state that any solution to the critical-section problem requires a simple tool, a **lock**. Race conditions are prevented by requiring that critical regions be protected by locks.

9.2 TestAndSet Instruction

The important characteristic of this instruction is that it is executed **atomically**. Thus, if two `TestAndSet()` instructions are executed simultaneously (each on a different CPU), they will be executed sequentially in some arbitrary order.

9.2.1 The TestAndSet Instruction

```
boolean TestAndSet(boolean *lock) {
    boolean rv = *lock;
    *lock = true;
    return rv;
}
```

If the machine supports the `TestAndSet()` instruction, then mutual exclusion can be provided.

9.3 Swap Instruction

The `Swap()` instruction operates on the contents of two words. Like the `TestAndSet()` instruction, it is executed atomically.

```
void Swap(boolean *lock, boolean *key) {
    boolean temp = *lock;
    *lock = *key;
    *key = temp;
}
```

9.4 Bounded-Waiting Mutual Exclusion with TestAndSet

```
do {
    waiting[i] = true;
    key = true;
    while (waiting[i] && key)
        key = TestAndSet(&lock);
    waiting[i] = false;

    critical section

    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;

    if (j == i)
        lock = false;
    else
        waiting[j] = false;

    remainder section
} while (true);
```

For the algorithm using the `TestAndSet()` instruction that satisfies all the critical-section requirements, the common data structures are:

```
boolean waiting[n];
boolean lock;
```

These data structures are initialized to `false`.

Chapter 10

Semaphores

10.1 Introduction

The various hardware-based solutions to the critical-section problem are complicated as well as generally inaccessible to application programmers. To overcome this difficulty, we can use a synchronization tool called a **semaphore**.

A semaphore S is an integer variable that, apart from initialization, is accessed only through two standard atomic operations: `wait()` and `signal()`.

The `wait()` operation was originally termed P and `signal()` was originally called V.

The definition of `wait()` is as follows:

```
wait(S) {
    while (S <= 0)
        ; /* No operation */
    S--;
}
```

The definition of `signal()` is as follows:

```
signal(S) {
    S++;
}
```

10.2 Binary Semaphores

The value of a binary semaphore can range only between 0 and 1. On some systems, binary semaphores are known as **mutex locks**, as they are locks that provide mutual exclusion.

We can use binary semaphores to deal with the critical-section problem for multiple processes. The n processes share a semaphore, `mutex`, initialized to 1.

10.3 Counting Semaphores

The value of a counting semaphore can range over an unrestricted domain. Counting semaphores can be used to control access to a given resource consisting of a finite number

of instances. The semaphore is initialized to the number of resources available. Each process that wishes to use a resource performs a `wait()` operation. When a process releases a resource, it performs a `signal()` operation.

When the count for the semaphore goes to 0, all resources are being used. Processes that wish to use a resource will block until the count becomes greater than 0.

10.4 Semaphore Implementation

The main disadvantage of the semaphore definition given here is that it requires **busy waiting**. While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the entry code. This type of semaphore is also called a **spinlock** because the process "spins" while waiting for the lock.

To overcome the need for busy waiting, we can modify the definition of the `wait()` and `signal()` semaphore operations.

10.4.1 Wait Operation Implementation

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}
```

10.4.2 Signal Operation Implementation

When some other process executes a `signal()` operation, which changes the process from the waiting state to the ready state, the process is placed in the ready queue.

```
signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```

Note: Although under the classical definition of semaphores with busy waiting, the semaphore value is never negative, this implementation may have negative semaphore values. If the semaphore value is negative, its magnitude is the number of processes waiting on that semaphore.

The critical aspect of semaphores is that they be executed atomically. We must guarantee that no two processes can execute `wait()` and `signal()` operations on the same semaphore at the same time.

This is a critical-section problem; and in a single-processor environment, we can solve it by simply inhibiting interrupts during the time the `wait()` and `signal()` operations are executing.

In a multiprocessor environment, disabling interrupts on every processor can be a difficult task. Therefore, SMP systems must provide alternative locking techniques to ensure that `wait()` and `signal()` are performed atomically.

10.5 Deadlocks and Starvation

The implementation of a semaphore with a waiting queue may result in a situation where two or more processes are waiting indefinitely for an event that can be caused only by one of the waiting processes. The event in question is the execution of a `signal()` operation.

When such a state is reached, these processes are said to be **deadlocked**.

Another problem related to deadlocks is **indefinite blocking or starvation**. A situation where processes wait indefinitely within the semaphore. Indefinite blocking may occur if we add and remove processes from the list associated with a semaphore in LIFO order.

Chapter 11

Code for Producer and Consumer

11.1 Producer-Consumer Problem with Race Condition

11.1.1 Code for Consumer

```
item nextConsumed;
while (true) {
    while (in == out)
        ; /* do nothing */
    nextConsumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    /* consume the item in nextConsumed */
}
```

The producer process has a local variable `nextProduced` in which the new item to be produced is stored. The consumer process has a local variable `nextConsumed` in which the item to be consumed is stored.

This scheme allows at most `BUFFER_SIZE - 1` items in the buffer at the same time. One possible remedy is to add an integer variable `counter`, initialized to 0. Counter is incremented every time we add a new item to the buffer and is decremented every time we remove one item from the buffer.

11.2 Modified Code for Producer

```
while (true) {
    /* produce an item in nextProduced */
    while (counter == BUFFER_SIZE)
        ; /* do nothing */
    buffer[in] = nextProduced;
    in = (in + 1) % BUFFER_SIZE;
    counter++;
}
```

11.3 Modified Code for Consumer

```
while (true) {
    while (counter == 0)
        ; /* do waiting */
    nextConsumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    counter--;
    /* Consume the item in nextConsumed */
}
```

Although both the producer and consumer routines are correct separately, they may not function correctly when executed concurrently.

Suppose that the value of the variable counter is currently 5 and that the producer and consumer processes execute the statements `counter++` and `counter--` concurrently. Following the execution of these two statements, the value of the variable counter may be 4, 5, or 6!

The only correct result, though, is `counter == 5`, which is generated correctly if the producer and consumer execute separately.

Chapter 12

File Systems

12.1 File System Structure

A file system is used to allow the data to be stored, located, and retrieved easily. The file system itself is generally composed of many different levels.

12.1.1 Levels of File System

- Application Programs
- Logical File System
- File-Organization Module
- Basic File System
- I/O Control
- Devices

12.2 I/O Control

I/O control consists of device drivers and interrupt handlers to transfer information between the main memory and the disk system.

12.3 Basic File System

It needs only to issue generic commands to the appropriate device driver to read and write physical blocks on the disk.

12.4 File Organization Module

It knows about files and their logical blocks as well as physical blocks. It translates logical block addresses to physical block addresses.

12.5 Logical File System

It manages the metadata information. It manages the directory structure via file-control blocks.

A **file control block (FCB)** contains information about the file, including ownership, permissions, and location of the file contents.

12.6 File Systems Used by Various Operating Systems

(i) **UNIX** → UNIX File System (UFS), which is based on Berkeley Fast File System (FFS).

(ii) **Windows** → FAT, FAT32, NTFS

(iii) **Linux** → Extended File System (ext2 & ext3)

12.7 On-Disk and In-Memory Structures

Several on-disk and in-memory structures are used to implement a file system. These structures vary depending on the operating system and the file system, but some general principles apply.

On disk, the file system may contain the following information:

(i) **Boot Control Block (per volume)**: Contains information needed by the system to boot an OS from that volume. In UFS, it is called the **boot block**. In NTFS, it is called the partition boot sector.

(ii) **Volume Control Block (per volume)**: Contains volume (or partition) details such as the number of blocks in the partition and the size of blocks, free-block count and free-block pointers. In UFS, it is called a **superblock**. In NTFS, it is stored in the master file table.

(iii) **Directory Structure (per file system)**: Used to organize the files. In UFS, this includes file names and associated inode numbers. In NTFS, it is stored in the master file table.

(iv) **File Control Block - FCB (per file)**: Contains details about the file, including file permissions, ownership, size, etc. In UFS, this is called the **inode block**. In NTFS, this information is actually stored within the master file table.

Chapter 13

Disk Allocation Methods

The major methods of allocating disk space are in wide use:

(1) Contiguous Allocation

(2) Linked Allocation

(3) Indexed Allocation

13.1 Contiguous Allocation

Chapter 14

File Organization and Indexes

14.1 File Organization

14.1.1 Spanned Records

If records can span more than one block in a disk, this organization is called spanned.

Whenever a record is larger than a block, we must use a spanned organization.

14.1.2 Unspanned Records

If records are not allowed to cross block boundaries, the organization is called unspanned. This is used with fixed-length records having $B > R$.

14.1.3 Blocking Factor

Suppose that the block size is B bytes. For a file of fixed-length records of size R bytes, with $B \geq R$, we can fit bfr $= \lfloor B/R \rfloor$ records per block.

The value bfr is called the blocking factor.

The maximum number of records that can fit in a disk block is called the blocking factor (bfr).

14.2 Types of Organization of Files

There are three types of organization of files:

(1) **Files of Unordered Records (Heap Files)**: The records are placed in the file in the order in which they are inserted, so new records are inserted at the end of the file.

(2) **Files of Ordered Records (Sorted Files)**: We can physically order the records of a file on disk based on the values of one of their fields, called the **ordering field**. If the ordering field is also a key field of the file, then the field is called the **ordering key field**.

(3) **Hash Files**: The idea behind hashing is to provide a function h, called a **hash function**, that is applied to the hash field value of a record and yields the address of the disk block in which the record is stored.

For most records, we need only a single-block access to retrieve that record.
The search condition must be an equality condition on a single field.
All the above organizations of files are called primary organization.

14.3 Indexes

The index structures typically provide secondary access paths to access the records without affecting the physical placement of records on disk.

14.3.1 Dense and Sparse Indexes

A **dense index** has an index entry for every search key value in the search key.

A **sparse index** has index entries for only some of the search values.

14.3.2 Types of Indexes

(1) Single-Level Ordered Indexes

   (a) Primary Indexes

   (b) Clustering Indexes

   (c) Secondary Indexes

(2) Multi-Level Indexes

14.4 Single-Level Ordered Indexes

14.4.1 Primary Indexes

A **primary index** is a non-dense (sparse) index.

A major problem with a primary index—as with any ordered file—is insertion and deletion of records.

14.4.2 Clustering Indexes

If records of a file are physically ordered on a non-key field, that field is called the **clustering field**.

A **clustering index** is also an ordered file with two fields. The first field is of the same data type as some non-ordering field of the data file, and the second field is a block pointer.

There is one entry in the clustering index for each distinct value of the clustering field, containing the value and the pointer to the first block in the data file that has a record with that value.

A clustering index is also a non-dense (sparse) index, because it has an entry for every distinct value of the indexing field, rather than for every record in the file.

14.4.3 Secondary Indexes

It is also an ordered file with two fields. The first field is of the same data type as some non-ordering field of the data file that is an indexing field. The second field is either a block pointer or a record pointer.

There can be many secondary indexes (and hence, secondary key fields) for the same file.

If the secondary index is on a key field, then there is one index entry for each record in the data file. Hence, such an index is dense.

If the secondary index is on a non-key field, then numerous records in the data file can have the same value for the indexing field. There are various options for implementing such an index.

A secondary index provides a logical ordering on the records by the indexing field.

14.5 Multilevel Indexes

A **multilevel index** considers the index file (first level) as an ordered file with a distinct value for each $K(i)$. Hence, we create a primary index for the first level.

The index to the first level is called the second level of the multilevel index, and so on.

We require a second level only if the first level needs more than one block of disk storage and so on.

Each level reduces the number of entries at the previous level by a factor of f_o - the index fan-out.

If the first level has r_1 entries and the blocking factor—which is also called the **fan-out**—of the index is f_o, then the first level needs $\lceil r_1/f_o \rceil$ blocks, which is therefore the number of entries needed at the second level of the index.

14.6 B-Trees

14.6.1 Introduction to B-Trees

The **B-tree** is a search tree with additional constraints that ensure that the tree is always balanced.

A B-tree of order p, when used as an access structure on a key field to search for records in a data file, can be defined as follows:

(1) Each internal node in the B-tree is of the form:

$$\langle P_1, \langle K_1, P_{r_1} \rangle, P_2, \langle K_2, P_{r_2} \rangle, \ldots, \langle K_{q-1}, P_{r_{q-1}} \rangle, P_q \rangle$$

where $q \leq p$. Each P_i is a tree pointer—a pointer to another node in the B-tree. Each P_{r_i} is a data pointer—a pointer to the record whose search key field value is equal to K_i (or to the data file block containing that record).

(2) Within each node, $K_1 < K_2 < \ldots < K_{q-1}$.

(3) For all search key field values X in the subtree pointed at by P_i, we have:

$$\begin{aligned} K_{i-1} < X < K_i & \quad \text{for } 1 < i < q, \\ X \leq K_i & \quad \text{for } i = 1, \text{ and} \\ K_{i-1} < X & \quad \text{for } i = q. \end{aligned}$$

(4) Each node has at most p tree pointers.

(5) Each node, except the root and leaf nodes, has at least $\lceil p/2 \rceil$ tree pointers. The root node has at least two tree pointers unless it is the only node in the tree.

(6) All leaf nodes are at the same level. Leaf nodes have the same structure as internal nodes except that all of their tree pointers P_i are null.

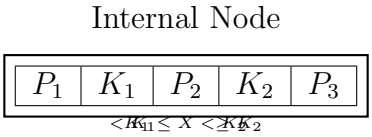


Figure 14.1: B-Tree Node Structure (order p)

14.6.2 B+ Tree Structure

In a B+ tree:

- Internal nodes contain only keys and tree pointers
- Leaf nodes contain keys and data pointers
- All data pointers are at the leaf level
- Leaf nodes are linked together for sequential access

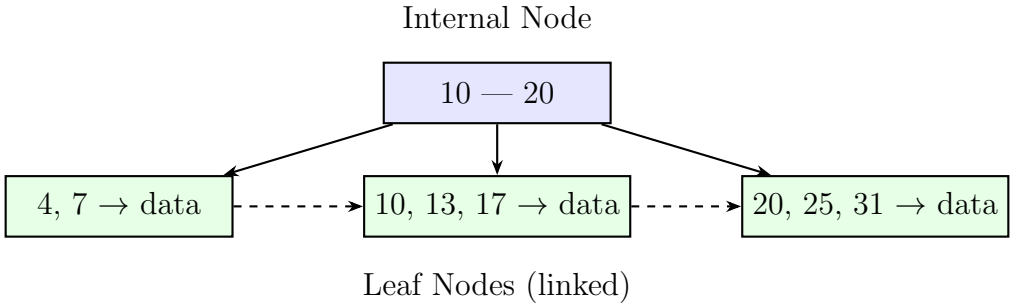


Figure 14.2: B+ Tree Example Structure

14.6.3 Insertion in B+ Trees

Overflow

When the number of search key values exceeds $p - 1$.

Splitting Leaf Node into Two Nodes

- Left node contains $\lceil p/2 \rceil$ values.
- 2nd node contains remaining values.
- Copy the maximum search key value of the 1st node to the parent node.

Example: B+ Tree Insertion with Splitting

Consider inserting values 7, 9, 13, 15 into a B+ tree of order $p = 4$ (can hold 3 keys per node).

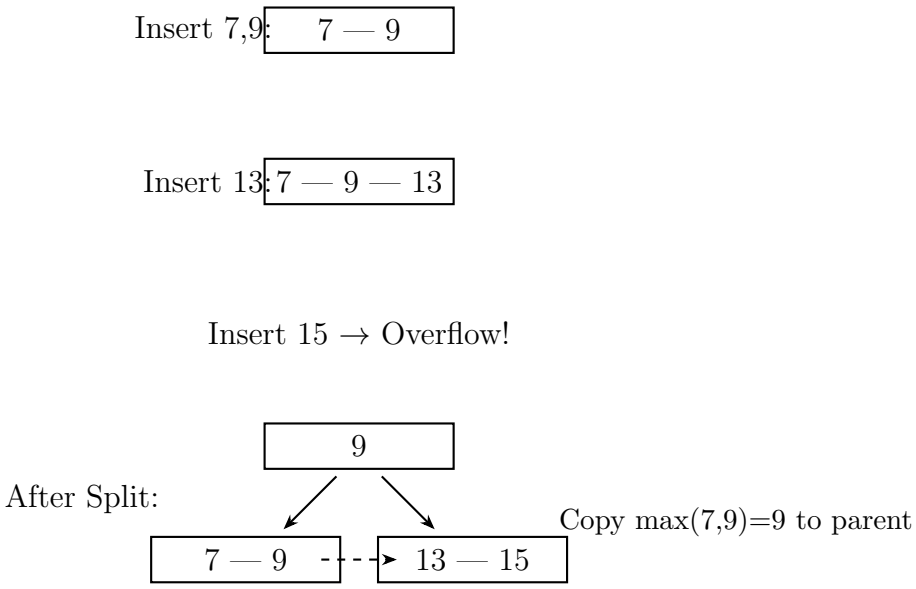


Figure 14.3: B+ Tree Insertion Example with Leaf Node Splitting

Splitting Non-Leaf Node into Two Nodes

- 1st node contains $\lceil p/2 \rceil$ values.
- Move the smallest of the remaining values to parent, with pointer to the parent.
- 2nd node contains the remaining values.

14.6.4 Deletion in B+ Trees

Underflow

When number of search key values $< \lceil p/2 \rceil - 1$.

Leaf Node Deletion

(a) **Redistribute to sibling**: Right node should not be less than left node.

(b) Move the smallest value of the right node to left node.

(c) Replace the between value in parent by the smallest value of the right node.

Merge Operation

(a) Both left and right nodes have $\lceil (p-1)/2 \rceil$ values.

(b) Move all values, pointers to left node.

(c) Delete the right node and pointers in parent.

Example: B+ Tree Deletion with Redistribution

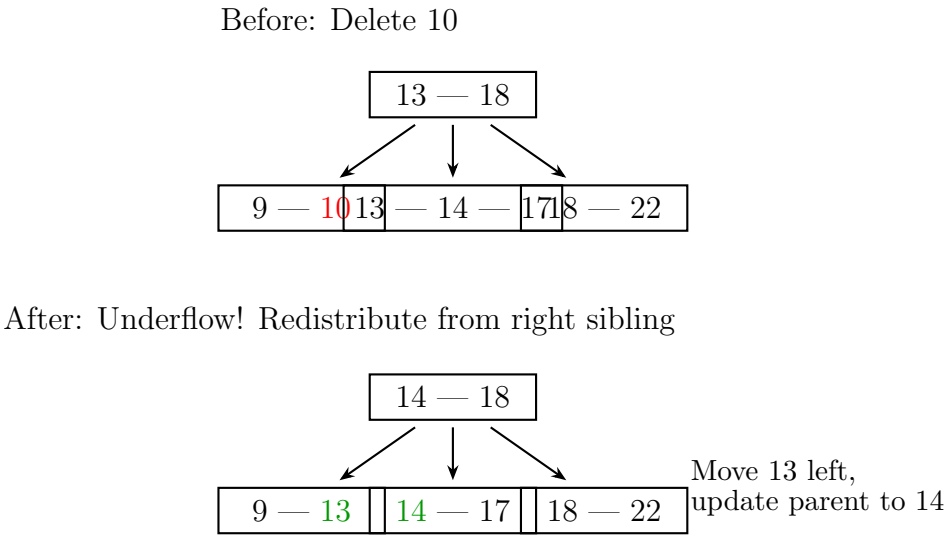


Figure 14.4: B+ Tree Deletion Example with Redistribution

Example: B+ Tree Deletion with Merge

Non-Leaf Node Deletion

(a) Redistribute to sibling: Through parent.

(b) Right node should not be less than left node.

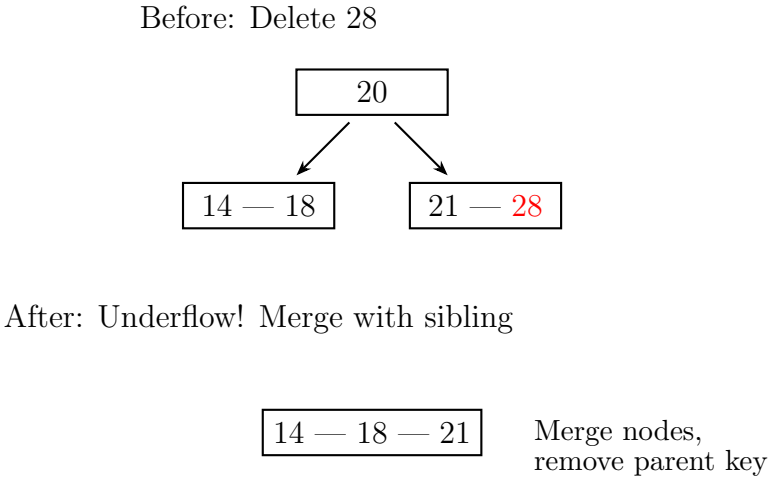


Figure 14.5: B+ Tree Deletion Example with Merge

Merge for Non-Leaf Nodes

(a) Both left and right node have $\lceil p/2 \rceil$ values.

(b) Move all values, pointers to left node.

(c) Delete the right node, and pointers in parent.

14.6.5 Shortest Seek Time First Scheduling

The SSTF algorithm selects the request with the minimum seek time from the head position. It may cause starvation of some requests.

This algorithm is not optimal.

14.6.6 SCAN Scheduling

The disk arm starts at one end of the disk and moves towards the other end, servicing requests as it reaches each cylinder. At the other end, the direction of head movement is reversed. It is sometimes called the elevator algorithm.

14.6.7 Circular SCAN Scheduling

It is similar to SCAN scheduling. However, when the head reaches the other end, it immediately returns to the beginning of the disk without servicing any requests on the return trip.

14.7 Disk Formatting

A new magnetic disk is a blank slate: it is just a platter of a magnetic recording material.

14.7.1 Physical Formatting

Before a disk can store data, it must be divided into sectors that the disk controller can read and write. This process is called **low-level formatting** or **physical formatting**. It fills the disk with a special data structure for each sector.

14.7.2 Logical Formatting

Here, the operating system stores the initial file-system data structures onto the disk.

These data structures may include maps of free and allocated space and an initial empty directory.

14.8 Raw Disk

Some operating systems give special programs the ability to use a disk partition as a large sequential array of logical blocks, without any file-system data structures. This array is sometimes called **raw disk**, and I/O to this array is termed **raw I/O**.

14.9 Disk Block

A disk block is the unit of data transfer between disk and memory.

Blocks must be multiple of sector size. Block size cannot exceed the size of track.

Theory of Automaton

Comprehensive Study Notes for GATE Computer Science

Ankur Gupta

www.ankurgupta.net

Note: These notes have been prepared primarily from the following books:

(1) **Theory of Computer Science: Automata, Languages and Computation** by K.L.P. Mishra, N. Chandrasekaran

Please refer to the above books for more details.

Contents

1 Finite Automata **5**
- 1.1 Introduction to Finite Automaton . . . 5
  - 1.1.1 Equivalence with NDFA . . . 5
- 1.2 Identities for Regular Expressions . . . 5
- 1.3 Arden's Theorem . . . 6
  - 1.3.1 Transition System Containing Λ-moves . . . 6
- 1.4 Finding the Regular Expression Recognized by a Transition System . . . 7
- 1.5 Equivalence of Two Finite Automata . . . 8

2 Minimization of Finite Automata **9**
- 2.1 Minimization Definitions . . . 9
- 2.2 Example: Construct the Minimum State Automaton . . . 9

3 Formal Languages **11**
- 3.1 Introduction . . . 11
  - 3.1.1 Grammar Definition . . . 11
- 3.2 Chomsky Classification of Languages . . . 11
- 3.3 Relation Between Languages . . . 12
- 3.4 Construction of a Transition System Accepting a Regular Grammar . . . 12

4 Context-Free Languages **14**
- 4.1 Property of Languages That Are Regular . . . 14
- 4.2 Ambiguity in Context-Free Grammars . . . 14
  - 4.2.1 Construction of Reduced Grammars . . . 14

5 Elimination of Null Productions **15**
- 5.1 Example: Find a Reduced Grammar Equivalent to the Given Grammar . 15
- 5.2 Lemma 1: Deleting a Variable B Appearing as the First Symbol on the R.H.S. . . . 16
- 5.3 Lemma 2: Eliminating Left Recursion . . . 16

6 Greibach Normal Form **17**
- 6.1 Definition . . . 17

7 Chomsky Normal Form for Context-Free Grammars **18**
- 7.1 Reduction to Chomsky Normal Form . . . 18
- 7.2 Remove A-production from the following grammar . . . 19

8 Pushdown Automata **21**
8.1 Introduction 21
8.1.1 Pushdown Automaton for $(a^n b^n \mid n \geq 1)$ (Initial Pushdown Symbol) 21
8.2 Infinite Context-Free Languages 21
8.3 Determinism and Parsing 21

9 Turing Machines and Recursive Enumerable Languages **23**
9.1 Turing Machines 23
9.2 Linear Bounded Automaton 23

10 Recursive and Recursive Enumerable Languages **24**
10.1 Definitions 24

11 Undecidability **25**
11.1 Halting Problem 25
11.2 Reduction 25
11.3 Unsolvable Problems about Grammars 26

12 Some Important Results **27**
12.1 Regular Expression Results 27

13 NP Completeness **28**
13.1 Tractable Problems 28
13.2 Intractable Problems 28
13.3 P-Class Problems 28
13.4 NP-Class Problems 28
13.5 NP-Hard Problems 29
13.5.1 P-class Problems 29
13.6 NP-Complete Problems 29
13.7 Examples of NP-Complete Problems 30

Chapter 1

Finite Automata

1.1 Introduction to Finite Automaton

Definition 1.1.1 (Finite Automaton)**.** The finite automaton has complete absence of memory outside its fixed control processor. It delivers an output at all except on an indication of whether or not the input is considered acceptable.

A deterministic finite state transducer is a device much like a DFA except that its purpose is not to accept strings or languages but to transform input strings into output strings.

1.1.1 Equivalence with NDFA

For every NDFA, there exists a DFA which simulates the behavior of NDFA. NDFA is a more general machine without being more powerful.

If an NDFA has n states, the corresponding finite automaton has 2^n states. However, we need not construct a DFA for all these 2^n states, but only for those states reachable from (q_0).

Two automata are considered to be equivalent if they accept the same language, even though they may use different methods to do so.

The class of languages accepted by finite automata is the same as the class of regular languages—those that can be described by regular expressions.

Definition 1.1.2 (Regular Language)**.** A language is regular if and only if it is accepted by a finite state automaton.

For any language L:

$$\emptyset L = L\emptyset = \emptyset$$

1.2 Identities for Regular Expressions

Let Φ, R, and S be regular expressions. The following identities hold:

(1) $\Phi + R = R$

(2) $\emptyset R = R\emptyset = \emptyset$

(3) $\Lambda R = R\Lambda = R$

(4) $\Lambda^* = \Lambda$ and $\Phi^* = \Lambda$

(5) $R + R = R$

(6) $R^*R^* = R^*$

(7) $(R^*)^* = R^*$

(8) $(R^*)^+ = R^*R^* = R^* = \Lambda + R^*R$

(9) $(PQ)^*P = P(QP)^*$

(10) $(P + Q)R = PR + QR$

(11) $R(P + Q) = RP + RQ$

(12) and $R(P + Q) = RP + RQ$

1.3 Arden's Theorem

Theorem 1.3.1 (Arden's Theorem)**.** Let P and Q be two regular expressions over Σ. If P does not contain Λ, then the following equation in R:

$$R = Q + RP$$

has a unique solution given by:

$$R = QP^*$$

1.3.1 Transition System Containing Λ-moves

Suppose we want to replace a Λ-move from vertex v_1 to vertex v_2. Then we proceed as follows:

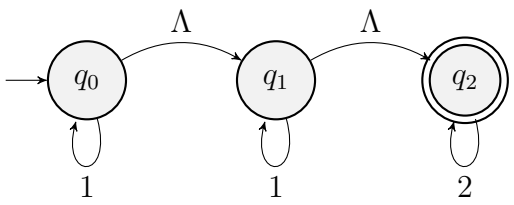
Step 1: Find all edges starting from v_2.

Step 2: Duplicate all those edges starting from v_1 without changing the edge labels.

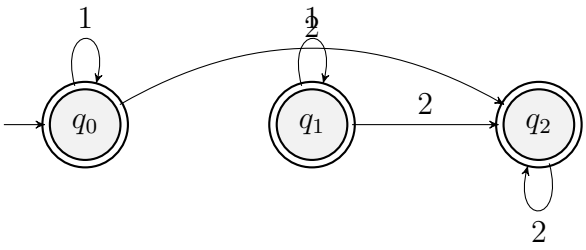
Step 3: If v_1 is an initial state, make v_2 also an initial state.

Step 4: If v_2 is a final state, make v_1 as the final state.

Example 1.3.1. Consider the following transition system:



After removing Λ-moves:



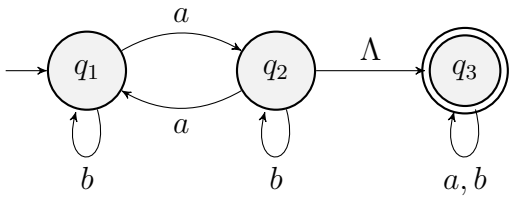
1.4 Finding the Regular Expression Recognized by a Transition System

The following assumptions are made regarding the transition system:

(i) The transition graph does not have Λ-moves.

(ii) It has only one initial state, say q_i.

Example 1.4.1. Consider the following DFA:



The equations for each state:

$$\begin{aligned} q_1 &= q_1a + q_2b + \Lambda \\ q_2 &= q_2a + q_2b + q_3a \\ q_3 &= q_2a(b + aa)^* \end{aligned}$$

Placing value of q_1 in equation (i):

$$\begin{aligned} q_2 &= q_1a + q_2b + q_3aa \\ &\Rightarrow q_2 = \Lambda q_1(a + a(b + aa)^*b) \\ &\Rightarrow q_1 = (a + a(b + aa)^*b)^* \end{aligned}$$

Now:

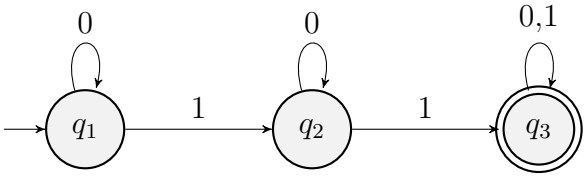
$$\begin{aligned} q_3 &= q_2a \\ &= q_1a(b + aa)^*a \\ &= (a + a(b + aa)^*b)^*a(b + aa)^*a \\ &\Rightarrow \mathcal{A} \end{aligned}$$

1.5 Equivalence of Two Finite Automata

Two finite automata over Σ are equivalent if they accept the same set of strings over Σ.

Example 1.5.1. Consider the following two DFAs M and M' over $\{0, 1\}$. Find out whether M and M' are equivalent.

Solution:



(q_i, q_i')	(q_0, q_0')	(q_3, q_1')
(q_1, q_1)	(q_1, q_1)	(q_3, q_3)
(q_3, q_1)	(q_3, q_3)	(q_3, q_3)
(q_3, q_2)	(q_3, q_2)	(q_3, q_2)
(q_2, q_2)	(q_3, q_2)	(q_3, q_4)

The first element of the first column must be (q_1, q_1), i.e., initial state of M and M'.

As we do not get a pair (q, q') where q is a final state and q' is a non-final state (or vice versa) at every row, we proceed until all the elements in the second and third column are also in the first column. Therefore, M and M' are equivalent.

Chapter 2

Minimization of Finite Automata

2.1 Minimization Definitions

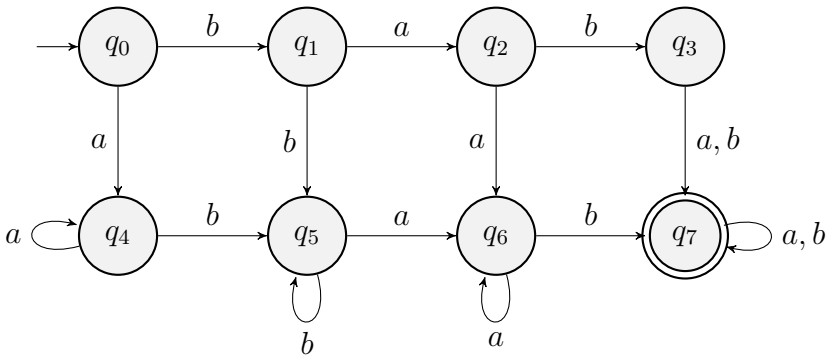
Definition 2.1.1. Two states q_1 and q_2 are equivalent if both $\delta(q_1, x)$ and $\delta(q_2, x)$ are final states or both of them are non-final states for all $x \in \Sigma^*$.

Definition 2.1.2. Two states q_1 and q_2 are k-equivalent ($k \geq 0$) if both $\delta(q_1, x)$ and $\delta(q_2, x)$ are final states or both of them are non-final states for all strings x of length k or less. In particular, any two final states are 0-equivalent and any two non-final states are also 0-equivalent.

2.2 Example: Construct the Minimum State Automaton

Example 2.2.1. Construct the minimum state automaton equivalent to the transition diagram given below:

Solution:

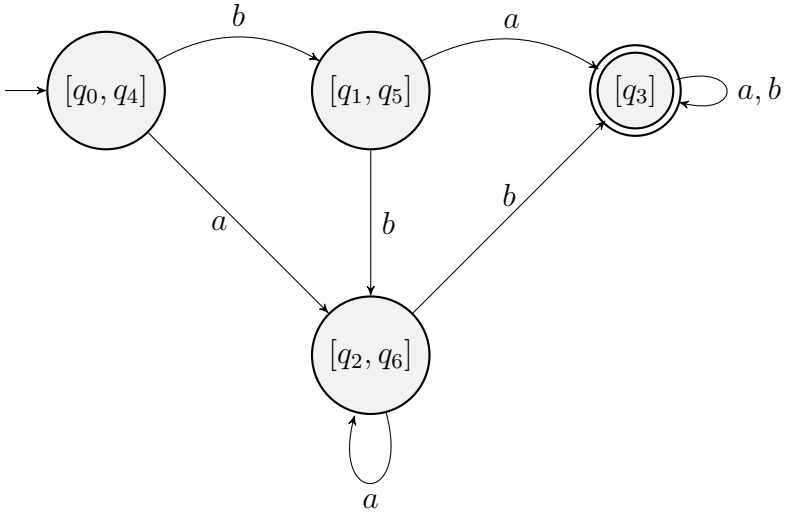


State/Σ	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_2	q_2
q_2	q_3	q_2
q_3	q_1	q_3
q_4	q_5	q_4
q_5	q_2	q_4
q_6	q_1	q_4

After minimization:

$$\Pi_0 = \{[q_3], [q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7]\}$$
$$\Pi_1 = \{[q_3], [q_0, q_2, q_4, q_5], [q_1, q_6, q_7]\}$$
$$\Pi_2 = \{[q_3], [q_0, q_2], [q_1, q_6], [q_4, q_5, q_7]\}$$
$$\Pi_3 = \{[q_3], [q_1, q_6], [q_2, q_5], [q_0, q_4]\}$$

The minimum state automaton:



Chapter 3

Formal Languages

3.1 Introduction

Definition 3.1.1 (Language)**.** A language generated by a grammar G (denoted by $L(G)$) is defined as:

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$$

If w contains non-terminals, it is called a sentential form.
If w does not contain non-terminals, it is called a sentence of G.

3.1.1 Grammar Definition

Definition 3.1.2. $L(G)$ is the set of all terminal strings derived from the start symbol S.

Two grammars are equivalent if they produce the same language.

3.2 Chomsky Classification of Languages

Definition 3.2.1 (Type-0)**.** A type-0 grammar is any phrase-structured grammar without any restriction.

A production without any restriction is called a type-0 production.

Definition 3.2.2 (Type-1)**.** A production of the form $\Phi A \Psi \to \Phi \alpha \Psi$ is called a type-1 production if $\alpha \neq \Lambda$.

In type-1 productions, erasing of A is not permitted.

Example 3.2.1. Examples of productions:

- (i) $AB \to A\underline{BC}$ is a type-1 production. The left context is A, right context is Λ.
- (ii) $A \to abA$ is a type-1 production. Here both the left and right context are Λ.

A grammar is called type-1 or context-sensitive if all its productions are type-1 productions. The production $S \to \Lambda$ is also allowed in a type-1 grammar, but in this case S does not appear on the right-hand side of any production.

The language generated by a type-1 grammar is called a type-1 or context-sensitive language.

In a context-sensitive grammar G, we allow $S \to \Lambda$ for including Λ in $L(G)$. Apart from $S \to \Lambda$, all the productions must be in the form where $\Lambda \in L(G)$. When $\Lambda \in L(G)$, we assume that S does not appear on the R.H.S. of any production.

Definition 3.2.3 (Type-2)**.** A type-2 production is a production of the form $A \to \alpha$ where $A \in V_N$ and $\alpha \in (V_N \cup \Sigma)^*$. In other words, the L.H.S. has no left context and right context.

Example 3.2.2. Examples:

- $S \to Aa$, $A \to a$, $B \to ab\epsilon$, $A \to \Lambda$

A grammar is called a type-2 grammar if it contains only type-2 productions. It is also called a context-free grammar.

A language generated by a context-free grammar is called a type-2 language or context-free language.

Definition 3.2.4 (Type-3)**.** A production of the form $A \to a$ or $A \to aB$ where $A, B \in V_N$ and $a \in \Sigma$, is called a type-3 production.

A grammar is called a type-3 or regular grammar if all its productions are type-3 productions. A production $S \to \Lambda$ is allowed in a type-3 grammar, but in this case S does not appear on the right-hand side of any production.

A language is regular if it can be described by a regular expression.

3.3 Relation Between Languages

$$L_{RL} \subseteq L_{CFL} \subseteq L_{CSL} \subseteq L_0$$

Two grammers of different types may generate the same language.

Operation	L_{RL}	L_{CFL}	L_{CSL}	L_0
Union	✓	✓	✓	✓
Concatenation	✓	✓	✓	✓
Transpose	✓	✓	✓	✓
Intersection	✓	×	✓	✓
Complement	✓	×	✓	✓
Kleene Star	✓	✓	✓	✓

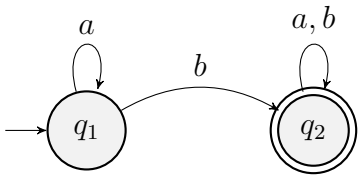
L_{CFL} is not closed under intersection, but the intersection of a context-free language and a regular language is context-free.

3.4 Construction of a Transition System Accepting a Regular Grammar

Example 3.4.1. Construct a regular grammar G generating the regular set represented by $P = a^*b(a+b)^*$.

Solution:

DFA corresponding to P is:



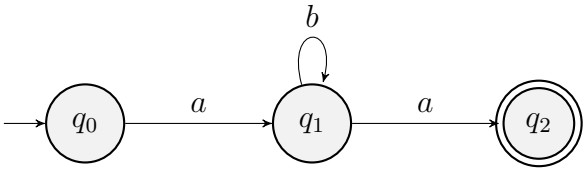
Regular grammar corresponding to P is:

$$A_0 \to aA_0, \quad A_0 \to bA_1,$$
$$A_1 \to aA_1, \quad A_1 \to bA_1, \quad A_1 \to \Lambda$$

Example 3.4.2. Construction of a transition system accepting a regular grammar G:

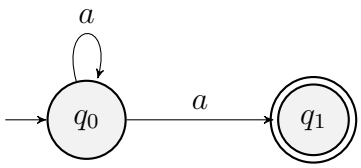
$$G = (\{A_0, A_1\}, \{a, b\}, P, A_0)$$
$$P = \{A_0 \to aA_1, A_1 \to bA_1, A_1 \to a, A_1 \to bA_1\}$$

Solution:



Example 3.4.3. $S \to aS \mid a$

Solution:



Chapter 4

Context-Free Languages

4.1 Property of Languages That Are Regular

As a string is scanned left to right, the amount of memory that is required in order to determine at the end whether or not the string is in the language is the language is bounded and independent on the language, not the particular input string.

Regular languages with an infinite number of strings are represented by automata with cycles and regular expressions involving the Kleene star. It means they must have some repetitive structure.

Example 4.1.1. $L(G) = \{a^n \mid n \geq 1\}$ is a prime) is not regular.
$L = \{w \in \{a, b\}^* \mid w$ has an equal number of a's and b's$\}$ is not regular.
$L = a^*b^*$ is regular.

Every finite subset of a non-regular set is regular.

4.2 Ambiguity in Context-Free Grammars

Definition 4.2.1 (Ambiguity in CFG)**.** A terminal string $w \in L(G)$ is ambiguous if there exists two or more derivation trees for w.

A context-free grammar G is ambiguous if there exists some $w \in L(G)$ which is ambiguous.

4.2.1 Construction of Reduced Grammars

For every CFG G, there exists a reduced grammar G' which is equivalent to G.

We construct the reduced grammar in two steps:

Step 1: We construct a grammar G_1 equivalent to the given grammar G so that every variable in G_1 derives some terminal string.

Step 2: We construct a grammar G' equivalent to G_1 so that every symbol in G' appears in some sentential form of G.

G' is the required reduced grammar.

Chapter 5

Elimination of Null Productions

Definition 5.0.1 (Null Production)**.** A production of the form $A \to \Lambda$ where A is a variable, is called a null production.

A variable A in a context-free grammar is nullable if $A \Rightarrow^* \Lambda$.

If $G = (V_N, \Sigma, P, S)$ is a context-free grammar, then we can find a context-free grammar G_1 having no null or unit productions such that $L(G_1) = L(G) - \{\Lambda\}$.

5.1 Example: Find a Reduced Grammar Equivalent to the Given Grammar

Example 5.1.1. Find a reduced grammar equivalent to the grammar G where productions are:

$$S \to AB \mid CA, \quad A \to a,$$
$$B \to BC \mid AB, \quad A \to aB \mid \epsilon \mid b, \quad C \to aB \mid b$$

Solution:

Step-1: Construction of set W of all nullable variables:

$$W_0 = \{S, A\}, \quad W_1 = \{S, A\} = W_0$$
$$S \therefore U = \{S, A\}$$

Step-2: Construction of P':

$$S \to ASB \mid SB \mid AB \mid B$$

$$A \to aAS \mid aS \mid aA \mid AB \mid B \mid a$$
$$B \to SbS \mid Sb \mid bS \mid b \mid Aa \mid a$$

Example 5.1.2. Remove left recursion from the following grammar:

$$S \to Aa \mid Aa\epsilon \mid Aa_1 \ldots Aa_n \mid B_n$$

then

$$P_1 = \{A \to BY \mid B_1Y_1 \mid \ldots \mid B_nY_n\}$$
$$A \to B_1 \mid B_2 \mid \ldots \mid B_n \mid \Lambda$$

5.2 Lemma 1: Deleting a Variable B Appearing as the First Symbol on the R.H.S.

Lemma 5.2.1. Let $P = \{A \to BY, B \to \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n\}$, then

$$P_1 = \{A \to \beta_1 Y \mid \beta_2 Y_1 \mid \ldots \mid \beta_n\}$$

5.3 Lemma 2: Eliminating Left Recursion

Lemma 5.3.1. Let $P = \{A \to A\alpha_1 \mid A\alpha_2 \mid \ldots \mid A\alpha_n \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_m\}$ where no β_i 's production $\alpha_i \in \alpha \in (\Sigma \cup V_N)^*$.

$P_1 = \{A \to \beta_1 Z \mid \beta_2 Z \mid \ldots \mid \beta_m Z, Z \to \alpha_1 \mid \alpha_2 Z_1 \mid \ldots \mid \alpha_n Z, Z \to \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_n Z_n\}$ ...

Chapter 6

Greibach Normal Form

6.1 Definition

Definition 6.1.1 (Greibach Normal Form)**.** A context-free grammar generating the set accepted by a pushdown automaton is in Greibach Normal Form.

A context-free grammar is in Greibach Normal Form if every production is of the form:

$$A \to a\alpha, \quad \text{where } \alpha \in V_N \text{ and } \alpha \in \Sigma$$

(or may be Λ), and $S \to \Lambda$ is an Λ and where S does not appear on R.

Chapter 7

Chomsky Normal Form for Context-Free Grammars

Definition 7.0.1 (Chomsky Normal Form)**.** A context-free grammar G is in Chomsky normal form if every production is of the form:

$$A \to a, \quad \text{or} \quad A \to BC,$$

and $S \to \Lambda$ is in G if $\Lambda \in L(G)$.

When Λ is in $L(G)$, we assume that S does not appear on the R.H.S of any production.

7.1 Reduction to Chomsky Normal Form

Step 1: Elimination of null productions and unit productions.
Step 2: Elimination of terminals on R.H.S.
We define $G_2 = (V_N, \Sigma, P_2, S)$ as follows:

(i) All productions in P_1 are added to P_2 if they are in the required form. All the variables in V_1 are added to V_N.

(ii) If $S \to aD$ gives rise to $S \to CaD$ and $Ca \to b$

(iii) $A \to bAB$ gives rise to $A \to CbAB$ and $Cb \to b$

(iv) $A \to bAB$ gives rise to $A \to CbAB$ and $Cb \to b$

(v) $V_N = \{S, A, B, D, Ca, Cb\}$

Step 3: $P_1 = \{S \to CaAD, A \to CaCb \mid CaCb\}$

$$Ca \to a, Cb \to b$$

$A \to CaB, B \to b, D \to d, Ca \to a, Cb \to b$ are added
$S \to CaAD$ is replaced by $S \to CaCb$ and $C \to AD$

$$A \to CbAB \text{ is replaced by } A \to CaC_1 \text{ and } C_2 \to AB$$

Now, $G_n = (\{S, A, B, D, Ca, Cb, C_1, C_2\}, \{a, b, d\}, P_3, S)$ where

$$P_3 = \{S \to CaC_1, A \to CaB \mid CaCb \mid CaC_2\}, \ldots$$

Example 7.1.1. Example: Remove left recursion from the following grammar:

$$A \to aBD \mid bDB \mid \epsilon, \quad A \to AB \mid AD$$

Solution:

$$A \to aBD \mid bDB \mid \epsilon, \quad A \to aBDZ \mid bDBZ \mid \epsilon Z$$
$$Z \to BZ \mid DZ \mid B \mid D$$

Example 7.1.2. Remove A-productions from the following grammar:

$$S \to ASB \mid A, \quad A \to aAS \mid aB \mid AB \mid B \mid a$$
$$B \to Sbs \mid Sb \mid bS \mid b \mid Aa \mid a$$

Solution: Step-1: Construction of set W of all nullable variables:

$$W = \{S, A\}, \quad W_1 = \{S, A\} = W_0$$
$$\therefore U = \{S, A\}$$

Step-2: Construction of P':

$$S \to ASB \mid AB \mid B$$
$$A \to a$$

7.2 Remove A-production from the following grammar

Example 7.2.1.

$$A \to aBDI \mid bDBZ \mid \epsilon$$
$$Z \to BZ \mid DZ \mid B \mid D$$

Solution:

$$A \to aBDIBDBZ \mid \epsilon, \quad A \to aBDZ \mid bDBZ \mid Z$$
$$Z \to BZ \mid DZ \mid B \mid D$$

Step-1: Construction of set W of all nullable variables:

$$W_0 = \{S, A\}, \quad W_1 = W_2(\epsilon S) = W_0(\epsilon S) \cup \phi = W_0(\epsilon S)$$

Hence $W(\epsilon S) = \{S\}$

Similarly:

$$\begin{aligned} W(A) &= \{A\}, \quad W(E) = \{E\} \\ W_0(B) &= \{B\}, \quad W(B) = \{B\} \\ W_0(C) &= \{C\}, \quad W_1(E) = \{B, C\} \cup \{E\} = \{B, C, E\} \\ W_2(B) &= \{B, C, DE\} \\ W_3(B) &= \{B, CDE\} \end{aligned}$$

Therefore $W(B) = \{B, C, DE\}$.
Step-2: The productions in G_1 are:

$$\begin{aligned} &S \to AB, \quad A \to a, \quad E \to a, \\ &B \to aD \mid D \quad \text{ or } B \to CaD \text{ and } C \to AD \end{aligned}$$

$S \to CaAD$ is replaced by $S \to CaC_1$ and $C \to AD$ ...
Now, $G_n = (\{S, A, B, D, Ca, Cb, C_1, C_2\}, \{a, b, d\}, P_3, S)$ where

$$P_3 = \{S \to CaC_1, A \to CaB \mid CaCb \mid Ca\}$$

Note: $L = \{a^n b^c \mid n \geq 1\}$ is not context-free but context-sensitive.
Also, $L = \{a^n \mid p \text{ is a prime}\}$ is not context-free.

Chapter 8

Pushdown Automata

8.1 Introduction

Definition 8.1.1 (Pushdown Automaton)**.** Each context-free language is accepted by some pushdown automaton.

If a language is accepted by a pushdown automaton, it is a context-free language.

8.1.1 Pushdown Automaton for $(a^n b^n \mid n \geq 1)$ (Initial Pushdown Symbol)

Let $A = (Q, \Sigma, \Gamma, \delta, q_0, \Sigma_0, F)$

$$\delta \Rightarrow Q \times (\Sigma \cup \{\Lambda\}) \times \Gamma \to Q \times \Gamma^*$$

where: $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{a, b\}$, $\Gamma = \{a, Z_0\}$,
$F = \{q_2\}$ and
S is given by:

$$\begin{aligned}
\delta(q_0, a, Z_0) &= (q_0, aZ_0) \\
\delta(q_0, a, a) &= (q_0, aa) \\
\delta(q_0, b, a) &= (q_1, \Lambda) \\
\delta(q_1, b, a) &= (q_1, \Lambda) \\
\delta(q_1, \Lambda, Z_0) &= (q_2, \Lambda)
\end{aligned}$$

8.2 Infinite Context-Free Languages

Infinite context-free languages display periodicity—of a somewhat subtler form than do regular languages.

8.3 Determinism and Parsing

A pushdown automaton is not of immediate use in parsing because it's a nondeterministic device.

A pushdown automaton M is deterministic if for each configuration, there is at most one configuration that can succeed it in a computation by M.

Deterministic Context-Free Languages are essentially those that are accepted by deterministic pushdown automaton.

There are some context-free languages that cannot be accepted by deterministic pushdown automata.

The machine to accept the language $\{wcw^R \mid w \in \{a, b\}^*\}$ is deterministic. We use the machine to accept $\{w \in \{a, b\}^* \mid w = w^R\}$ is nondeterministic.

The class of deterministic context-free languages is closed under complement.

Nondeterminism is more powerful than determinism in the context of pushdown automaton.

Chapter 9

Turing Machines and Recursive Enumerable Languages

9.1 Turing Machines

Definition 9.1.1 (Turing Machine)**.** A Turing machine consists of a finite control, a tape, and a head that can be used for reading or writing on that tape.

The Turing machines can form a stable and maximal class of computational devices, in terms of the computations they can perform.

A Turing machine M does not accept w if the machine either halts in a non-accepting state or does not halt.

A general purpose Turing machine is usually called a universal Turing machine which is powerful enough to simulate the behavior of any computer including any Turing machine itself.

If we increase the number of heads or make the tape two or three dimensional, the Turing machine will almost speedup the operation of the machine, but will not increase the computing power of doing something more than basic model.

9.2 Linear Bounded Automaton

Definition 9.2.1 (Linear Bounded Automaton)**.** A Linear Bounded Automaton is a nondeterministic Turing machine which has a single tape whose length is not infinite but bounded by a linear function of the length of the input string.

The set of languages accepted by nondeterministic Turing machine is same as the set of languages accepted by deterministic Turing machine.

In linear bounded automaton, the length of the tape is restricted.

If L is a context-sensitive language, then L is accepted by a Linear Bounded Automaton (LBA). The converse is also true.

Chapter 10

Recursive and Recursive Enumerable Languages

10.1 Definitions

Definition 10.1.1 (Recursive Enumerable Language)**.** A language is recursive enumerable if and only if there is a Turing machine M that semidecides it.

OR

A recursive enumerable set is a set X for which we have a procedure to determine whether a given element belongs to X or not.

Every recursive language is recursive enumerable, but not the two terms are not the same.

Recursive Language $\subseteq$ Recursive Enumerable Language

The class of recursive languages is a proper subset of the class of recursively enumerable languages.

If a L is a recursive language, then its complement L^c is also recursive.

A language is recursive if and only if both it and its complement are recursively enumerable.

The class of recursively enumerable languages is not closed under complement.

A language is generated by a grammar if and only if it is recursively enumerable.

Recursive Enumerable Language = Type-0 Language.

Chapter 11

Undecidability

11.1 Halting Problem

Definition 11.1.1 (Halting Problem)**.** Halting Machine likes as input an encoding of a Turing machine $e(M)$ and an encoding of an input string $e(w)$, and returns "yes" if M halts on w and "no" if M does not halt on w.

The halting problem is undecidable:

→ There exists no Turing machine that decides it.

→ There is no Turing machine that halts on all inputs, and always say "yes" if M halts on w, and always say "no" if M does not halt on w.

The following problems about Turing machines are undecidable:

(i) Given a Turing machine M, does M halt on the empty tape?

(ii) Given two Turing machines M_1 and M_2, do they have the same input strings?

(iii) Given a Turing machine M, is the language that M semidecides regular? Is it context-free? Is it recursive?

11.2 Reduction

To solve a problem B as undecidable:

(i) Start with an instance of a known undecidable problem.

(ii) Create an instance of problem B, such that the answer to the instance of problem B gives the answer to the undecidable problem.

(iii) If we could solve problem B, we could solve the halting problem.

(iv) Thus, problem B must be undecidable.

11.3 Unsolvable Problems about Grammars

If we can reduce problem A to problem B, and problem A is undecidable, then:
it tells nothing about problem B.
If we can reduce problem B to problem A, and problem A is undecidable, then:
B tells nothing about problem B.
Each of the following problems is undecidable:

(1) Given a grammar G and a string w, is $w \in L(G)$?

(2) Given a CFG G, is $L(G) = \Sigma^*$?

(3) Given two CFGs G_1 and G_2, is $L(G_1) = L(G_2)$?

(4) Given two PDA M_1 and M_2, is $L(M_1) = L(M_2)$?

However, there are some problems about CFGs that are undecidable:

(1) Given any CFG G, is $L(G) = \emptyset$?

(2) Given any two CFGs G_1 and G_2, is $L(G_1) = L(G_2)$?

(3) Given two PDA M_1 and M_2, is $L(M_1) = L(M_2)$?

(4) Given a PDA M, find an equivalent PDA with the smallest possible number of states.

(5) Tiling problem is undecidable.

(6) Ambiguity problem for CFGs is undecidable.

The following questions about Context-Free Grammars are decidable:

(1) Given a C.F.G. G and string w, is $w \in L(G)$?

(2) Given a CFG G, is $w\wedge \in L(G)$?

Chapter 12

Some Important Results

12.1 Regular Expression Results

(1) Regular Expression for all strings over $\{0, 1\}$:

- have an even number of 1's: $0^*(10^*10^*)^*$
- have no occurrences of 00: $1^*(01^*)^*(01^*)$
- have exactly one occurrence of 00: $1^*(01^*)^*00(1 + 01)^*1^*$
- do not contain 01: 1^*0^*

(2) $L = \{a^n b^c \mid n \geq 1\}$ is not context-free, but context-sensitive.

(3) $L = \{a^n \mid p \text{ is a prime}\}$ is not a context-free language.

(4) $L = \{a^n l^n \mid n \geq 1\}$ is not regular, but context-free.

(5) $L = \{a^n \mid n \geq 0\}$ is not context-free.

(6) $L = \{a^n \mid n \geq 1\}$ is regular.

(7) $L = \{babaabaaa \ldots ba^{2n}a^n \mid n \geq 1\}$ is not context-free.

(8) $L = \{wxw^T \mid w \in \{a, b\}^*\}$ is regular.

(9) $L = \{ww^T \mid w \in \{a, b\}^*\}$ is context-free.

Chapter 13

NP Completeness

13.1 Tractable Problems

Definition 13.1.1 (Tractable Problems)**.** The set of all the problems that can be solved within polynomial amount of time using deterministic machines is of type $O(n^k)$.

13.2 Intractable Problems

Definition 13.2.1 (Intractable Problems)**.** The set of all problems that can't be solved within polynomial amount of time using deterministic machines is of type $O(c^n)$.

13.3 P-Class Problems

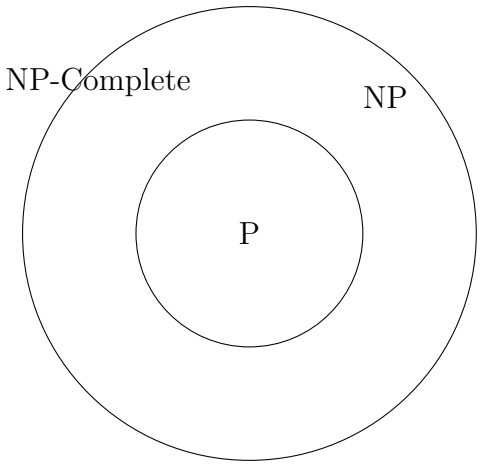
The set of problems that can be solved in rational polynomial amount of time by deterministic machines, is called the P-class or polynomial problems.

13.4 NP-Class Problems

The set of problems that can be solved in polynomial time using non-deterministic machines.

$$NP = \text{Non-deterministic Polynomial Time}$$

$$NP \neq \text{Non-polynomial}$$



$$P \subseteq NP$$

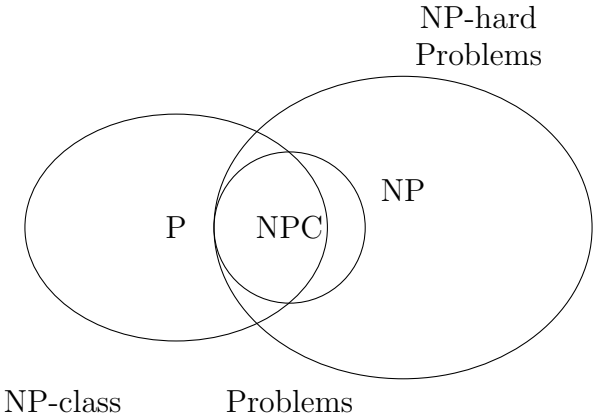
The class NP consists of those problems that are verifiable in polynomial time.

13.5 NP-Hard Problems

These problems are at least as hard as the hardest problems in NP, but not necessarily in NP.

The problems to which all NP-class problems are reducible in polynomial time, are known as NP-hard class of problems.

13.5.1 P-class Problems



13.6 NP-Complete Problems

NP-Complete problems are those NP-hard problems which are within NP.

These are not of the hardest problems in NP, i.e., in real sense these are the ones that most likely not to be in P.

NP-Complete problems are solvable using heuristic.

It is not known whether NP problems are tractable or intractable.

If a problem in a class of NP-Complete can be solved in polynomial time, then all the problems in that class can be solved in a polynomial time using the same algorithm.

If a deterministic polynomial time algorithm is found for an NP-Complete problem, then every problem in NP-class can be solved in deterministic polynomial time.

The class of all polynomially decidable languages denoted by P is closed under union, intersection, concatenation, complement, and Kleene Star.

13.7 Examples of NP-Complete Problems

(1) Bounded Tiling

(2) 3-Satisfiability / Satisfiability (SAT)

(3) MAX-SAT

(4) Hamiltonian Cycle

(5) Traveling Salesman Problem

(6) Exact Cover and Node Cover

(7) Independent Set

(8) 0-1 Knapsack

(9) Bin Packing

(10) Printed Circuit Board (PCB)

(11) CLIQUE

www.ankurgupta.net